DOCKER SCENARIO WORKBOOK

550 Guided Labs with Step-by-Step Commands & Conclusions

Scenario-based practice

Do this → verify it → diagnose the result → write a conclusion

Every lab is organized around a practical situation, a sequence of commands, verification evidence, and a final conclusion prompt. Practice in a disposable environment and write your own findings before reading the model conclusion guidance.

Coverage includes Docker CLI and Engine, images, containers, volumes, networking, builds, Compose, registries, security, Swarm, troubleshooting, and automation.

Docker changes across versions and platforms. Use the built-in help for your installed CLI. A few labs require a Dockerfile, Compose file, registry, existing container, or disposable Swarm cluster; these prerequisites are stated in the lab steps.

How to work through each scenario

1. Read the situation. Treat it as a small ticket from a developer, operator, or teammate.

2. Perform the steps in order. Run one command at a time and observe the output before moving on.

3. Verify each result. A command returning successfully does not necessarily mean the application is healthy or reachable.

4. Write a conclusion. State what the evidence proves, what remains uncertain, and the next safe action.

5. Protect data. Use only disposable resources. Do not use real credentials, production volumes, or shared cluster resources in destructive exercises.

Scenario index

01. CLI & Engine — Investigate a Docker host before deployment
02. CLI & Engine — Learn a command from built-in help
03. CLI & Engine — Track lifecycle events during a deployment
04. CLI & Engine — Choose the right cleanup target
05. CLI & Engine — Compare active and inactive contexts
06. Images — Pull and verify a base image
07. Images — Inspect image defaults before running
08. Images — Tag an image for a local release
09. Images — Save and reload an image for offline transfer
10. Images — Compare image history and size
11. Images — Publish an image to a registry you control
12. Containers — Run a one-shot job
13. Containers — Launch and expose a web server
14. Containers — Configure a container with environment variables
15. Containers — Use a read-only container filesystem
16. Containers — Pass a command and understand exit codes
17. Containers — Inspect, rename, and identify a container
18. Containers — Execute a command in a running container
19. Containers — Copy a file out and into a container
20. Containers — Graceful stop versus immediate kill
21. Containers — Set restart behavior and observe it
22. Containers — Apply CPU and memory limits
23. Containers — Run with a non-root user and fewer capabilities
24. Volumes — Prove that named-volume data persists
25. Volumes — Inspect a volume before using it
26. Volumes — Back up a named volume to an archive
27. Volumes — Restore a volume backup
28. Volumes — Compare bind mounts with named volumes
29. Volumes — Clean up volume data safely
30. Networks — Create a custom bridge network
31. Networks — Connect and disconnect a running container
32. Networks — Diagnose a published-port problem
33. Networks — Compare network isolation
34. Networks — Remove an unused custom network

35. Builds — Build an image from a Dockerfile
36. Builds — Diagnose a failed Docker build
37. Builds — Understand build cache behavior
38. Builds — Use build arguments correctly
39. Builds — Build a named multi-stage target
40. Builds — Reduce unnecessary files in the build context
41. Builds — Inspect a Buildx builder
42. Compose — Validate a Compose project before launch
43. Compose — Bring up a multi-service app
44. Compose — Diagnose a service that exits
45. Compose — Execute a command in a running service
46. Compose — Run a one-off migration or maintenance job
47. Compose — Update a Compose service after a configuration change
48. Compose — Tear down a lab Compose project safely
49. Compose — Separate projects with project names
50. Registries — Diagnose an image-pull failure
51. Registries — Promote an image from test to release tag
52. Security — Run a container without extra privileges
53. Security — Avoid embedding credentials in images
54. Security — Audit exposed ports and running containers
55. Security — Check health status instead of only process status

The workbook contains 55 scenario patterns, each repeated with a unique lab case number and resource names, for 550 sequential scenario exercises.

SC-001 · CLI & Engine · Investigate a Docker host before deployment

Situation: You have inherited a development machine and must decide whether it is ready for a new workload.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check which Docker client and server versions are installed.

```
docker version
```

Verify / observe: Identify both Client and Server versions; note if the daemon is unreachable.

Step 2 — Review the Engine configuration and runtime environment.

```
docker info
```

Verify / observe: Record the storage driver, runtimes, operating system, and security settings.

Step 3 — Check current disk consumption before making changes.

```
docker system df -v
```

Verify / observe: Identify which object types use the most space; do not delete anything yet.

Final conclusion: Write a short readiness verdict: what is healthy, what is unknown, and what should be checked next?

Concept to retain: Diagnosis should precede changes. Version and host details determine which options and features are available.

My notes: _____

SC-002 · CLI & Engine · Investigate a Docker host before deployment

Situation: You have inherited a development machine and must decide whether it is ready for a new workload.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check which Docker client and server versions are installed.

```
docker version
```

Verify / observe: Identify both Client and Server versions; note if the daemon is unreachable.

Step 2 — Review the Engine configuration and runtime environment.

```
docker info
```

Verify / observe: Record the storage driver, runtimes, operating system, and security settings.

Step 3 — Check current disk consumption before making changes.

```
docker system df -v
```

Verify / observe: Identify which object types use the most space; do not delete anything yet.

Final conclusion: Write a short readiness verdict: what is healthy, what is unknown, and what should be checked next?

Concept to retain: Diagnosis should precede changes. Version and host details determine which options and features are available.

My notes: _____

SC-003 · CLI & Engine · Investigate a Docker host before deployment

Situation: You have inherited a development machine and must decide whether it is ready for a new workload.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check which Docker client and server versions are installed.

```
docker version
```

Verify / observe: Identify both Client and Server versions; note if the daemon is unreachable.

Step 2 — Review the Engine configuration and runtime environment.

```
docker info
```

Verify / observe: Record the storage driver, runtimes, operating system, and security settings.

Step 3 — Check current disk consumption before making changes.

```
docker system df -v
```

Verify / observe: Identify which object types use the most space; do not delete anything yet.

Final conclusion: Write a short readiness verdict: what is healthy, what is unknown, and what should be checked next?

Concept to retain: Diagnosis should precede changes. Version and host details determine which options and features are available.

My notes: _____

SC-004 · CLI & Engine · Investigate a Docker host before deployment

Situation: You have inherited a development machine and must decide whether it is ready for a new workload.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check which Docker client and server versions are installed.

```
docker version
```

Verify / observe: Identify both Client and Server versions; note if the daemon is unreachable.

Step 2 — Review the Engine configuration and runtime environment.

```
docker info
```

Verify / observe: Record the storage driver, runtimes, operating system, and security settings.

Step 3 — Check current disk consumption before making changes.

```
docker system df -v
```

Verify / observe: Identify which object types use the most space; do not delete anything yet.

Final conclusion: Write a short readiness verdict: what is healthy, what is unknown, and what should be checked next?

Concept to retain: Diagnosis should precede changes. Version and host details determine which options and features are available.

My notes: _____

SC-005 · CLI & Engine · Investigate a Docker host before deployment

Situation: You have inherited a development machine and must decide whether it is ready for a new workload.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check which Docker client and server versions are installed.

```
docker version
```

Verify / observe: Identify both Client and Server versions; note if the daemon is unreachable.

Step 2 — Review the Engine configuration and runtime environment.

```
docker info
```

Verify / observe: Record the storage driver, runtimes, operating system, and security settings.

Step 3 — Check current disk consumption before making changes.

```
docker system df -v
```

Verify / observe: Identify which object types use the most space; do not delete anything yet.

Final conclusion: Write a short readiness verdict: what is healthy, what is unknown, and what should be checked next?

Concept to retain: Diagnosis should precede changes. Version and host details determine which options and features are available.

My notes: _____

SC-006 · CLI & Engine · Investigate a Docker host before deployment

Situation: You have inherited a development machine and must decide whether it is ready for a new workload.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check which Docker client and server versions are installed.

```
docker version
```

Verify / observe: Identify both Client and Server versions; note if the daemon is unreachable.

Step 2 — Review the Engine configuration and runtime environment.

```
docker info
```

Verify / observe: Record the storage driver, runtimes, operating system, and security settings.

Step 3 — Check current disk consumption before making changes.

```
docker system df -v
```

Verify / observe: Identify which object types use the most space; do not delete anything yet.

Final conclusion: Write a short readiness verdict: what is healthy, what is unknown, and what should be checked next?

Concept to retain: Diagnosis should precede changes. Version and host details determine which options and features are available.

My notes: _____

SC-007 · CLI & Engine · Investigate a Docker host before deployment

Situation: You have inherited a development machine and must decide whether it is ready for a new workload.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check which Docker client and server versions are installed.

```
docker version
```

Verify / observe: Identify both Client and Server versions; note if the daemon is unreachable.

Step 2 — Review the Engine configuration and runtime environment.

```
docker info
```

Verify / observe: Record the storage driver, runtimes, operating system, and security settings.

Step 3 — Check current disk consumption before making changes.

```
docker system df -v
```

Verify / observe: Identify which object types use the most space; do not delete anything yet.

Final conclusion: Write a short readiness verdict: what is healthy, what is unknown, and what should be checked next?

Concept to retain: Diagnosis should precede changes. Version and host details determine which options and features are available.

My notes: _____

SC-008 · CLI & Engine · Investigate a Docker host before deployment

Situation: You have inherited a development machine and must decide whether it is ready for a new workload.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check which Docker client and server versions are installed.

```
docker version
```

Verify / observe: Identify both Client and Server versions; note if the daemon is unreachable.

Step 2 — Review the Engine configuration and runtime environment.

```
docker info
```

Verify / observe: Record the storage driver, runtimes, operating system, and security settings.

Step 3 — Check current disk consumption before making changes.

```
docker system df -v
```

Verify / observe: Identify which object types use the most space; do not delete anything yet.

Final conclusion: Write a short readiness verdict: what is healthy, what is unknown, and what should be checked next?

Concept to retain: Diagnosis should precede changes. Version and host details determine which options and features are available.

My notes: _____

SC-009 · CLI & Engine · Investigate a Docker host before deployment

Situation: You have inherited a development machine and must decide whether it is ready for a new workload.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check which Docker client and server versions are installed.

```
docker version
```

Verify / observe: Identify both Client and Server versions; note if the daemon is unreachable.

Step 2 — Review the Engine configuration and runtime environment.

```
docker info
```

Verify / observe: Record the storage driver, runtimes, operating system, and security settings.

Step 3 — Check current disk consumption before making changes.

```
docker system df -v
```

Verify / observe: Identify which object types use the most space; do not delete anything yet.

Final conclusion: Write a short readiness verdict: what is healthy, what is unknown, and what should be checked next?

Concept to retain: Diagnosis should precede changes. Version and host details determine which options and features are available.

My notes: _____

SC-010 · CLI & Engine · Investigate a Docker host before deployment

Situation: You have inherited a development machine and must decide whether it is ready for a new workload.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check which Docker client and server versions are installed.

```
docker version
```

Verify / observe: Identify both Client and Server versions; note if the daemon is unreachable.

Step 2 — Review the Engine configuration and runtime environment.

```
docker info
```

Verify / observe: Record the storage driver, runtimes, operating system, and security settings.

Step 3 — Check current disk consumption before making changes.

```
docker system df -v
```

Verify / observe: Identify which object types use the most space; do not delete anything yet.

Final conclusion: Write a short readiness verdict: what is healthy, what is unknown, and what should be checked next?

Concept to retain: Diagnosis should precede changes. Version and host details determine which options and features are available.

My notes: _____

SC-011 · CLI & Engine · Learn a command from built-in help

Situation: A teammate gave you a command but did not explain its flags. You must verify the syntax rather than guess.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List top-level Docker commands.

```
docker --help
```

Verify / observe: Find the command family relevant to the task.

Step 2 — Open help for the command family you plan to use.

```
docker container --help
```

Verify / observe: Locate the matching subcommand.

Step 3 — Read the options for a concrete operation.

```
docker run --help
```

Verify / observe: Identify the meaning of each option you intend to use.

Final conclusion: Which option changes behavior, and which argument names the target? Record the installed Docker version too.

Concept to retain: Built-in help is the most reliable first check for version-specific syntax.

My notes: _____

SC-012 · CLI & Engine · Learn a command from built-in help

Situation: A teammate gave you a command but did not explain its flags. You must verify the syntax rather than guess.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List top-level Docker commands.

```
docker --help
```

Verify / observe: Find the command family relevant to the task.

Step 2 — Open help for the command family you plan to use.

```
docker container --help
```

Verify / observe: Locate the matching subcommand.

Step 3 — Read the options for a concrete operation.

```
docker run --help
```

Verify / observe: Identify the meaning of each option you intend to use.

Final conclusion: Which option changes behavior, and which argument names the target? Record the installed Docker version too.

Concept to retain: Built-in help is the most reliable first check for version-specific syntax.

My notes: _____

SC-013 · CLI & Engine · Learn a command from built-in help

Situation: A teammate gave you a command but did not explain its flags. You must verify the syntax rather than guess.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List top-level Docker commands.

```
docker --help
```

Verify / observe: Find the command family relevant to the task.

Step 2 — Open help for the command family you plan to use.

```
docker container --help
```

Verify / observe: Locate the matching subcommand.

Step 3 — Read the options for a concrete operation.

```
docker run --help
```

Verify / observe: Identify the meaning of each option you intend to use.

Final conclusion: Which option changes behavior, and which argument names the target? Record the installed Docker version too.

Concept to retain: Built-in help is the most reliable first check for version-specific syntax.

My notes: _____

SC-014 · CLI & Engine · Learn a command from built-in help

Situation: A teammate gave you a command but did not explain its flags. You must verify the syntax rather than guess.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List top-level Docker commands.

```
docker --help
```

Verify / observe: Find the command family relevant to the task.

Step 2 — Open help for the command family you plan to use.

```
docker container --help
```

Verify / observe: Locate the matching subcommand.

Step 3 — Read the options for a concrete operation.

```
docker run --help
```

Verify / observe: Identify the meaning of each option you intend to use.

Final conclusion: Which option changes behavior, and which argument names the target? Record the installed Docker version too.

Concept to retain: Built-in help is the most reliable first check for version-specific syntax.

My notes: _____

SC-015 · CLI & Engine · Learn a command from built-in help

Situation: A teammate gave you a command but did not explain its flags. You must verify the syntax rather than guess.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List top-level Docker commands.

```
docker --help
```

Verify / observe: Find the command family relevant to the task.

Step 2 — Open help for the command family you plan to use.

```
docker container --help
```

Verify / observe: Locate the matching subcommand.

Step 3 — Read the options for a concrete operation.

```
docker run --help
```

Verify / observe: Identify the meaning of each option you intend to use.

Final conclusion: Which option changes behavior, and which argument names the target? Record the installed Docker version too.

Concept to retain: Built-in help is the most reliable first check for version-specific syntax.

My notes: _____

SC-016 · CLI & Engine · Learn a command from built-in help

Situation: A teammate gave you a command but did not explain its flags. You must verify the syntax rather than guess.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List top-level Docker commands.

```
docker --help
```

Verify / observe: Find the command family relevant to the task.

Step 2 — Open help for the command family you plan to use.

```
docker container --help
```

Verify / observe: Locate the matching subcommand.

Step 3 — Read the options for a concrete operation.

```
docker run --help
```

Verify / observe: Identify the meaning of each option you intend to use.

Final conclusion: Which option changes behavior, and which argument names the target? Record the installed Docker version too.

Concept to retain: Built-in help is the most reliable first check for version-specific syntax.

My notes: _____

SC-017 · CLI & Engine · Learn a command from built-in help

Situation: A teammate gave you a command but did not explain its flags. You must verify the syntax rather than guess.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List top-level Docker commands.

```
docker --help
```

Verify / observe: Find the command family relevant to the task.

Step 2 — Open help for the command family you plan to use.

```
docker container --help
```

Verify / observe: Locate the matching subcommand.

Step 3 — Read the options for a concrete operation.

```
docker run --help
```

Verify / observe: Identify the meaning of each option you intend to use.

Final conclusion: Which option changes behavior, and which argument names the target? Record the installed Docker version too.

Concept to retain: Built-in help is the most reliable first check for version-specific syntax.

My notes: _____

SC-018 · CLI & Engine · Learn a command from built-in help

Situation: A teammate gave you a command but did not explain its flags. You must verify the syntax rather than guess.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List top-level Docker commands.

```
docker --help
```

Verify / observe: Find the command family relevant to the task.

Step 2 — Open help for the command family you plan to use.

```
docker container --help
```

Verify / observe: Locate the matching subcommand.

Step 3 — Read the options for a concrete operation.

```
docker run --help
```

Verify / observe: Identify the meaning of each option you intend to use.

Final conclusion: Which option changes behavior, and which argument names the target? Record the installed Docker version too.

Concept to retain: Built-in help is the most reliable first check for version-specific syntax.

My notes: _____

SC-019 · CLI & Engine · Learn a command from built-in help

Situation: A teammate gave you a command but did not explain its flags. You must verify the syntax rather than guess.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List top-level Docker commands.

```
docker --help
```

Verify / observe: Find the command family relevant to the task.

Step 2 — Open help for the command family you plan to use.

```
docker container --help
```

Verify / observe: Locate the matching subcommand.

Step 3 — Read the options for a concrete operation.

```
docker run --help
```

Verify / observe: Identify the meaning of each option you intend to use.

Final conclusion: Which option changes behavior, and which argument names the target? Record the installed Docker version too.

Concept to retain: Built-in help is the most reliable first check for version-specific syntax.

My notes: _____

SC-020 · CLI & Engine · Learn a command from built-in help

Situation: A teammate gave you a command but did not explain its flags. You must verify the syntax rather than guess.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List top-level Docker commands.

```
docker --help
```

Verify / observe: Find the command family relevant to the task.

Step 2 — Open help for the command family you plan to use.

```
docker container --help
```

Verify / observe: Locate the matching subcommand.

Step 3 — Read the options for a concrete operation.

```
docker run --help
```

Verify / observe: Identify the meaning of each option you intend to use.

Final conclusion: Which option changes behavior, and which argument names the target? Record the installed Docker version too.

Concept to retain: Built-in help is the most reliable first check for version-specific syntax.

My notes: _____

SC-021 · CLI & Engine · Track lifecycle events during a deployment

Situation: A container unexpectedly restarts during a test. Capture the lifecycle events while reproducing the issue.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start an event stream in one terminal.

```
docker events --since 10m
```

Verify / observe: Keep this terminal open while you reproduce the lifecycle.

Step 2 — In another terminal, start a disposable test container.

```
docker run -d --name event-lab-1 alpine:3.20 sh -c 'sleep 20'
```

Verify / observe: A container ID should be returned.

Step 3 — Inspect its state after the short run.

```
docker ps -a --filter name=event-lab-1
```

Verify / observe: Compare create, start, die, and destroy events with the container status.

Final conclusion: What sequence of events explains the container's lifecycle? Stop following events with Ctrl+C.

Concept to retain: Events help connect observed behavior to container state changes.

My notes: _____

SC-022 · CLI & Engine · Track lifecycle events during a deployment

Situation: A container unexpectedly restarts during a test. Capture the lifecycle events while reproducing the issue.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start an event stream in one terminal.

```
docker events --since 10m
```

Verify / observe: Keep this terminal open while you reproduce the lifecycle.

Step 2 — In another terminal, start a disposable test container.

```
docker run -d --name event-lab-2 alpine:3.20 sh -c 'sleep 20'
```

Verify / observe: A container ID should be returned.

Step 3 — Inspect its state after the short run.

```
docker ps -a --filter name=event-lab-2
```

Verify / observe: Compare create, start, die, and destroy events with the container status.

Final conclusion: What sequence of events explains the container's lifecycle? Stop following events with Ctrl+C.

Concept to retain: Events help connect observed behavior to container state changes.

My notes: _____

SC-023 · CLI & Engine · Track lifecycle events during a deployment

Situation: A container unexpectedly restarts during a test. Capture the lifecycle events while reproducing the issue.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start an event stream in one terminal.

```
docker events --since 10m
```

Verify / observe: Keep this terminal open while you reproduce the lifecycle.

Step 2 — In another terminal, start a disposable test container.

```
docker run -d --name event-lab-3 alpine:3.20 sh -c 'sleep 20'
```

Verify / observe: A container ID should be returned.

Step 3 — Inspect its state after the short run.

```
docker ps -a --filter name=event-lab-3
```

Verify / observe: Compare create, start, die, and destroy events with the container status.

Final conclusion: What sequence of events explains the container's lifecycle? Stop following events with Ctrl+C.

Concept to retain: Events help connect observed behavior to container state changes.

My notes: _____

SC-024 · CLI & Engine · Track lifecycle events during a deployment

Situation: A container unexpectedly restarts during a test. Capture the lifecycle events while reproducing the issue.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start an event stream in one terminal.

```
docker events --since 10m
```

Verify / observe: Keep this terminal open while you reproduce the lifecycle.

Step 2 — In another terminal, start a disposable test container.

```
docker run -d --name event-lab-4 alpine:3.20 sh -c 'sleep 20'
```

Verify / observe: A container ID should be returned.

Step 3 — Inspect its state after the short run.

```
docker ps -a --filter name=event-lab-4
```

Verify / observe: Compare create, start, die, and destroy events with the container status.

Final conclusion: What sequence of events explains the container's lifecycle? Stop following events with Ctrl+C.

Concept to retain: Events help connect observed behavior to container state changes.

My notes: _____

SC-025 · CLI & Engine · Track lifecycle events during a deployment

Situation: A container unexpectedly restarts during a test. Capture the lifecycle events while reproducing the issue.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start an event stream in one terminal.

```
docker events --since 10m
```

Verify / observe: Keep this terminal open while you reproduce the lifecycle.

Step 2 — In another terminal, start a disposable test container.

```
docker run -d --name event-lab-5 alpine:3.20 sh -c 'sleep 20'
```

Verify / observe: A container ID should be returned.

Step 3 — Inspect its state after the short run.

```
docker ps -a --filter name=event-lab-5
```

Verify / observe: Compare create, start, die, and destroy events with the container status.

Final conclusion: What sequence of events explains the container's lifecycle? Stop following events with Ctrl+C.

Concept to retain: Events help connect observed behavior to container state changes.

My notes: _____

SC-026 · CLI & Engine · Track lifecycle events during a deployment

Situation: A container unexpectedly restarts during a test. Capture the lifecycle events while reproducing the issue.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start an event stream in one terminal.

```
docker events --since 10m
```

Verify / observe: Keep this terminal open while you reproduce the lifecycle.

Step 2 — In another terminal, start a disposable test container.

```
docker run -d --name event-lab-6 alpine:3.20 sh -c 'sleep 20'
```

Verify / observe: A container ID should be returned.

Step 3 — Inspect its state after the short run.

```
docker ps -a --filter name=event-lab-6
```

Verify / observe: Compare create, start, die, and destroy events with the container status.

Final conclusion: What sequence of events explains the container's lifecycle? Stop following events with Ctrl+C.

Concept to retain: Events help connect observed behavior to container state changes.

My notes: _____

SC-027 · CLI & Engine · Track lifecycle events during a deployment

Situation: A container unexpectedly restarts during a test. Capture the lifecycle events while reproducing the issue.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start an event stream in one terminal.

```
docker events --since 10m
```

Verify / observe: Keep this terminal open while you reproduce the lifecycle.

Step 2 — In another terminal, start a disposable test container.

```
docker run -d --name event-lab-7 alpine:3.20 sh -c 'sleep 20'
```

Verify / observe: A container ID should be returned.

Step 3 — Inspect its state after the short run.

```
docker ps -a --filter name=event-lab-7
```

Verify / observe: Compare create, start, die, and destroy events with the container status.

Final conclusion: What sequence of events explains the container's lifecycle? Stop following events with Ctrl+C.

Concept to retain: Events help connect observed behavior to container state changes.

My notes: _____

SC-028 · CLI & Engine · Track lifecycle events during a deployment

Situation: A container unexpectedly restarts during a test. Capture the lifecycle events while reproducing the issue.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start an event stream in one terminal.

```
docker events --since 10m
```

Verify / observe: Keep this terminal open while you reproduce the lifecycle.

Step 2 — In another terminal, start a disposable test container.

```
docker run -d --name event-lab-8 alpine:3.20 sh -c 'sleep 20'
```

Verify / observe: A container ID should be returned.

Step 3 — Inspect its state after the short run.

```
docker ps -a --filter name=event-lab-8
```

Verify / observe: Compare create, start, die, and destroy events with the container status.

Final conclusion: What sequence of events explains the container's lifecycle? Stop following events with Ctrl+C.

Concept to retain: Events help connect observed behavior to container state changes.

My notes: _____

SC-029 · CLI & Engine · Track lifecycle events during a deployment

Situation: A container unexpectedly restarts during a test. Capture the lifecycle events while reproducing the issue.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start an event stream in one terminal.

```
docker events --since 10m
```

Verify / observe: Keep this terminal open while you reproduce the lifecycle.

Step 2 — In another terminal, start a disposable test container.

```
docker run -d --name event-lab-9 alpine:3.20 sh -c 'sleep 20'
```

Verify / observe: A container ID should be returned.

Step 3 — Inspect its state after the short run.

```
docker ps -a --filter name=event-lab-9
```

Verify / observe: Compare create, start, die, and destroy events with the container status.

Final conclusion: What sequence of events explains the container's lifecycle? Stop following events with Ctrl+C.

Concept to retain: Events help connect observed behavior to container state changes.

My notes: _____

SC-030 · CLI & Engine · Track lifecycle events during a deployment

Situation: A container unexpectedly restarts during a test. Capture the lifecycle events while reproducing the issue.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start an event stream in one terminal.

```
docker events --since 10m
```

Verify / observe: Keep this terminal open while you reproduce the lifecycle.

Step 2 — In another terminal, start a disposable test container.

```
docker run -d --name event-lab-10 alpine:3.20 sh -c 'sleep 20'
```

Verify / observe: A container ID should be returned.

Step 3 — Inspect its state after the short run.

```
docker ps -a --filter name=event-lab-10
```

Verify / observe: Compare create, start, die, and destroy events with the container status.

Final conclusion: What sequence of events explains the container's lifecycle? Stop following events with Ctrl+C.

Concept to retain: Events help connect observed behavior to container state changes.

My notes: _____

SC-031 · CLI & Engine · Choose the right cleanup target

Situation: The host is low on disk space. You must identify reclaimable space without risking active workloads.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review Docker disk usage.

```
docker system df -v
```

Verify / observe: Record image, container, local volume, and build-cache usage.

Step 2 — List stopped containers.

```
docker ps -a --filter status=exited
```

Verify / observe: Check each container's purpose before deciding whether it is disposable.

Step 3 — List dangling images for review.

```
docker image ls --filter dangling=true
```

Verify / observe: Do not prune until ownership and reuse needs are understood.

Final conclusion: Recommend the safest cleanup target and justify why broad pruning is not your first step.

Concept to retain: Inspect first; prune commands can remove data or reusable artifacts.

My notes: _____

SC-032 · CLI & Engine · Choose the right cleanup target

Situation: The host is low on disk space. You must identify reclaimable space without risking active workloads.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review Docker disk usage.

```
docker system df -v
```

Verify / observe: Record image, container, local volume, and build-cache usage.

Step 2 — List stopped containers.

```
docker ps -a --filter status=exited
```

Verify / observe: Check each container's purpose before deciding whether it is disposable.

Step 3 — List dangling images for review.

```
docker image ls --filter dangling=true
```

Verify / observe: Do not prune until ownership and reuse needs are understood.

Final conclusion: Recommend the safest cleanup target and justify why broad pruning is not your first step.

Concept to retain: Inspect first; prune commands can remove data or reusable artifacts.

My notes: _____

SC-033 · CLI & Engine · Choose the right cleanup target

Situation: The host is low on disk space. You must identify reclaimable space without risking active workloads.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review Docker disk usage.

```
docker system df -v
```

Verify / observe: Record image, container, local volume, and build-cache usage.

Step 2 — List stopped containers.

```
docker ps -a --filter status=exited
```

Verify / observe: Check each container's purpose before deciding whether it is disposable.

Step 3 — List dangling images for review.

```
docker image ls --filter dangling=true
```

Verify / observe: Do not prune until ownership and reuse needs are understood.

Final conclusion: Recommend the safest cleanup target and justify why broad pruning is not your first step.

Concept to retain: Inspect first; prune commands can remove data or reusable artifacts.

My notes: _____

SC-034 · CLI & Engine · Choose the right cleanup target

Situation: The host is low on disk space. You must identify reclaimable space without risking active workloads.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review Docker disk usage.

```
docker system df -v
```

Verify / observe: Record image, container, local volume, and build-cache usage.

Step 2 — List stopped containers.

```
docker ps -a --filter status=exited
```

Verify / observe: Check each container's purpose before deciding whether it is disposable.

Step 3 — List dangling images for review.

```
docker image ls --filter dangling=true
```

Verify / observe: Do not prune until ownership and reuse needs are understood.

Final conclusion: Recommend the safest cleanup target and justify why broad pruning is not your first step.

Concept to retain: Inspect first; prune commands can remove data or reusable artifacts.

My notes: _____

SC-035 · CLI & Engine · Choose the right cleanup target

Situation: The host is low on disk space. You must identify reclaimable space without risking active workloads.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review Docker disk usage.

```
docker system df -v
```

Verify / observe: Record image, container, local volume, and build-cache usage.

Step 2 — List stopped containers.

```
docker ps -a --filter status=exited
```

Verify / observe: Check each container's purpose before deciding whether it is disposable.

Step 3 — List dangling images for review.

```
docker image ls --filter dangling=true
```

Verify / observe: Do not prune until ownership and reuse needs are understood.

Final conclusion: Recommend the safest cleanup target and justify why broad pruning is not your first step.

Concept to retain: Inspect first; prune commands can remove data or reusable artifacts.

My notes: _____

SC-036 · CLI & Engine · Choose the right cleanup target

Situation: The host is low on disk space. You must identify reclaimable space without risking active workloads.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review Docker disk usage.

```
docker system df -v
```

Verify / observe: Record image, container, local volume, and build-cache usage.

Step 2 — List stopped containers.

```
docker ps -a --filter status=exited
```

Verify / observe: Check each container's purpose before deciding whether it is disposable.

Step 3 — List dangling images for review.

```
docker image ls --filter dangling=true
```

Verify / observe: Do not prune until ownership and reuse needs are understood.

Final conclusion: Recommend the safest cleanup target and justify why broad pruning is not your first step.

Concept to retain: Inspect first; prune commands can remove data or reusable artifacts.

My notes: _____

SC-037 · CLI & Engine · Choose the right cleanup target

Situation: The host is low on disk space. You must identify reclaimable space without risking active workloads.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review Docker disk usage.

```
docker system df -v
```

Verify / observe: Record image, container, local volume, and build-cache usage.

Step 2 — List stopped containers.

```
docker ps -a --filter status=exited
```

Verify / observe: Check each container's purpose before deciding whether it is disposable.

Step 3 — List dangling images for review.

```
docker image ls --filter dangling=true
```

Verify / observe: Do not prune until ownership and reuse needs are understood.

Final conclusion: Recommend the safest cleanup target and justify why broad pruning is not your first step.

Concept to retain: Inspect first; prune commands can remove data or reusable artifacts.

My notes: _____

SC-038 · CLI & Engine · Choose the right cleanup target

Situation: The host is low on disk space. You must identify reclaimable space without risking active workloads.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review Docker disk usage.

```
docker system df -v
```

Verify / observe: Record image, container, local volume, and build-cache usage.

Step 2 — List stopped containers.

```
docker ps -a --filter status=exited
```

Verify / observe: Check each container's purpose before deciding whether it is disposable.

Step 3 — List dangling images for review.

```
docker image ls --filter dangling=true
```

Verify / observe: Do not prune until ownership and reuse needs are understood.

Final conclusion: Recommend the safest cleanup target and justify why broad pruning is not your first step.

Concept to retain: Inspect first; prune commands can remove data or reusable artifacts.

My notes: _____

SC-039 · CLI & Engine · Choose the right cleanup target

Situation: The host is low on disk space. You must identify reclaimable space without risking active workloads.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review Docker disk usage.

```
docker system df -v
```

Verify / observe: Record image, container, local volume, and build-cache usage.

Step 2 — List stopped containers.

```
docker ps -a --filter status=exited
```

Verify / observe: Check each container's purpose before deciding whether it is disposable.

Step 3 — List dangling images for review.

```
docker image ls --filter dangling=true
```

Verify / observe: Do not prune until ownership and reuse needs are understood.

Final conclusion: Recommend the safest cleanup target and justify why broad pruning is not your first step.

Concept to retain: Inspect first; prune commands can remove data or reusable artifacts.

My notes: _____

SC-040 · CLI & Engine · Choose the right cleanup target

Situation: The host is low on disk space. You must identify reclaimable space without risking active workloads.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review Docker disk usage.

```
docker system df -v
```

Verify / observe: Record image, container, local volume, and build-cache usage.

Step 2 — List stopped containers.

```
docker ps -a --filter status=exited
```

Verify / observe: Check each container's purpose before deciding whether it is disposable.

Step 3 — List dangling images for review.

```
docker image ls --filter dangling=true
```

Verify / observe: Do not prune until ownership and reuse needs are understood.

Final conclusion: Recommend the safest cleanup target and justify why broad pruning is not your first step.

Concept to retain: Inspect first; prune commands can remove data or reusable artifacts.

My notes: _____

SC-041 · CLI & Engine · Compare active and inactive contexts

Situation: You suspect Docker commands might be targeting the wrong daemon.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List configured contexts.

```
docker context ls
```

Verify / observe: Identify the active context marked with an asterisk.

Step 2 — Print the active context name.

```
docker context show
```

Verify / observe: Compare it with the intended lab environment.

Step 3 — Inspect the selected context endpoint.

```
docker context inspect
```

Verify / observe: Check which endpoint the client will contact.

Final conclusion: Explain how an incorrect context could affect a deployment, and state what you would verify before switching.

Concept to retain: Contexts can redirect commands to another host; verify the target before making changes.

My notes: _____

SC-042 · CLI & Engine · Compare active and inactive contexts

Situation: You suspect Docker commands might be targeting the wrong daemon.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List configured contexts.

```
docker context ls
```

Verify / observe: Identify the active context marked with an asterisk.

Step 2 — Print the active context name.

```
docker context show
```

Verify / observe: Compare it with the intended lab environment.

Step 3 — Inspect the selected context endpoint.

```
docker context inspect
```

Verify / observe: Check which endpoint the client will contact.

Final conclusion: Explain how an incorrect context could affect a deployment, and state what you would verify before switching.

Concept to retain: Contexts can redirect commands to another host; verify the target before making changes.

My notes: _____

SC-043 · CLI & Engine · Compare active and inactive contexts

Situation: You suspect Docker commands might be targeting the wrong daemon.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List configured contexts.

```
docker context ls
```

Verify / observe: Identify the active context marked with an asterisk.

Step 2 — Print the active context name.

```
docker context show
```

Verify / observe: Compare it with the intended lab environment.

Step 3 — Inspect the selected context endpoint.

```
docker context inspect
```

Verify / observe: Check which endpoint the client will contact.

Final conclusion: Explain how an incorrect context could affect a deployment, and state what you would verify before switching.

Concept to retain: Contexts can redirect commands to another host; verify the target before making changes.

My notes: _____

SC-044 · CLI & Engine · Compare active and inactive contexts

Situation: You suspect Docker commands might be targeting the wrong daemon.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List configured contexts.

```
docker context ls
```

Verify / observe: Identify the active context marked with an asterisk.

Step 2 — Print the active context name.

```
docker context show
```

Verify / observe: Compare it with the intended lab environment.

Step 3 — Inspect the selected context endpoint.

```
docker context inspect
```

Verify / observe: Check which endpoint the client will contact.

Final conclusion: Explain how an incorrect context could affect a deployment, and state what you would verify before switching.

Concept to retain: Contexts can redirect commands to another host; verify the target before making changes.

My notes: _____

SC-045 · CLI & Engine · Compare active and inactive contexts

Situation: You suspect Docker commands might be targeting the wrong daemon.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List configured contexts.

```
docker context ls
```

Verify / observe: Identify the active context marked with an asterisk.

Step 2 — Print the active context name.

```
docker context show
```

Verify / observe: Compare it with the intended lab environment.

Step 3 — Inspect the selected context endpoint.

```
docker context inspect
```

Verify / observe: Check which endpoint the client will contact.

Final conclusion: Explain how an incorrect context could affect a deployment, and state what you would verify before switching.

Concept to retain: Contexts can redirect commands to another host; verify the target before making changes.

My notes: _____

SC-046 · CLI & Engine · Compare active and inactive contexts

Situation: You suspect Docker commands might be targeting the wrong daemon.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List configured contexts.

```
docker context ls
```

Verify / observe: Identify the active context marked with an asterisk.

Step 2 — Print the active context name.

```
docker context show
```

Verify / observe: Compare it with the intended lab environment.

Step 3 — Inspect the selected context endpoint.

```
docker context inspect
```

Verify / observe: Check which endpoint the client will contact.

Final conclusion: Explain how an incorrect context could affect a deployment, and state what you would verify before switching.

Concept to retain: Contexts can redirect commands to another host; verify the target before making changes.

My notes: _____

SC-047 · CLI & Engine · Compare active and inactive contexts

Situation: You suspect Docker commands might be targeting the wrong daemon.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List configured contexts.

```
docker context ls
```

Verify / observe: Identify the active context marked with an asterisk.

Step 2 — Print the active context name.

```
docker context show
```

Verify / observe: Compare it with the intended lab environment.

Step 3 — Inspect the selected context endpoint.

```
docker context inspect
```

Verify / observe: Check which endpoint the client will contact.

Final conclusion: Explain how an incorrect context could affect a deployment, and state what you would verify before switching.

Concept to retain: Contexts can redirect commands to another host; verify the target before making changes.

My notes: _____

SC-048 · CLI & Engine · Compare active and inactive contexts

Situation: You suspect Docker commands might be targeting the wrong daemon.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List configured contexts.

```
docker context ls
```

Verify / observe: Identify the active context marked with an asterisk.

Step 2 — Print the active context name.

```
docker context show
```

Verify / observe: Compare it with the intended lab environment.

Step 3 — Inspect the selected context endpoint.

```
docker context inspect
```

Verify / observe: Check which endpoint the client will contact.

Final conclusion: Explain how an incorrect context could affect a deployment, and state what you would verify before switching.

Concept to retain: Contexts can redirect commands to another host; verify the target before making changes.

My notes: _____

SC-049 · CLI & Engine · Compare active and inactive contexts

Situation: You suspect Docker commands might be targeting the wrong daemon.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List configured contexts.

```
docker context ls
```

Verify / observe: Identify the active context marked with an asterisk.

Step 2 — Print the active context name.

```
docker context show
```

Verify / observe: Compare it with the intended lab environment.

Step 3 — Inspect the selected context endpoint.

```
docker context inspect
```

Verify / observe: Check which endpoint the client will contact.

Final conclusion: Explain how an incorrect context could affect a deployment, and state what you would verify before switching.

Concept to retain: Contexts can redirect commands to another host; verify the target before making changes.

My notes: _____

SC-050 · CLI & Engine · Compare active and inactive contexts

Situation: You suspect Docker commands might be targeting the wrong daemon.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List configured contexts.

```
docker context ls
```

Verify / observe: Identify the active context marked with an asterisk.

Step 2 — Print the active context name.

```
docker context show
```

Verify / observe: Compare it with the intended lab environment.

Step 3 — Inspect the selected context endpoint.

```
docker context inspect
```

Verify / observe: Check which endpoint the client will contact.

Final conclusion: Explain how an incorrect context could affect a deployment, and state what you would verify before switching.

Concept to retain: Contexts can redirect commands to another host; verify the target before making changes.

My notes: _____

SC-051 · Images · Pull and verify a base image

Situation: A project needs a known Alpine base image before a container can be tested.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Pull the chosen tag.

```
docker pull alpine:3.20
```

Verify / observe: Confirm the pull completes successfully.

Step 2 — List the matching local image.

```
docker image ls alpine
```

Verify / observe: Check repository, tag, image ID, and size.

Step 3 — Inspect operating system and architecture.

```
docker image inspect --format '{{.Os}}/{{.Architecture}}' alpine:3.20
```

Verify / observe: Compare the result with your host and workload needs.

Final conclusion: Is the image available locally, and is its platform suitable for this lab?

Concept to retain: Pulling downloads an image; listing and inspecting verify what is actually present.

My notes: _____

SC-052 · Images · Pull and verify a base image

Situation: A project needs a known Alpine base image before a container can be tested.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Pull the chosen tag.

```
docker pull alpine:3.20
```

Verify / observe: Confirm the pull completes successfully.

Step 2 — List the matching local image.

```
docker image ls alpine
```

Verify / observe: Check repository, tag, image ID, and size.

Step 3 — Inspect operating system and architecture.

```
docker image inspect --format '{{.Os}}/{{.Architecture}}' alpine:3.20
```

Verify / observe: Compare the result with your host and workload needs.

Final conclusion: Is the image available locally, and is its platform suitable for this lab?

Concept to retain: Pulling downloads an image; listing and inspecting verify what is actually present.

My notes: _____

SC-053 · Images · Pull and verify a base image

Situation: A project needs a known Alpine base image before a container can be tested.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Pull the chosen tag.

```
docker pull alpine:3.20
```

Verify / observe: Confirm the pull completes successfully.

Step 2 — List the matching local image.

```
docker image ls alpine
```

Verify / observe: Check repository, tag, image ID, and size.

Step 3 — Inspect operating system and architecture.

```
docker image inspect --format '{{.Os}}/{{.Architecture}}' alpine:3.20
```

Verify / observe: Compare the result with your host and workload needs.

Final conclusion: Is the image available locally, and is its platform suitable for this lab?

Concept to retain: Pulling downloads an image; listing and inspecting verify what is actually present.

My notes: _____

SC-054 · Images · Pull and verify a base image

Situation: A project needs a known Alpine base image before a container can be tested.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Pull the chosen tag.

```
docker pull alpine:3.20
```

Verify / observe: Confirm the pull completes successfully.

Step 2 — List the matching local image.

```
docker image ls alpine
```

Verify / observe: Check repository, tag, image ID, and size.

Step 3 — Inspect operating system and architecture.

```
docker image inspect --format '{{.Os}}/{{.Architecture}}' alpine:3.20
```

Verify / observe: Compare the result with your host and workload needs.

Final conclusion: Is the image available locally, and is its platform suitable for this lab?

Concept to retain: Pulling downloads an image; listing and inspecting verify what is actually present.

My notes: _____

SC-055 · Images · Pull and verify a base image

Situation: A project needs a known Alpine base image before a container can be tested.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Pull the chosen tag.

```
docker pull alpine:3.20
```

Verify / observe: Confirm the pull completes successfully.

Step 2 — List the matching local image.

```
docker image ls alpine
```

Verify / observe: Check repository, tag, image ID, and size.

Step 3 — Inspect operating system and architecture.

```
docker image inspect --format '{{.Os}}/{{.Architecture}}' alpine:3.20
```

Verify / observe: Compare the result with your host and workload needs.

Final conclusion: Is the image available locally, and is its platform suitable for this lab?

Concept to retain: Pulling downloads an image; listing and inspecting verify what is actually present.

My notes: _____

SC-056 · Images · Pull and verify a base image

Situation: A project needs a known Alpine base image before a container can be tested.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Pull the chosen tag.

```
docker pull alpine:3.20
```

Verify / observe: Confirm the pull completes successfully.

Step 2 — List the matching local image.

```
docker image ls alpine
```

Verify / observe: Check repository, tag, image ID, and size.

Step 3 — Inspect operating system and architecture.

```
docker image inspect --format '{{.Os}}/{{.Architecture}}' alpine:3.20
```

Verify / observe: Compare the result with your host and workload needs.

Final conclusion: Is the image available locally, and is its platform suitable for this lab?

Concept to retain: Pulling downloads an image; listing and inspecting verify what is actually present.

My notes: _____

SC-057 · Images · Pull and verify a base image

Situation: A project needs a known Alpine base image before a container can be tested.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Pull the chosen tag.

```
docker pull alpine:3.20
```

Verify / observe: Confirm the pull completes successfully.

Step 2 — List the matching local image.

```
docker image ls alpine
```

Verify / observe: Check repository, tag, image ID, and size.

Step 3 — Inspect operating system and architecture.

```
docker image inspect --format '{{.Os}}/{{.Architecture}}' alpine:3.20
```

Verify / observe: Compare the result with your host and workload needs.

Final conclusion: Is the image available locally, and is its platform suitable for this lab?

Concept to retain: Pulling downloads an image; listing and inspecting verify what is actually present.

My notes: _____

SC-058 · Images · Pull and verify a base image

Situation: A project needs a known Alpine base image before a container can be tested.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Pull the chosen tag.

```
docker pull alpine:3.20
```

Verify / observe: Confirm the pull completes successfully.

Step 2 — List the matching local image.

```
docker image ls alpine
```

Verify / observe: Check repository, tag, image ID, and size.

Step 3 — Inspect operating system and architecture.

```
docker image inspect --format '{{.Os}}/{{.Architecture}}' alpine:3.20
```

Verify / observe: Compare the result with your host and workload needs.

Final conclusion: Is the image available locally, and is its platform suitable for this lab?

Concept to retain: Pulling downloads an image; listing and inspecting verify what is actually present.

My notes: _____

SC-059 · Images · Pull and verify a base image

Situation: A project needs a known Alpine base image before a container can be tested.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Pull the chosen tag.

```
docker pull alpine:3.20
```

Verify / observe: Confirm the pull completes successfully.

Step 2 — List the matching local image.

```
docker image ls alpine
```

Verify / observe: Check repository, tag, image ID, and size.

Step 3 — Inspect operating system and architecture.

```
docker image inspect --format '{{.Os}}/{{.Architecture}}' alpine:3.20
```

Verify / observe: Compare the result with your host and workload needs.

Final conclusion: Is the image available locally, and is its platform suitable for this lab?

Concept to retain: Pulling downloads an image; listing and inspecting verify what is actually present.

My notes: _____

SC-060 · Images · Pull and verify a base image

Situation: A project needs a known Alpine base image before a container can be tested.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Pull the chosen tag.

```
docker pull alpine:3.20
```

Verify / observe: Confirm the pull completes successfully.

Step 2 — List the matching local image.

```
docker image ls alpine
```

Verify / observe: Check repository, tag, image ID, and size.

Step 3 — Inspect operating system and architecture.

```
docker image inspect --format '{{.Os}}/{{.Architecture}}' alpine:3.20
```

Verify / observe: Compare the result with your host and workload needs.

Final conclusion: Is the image available locally, and is its platform suitable for this lab?

Concept to retain: Pulling downloads an image; listing and inspecting verify what is actually present.

My notes: _____

SC-061 · Images · Inspect image defaults before running

Situation: A container exits immediately and you need to understand the image's default process.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the image's configured command and entrypoint.

```
docker image inspect --format 'Cmd={{json .Config.Cmd}} Entrypoint={{json .Config.Entrypoint}}' alpine:3.20
```

Verify / observe: Record the defaults shown by the image.

Step 2 — Read the layer history.

```
docker image history --no-trunc alpine:3.20
```

Verify / observe: Review the layer creation commands and sizes.

Step 3 — Run an explicit short command.

```
docker run --rm alpine:3.20 sh -c 'echo process-ok'
```

Verify / observe: Confirm the explicit command runs and prints process-ok.

Final conclusion: Explain how CMD and ENTRYPOINT influence the process that runs in a container.

Concept to retain: Image defaults and runtime command overrides explain many unexpected exits.

My notes: _____

SC-062 · Images · Inspect image defaults before running

Situation: A container exits immediately and you need to understand the image's default process.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the image's configured command and entrypoint.

```
docker image inspect --format 'Cmd={{json .Config.Cmd}} Entrypoint={{json .Config.Entrypoint}}' alpine:3.20
```

Verify / observe: Record the defaults shown by the image.

Step 2 — Read the layer history.

```
docker image history --no-trunc alpine:3.20
```

Verify / observe: Review the layer creation commands and sizes.

Step 3 — Run an explicit short command.

```
docker run --rm alpine:3.20 sh -c 'echo process-ok'
```

Verify / observe: Confirm the explicit command runs and prints process-ok.

Final conclusion: Explain how CMD and ENTRYPOINT influence the process that runs in a container.

Concept to retain: Image defaults and runtime command overrides explain many unexpected exits.

My notes: _____

SC-063 · Images · Inspect image defaults before running

Situation: A container exits immediately and you need to understand the image's default process.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the image's configured command and entrypoint.

```
docker image inspect --format 'Cmd={{json .Config.Cmd}} Entrypoint={{json .Config.Entrypoint}}' alpine:3.20
```

Verify / observe: Record the defaults shown by the image.

Step 2 — Read the layer history.

```
docker image history --no-trunc alpine:3.20
```

Verify / observe: Review the layer creation commands and sizes.

Step 3 — Run an explicit short command.

```
docker run --rm alpine:3.20 sh -c 'echo process-ok'
```

Verify / observe: Confirm the explicit command runs and prints process-ok.

Final conclusion: Explain how CMD and ENTRYPOINT influence the process that runs in a container.

Concept to retain: Image defaults and runtime command overrides explain many unexpected exits.

My notes: _____

SC-064 · Images · Inspect image defaults before running

Situation: A container exits immediately and you need to understand the image's default process.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the image's configured command and entrypoint.

```
docker image inspect --format 'Cmd={{json .Config.Cmd}} Entrypoint={{json .Config.Entrypoint}}' alpine:3.20
```

Verify / observe: Record the defaults shown by the image.

Step 2 — Read the layer history.

```
docker image history --no-trunc alpine:3.20
```

Verify / observe: Review the layer creation commands and sizes.

Step 3 — Run an explicit short command.

```
docker run --rm alpine:3.20 sh -c 'echo process-ok'
```

Verify / observe: Confirm the explicit command runs and prints process-ok.

Final conclusion: Explain how CMD and ENTRYPOINT influence the process that runs in a container.

Concept to retain: Image defaults and runtime command overrides explain many unexpected exits.

My notes: _____

SC-065 · Images · Inspect image defaults before running

Situation: A container exits immediately and you need to understand the image's default process.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the image's configured command and entrypoint.

```
docker image inspect --format 'Cmd={{json .Config.Cmd}} Entrypoint={{json .Config.Entrypoint}}' alpine:3.20
```

Verify / observe: Record the defaults shown by the image.

Step 2 — Read the layer history.

```
docker image history --no-trunc alpine:3.20
```

Verify / observe: Review the layer creation commands and sizes.

Step 3 — Run an explicit short command.

```
docker run --rm alpine:3.20 sh -c 'echo process-ok'
```

Verify / observe: Confirm the explicit command runs and prints process-ok.

Final conclusion: Explain how CMD and ENTRYPOINT influence the process that runs in a container.

Concept to retain: Image defaults and runtime command overrides explain many unexpected exits.

My notes: _____

SC-066 · Images · Inspect image defaults before running

Situation: A container exits immediately and you need to understand the image's default process.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the image's configured command and entrypoint.

```
docker image inspect --format 'Cmd={{json .Config.Cmd}} Entrypoint={{json .Config.Entrypoint}}' alpine:3.20
```

Verify / observe: Record the defaults shown by the image.

Step 2 — Read the layer history.

```
docker image history --no-trunc alpine:3.20
```

Verify / observe: Review the layer creation commands and sizes.

Step 3 — Run an explicit short command.

```
docker run --rm alpine:3.20 sh -c 'echo process-ok'
```

Verify / observe: Confirm the explicit command runs and prints process-ok.

Final conclusion: Explain how CMD and ENTRYPOINT influence the process that runs in a container.

Concept to retain: Image defaults and runtime command overrides explain many unexpected exits.

My notes: _____

SC-067 · Images · Inspect image defaults before running

Situation: A container exits immediately and you need to understand the image's default process.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the image's configured command and entrypoint.

```
docker image inspect --format 'Cmd={{json .Config.Cmd}} Entrypoint={{json .Config.Entrypoint}}' alpine:3.20
```

Verify / observe: Record the defaults shown by the image.

Step 2 — Read the layer history.

```
docker image history --no-trunc alpine:3.20
```

Verify / observe: Review the layer creation commands and sizes.

Step 3 — Run an explicit short command.

```
docker run --rm alpine:3.20 sh -c 'echo process-ok'
```

Verify / observe: Confirm the explicit command runs and prints process-ok.

Final conclusion: Explain how CMD and ENTRYPOINT influence the process that runs in a container.

Concept to retain: Image defaults and runtime command overrides explain many unexpected exits.

My notes: _____

SC-068 · Images · Inspect image defaults before running

Situation: A container exits immediately and you need to understand the image's default process.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the image's configured command and entrypoint.

```
docker image inspect --format 'Cmd={{json .Config.Cmd}} Entrypoint={{json .Config.Entrypoint}}' alpine:3.20
```

Verify / observe: Record the defaults shown by the image.

Step 2 — Read the layer history.

```
docker image history --no-trunc alpine:3.20
```

Verify / observe: Review the layer creation commands and sizes.

Step 3 — Run an explicit short command.

```
docker run --rm alpine:3.20 sh -c 'echo process-ok'
```

Verify / observe: Confirm the explicit command runs and prints process-ok.

Final conclusion: Explain how CMD and ENTRYPOINT influence the process that runs in a container.

Concept to retain: Image defaults and runtime command overrides explain many unexpected exits.

My notes: _____

SC-069 · Images · Inspect image defaults before running

Situation: A container exits immediately and you need to understand the image's default process.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the image's configured command and entrypoint.

```
docker image inspect --format 'Cmd={{json .Config.Cmd}} Entrypoint={{json .Config.Entrypoint}}' alpine:3.20
```

Verify / observe: Record the defaults shown by the image.

Step 2 — Read the layer history.

```
docker image history --no-trunc alpine:3.20
```

Verify / observe: Review the layer creation commands and sizes.

Step 3 — Run an explicit short command.

```
docker run --rm alpine:3.20 sh -c 'echo process-ok'
```

Verify / observe: Confirm the explicit command runs and prints process-ok.

Final conclusion: Explain how CMD and ENTRYPOINT influence the process that runs in a container.

Concept to retain: Image defaults and runtime command overrides explain many unexpected exits.

My notes: _____

SC-070 · Images · Inspect image defaults before running

Situation: A container exits immediately and you need to understand the image's default process.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the image's configured command and entrypoint.

```
docker image inspect --format 'Cmd={{json .Config.Cmd}} Entrypoint={{json .Config.Entrypoint}}' alpine:3.20
```

Verify / observe: Record the defaults shown by the image.

Step 2 — Read the layer history.

```
docker image history --no-trunc alpine:3.20
```

Verify / observe: Review the layer creation commands and sizes.

Step 3 — Run an explicit short command.

```
docker run --rm alpine:3.20 sh -c 'echo process-ok'
```

Verify / observe: Confirm the explicit command runs and prints process-ok.

Final conclusion: Explain how CMD and ENTRYPOINT influence the process that runs in a container.

Concept to retain: Image defaults and runtime command overrides explain many unexpected exits.

My notes: _____

SC-071 · Images · Tag an image for a local release

Situation: You have tested an image and need a clear local release tag before handing it to a teammate.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Pull the source image.

```
docker pull alpine:3.20
```

Verify / observe: Confirm the source tag exists.

Step 2 — Create a new local tag.

```
docker image tag alpine:3.20 local/training-base:release-1
```

Verify / observe: Tagging should complete without copying image layers.

Step 3 — List both references.

```
docker image ls local/training-base
```

Verify / observe: Compare the new tag's image ID with the source image ID.

Final conclusion: What does a tag change, and what does it not change?

Concept to retain: Tags are references to image content, not independent copies.

My notes: _____

SC-072 · Images · Tag an image for a local release

Situation: You have tested an image and need a clear local release tag before handing it to a teammate.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Pull the source image.

```
docker pull alpine:3.20
```

Verify / observe: Confirm the source tag exists.

Step 2 — Create a new local tag.

```
docker image tag alpine:3.20 local/training-base:release-2
```

Verify / observe: Tagging should complete without copying image layers.

Step 3 — List both references.

```
docker image ls local/training-base
```

Verify / observe: Compare the new tag's image ID with the source image ID.

Final conclusion: What does a tag change, and what does it not change?

Concept to retain: Tags are references to image content, not independent copies.

My notes: _____

SC-073 · Images · Tag an image for a local release

Situation: You have tested an image and need a clear local release tag before handing it to a teammate.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Pull the source image.

```
docker pull alpine:3.20
```

Verify / observe: Confirm the source tag exists.

Step 2 — Create a new local tag.

```
docker image tag alpine:3.20 local/training-base:release-3
```

Verify / observe: Tagging should complete without copying image layers.

Step 3 — List both references.

```
docker image ls local/training-base
```

Verify / observe: Compare the new tag's image ID with the source image ID.

Final conclusion: What does a tag change, and what does it not change?

Concept to retain: Tags are references to image content, not independent copies.

My notes: _____

SC-074 · Images · Tag an image for a local release

Situation: You have tested an image and need a clear local release tag before handing it to a teammate.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Pull the source image.

```
docker pull alpine:3.20
```

Verify / observe: Confirm the source tag exists.

Step 2 — Create a new local tag.

```
docker image tag alpine:3.20 local/training-base:release-4
```

Verify / observe: Tagging should complete without copying image layers.

Step 3 — List both references.

```
docker image ls local/training-base
```

Verify / observe: Compare the new tag's image ID with the source image ID.

Final conclusion: What does a tag change, and what does it not change?

Concept to retain: Tags are references to image content, not independent copies.

My notes: _____

SC-075 · Images · Tag an image for a local release

Situation: You have tested an image and need a clear local release tag before handing it to a teammate.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Pull the source image.

```
docker pull alpine:3.20
```

Verify / observe: Confirm the source tag exists.

Step 2 — Create a new local tag.

```
docker image tag alpine:3.20 local/training-base:release-5
```

Verify / observe: Tagging should complete without copying image layers.

Step 3 — List both references.

```
docker image ls local/training-base
```

Verify / observe: Compare the new tag's image ID with the source image ID.

Final conclusion: What does a tag change, and what does it not change?

Concept to retain: Tags are references to image content, not independent copies.

My notes: _____

SC-076 · Images · Tag an image for a local release

Situation: You have tested an image and need a clear local release tag before handing it to a teammate.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Pull the source image.

```
docker pull alpine:3.20
```

Verify / observe: Confirm the source tag exists.

Step 2 — Create a new local tag.

```
docker image tag alpine:3.20 local/training-base:release-6
```

Verify / observe: Tagging should complete without copying image layers.

Step 3 — List both references.

```
docker image ls local/training-base
```

Verify / observe: Compare the new tag's image ID with the source image ID.

Final conclusion: What does a tag change, and what does it not change?

Concept to retain: Tags are references to image content, not independent copies.

My notes: _____

SC-077 · Images · Tag an image for a local release

Situation: You have tested an image and need a clear local release tag before handing it to a teammate.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Pull the source image.

```
docker pull alpine:3.20
```

Verify / observe: Confirm the source tag exists.

Step 2 — Create a new local tag.

```
docker image tag alpine:3.20 local/training-base:release-7
```

Verify / observe: Tagging should complete without copying image layers.

Step 3 — List both references.

```
docker image ls local/training-base
```

Verify / observe: Compare the new tag's image ID with the source image ID.

Final conclusion: What does a tag change, and what does it not change?

Concept to retain: Tags are references to image content, not independent copies.

My notes: _____

SC-078 · Images · Tag an image for a local release

Situation: You have tested an image and need a clear local release tag before handing it to a teammate.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Pull the source image.

```
docker pull alpine:3.20
```

Verify / observe: Confirm the source tag exists.

Step 2 — Create a new local tag.

```
docker image tag alpine:3.20 local/training-base:release-8
```

Verify / observe: Tagging should complete without copying image layers.

Step 3 — List both references.

```
docker image ls local/training-base
```

Verify / observe: Compare the new tag's image ID with the source image ID.

Final conclusion: What does a tag change, and what does it not change?

Concept to retain: Tags are references to image content, not independent copies.

My notes: _____

SC-079 · Images · Tag an image for a local release

Situation: You have tested an image and need a clear local release tag before handing it to a teammate.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Pull the source image.

```
docker pull alpine:3.20
```

Verify / observe: Confirm the source tag exists.

Step 2 — Create a new local tag.

```
docker image tag alpine:3.20 local/training-base:release-9
```

Verify / observe: Tagging should complete without copying image layers.

Step 3 — List both references.

```
docker image ls local/training-base
```

Verify / observe: Compare the new tag's image ID with the source image ID.

Final conclusion: What does a tag change, and what does it not change?

Concept to retain: Tags are references to image content, not independent copies.

My notes: _____

SC-080 · Images · Tag an image for a local release

Situation: You have tested an image and need a clear local release tag before handing it to a teammate.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Pull the source image.

```
docker pull alpine:3.20
```

Verify / observe: Confirm the source tag exists.

Step 2 — Create a new local tag.

```
docker image tag alpine:3.20 local/training-base:release-10
```

Verify / observe: Tagging should complete without copying image layers.

Step 3 — List both references.

```
docker image ls local/training-base
```

Verify / observe: Compare the new tag's image ID with the source image ID.

Final conclusion: What does a tag change, and what does it not change?

Concept to retain: Tags are references to image content, not independent copies.

My notes: _____

SC-081 · Images · Save and reload an image for offline transfer

Situation: A training host has no registry access, so you need to transfer an image using an archive.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Save the image to an archive.

```
docker image save -o alpine-transfer-1.tar alpine:3.20
```

Verify / observe: Confirm the archive exists and has a plausible size.

Step 2 — Remove only the training tag if it is safe to do so.

```
docker image rm alpine:3.20
```

Verify / observe: Proceed only if no important container or workflow depends on the tag.

Step 3 — Load the image archive again.

```
docker image load -i alpine-transfer-1.tar
```

Verify / observe: Verify the restored tag with docker image ls.

Final conclusion: Explain why `docker image save/load` differs from exporting a container filesystem.

Concept to retain: Save/load transfers image layers and metadata; container export is a different operation.

My notes: _____

SC-082 · Images · Save and reload an image for offline transfer

Situation: A training host has no registry access, so you need to transfer an image using an archive.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Save the image to an archive.

```
docker image save -o alpine-transfer-2.tar alpine:3.20
```

Verify / observe: Confirm the archive exists and has a plausible size.

Step 2 — Remove only the training tag if it is safe to do so.

```
docker image rm alpine:3.20
```

Verify / observe: Proceed only if no important container or workflow depends on the tag.

Step 3 — Load the image archive again.

```
docker image load -i alpine-transfer-2.tar
```

Verify / observe: Verify the restored tag with docker image ls.

Final conclusion: Explain why `docker image save/load` differs from exporting a container filesystem.

Concept to retain: Save/load transfers image layers and metadata; container export is a different operation.

My notes: _____

SC-083 · Images · Save and reload an image for offline transfer

Situation: A training host has no registry access, so you need to transfer an image using an archive.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Save the image to an archive.

```
docker image save -o alpine-transfer-3.tar alpine:3.20
```

Verify / observe: Confirm the archive exists and has a plausible size.

Step 2 — Remove only the training tag if it is safe to do so.

```
docker image rm alpine:3.20
```

Verify / observe: Proceed only if no important container or workflow depends on the tag.

Step 3 — Load the image archive again.

```
docker image load -i alpine-transfer-3.tar
```

Verify / observe: Verify the restored tag with docker image ls.

Final conclusion: Explain why `docker image save/load` differs from exporting a container filesystem.

Concept to retain: Save/load transfers image layers and metadata; container export is a different operation.

My notes: _____

SC-084 · Images · Save and reload an image for offline transfer

Situation: A training host has no registry access, so you need to transfer an image using an archive.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Save the image to an archive.

```
docker image save -o alpine-transfer-4.tar alpine:3.20
```

Verify / observe: Confirm the archive exists and has a plausible size.

Step 2 — Remove only the training tag if it is safe to do so.

```
docker image rm alpine:3.20
```

Verify / observe: Proceed only if no important container or workflow depends on the tag.

Step 3 — Load the image archive again.

```
docker image load -i alpine-transfer-4.tar
```

Verify / observe: Verify the restored tag with docker image ls.

Final conclusion: Explain why `docker image save/load` differs from exporting a container filesystem.

Concept to retain: Save/load transfers image layers and metadata; container export is a different operation.

My notes: _____

SC-085 · Images · Save and reload an image for offline transfer

Situation: A training host has no registry access, so you need to transfer an image using an archive.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Save the image to an archive.

```
docker image save -o alpine-transfer-5.tar alpine:3.20
```

Verify / observe: Confirm the archive exists and has a plausible size.

Step 2 — Remove only the training tag if it is safe to do so.

```
docker image rm alpine:3.20
```

Verify / observe: Proceed only if no important container or workflow depends on the tag.

Step 3 — Load the image archive again.

```
docker image load -i alpine-transfer-5.tar
```

Verify / observe: Verify the restored tag with docker image ls.

Final conclusion: Explain why `docker image save/load` differs from exporting a container filesystem.

Concept to retain: Save/load transfers image layers and metadata; container export is a different operation.

My notes: _____

SC-086 · Images · Save and reload an image for offline transfer

Situation: A training host has no registry access, so you need to transfer an image using an archive.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Save the image to an archive.

```
docker image save -o alpine-transfer-6.tar alpine:3.20
```

Verify / observe: Confirm the archive exists and has a plausible size.

Step 2 — Remove only the training tag if it is safe to do so.

```
docker image rm alpine:3.20
```

Verify / observe: Proceed only if no important container or workflow depends on the tag.

Step 3 — Load the image archive again.

```
docker image load -i alpine-transfer-6.tar
```

Verify / observe: Verify the restored tag with docker image ls.

Final conclusion: Explain why `docker image save/load` differs from exporting a container filesystem.

Concept to retain: Save/load transfers image layers and metadata; container export is a different operation.

My notes: _____

SC-087 · Images · Save and reload an image for offline transfer

Situation: A training host has no registry access, so you need to transfer an image using an archive.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Save the image to an archive.

```
docker image save -o alpine-transfer-7.tar alpine:3.20
```

Verify / observe: Confirm the archive exists and has a plausible size.

Step 2 — Remove only the training tag if it is safe to do so.

```
docker image rm alpine:3.20
```

Verify / observe: Proceed only if no important container or workflow depends on the tag.

Step 3 — Load the image archive again.

```
docker image load -i alpine-transfer-7.tar
```

Verify / observe: Verify the restored tag with docker image ls.

Final conclusion: Explain why `docker image save/load` differs from exporting a container filesystem.

Concept to retain: Save/load transfers image layers and metadata; container export is a different operation.

My notes: _____

SC-088 · Images · Save and reload an image for offline transfer

Situation: A training host has no registry access, so you need to transfer an image using an archive.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Save the image to an archive.

```
docker image save -o alpine-transfer-8.tar alpine:3.20
```

Verify / observe: Confirm the archive exists and has a plausible size.

Step 2 — Remove only the training tag if it is safe to do so.

```
docker image rm alpine:3.20
```

Verify / observe: Proceed only if no important container or workflow depends on the tag.

Step 3 — Load the image archive again.

```
docker image load -i alpine-transfer-8.tar
```

Verify / observe: Verify the restored tag with docker image ls.

Final conclusion: Explain why `docker image save/load` differs from exporting a container filesystem.

Concept to retain: Save/load transfers image layers and metadata; container export is a different operation.

My notes: _____

SC-089 · Images · Save and reload an image for offline transfer

Situation: A training host has no registry access, so you need to transfer an image using an archive.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Save the image to an archive.

```
docker image save -o alpine-transfer-9.tar alpine:3.20
```

Verify / observe: Confirm the archive exists and has a plausible size.

Step 2 — Remove only the training tag if it is safe to do so.

```
docker image rm alpine:3.20
```

Verify / observe: Proceed only if no important container or workflow depends on the tag.

Step 3 — Load the image archive again.

```
docker image load -i alpine-transfer-9.tar
```

Verify / observe: Verify the restored tag with docker image ls.

Final conclusion: Explain why `docker image save/load` differs from exporting a container filesystem.

Concept to retain: Save/load transfers image layers and metadata; container export is a different operation.

My notes: _____

SC-090 · Images · Save and reload an image for offline transfer

Situation: A training host has no registry access, so you need to transfer an image using an archive.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Save the image to an archive.

```
docker image save -o alpine-transfer-10.tar alpine:3.20
```

Verify / observe: Confirm the archive exists and has a plausible size.

Step 2 — Remove only the training tag if it is safe to do so.

```
docker image rm alpine:3.20
```

Verify / observe: Proceed only if no important container or workflow depends on the tag.

Step 3 — Load the image archive again.

```
docker image load -i alpine-transfer-10.tar
```

Verify / observe: Verify the restored tag with docker image ls.

Final conclusion: Explain why `docker image save/load` differs from exporting a container filesystem.

Concept to retain: Save/load transfers image layers and metadata; container export is a different operation.

My notes: _____

SC-091 · Images · Compare image history and size

Situation: Your team wants to understand why an image is larger than expected.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review image size and metadata.

```
docker image ls
```

Verify / observe: Record the target image's reported size.

Step 2 — Inspect layer history.

```
docker image history --no-trunc alpine:3.20
```

Verify / observe: Identify large layers and commands that created them.

Step 3 — Inspect detailed image configuration.

```
docker image inspect alpine:3.20
```

Verify / observe: Review environment, command, entrypoint, and architecture.

Final conclusion: Name one potential image-size improvement and one item you would investigate before changing the Dockerfile.

Concept to retain: Layer history is useful evidence, but it does not replace reviewing the Dockerfile and build context.

My notes: _____

SC-092 · Images · Compare image history and size

Situation: Your team wants to understand why an image is larger than expected.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review image size and metadata.

```
docker image ls
```

Verify / observe: Record the target image's reported size.

Step 2 — Inspect layer history.

```
docker image history --no-trunc alpine:3.20
```

Verify / observe: Identify large layers and commands that created them.

Step 3 — Inspect detailed image configuration.

```
docker image inspect alpine:3.20
```

Verify / observe: Review environment, command, entrypoint, and architecture.

Final conclusion: Name one potential image-size improvement and one item you would investigate before changing the Dockerfile.

Concept to retain: Layer history is useful evidence, but it does not replace reviewing the Dockerfile and build context.

My notes: _____

SC-093 · Images · Compare image history and size

Situation: Your team wants to understand why an image is larger than expected.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review image size and metadata.

```
docker image ls
```

Verify / observe: Record the target image's reported size.

Step 2 — Inspect layer history.

```
docker image history --no-trunc alpine:3.20
```

Verify / observe: Identify large layers and commands that created them.

Step 3 — Inspect detailed image configuration.

```
docker image inspect alpine:3.20
```

Verify / observe: Review environment, command, entrypoint, and architecture.

Final conclusion: Name one potential image-size improvement and one item you would investigate before changing the Dockerfile.

Concept to retain: Layer history is useful evidence, but it does not replace reviewing the Dockerfile and build context.

My notes: _____

SC-094 · Images · Compare image history and size

Situation: Your team wants to understand why an image is larger than expected.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review image size and metadata.

```
docker image ls
```

Verify / observe: Record the target image's reported size.

Step 2 — Inspect layer history.

```
docker image history --no-trunc alpine:3.20
```

Verify / observe: Identify large layers and commands that created them.

Step 3 — Inspect detailed image configuration.

```
docker image inspect alpine:3.20
```

Verify / observe: Review environment, command, entrypoint, and architecture.

Final conclusion: Name one potential image-size improvement and one item you would investigate before changing the Dockerfile.

Concept to retain: Layer history is useful evidence, but it does not replace reviewing the Dockerfile and build context.

My notes: _____

SC-095 · Images · Compare image history and size

Situation: Your team wants to understand why an image is larger than expected.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review image size and metadata.

```
docker image ls
```

Verify / observe: Record the target image's reported size.

Step 2 — Inspect layer history.

```
docker image history --no-trunc alpine:3.20
```

Verify / observe: Identify large layers and commands that created them.

Step 3 — Inspect detailed image configuration.

```
docker image inspect alpine:3.20
```

Verify / observe: Review environment, command, entrypoint, and architecture.

Final conclusion: Name one potential image-size improvement and one item you would investigate before changing the Dockerfile.

Concept to retain: Layer history is useful evidence, but it does not replace reviewing the Dockerfile and build context.

My notes: _____

SC-096 · Images · Compare image history and size

Situation: Your team wants to understand why an image is larger than expected.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review image size and metadata.

```
docker image ls
```

Verify / observe: Record the target image's reported size.

Step 2 — Inspect layer history.

```
docker image history --no-trunc alpine:3.20
```

Verify / observe: Identify large layers and commands that created them.

Step 3 — Inspect detailed image configuration.

```
docker image inspect alpine:3.20
```

Verify / observe: Review environment, command, entrypoint, and architecture.

Final conclusion: Name one potential image-size improvement and one item you would investigate before changing the Dockerfile.

Concept to retain: Layer history is useful evidence, but it does not replace reviewing the Dockerfile and build context.

My notes: _____

SC-097 · Images · Compare image history and size

Situation: Your team wants to understand why an image is larger than expected.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review image size and metadata.

```
docker image ls
```

Verify / observe: Record the target image's reported size.

Step 2 — Inspect layer history.

```
docker image history --no-trunc alpine:3.20
```

Verify / observe: Identify large layers and commands that created them.

Step 3 — Inspect detailed image configuration.

```
docker image inspect alpine:3.20
```

Verify / observe: Review environment, command, entrypoint, and architecture.

Final conclusion: Name one potential image-size improvement and one item you would investigate before changing the Dockerfile.

Concept to retain: Layer history is useful evidence, but it does not replace reviewing the Dockerfile and build context.

My notes: _____

SC-098 · Images · Compare image history and size

Situation: Your team wants to understand why an image is larger than expected.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review image size and metadata.

```
docker image ls
```

Verify / observe: Record the target image's reported size.

Step 2 — Inspect layer history.

```
docker image history --no-trunc alpine:3.20
```

Verify / observe: Identify large layers and commands that created them.

Step 3 — Inspect detailed image configuration.

```
docker image inspect alpine:3.20
```

Verify / observe: Review environment, command, entrypoint, and architecture.

Final conclusion: Name one potential image-size improvement and one item you would investigate before changing the Dockerfile.

Concept to retain: Layer history is useful evidence, but it does not replace reviewing the Dockerfile and build context.

My notes: _____

SC-099 · Images · Compare image history and size

Situation: Your team wants to understand why an image is larger than expected.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review image size and metadata.

```
docker image ls
```

Verify / observe: Record the target image's reported size.

Step 2 — Inspect layer history.

```
docker image history --no-trunc alpine:3.20
```

Verify / observe: Identify large layers and commands that created them.

Step 3 — Inspect detailed image configuration.

```
docker image inspect alpine:3.20
```

Verify / observe: Review environment, command, entrypoint, and architecture.

Final conclusion: Name one potential image-size improvement and one item you would investigate before changing the Dockerfile.

Concept to retain: Layer history is useful evidence, but it does not replace reviewing the Dockerfile and build context.

My notes: _____

SC-100 · Images · Compare image history and size

Situation: Your team wants to understand why an image is larger than expected.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review image size and metadata.

```
docker image ls
```

Verify / observe: Record the target image's reported size.

Step 2 — Inspect layer history.

```
docker image history --no-trunc alpine:3.20
```

Verify / observe: Identify large layers and commands that created them.

Step 3 — Inspect detailed image configuration.

```
docker image inspect alpine:3.20
```

Verify / observe: Review environment, command, entrypoint, and architecture.

Final conclusion: Name one potential image-size improvement and one item you would investigate before changing the Dockerfile.

Concept to retain: Layer history is useful evidence, but it does not replace reviewing the Dockerfile and build context.

My notes: _____

SC-101 · Images · Publish an image to a registry you control

Situation: A tested image must be made available to another machine through a registry.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Authenticate interactively to your registry.

```
docker login REGISTRY
```

Verify / observe: Replace REGISTRY with a registry you control; use a credential helper or short-lived token where possible.

Step 2 — Tag the image with your registry namespace.

```
docker image tag alpine:3.20 REGISTRY/USERNAME/training-base:release-1
```

Verify / observe: Replace placeholders with your registry and account namespace.

Step 3 — Push the exact tag.

```
docker image push REGISTRY/USERNAME/training-base:release-1
```

Verify / observe: Confirm the registry reports the layers and manifest were uploaded.

Final conclusion: What tag will the receiving machine pull, and how will you avoid publishing to the wrong namespace?

Concept to retain: Registry publishing requires correct authentication, namespace, and tag.

My notes: _____

SC-102 · Images · Publish an image to a registry you control

Situation: A tested image must be made available to another machine through a registry.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Authenticate interactively to your registry.

```
docker login REGISTRY
```

Verify / observe: Replace REGISTRY with a registry you control; use a credential helper or short-lived token where possible.

Step 2 — Tag the image with your registry namespace.

```
docker image tag alpine:3.20 REGISTRY/USERNAME/training-base:release-2
```

Verify / observe: Replace placeholders with your registry and account namespace.

Step 3 — Push the exact tag.

```
docker image push REGISTRY/USERNAME/training-base:release-2
```

Verify / observe: Confirm the registry reports the layers and manifest were uploaded.

Final conclusion: What tag will the receiving machine pull, and how will you avoid publishing to the wrong namespace?

Concept to retain: Registry publishing requires correct authentication, namespace, and tag.

My notes: _____

SC-103 · Images · Publish an image to a registry you control

Situation: A tested image must be made available to another machine through a registry.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Authenticate interactively to your registry.

```
docker login REGISTRY
```

Verify / observe: Replace REGISTRY with a registry you control; use a credential helper or short-lived token where possible.

Step 2 — Tag the image with your registry namespace.

```
docker image tag alpine:3.20 REGISTRY/USERNAME/training-base:release-3
```

Verify / observe: Replace placeholders with your registry and account namespace.

Step 3 — Push the exact tag.

```
docker image push REGISTRY/USERNAME/training-base:release-3
```

Verify / observe: Confirm the registry reports the layers and manifest were uploaded.

Final conclusion: What tag will the receiving machine pull, and how will you avoid publishing to the wrong namespace?

Concept to retain: Registry publishing requires correct authentication, namespace, and tag.

My notes: _____

SC-104 · Images · Publish an image to a registry you control

Situation: A tested image must be made available to another machine through a registry.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Authenticate interactively to your registry.

```
docker login REGISTRY
```

Verify / observe: Replace REGISTRY with a registry you control; use a credential helper or short-lived token where possible.

Step 2 — Tag the image with your registry namespace.

```
docker image tag alpine:3.20 REGISTRY/USERNAME/training-base:release-4
```

Verify / observe: Replace placeholders with your registry and account namespace.

Step 3 — Push the exact tag.

```
docker image push REGISTRY/USERNAME/training-base:release-4
```

Verify / observe: Confirm the registry reports the layers and manifest were uploaded.

Final conclusion: What tag will the receiving machine pull, and how will you avoid publishing to the wrong namespace?

Concept to retain: Registry publishing requires correct authentication, namespace, and tag.

My notes: _____

SC-105 · Images · Publish an image to a registry you control

Situation: A tested image must be made available to another machine through a registry.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Authenticate interactively to your registry.

```
docker login REGISTRY
```

Verify / observe: Replace REGISTRY with a registry you control; use a credential helper or short-lived token where possible.

Step 2 — Tag the image with your registry namespace.

```
docker image tag alpine:3.20 REGISTRY/USERNAME/training-base:release-5
```

Verify / observe: Replace placeholders with your registry and account namespace.

Step 3 — Push the exact tag.

```
docker image push REGISTRY/USERNAME/training-base:release-5
```

Verify / observe: Confirm the registry reports the layers and manifest were uploaded.

Final conclusion: What tag will the receiving machine pull, and how will you avoid publishing to the wrong namespace?

Concept to retain: Registry publishing requires correct authentication, namespace, and tag.

My notes: _____

SC-106 · Images · Publish an image to a registry you control

Situation: A tested image must be made available to another machine through a registry.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Authenticate interactively to your registry.

```
docker login REGISTRY
```

Verify / observe: Replace REGISTRY with a registry you control; use a credential helper or short-lived token where possible.

Step 2 — Tag the image with your registry namespace.

```
docker image tag alpine:3.20 REGISTRY/USERNAME/training-base:release-6
```

Verify / observe: Replace placeholders with your registry and account namespace.

Step 3 — Push the exact tag.

```
docker image push REGISTRY/USERNAME/training-base:release-6
```

Verify / observe: Confirm the registry reports the layers and manifest were uploaded.

Final conclusion: What tag will the receiving machine pull, and how will you avoid publishing to the wrong namespace?

Concept to retain: Registry publishing requires correct authentication, namespace, and tag.

My notes: _____

SC-107 · Images · Publish an image to a registry you control

Situation: A tested image must be made available to another machine through a registry.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Authenticate interactively to your registry.

```
docker login REGISTRY
```

Verify / observe: Replace REGISTRY with a registry you control; use a credential helper or short-lived token where possible.

Step 2 — Tag the image with your registry namespace.

```
docker image tag alpine:3.20 REGISTRY/USERNAME/training-base:release-7
```

Verify / observe: Replace placeholders with your registry and account namespace.

Step 3 — Push the exact tag.

```
docker image push REGISTRY/USERNAME/training-base:release-7
```

Verify / observe: Confirm the registry reports the layers and manifest were uploaded.

Final conclusion: What tag will the receiving machine pull, and how will you avoid publishing to the wrong namespace?

Concept to retain: Registry publishing requires correct authentication, namespace, and tag.

My notes: _____

SC-108 · Images · Publish an image to a registry you control

Situation: A tested image must be made available to another machine through a registry.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Authenticate interactively to your registry.

```
docker login REGISTRY
```

Verify / observe: Replace REGISTRY with a registry you control; use a credential helper or short-lived token where possible.

Step 2 — Tag the image with your registry namespace.

```
docker image tag alpine:3.20 REGISTRY/USERNAME/training-base:release-8
```

Verify / observe: Replace placeholders with your registry and account namespace.

Step 3 — Push the exact tag.

```
docker image push REGISTRY/USERNAME/training-base:release-8
```

Verify / observe: Confirm the registry reports the layers and manifest were uploaded.

Final conclusion: What tag will the receiving machine pull, and how will you avoid publishing to the wrong namespace?

Concept to retain: Registry publishing requires correct authentication, namespace, and tag.

My notes: _____

SC-109 · Images · Publish an image to a registry you control

Situation: A tested image must be made available to another machine through a registry.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Authenticate interactively to your registry.

```
docker login REGISTRY
```

Verify / observe: Replace REGISTRY with a registry you control; use a credential helper or short-lived token where possible.

Step 2 — Tag the image with your registry namespace.

```
docker image tag alpine:3.20 REGISTRY/USERNAME/training-base:release-9
```

Verify / observe: Replace placeholders with your registry and account namespace.

Step 3 — Push the exact tag.

```
docker image push REGISTRY/USERNAME/training-base:release-9
```

Verify / observe: Confirm the registry reports the layers and manifest were uploaded.

Final conclusion: What tag will the receiving machine pull, and how will you avoid publishing to the wrong namespace?

Concept to retain: Registry publishing requires correct authentication, namespace, and tag.

My notes: _____

SC-110 · Images · Publish an image to a registry you control

Situation: A tested image must be made available to another machine through a registry.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Authenticate interactively to your registry.

```
docker login REGISTRY
```

Verify / observe: Replace REGISTRY with a registry you control; use a credential helper or short-lived token where possible.

Step 2 — Tag the image with your registry namespace.

```
docker image tag alpine:3.20 REGISTRY/USERNAME/training-base:release-10
```

Verify / observe: Replace placeholders with your registry and account namespace.

Step 3 — Push the exact tag.

```
docker image push REGISTRY/USERNAME/training-base:release-10
```

Verify / observe: Confirm the registry reports the layers and manifest were uploaded.

Final conclusion: What tag will the receiving machine pull, and how will you avoid publishing to the wrong namespace?

Concept to retain: Registry publishing requires correct authentication, namespace, and tag.

My notes: _____

SC-111 · Containers · Run a one-shot job

Situation: You need a disposable container to print a result and then leave no stopped container behind.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a one-shot command with automatic cleanup.

```
docker run --rm alpine:3.20 sh -c 'echo job-1-complete'
```

Verify / observe: Confirm the message appears and the command exits.

Step 2 — Check whether the container remains.

```
docker ps -a --filter ancestor=alpine:3.20
```

Verify / observe: The just-completed --rm container should no longer be listed.

Step 3 — Repeat with a nonzero exit status in a lab.

```
docker run --rm alpine:3.20 sh -c 'exit 7'; echo $?
```

Verify / observe: On POSIX shells, check the reported exit status; shell syntax differs on Windows.

Final conclusion: Why is `--rm` useful for jobs, and when would you avoid it?

Concept to retain: Disposable jobs benefit from automatic cleanup, but debugging may require retaining the stopped container.

My notes: _____

SC-112 · Containers · Run a one-shot job

Situation: You need a disposable container to print a result and then leave no stopped container behind.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a one-shot command with automatic cleanup.

```
docker run --rm alpine:3.20 sh -c 'echo job-2-complete'
```

Verify / observe: Confirm the message appears and the command exits.

Step 2 — Check whether the container remains.

```
docker ps -a --filter ancestor=alpine:3.20
```

Verify / observe: The just-completed --rm container should no longer be listed.

Step 3 — Repeat with a nonzero exit status in a lab.

```
docker run --rm alpine:3.20 sh -c 'exit 7'; echo $?
```

Verify / observe: On POSIX shells, check the reported exit status; shell syntax differs on Windows.

Final conclusion: Why is `--rm` useful for jobs, and when would you avoid it?

Concept to retain: Disposable jobs benefit from automatic cleanup, but debugging may require retaining the stopped container.

My notes: _____

SC-113 · Containers · Run a one-shot job

Situation: You need a disposable container to print a result and then leave no stopped container behind.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a one-shot command with automatic cleanup.

```
docker run --rm alpine:3.20 sh -c 'echo job-3-complete'
```

Verify / observe: Confirm the message appears and the command exits.

Step 2 — Check whether the container remains.

```
docker ps -a --filter ancestor=alpine:3.20
```

Verify / observe: The just-completed --rm container should no longer be listed.

Step 3 — Repeat with a nonzero exit status in a lab.

```
docker run --rm alpine:3.20 sh -c 'exit 7'; echo $?
```

Verify / observe: On POSIX shells, check the reported exit status; shell syntax differs on Windows.

Final conclusion: Why is `--rm` useful for jobs, and when would you avoid it?

Concept to retain: Disposable jobs benefit from automatic cleanup, but debugging may require retaining the stopped container.

My notes: _____

SC-114 · Containers · Run a one-shot job

Situation: You need a disposable container to print a result and then leave no stopped container behind.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a one-shot command with automatic cleanup.

```
docker run --rm alpine:3.20 sh -c 'echo job-4-complete'
```

Verify / observe: Confirm the message appears and the command exits.

Step 2 — Check whether the container remains.

```
docker ps -a --filter ancestor=alpine:3.20
```

Verify / observe: The just-completed --rm container should no longer be listed.

Step 3 — Repeat with a nonzero exit status in a lab.

```
docker run --rm alpine:3.20 sh -c 'exit 7'; echo $?
```

Verify / observe: On POSIX shells, check the reported exit status; shell syntax differs on Windows.

Final conclusion: Why is `--rm` useful for jobs, and when would you avoid it?

Concept to retain: Disposable jobs benefit from automatic cleanup, but debugging may require retaining the stopped container.

My notes: _____

SC-115 · Containers · Run a one-shot job

Situation: You need a disposable container to print a result and then leave no stopped container behind.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a one-shot command with automatic cleanup.

```
docker run --rm alpine:3.20 sh -c 'echo job-5-complete'
```

Verify / observe: Confirm the message appears and the command exits.

Step 2 — Check whether the container remains.

```
docker ps -a --filter ancestor=alpine:3.20
```

Verify / observe: The just-completed --rm container should no longer be listed.

Step 3 — Repeat with a nonzero exit status in a lab.

```
docker run --rm alpine:3.20 sh -c 'exit 7'; echo $?
```

Verify / observe: On POSIX shells, check the reported exit status; shell syntax differs on Windows.

Final conclusion: Why is `--rm` useful for jobs, and when would you avoid it?

Concept to retain: Disposable jobs benefit from automatic cleanup, but debugging may require retaining the stopped container.

My notes: _____

SC-116 · Containers · Run a one-shot job

Situation: You need a disposable container to print a result and then leave no stopped container behind.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a one-shot command with automatic cleanup.

```
docker run --rm alpine:3.20 sh -c 'echo job-6-complete'
```

Verify / observe: Confirm the message appears and the command exits.

Step 2 — Check whether the container remains.

```
docker ps -a --filter ancestor=alpine:3.20
```

Verify / observe: The just-completed --rm container should no longer be listed.

Step 3 — Repeat with a nonzero exit status in a lab.

```
docker run --rm alpine:3.20 sh -c 'exit 7'; echo $?
```

Verify / observe: On POSIX shells, check the reported exit status; shell syntax differs on Windows.

Final conclusion: Why is `--rm` useful for jobs, and when would you avoid it?

Concept to retain: Disposable jobs benefit from automatic cleanup, but debugging may require retaining the stopped container.

My notes: _____

SC-117 · Containers · Run a one-shot job

Situation: You need a disposable container to print a result and then leave no stopped container behind.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a one-shot command with automatic cleanup.

```
docker run --rm alpine:3.20 sh -c 'echo job-7-complete'
```

Verify / observe: Confirm the message appears and the command exits.

Step 2 — Check whether the container remains.

```
docker ps -a --filter ancestor=alpine:3.20
```

Verify / observe: The just-completed --rm container should no longer be listed.

Step 3 — Repeat with a nonzero exit status in a lab.

```
docker run --rm alpine:3.20 sh -c 'exit 7'; echo $?
```

Verify / observe: On POSIX shells, check the reported exit status; shell syntax differs on Windows.

Final conclusion: Why is `--rm` useful for jobs, and when would you avoid it?

Concept to retain: Disposable jobs benefit from automatic cleanup, but debugging may require retaining the stopped container.

My notes: _____

SC-118 · Containers · Run a one-shot job

Situation: You need a disposable container to print a result and then leave no stopped container behind.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a one-shot command with automatic cleanup.

```
docker run --rm alpine:3.20 sh -c 'echo job-8-complete'
```

Verify / observe: Confirm the message appears and the command exits.

Step 2 — Check whether the container remains.

```
docker ps -a --filter ancestor=alpine:3.20
```

Verify / observe: The just-completed --rm container should no longer be listed.

Step 3 — Repeat with a nonzero exit status in a lab.

```
docker run --rm alpine:3.20 sh -c 'exit 7'; echo $?
```

Verify / observe: On POSIX shells, check the reported exit status; shell syntax differs on Windows.

Final conclusion: Why is `--rm` useful for jobs, and when would you avoid it?

Concept to retain: Disposable jobs benefit from automatic cleanup, but debugging may require retaining the stopped container.

My notes: _____

SC-119 · Containers · Run a one-shot job

Situation: You need a disposable container to print a result and then leave no stopped container behind.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a one-shot command with automatic cleanup.

```
docker run --rm alpine:3.20 sh -c 'echo job-9-complete'
```

Verify / observe: Confirm the message appears and the command exits.

Step 2 — Check whether the container remains.

```
docker ps -a --filter ancestor=alpine:3.20
```

Verify / observe: The just-completed --rm container should no longer be listed.

Step 3 — Repeat with a nonzero exit status in a lab.

```
docker run --rm alpine:3.20 sh -c 'exit 7'; echo $?
```

Verify / observe: On POSIX shells, check the reported exit status; shell syntax differs on Windows.

Final conclusion: Why is `--rm` useful for jobs, and when would you avoid it?

Concept to retain: Disposable jobs benefit from automatic cleanup, but debugging may require retaining the stopped container.

My notes: _____

SC-120 · Containers · Run a one-shot job

Situation: You need a disposable container to print a result and then leave no stopped container behind.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a one-shot command with automatic cleanup.

```
docker run --rm alpine:3.20 sh -c 'echo job-10-complete'
```

Verify / observe: Confirm the message appears and the command exits.

Step 2 — Check whether the container remains.

```
docker ps -a --filter ancestor=alpine:3.20
```

Verify / observe: The just-completed --rm container should no longer be listed.

Step 3 — Repeat with a nonzero exit status in a lab.

```
docker run --rm alpine:3.20 sh -c 'exit 7'; echo $?
```

Verify / observe: On POSIX shells, check the reported exit status; shell syntax differs on Windows.

Final conclusion: Why is `--rm` useful for jobs, and when would you avoid it?

Concept to retain: Disposable jobs benefit from automatic cleanup, but debugging may require retaining the stopped container.

My notes: _____

SC-121 · Containers · Launch and expose a web server

Situation: You must start an isolated Nginx web server for a local demo.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start Nginx detached with a unique name and host port.

```
docker run -d --name web-1 -p 1881:80 nginx:alpine
```

Verify / observe: Use an available host port; if the generated port is unavailable, replace it with a free one.

Step 2 — Confirm status and port mapping.

```
docker ps --filter name=web-1
```

Verify / observe: Look for Up status and a host-to-container port mapping.

Step 3 — Read recent logs.

```
docker logs --tail 30 web-1
```

Verify / observe: Look for startup errors or request logs.

Step 4 — Stop and remove only this lab container.

```
docker rm -f web-1
```

Verify / observe: Verify it no longer appears in docker ps -a.

Final conclusion: Was the container running, and was the host port correctly published? Explain the difference between `-p HOST:CONTAINER` and exposing a port in image metadata.

Concept to retain: Port publishing creates host access; EXPOSE metadata alone does not publish a port.

My notes: _____

SC-122 · Containers · Launch and expose a web server

Situation: You must start an isolated Nginx web server for a local demo.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start Nginx detached with a unique name and host port.

```
docker run -d --name web-2 -p 1882:80 nginx:alpine
```

Verify / observe: Use an available host port; if the generated port is unavailable, replace it with a free one.

Step 2 — Confirm status and port mapping.

```
docker ps --filter name=web-2
```

Verify / observe: Look for Up status and a host-to-container port mapping.

Step 3 — Read recent logs.

```
docker logs --tail 30 web-2
```

Verify / observe: Look for startup errors or request logs.

Step 4 — Stop and remove only this lab container.

```
docker rm -f web-2
```

Verify / observe: Verify it no longer appears in docker ps -a.

Final conclusion: Was the container running, and was the host port correctly published? Explain the difference between `-p HOST:CONTAINER` and exposing a port in image metadata.

Concept to retain: Port publishing creates host access; EXPOSE metadata alone does not publish a port.

My notes: _____

SC-123 · Containers · Launch and expose a web server

Situation: You must start an isolated Nginx web server for a local demo.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start Nginx detached with a unique name and host port.

```
docker run -d --name web-3 -p 1883:80 nginx:alpine
```

Verify / observe: Use an available host port; if the generated port is unavailable, replace it with a free one.

Step 2 — Confirm status and port mapping.

```
docker ps --filter name=web-3
```

Verify / observe: Look for Up status and a host-to-container port mapping.

Step 3 — Read recent logs.

```
docker logs --tail 30 web-3
```

Verify / observe: Look for startup errors or request logs.

Step 4 — Stop and remove only this lab container.

```
docker rm -f web-3
```

Verify / observe: Verify it no longer appears in docker ps -a.

Final conclusion: Was the container running, and was the host port correctly published? Explain the difference between `-p HOST:CONTAINER` and exposing a port in image metadata.

Concept to retain: Port publishing creates host access; EXPOSE metadata alone does not publish a port.

My notes: _____

SC-124 · Containers · Launch and expose a web server

Situation: You must start an isolated Nginx web server for a local demo.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start Nginx detached with a unique name and host port.

```
docker run -d --name web-4 -p 1884:80 nginx:alpine
```

Verify / observe: Use an available host port; if the generated port is unavailable, replace it with a free one.

Step 2 — Confirm status and port mapping.

```
docker ps --filter name=web-4
```

Verify / observe: Look for Up status and a host-to-container port mapping.

Step 3 — Read recent logs.

```
docker logs --tail 30 web-4
```

Verify / observe: Look for startup errors or request logs.

Step 4 — Stop and remove only this lab container.

```
docker rm -f web-4
```

Verify / observe: Verify it no longer appears in docker ps -a.

Final conclusion: Was the container running, and was the host port correctly published? Explain the difference between `-p HOST:CONTAINER` and exposing a port in image metadata.

Concept to retain: Port publishing creates host access; EXPOSE metadata alone does not publish a port.

My notes: _____

SC-125 · Containers · Launch and expose a web server

Situation: You must start an isolated Nginx web server for a local demo.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start Nginx detached with a unique name and host port.

```
docker run -d --name web-5 -p 1885:80 nginx:alpine
```

Verify / observe: Use an available host port; if the generated port is unavailable, replace it with a free one.

Step 2 — Confirm status and port mapping.

```
docker ps --filter name=web-5
```

Verify / observe: Look for Up status and a host-to-container port mapping.

Step 3 — Read recent logs.

```
docker logs --tail 30 web-5
```

Verify / observe: Look for startup errors or request logs.

Step 4 — Stop and remove only this lab container.

```
docker rm -f web-5
```

Verify / observe: Verify it no longer appears in docker ps -a.

Final conclusion: Was the container running, and was the host port correctly published? Explain the difference between `-p HOST:CONTAINER` and exposing a port in image metadata.

Concept to retain: Port publishing creates host access; EXPOSE metadata alone does not publish a port.

My notes: _____

SC-126 · Containers · Launch and expose a web server

Situation: You must start an isolated Nginx web server for a local demo.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start Nginx detached with a unique name and host port.

```
docker run -d --name web-6 -p 1886:80 nginx:alpine
```

Verify / observe: Use an available host port; if the generated port is unavailable, replace it with a free one.

Step 2 — Confirm status and port mapping.

```
docker ps --filter name=web-6
```

Verify / observe: Look for Up status and a host-to-container port mapping.

Step 3 — Read recent logs.

```
docker logs --tail 30 web-6
```

Verify / observe: Look for startup errors or request logs.

Step 4 — Stop and remove only this lab container.

```
docker rm -f web-6
```

Verify / observe: Verify it no longer appears in docker ps -a.

Final conclusion: Was the container running, and was the host port correctly published? Explain the difference between `-p HOST:CONTAINER` and exposing a port in image metadata.

Concept to retain: Port publishing creates host access; EXPOSE metadata alone does not publish a port.

My notes: _____

SC-127 · Containers · Launch and expose a web server

Situation: You must start an isolated Nginx web server for a local demo.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start Nginx detached with a unique name and host port.

```
docker run -d --name web-7 -p 1887:80 nginx:alpine
```

Verify / observe: Use an available host port; if the generated port is unavailable, replace it with a free one.

Step 2 — Confirm status and port mapping.

```
docker ps --filter name=web-7
```

Verify / observe: Look for Up status and a host-to-container port mapping.

Step 3 — Read recent logs.

```
docker logs --tail 30 web-7
```

Verify / observe: Look for startup errors or request logs.

Step 4 — Stop and remove only this lab container.

```
docker rm -f web-7
```

Verify / observe: Verify it no longer appears in docker ps -a.

Final conclusion: Was the container running, and was the host port correctly published? Explain the difference between `-p HOST:CONTAINER` and exposing a port in image metadata.

Concept to retain: Port publishing creates host access; EXPOSE metadata alone does not publish a port.

My notes: _____

SC-128 · Containers · Launch and expose a web server

Situation: You must start an isolated Nginx web server for a local demo.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start Nginx detached with a unique name and host port.

```
docker run -d --name web-8 -p 1888:80 nginx:alpine
```

Verify / observe: Use an available host port; if the generated port is unavailable, replace it with a free one.

Step 2 — Confirm status and port mapping.

```
docker ps --filter name=web-8
```

Verify / observe: Look for Up status and a host-to-container port mapping.

Step 3 — Read recent logs.

```
docker logs --tail 30 web-8
```

Verify / observe: Look for startup errors or request logs.

Step 4 — Stop and remove only this lab container.

```
docker rm -f web-8
```

Verify / observe: Verify it no longer appears in docker ps -a.

Final conclusion: Was the container running, and was the host port correctly published? Explain the difference between `-p HOST:CONTAINER` and exposing a port in image metadata.

Concept to retain: Port publishing creates host access; EXPOSE metadata alone does not publish a port.

My notes: _____

SC-129 · Containers · Launch and expose a web server

Situation: You must start an isolated Nginx web server for a local demo.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start Nginx detached with a unique name and host port.

```
docker run -d --name web-9 -p 1889:80 nginx:alpine
```

Verify / observe: Use an available host port; if the generated port is unavailable, replace it with a free one.

Step 2 — Confirm status and port mapping.

```
docker ps --filter name=web-9
```

Verify / observe: Look for Up status and a host-to-container port mapping.

Step 3 — Read recent logs.

```
docker logs --tail 30 web-9
```

Verify / observe: Look for startup errors or request logs.

Step 4 — Stop and remove only this lab container.

```
docker rm -f web-9
```

Verify / observe: Verify it no longer appears in docker ps -a.

Final conclusion: Was the container running, and was the host port correctly published? Explain the difference between `-p HOST:CONTAINER` and exposing a port in image metadata.

Concept to retain: Port publishing creates host access; EXPOSE metadata alone does not publish a port.

My notes: _____

SC-130 · Containers · Launch and expose a web server

Situation: You must start an isolated Nginx web server for a local demo.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start Nginx detached with a unique name and host port.

```
docker run -d --name web-10 -p 1890:80 nginx:alpine
```

Verify / observe: Use an available host port; if the generated port is unavailable, replace it with a free one.

Step 2 — Confirm status and port mapping.

```
docker ps --filter name=web-10
```

Verify / observe: Look for Up status and a host-to-container port mapping.

Step 3 — Read recent logs.

```
docker logs --tail 30 web-10
```

Verify / observe: Look for startup errors or request logs.

Step 4 — Stop and remove only this lab container.

```
docker rm -f web-10
```

Verify / observe: Verify it no longer appears in docker ps -a.

Final conclusion: Was the container running, and was the host port correctly published? Explain the difference between `-p HOST:CONTAINER` and exposing a port in image metadata.

Concept to retain: Port publishing creates host access; EXPOSE metadata alone does not publish a port.

My notes: _____

SC-131 · Containers · Configure a container with environment variables

Situation: An application needs a runtime setting without rebuilding its image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run Alpine with a named environment variable.

```
docker run --rm -e APP_MODE=training-1 alpine:3.20 sh -c 'echo $APP_MODE'
```

Verify / observe: Confirm the expected value is printed.

Step 2 — Pass a second variable using an explicit name/value pair.

```
docker run --rm -e CASE_ID=1 alpine:3.20 sh -c 'env | grep CASE_ID'
```

Verify / observe: Confirm CASE_ID appears in the container environment.

Step 3 — Compare with image-level environment configuration.

```
docker image inspect --format '{{json .Config.Env}}' alpine:3.20
```

Verify / observe: Separate image defaults from runtime overrides.

Final conclusion: Which value is supplied at runtime, and why should secrets not be placed casually in command history or logs?

Concept to retain: Runtime environment configuration avoids rebuilding an image, but ordinary environment variables are not a secret-management system.

My notes: _____

SC-132 · Containers · Configure a container with environment variables

Situation: An application needs a runtime setting without rebuilding its image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run Alpine with a named environment variable.

```
docker run --rm -e APP_MODE=training-2 alpine:3.20 sh -c 'echo $APP_MODE'
```

Verify / observe: Confirm the expected value is printed.

Step 2 — Pass a second variable using an explicit name/value pair.

```
docker run --rm -e CASE_ID=2 alpine:3.20 sh -c 'env | grep CASE_ID'
```

Verify / observe: Confirm CASE_ID appears in the container environment.

Step 3 — Compare with image-level environment configuration.

```
docker image inspect --format '{{json .Config.Env}}' alpine:3.20
```

Verify / observe: Separate image defaults from runtime overrides.

Final conclusion: Which value is supplied at runtime, and why should secrets not be placed casually in command history or logs?

Concept to retain: Runtime environment configuration avoids rebuilding an image, but ordinary environment variables are not a secret-management system.

My notes: _____

SC-133 · Containers · Configure a container with environment variables

Situation: An application needs a runtime setting without rebuilding its image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run Alpine with a named environment variable.

```
docker run --rm -e APP_MODE=training-3 alpine:3.20 sh -c 'echo $APP_MODE'
```

Verify / observe: Confirm the expected value is printed.

Step 2 — Pass a second variable using an explicit name/value pair.

```
docker run --rm -e CASE_ID=3 alpine:3.20 sh -c 'env | grep CASE_ID'
```

Verify / observe: Confirm CASE_ID appears in the container environment.

Step 3 — Compare with image-level environment configuration.

```
docker image inspect --format '{{json .Config.Env}}' alpine:3.20
```

Verify / observe: Separate image defaults from runtime overrides.

Final conclusion: Which value is supplied at runtime, and why should secrets not be placed casually in command history or logs?

Concept to retain: Runtime environment configuration avoids rebuilding an image, but ordinary environment variables are not a secret-management system.

My notes: _____

SC-134 · Containers · Configure a container with environment variables

Situation: An application needs a runtime setting without rebuilding its image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run Alpine with a named environment variable.

```
docker run --rm -e APP_MODE=training-4 alpine:3.20 sh -c 'echo $APP_MODE'
```

Verify / observe: Confirm the expected value is printed.

Step 2 — Pass a second variable using an explicit name/value pair.

```
docker run --rm -e CASE_ID=4 alpine:3.20 sh -c 'env | grep CASE_ID'
```

Verify / observe: Confirm CASE_ID appears in the container environment.

Step 3 — Compare with image-level environment configuration.

```
docker image inspect --format '{{json .Config.Env}}' alpine:3.20
```

Verify / observe: Separate image defaults from runtime overrides.

Final conclusion: Which value is supplied at runtime, and why should secrets not be placed casually in command history or logs?

Concept to retain: Runtime environment configuration avoids rebuilding an image, but ordinary environment variables are not a secret-management system.

My notes: _____

SC-135 · Containers · Configure a container with environment variables

Situation: An application needs a runtime setting without rebuilding its image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run Alpine with a named environment variable.

```
docker run --rm -e APP_MODE=training-5 alpine:3.20 sh -c 'echo $APP_MODE'
```

Verify / observe: Confirm the expected value is printed.

Step 2 — Pass a second variable using an explicit name/value pair.

```
docker run --rm -e CASE_ID=5 alpine:3.20 sh -c 'env | grep CASE_ID'
```

Verify / observe: Confirm CASE_ID appears in the container environment.

Step 3 — Compare with image-level environment configuration.

```
docker image inspect --format '{{json .Config.Env}}' alpine:3.20
```

Verify / observe: Separate image defaults from runtime overrides.

Final conclusion: Which value is supplied at runtime, and why should secrets not be placed casually in command history or logs?

Concept to retain: Runtime environment configuration avoids rebuilding an image, but ordinary environment variables are not a secret-management system.

My notes: _____

SC-136 · Containers · Configure a container with environment variables

Situation: An application needs a runtime setting without rebuilding its image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run Alpine with a named environment variable.

```
docker run --rm -e APP_MODE=training-6 alpine:3.20 sh -c 'echo $APP_MODE'
```

Verify / observe: Confirm the expected value is printed.

Step 2 — Pass a second variable using an explicit name/value pair.

```
docker run --rm -e CASE_ID=6 alpine:3.20 sh -c 'env | grep CASE_ID'
```

Verify / observe: Confirm CASE_ID appears in the container environment.

Step 3 — Compare with image-level environment configuration.

```
docker image inspect --format '{{json .Config.Env}}' alpine:3.20
```

Verify / observe: Separate image defaults from runtime overrides.

Final conclusion: Which value is supplied at runtime, and why should secrets not be placed casually in command history or logs?

Concept to retain: Runtime environment configuration avoids rebuilding an image, but ordinary environment variables are not a secret-management system.

My notes: _____

SC-137 · Containers · Configure a container with environment variables

Situation: An application needs a runtime setting without rebuilding its image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run Alpine with a named environment variable.

```
docker run --rm -e APP_MODE=training-7 alpine:3.20 sh -c 'echo $APP_MODE'
```

Verify / observe: Confirm the expected value is printed.

Step 2 — Pass a second variable using an explicit name/value pair.

```
docker run --rm -e CASE_ID=7 alpine:3.20 sh -c 'env | grep CASE_ID'
```

Verify / observe: Confirm CASE_ID appears in the container environment.

Step 3 — Compare with image-level environment configuration.

```
docker image inspect --format '{{json .Config.Env}}' alpine:3.20
```

Verify / observe: Separate image defaults from runtime overrides.

Final conclusion: Which value is supplied at runtime, and why should secrets not be placed casually in command history or logs?

Concept to retain: Runtime environment configuration avoids rebuilding an image, but ordinary environment variables are not a secret-management system.

My notes: _____

SC-138 · Containers · Configure a container with environment variables

Situation: An application needs a runtime setting without rebuilding its image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run Alpine with a named environment variable.

```
docker run --rm -e APP_MODE=training-8 alpine:3.20 sh -c 'echo $APP_MODE'
```

Verify / observe: Confirm the expected value is printed.

Step 2 — Pass a second variable using an explicit name/value pair.

```
docker run --rm -e CASE_ID=8 alpine:3.20 sh -c 'env | grep CASE_ID'
```

Verify / observe: Confirm CASE_ID appears in the container environment.

Step 3 — Compare with image-level environment configuration.

```
docker image inspect --format '{{json .Config.Env}}' alpine:3.20
```

Verify / observe: Separate image defaults from runtime overrides.

Final conclusion: Which value is supplied at runtime, and why should secrets not be placed casually in command history or logs?

Concept to retain: Runtime environment configuration avoids rebuilding an image, but ordinary environment variables are not a secret-management system.

My notes: _____

SC-139 · Containers · Configure a container with environment variables

Situation: An application needs a runtime setting without rebuilding its image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run Alpine with a named environment variable.

```
docker run --rm -e APP_MODE=training-9 alpine:3.20 sh -c 'echo $APP_MODE'
```

Verify / observe: Confirm the expected value is printed.

Step 2 — Pass a second variable using an explicit name/value pair.

```
docker run --rm -e CASE_ID=9 alpine:3.20 sh -c 'env | grep CASE_ID'
```

Verify / observe: Confirm CASE_ID appears in the container environment.

Step 3 — Compare with image-level environment configuration.

```
docker image inspect --format '{{json .Config.Env}}' alpine:3.20
```

Verify / observe: Separate image defaults from runtime overrides.

Final conclusion: Which value is supplied at runtime, and why should secrets not be placed casually in command history or logs?

Concept to retain: Runtime environment configuration avoids rebuilding an image, but ordinary environment variables are not a secret-management system.

My notes: _____

SC-140 · Containers · Configure a container with environment variables

Situation: An application needs a runtime setting without rebuilding its image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run Alpine with a named environment variable.

```
docker run --rm -e APP_MODE=training-10 alpine:3.20 sh -c 'echo $APP_MODE'
```

Verify / observe: Confirm the expected value is printed.

Step 2 — Pass a second variable using an explicit name/value pair.

```
docker run --rm -e CASE_ID=10 alpine:3.20 sh -c 'env | grep CASE_ID'
```

Verify / observe: Confirm CASE_ID appears in the container environment.

Step 3 — Compare with image-level environment configuration.

```
docker image inspect --format '{{json .Config.Env}}' alpine:3.20
```

Verify / observe: Separate image defaults from runtime overrides.

Final conclusion: Which value is supplied at runtime, and why should secrets not be placed casually in command history or logs?

Concept to retain: Runtime environment configuration avoids rebuilding an image, but ordinary environment variables are not a secret-management system.

My notes: _____

SC-141 · Containers · Use a read-only container filesystem

Situation: You want to reduce accidental writes from a simple inspection workload.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a read-only container and read a system file.

```
docker run --rm --read-only alpine:3.20 cat /etc/alpine-release
```

Verify / observe: Reading should succeed.

Step 2 — Attempt a write to the root filesystem.

```
docker run --rm --read-only alpine:3.20 sh -c 'echo test > /root/test.txt'
```

Verify / observe: The write should fail because the root filesystem is read-only.

Step 3 — Repeat with an explicit temporary writable area.

```
docker run --rm --read-only --tmpfs /tmp alpine:3.20 sh -c 'echo ok > /tmp/test.txt && cat /tmp/test.txt'
```

Verify / observe: Confirm writes to /tmp work while the root filesystem remains read-only.

Final conclusion: What changed between the second and third run, and what kinds of application writes need explicit mounts?

Concept to retain: Read-only root filesystems reduce writable surface area; temporary data needs a deliberate writable mount.

My notes: _____

SC-142 · Containers · Use a read-only container filesystem

Situation: You want to reduce accidental writes from a simple inspection workload.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a read-only container and read a system file.

```
docker run --rm --read-only alpine:3.20 cat /etc/alpine-release
```

Verify / observe: Reading should succeed.

Step 2 — Attempt a write to the root filesystem.

```
docker run --rm --read-only alpine:3.20 sh -c 'echo test > /root/test.txt'
```

Verify / observe: The write should fail because the root filesystem is read-only.

Step 3 — Repeat with an explicit temporary writable area.

```
docker run --rm --read-only --tmpfs /tmp alpine:3.20 sh -c 'echo ok > /tmp/test.txt && cat /tmp/test.txt'
```

Verify / observe: Confirm writes to /tmp work while the root filesystem remains read-only.

Final conclusion: What changed between the second and third run, and what kinds of application writes need explicit mounts?

Concept to retain: Read-only root filesystems reduce writable surface area; temporary data needs a deliberate writable mount.

My notes: _____

SC-143 · Containers · Use a read-only container filesystem

Situation: You want to reduce accidental writes from a simple inspection workload.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a read-only container and read a system file.

```
docker run --rm --read-only alpine:3.20 cat /etc/alpine-release
```

Verify / observe: Reading should succeed.

Step 2 — Attempt a write to the root filesystem.

```
docker run --rm --read-only alpine:3.20 sh -c 'echo test > /root/test.txt'
```

Verify / observe: The write should fail because the root filesystem is read-only.

Step 3 — Repeat with an explicit temporary writable area.

```
docker run --rm --read-only --tmpfs /tmp alpine:3.20 sh -c 'echo ok > /tmp/test.txt && cat /tmp/test.txt'
```

Verify / observe: Confirm writes to /tmp work while the root filesystem remains read-only.

Final conclusion: What changed between the second and third run, and what kinds of application writes need explicit mounts?

Concept to retain: Read-only root filesystems reduce writable surface area; temporary data needs a deliberate writable mount.

My notes: _____

SC-144 · Containers · Use a read-only container filesystem

Situation: You want to reduce accidental writes from a simple inspection workload.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a read-only container and read a system file.

```
docker run --rm --read-only alpine:3.20 cat /etc/alpine-release
```

Verify / observe: Reading should succeed.

Step 2 — Attempt a write to the root filesystem.

```
docker run --rm --read-only alpine:3.20 sh -c 'echo test > /root/test.txt'
```

Verify / observe: The write should fail because the root filesystem is read-only.

Step 3 — Repeat with an explicit temporary writable area.

```
docker run --rm --read-only --tmpfs /tmp alpine:3.20 sh -c 'echo ok > /tmp/test.txt && cat /tmp/test.txt'
```

Verify / observe: Confirm writes to /tmp work while the root filesystem remains read-only.

Final conclusion: What changed between the second and third run, and what kinds of application writes need explicit mounts?

Concept to retain: Read-only root filesystems reduce writable surface area; temporary data needs a deliberate writable mount.

My notes: _____

SC-145 · Containers · Use a read-only container filesystem

Situation: You want to reduce accidental writes from a simple inspection workload.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a read-only container and read a system file.

```
docker run --rm --read-only alpine:3.20 cat /etc/alpine-release
```

Verify / observe: Reading should succeed.

Step 2 — Attempt a write to the root filesystem.

```
docker run --rm --read-only alpine:3.20 sh -c 'echo test > /root/test.txt'
```

Verify / observe: The write should fail because the root filesystem is read-only.

Step 3 — Repeat with an explicit temporary writable area.

```
docker run --rm --read-only --tmpfs /tmp alpine:3.20 sh -c 'echo ok > /tmp/test.txt && cat /tmp/test.txt'
```

Verify / observe: Confirm writes to /tmp work while the root filesystem remains read-only.

Final conclusion: What changed between the second and third run, and what kinds of application writes need explicit mounts?

Concept to retain: Read-only root filesystems reduce writable surface area; temporary data needs a deliberate writable mount.

My notes: _____

SC-146 · Containers · Use a read-only container filesystem

Situation: You want to reduce accidental writes from a simple inspection workload.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a read-only container and read a system file.

```
docker run --rm --read-only alpine:3.20 cat /etc/alpine-release
```

Verify / observe: Reading should succeed.

Step 2 — Attempt a write to the root filesystem.

```
docker run --rm --read-only alpine:3.20 sh -c 'echo test > /root/test.txt'
```

Verify / observe: The write should fail because the root filesystem is read-only.

Step 3 — Repeat with an explicit temporary writable area.

```
docker run --rm --read-only --tmpfs /tmp alpine:3.20 sh -c 'echo ok > /tmp/test.txt && cat /tmp/test.txt'
```

Verify / observe: Confirm writes to /tmp work while the root filesystem remains read-only.

Final conclusion: What changed between the second and third run, and what kinds of application writes need explicit mounts?

Concept to retain: Read-only root filesystems reduce writable surface area; temporary data needs a deliberate writable mount.

My notes: _____

SC-147 · Containers · Use a read-only container filesystem

Situation: You want to reduce accidental writes from a simple inspection workload.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a read-only container and read a system file.

```
docker run --rm --read-only alpine:3.20 cat /etc/alpine-release
```

Verify / observe: Reading should succeed.

Step 2 — Attempt a write to the root filesystem.

```
docker run --rm --read-only alpine:3.20 sh -c 'echo test > /root/test.txt'
```

Verify / observe: The write should fail because the root filesystem is read-only.

Step 3 — Repeat with an explicit temporary writable area.

```
docker run --rm --read-only --tmpfs /tmp alpine:3.20 sh -c 'echo ok > /tmp/test.txt && cat /tmp/test.txt'
```

Verify / observe: Confirm writes to /tmp work while the root filesystem remains read-only.

Final conclusion: What changed between the second and third run, and what kinds of application writes need explicit mounts?

Concept to retain: Read-only root filesystems reduce writable surface area; temporary data needs a deliberate writable mount.

My notes: _____

SC-148 · Containers · Use a read-only container filesystem

Situation: You want to reduce accidental writes from a simple inspection workload.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a read-only container and read a system file.

```
docker run --rm --read-only alpine:3.20 cat /etc/alpine-release
```

Verify / observe: Reading should succeed.

Step 2 — Attempt a write to the root filesystem.

```
docker run --rm --read-only alpine:3.20 sh -c 'echo test > /root/test.txt'
```

Verify / observe: The write should fail because the root filesystem is read-only.

Step 3 — Repeat with an explicit temporary writable area.

```
docker run --rm --read-only --tmpfs /tmp alpine:3.20 sh -c 'echo ok > /tmp/test.txt && cat /tmp/test.txt'
```

Verify / observe: Confirm writes to /tmp work while the root filesystem remains read-only.

Final conclusion: What changed between the second and third run, and what kinds of application writes need explicit mounts?

Concept to retain: Read-only root filesystems reduce writable surface area; temporary data needs a deliberate writable mount.

My notes: _____

SC-149 · Containers · Use a read-only container filesystem

Situation: You want to reduce accidental writes from a simple inspection workload.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a read-only container and read a system file.

```
docker run --rm --read-only alpine:3.20 cat /etc/alpine-release
```

Verify / observe: Reading should succeed.

Step 2 — Attempt a write to the root filesystem.

```
docker run --rm --read-only alpine:3.20 sh -c 'echo test > /root/test.txt'
```

Verify / observe: The write should fail because the root filesystem is read-only.

Step 3 — Repeat with an explicit temporary writable area.

```
docker run --rm --read-only --tmpfs /tmp alpine:3.20 sh -c 'echo ok > /tmp/test.txt && cat /tmp/test.txt'
```

Verify / observe: Confirm writes to /tmp work while the root filesystem remains read-only.

Final conclusion: What changed between the second and third run, and what kinds of application writes need explicit mounts?

Concept to retain: Read-only root filesystems reduce writable surface area; temporary data needs a deliberate writable mount.

My notes: _____

SC-150 · Containers · Use a read-only container filesystem

Situation: You want to reduce accidental writes from a simple inspection workload.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a read-only container and read a system file.

```
docker run --rm --read-only alpine:3.20 cat /etc/alpine-release
```

Verify / observe: Reading should succeed.

Step 2 — Attempt a write to the root filesystem.

```
docker run --rm --read-only alpine:3.20 sh -c 'echo test > /root/test.txt'
```

Verify / observe: The write should fail because the root filesystem is read-only.

Step 3 — Repeat with an explicit temporary writable area.

```
docker run --rm --read-only --tmpfs /tmp alpine:3.20 sh -c 'echo ok > /tmp/test.txt && cat /tmp/test.txt'
```

Verify / observe: Confirm writes to /tmp work while the root filesystem remains read-only.

Final conclusion: What changed between the second and third run, and what kinds of application writes need explicit mounts?

Concept to retain: Read-only root filesystems reduce writable surface area; temporary data needs a deliberate writable mount.

My notes: _____

SC-151 · Containers · Pass a command and understand exit codes

Situation: A job runner needs to distinguish success from failure.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a command that succeeds.

```
docker run --rm alpine:3.20 sh -c 'exit 0'; echo $?
```

Verify / observe: Expect exit code 0 in a POSIX shell.

Step 2 — Run a command that deliberately fails.

```
docker run --rm alpine:3.20 sh -c 'exit 5'; echo $?
```

Verify / observe: Expect exit code 5 in a POSIX shell.

Step 3 — Run a command that prints context before exiting.

```
docker run --rm alpine:3.20 sh -c 'echo case-1; exit 2'; echo $?
```

Verify / observe: Confirm output and exit status are both visible.

Final conclusion: How should an automated pipeline interpret exit code 0 versus a nonzero code?

Concept to retain: Exit status is a key signal for scripting and automation.

My notes: _____

SC-152 · Containers · Pass a command and understand exit codes

Situation: A job runner needs to distinguish success from failure.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a command that succeeds.

```
docker run --rm alpine:3.20 sh -c 'exit 0'; echo $?
```

Verify / observe: Expect exit code 0 in a POSIX shell.

Step 2 — Run a command that deliberately fails.

```
docker run --rm alpine:3.20 sh -c 'exit 5'; echo $?
```

Verify / observe: Expect exit code 5 in a POSIX shell.

Step 3 — Run a command that prints context before exiting.

```
docker run --rm alpine:3.20 sh -c 'echo case-2; exit 2'; echo $?
```

Verify / observe: Confirm output and exit status are both visible.

Final conclusion: How should an automated pipeline interpret exit code 0 versus a nonzero code?

Concept to retain: Exit status is a key signal for scripting and automation.

My notes: _____

SC-153 · Containers · Pass a command and understand exit codes

Situation: A job runner needs to distinguish success from failure.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a command that succeeds.

```
docker run --rm alpine:3.20 sh -c 'exit 0'; echo $?
```

Verify / observe: Expect exit code 0 in a POSIX shell.

Step 2 — Run a command that deliberately fails.

```
docker run --rm alpine:3.20 sh -c 'exit 5'; echo $?
```

Verify / observe: Expect exit code 5 in a POSIX shell.

Step 3 — Run a command that prints context before exiting.

```
docker run --rm alpine:3.20 sh -c 'echo case-3; exit 2'; echo $?
```

Verify / observe: Confirm output and exit status are both visible.

Final conclusion: How should an automated pipeline interpret exit code 0 versus a nonzero code?

Concept to retain: Exit status is a key signal for scripting and automation.

My notes: _____

SC-154 · Containers · Pass a command and understand exit codes

Situation: A job runner needs to distinguish success from failure.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a command that succeeds.

```
docker run --rm alpine:3.20 sh -c 'exit 0'; echo $?
```

Verify / observe: Expect exit code 0 in a POSIX shell.

Step 2 — Run a command that deliberately fails.

```
docker run --rm alpine:3.20 sh -c 'exit 5'; echo $?
```

Verify / observe: Expect exit code 5 in a POSIX shell.

Step 3 — Run a command that prints context before exiting.

```
docker run --rm alpine:3.20 sh -c 'echo case-4; exit 2'; echo $?
```

Verify / observe: Confirm output and exit status are both visible.

Final conclusion: How should an automated pipeline interpret exit code 0 versus a nonzero code?

Concept to retain: Exit status is a key signal for scripting and automation.

My notes: _____

SC-155 · Containers · Pass a command and understand exit codes

Situation: A job runner needs to distinguish success from failure.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a command that succeeds.

```
docker run --rm alpine:3.20 sh -c 'exit 0'; echo $?
```

Verify / observe: Expect exit code 0 in a POSIX shell.

Step 2 — Run a command that deliberately fails.

```
docker run --rm alpine:3.20 sh -c 'exit 5'; echo $?
```

Verify / observe: Expect exit code 5 in a POSIX shell.

Step 3 — Run a command that prints context before exiting.

```
docker run --rm alpine:3.20 sh -c 'echo case-5; exit 2'; echo $?
```

Verify / observe: Confirm output and exit status are both visible.

Final conclusion: How should an automated pipeline interpret exit code 0 versus a nonzero code?

Concept to retain: Exit status is a key signal for scripting and automation.

My notes: _____

SC-156 · Containers · Pass a command and understand exit codes

Situation: A job runner needs to distinguish success from failure.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a command that succeeds.

```
docker run --rm alpine:3.20 sh -c 'exit 0'; echo $?
```

Verify / observe: Expect exit code 0 in a POSIX shell.

Step 2 — Run a command that deliberately fails.

```
docker run --rm alpine:3.20 sh -c 'exit 5'; echo $?
```

Verify / observe: Expect exit code 5 in a POSIX shell.

Step 3 — Run a command that prints context before exiting.

```
docker run --rm alpine:3.20 sh -c 'echo case-6; exit 2'; echo $?
```

Verify / observe: Confirm output and exit status are both visible.

Final conclusion: How should an automated pipeline interpret exit code 0 versus a nonzero code?

Concept to retain: Exit status is a key signal for scripting and automation.

My notes: _____

SC-157 · Containers · Pass a command and understand exit codes

Situation: A job runner needs to distinguish success from failure.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a command that succeeds.

```
docker run --rm alpine:3.20 sh -c 'exit 0'; echo $?
```

Verify / observe: Expect exit code 0 in a POSIX shell.

Step 2 — Run a command that deliberately fails.

```
docker run --rm alpine:3.20 sh -c 'exit 5'; echo $?
```

Verify / observe: Expect exit code 5 in a POSIX shell.

Step 3 — Run a command that prints context before exiting.

```
docker run --rm alpine:3.20 sh -c 'echo case-7; exit 2'; echo $?
```

Verify / observe: Confirm output and exit status are both visible.

Final conclusion: How should an automated pipeline interpret exit code 0 versus a nonzero code?

Concept to retain: Exit status is a key signal for scripting and automation.

My notes: _____

SC-158 · Containers · Pass a command and understand exit codes

Situation: A job runner needs to distinguish success from failure.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a command that succeeds.

```
docker run --rm alpine:3.20 sh -c 'exit 0'; echo $?
```

Verify / observe: Expect exit code 0 in a POSIX shell.

Step 2 — Run a command that deliberately fails.

```
docker run --rm alpine:3.20 sh -c 'exit 5'; echo $?
```

Verify / observe: Expect exit code 5 in a POSIX shell.

Step 3 — Run a command that prints context before exiting.

```
docker run --rm alpine:3.20 sh -c 'echo case-8; exit 2'; echo $?
```

Verify / observe: Confirm output and exit status are both visible.

Final conclusion: How should an automated pipeline interpret exit code 0 versus a nonzero code?

Concept to retain: Exit status is a key signal for scripting and automation.

My notes: _____

SC-159 · Containers · Pass a command and understand exit codes

Situation: A job runner needs to distinguish success from failure.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a command that succeeds.

```
docker run --rm alpine:3.20 sh -c 'exit 0'; echo $?
```

Verify / observe: Expect exit code 0 in a POSIX shell.

Step 2 — Run a command that deliberately fails.

```
docker run --rm alpine:3.20 sh -c 'exit 5'; echo $?
```

Verify / observe: Expect exit code 5 in a POSIX shell.

Step 3 — Run a command that prints context before exiting.

```
docker run --rm alpine:3.20 sh -c 'echo case-9; exit 2'; echo $?
```

Verify / observe: Confirm output and exit status are both visible.

Final conclusion: How should an automated pipeline interpret exit code 0 versus a nonzero code?

Concept to retain: Exit status is a key signal for scripting and automation.

My notes: _____

SC-160 · Containers · Pass a command and understand exit codes

Situation: A job runner needs to distinguish success from failure.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a command that succeeds.

```
docker run --rm alpine:3.20 sh -c 'exit 0'; echo $?
```

Verify / observe: Expect exit code 0 in a POSIX shell.

Step 2 — Run a command that deliberately fails.

```
docker run --rm alpine:3.20 sh -c 'exit 5'; echo $?
```

Verify / observe: Expect exit code 5 in a POSIX shell.

Step 3 — Run a command that prints context before exiting.

```
docker run --rm alpine:3.20 sh -c 'echo case-10; exit 2'; echo $?
```

Verify / observe: Confirm output and exit status are both visible.

Final conclusion: How should an automated pipeline interpret exit code 0 versus a nonzero code?

Concept to retain: Exit status is a key signal for scripting and automation.

My notes: _____

SC-161 · Containers · Inspect, rename, and identify a container

Situation: A teammate created a container with a confusing name and you need to document its identity.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List all containers and find the target.

```
docker ps -a
```

Verify / observe: Identify the exact container ID and current name.

Step 2 — Inspect its image and state.

```
docker inspect --format '{{.Name}} {{.Config.Image}} {{.State.Status}}' worker-1
```

Verify / observe: Confirm the name exists; substitute the actual name if necessary.

Step 3 — Rename the lab container.

```
docker rename worker-1 worker-renamed-1
```

Verify / observe: Verify the new name appears in docker ps -a.

Final conclusion: What remains unchanged when a container is renamed?

Concept to retain: A container name is a manageable reference; renaming does not rebuild the image or change its filesystem.

My notes: _____

SC-162 · Containers · Inspect, rename, and identify a container

Situation: A teammate created a container with a confusing name and you need to document its identity.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List all containers and find the target.

```
docker ps -a
```

Verify / observe: Identify the exact container ID and current name.

Step 2 — Inspect its image and state.

```
docker inspect --format '{{.Name}} {{.Config.Image}} {{.State.Status}}' worker-2
```

Verify / observe: Confirm the name exists; substitute the actual name if necessary.

Step 3 — Rename the lab container.

```
docker rename worker-2 worker-renamed-2
```

Verify / observe: Verify the new name appears in docker ps -a.

Final conclusion: What remains unchanged when a container is renamed?

Concept to retain: A container name is a manageable reference; renaming does not rebuild the image or change its filesystem.

My notes: _____

SC-163 · Containers · Inspect, rename, and identify a container

Situation: A teammate created a container with a confusing name and you need to document its identity.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List all containers and find the target.

```
docker ps -a
```

Verify / observe: Identify the exact container ID and current name.

Step 2 — Inspect its image and state.

```
docker inspect --format '{{.Name}} {{.Config.Image}} {{.State.Status}}' worker-3
```

Verify / observe: Confirm the name exists; substitute the actual name if necessary.

Step 3 — Rename the lab container.

```
docker rename worker-3 worker-renamed-3
```

Verify / observe: Verify the new name appears in docker ps -a.

Final conclusion: What remains unchanged when a container is renamed?

Concept to retain: A container name is a manageable reference; renaming does not rebuild the image or change its filesystem.

My notes: _____

SC-164 · Containers · Inspect, rename, and identify a container

Situation: A teammate created a container with a confusing name and you need to document its identity.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List all containers and find the target.

```
docker ps -a
```

Verify / observe: Identify the exact container ID and current name.

Step 2 — Inspect its image and state.

```
docker inspect --format '{{.Name}} {{.Config.Image}} {{.State.Status}}' worker-4
```

Verify / observe: Confirm the name exists; substitute the actual name if necessary.

Step 3 — Rename the lab container.

```
docker rename worker-4 worker-renamed-4
```

Verify / observe: Verify the new name appears in docker ps -a.

Final conclusion: What remains unchanged when a container is renamed?

Concept to retain: A container name is a manageable reference; renaming does not rebuild the image or change its filesystem.

My notes: _____

SC-165 · Containers · Inspect, rename, and identify a container

Situation: A teammate created a container with a confusing name and you need to document its identity.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List all containers and find the target.

```
docker ps -a
```

Verify / observe: Identify the exact container ID and current name.

Step 2 — Inspect its image and state.

```
docker inspect --format '{{.Name}} {{.Config.Image}} {{.State.Status}}' worker-5
```

Verify / observe: Confirm the name exists; substitute the actual name if necessary.

Step 3 — Rename the lab container.

```
docker rename worker-5 worker-renamed-5
```

Verify / observe: Verify the new name appears in docker ps -a.

Final conclusion: What remains unchanged when a container is renamed?

Concept to retain: A container name is a manageable reference; renaming does not rebuild the image or change its filesystem.

My notes: _____

SC-166 · Containers · Inspect, rename, and identify a container

Situation: A teammate created a container with a confusing name and you need to document its identity.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List all containers and find the target.

```
docker ps -a
```

Verify / observe: Identify the exact container ID and current name.

Step 2 — Inspect its image and state.

```
docker inspect --format '{{.Name}} {{.Config.Image}} {{.State.Status}}' worker-6
```

Verify / observe: Confirm the name exists; substitute the actual name if necessary.

Step 3 — Rename the lab container.

```
docker rename worker-6 worker-renamed-6
```

Verify / observe: Verify the new name appears in docker ps -a.

Final conclusion: What remains unchanged when a container is renamed?

Concept to retain: A container name is a manageable reference; renaming does not rebuild the image or change its filesystem.

My notes: _____

SC-167 · Containers · Inspect, rename, and identify a container

Situation: A teammate created a container with a confusing name and you need to document its identity.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List all containers and find the target.

```
docker ps -a
```

Verify / observe: Identify the exact container ID and current name.

Step 2 — Inspect its image and state.

```
docker inspect --format '{{.Name}} {{.Config.Image}} {{.State.Status}}' worker-7
```

Verify / observe: Confirm the name exists; substitute the actual name if necessary.

Step 3 — Rename the lab container.

```
docker rename worker-7 worker-renamed-7
```

Verify / observe: Verify the new name appears in docker ps -a.

Final conclusion: What remains unchanged when a container is renamed?

Concept to retain: A container name is a manageable reference; renaming does not rebuild the image or change its filesystem.

My notes: _____

SC-168 · Containers · Inspect, rename, and identify a container

Situation: A teammate created a container with a confusing name and you need to document its identity.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List all containers and find the target.

```
docker ps -a
```

Verify / observe: Identify the exact container ID and current name.

Step 2 — Inspect its image and state.

```
docker inspect --format '{{.Name}} {{.Config.Image}} {{.State.Status}}' worker-8
```

Verify / observe: Confirm the name exists; substitute the actual name if necessary.

Step 3 — Rename the lab container.

```
docker rename worker-8 worker-renamed-8
```

Verify / observe: Verify the new name appears in docker ps -a.

Final conclusion: What remains unchanged when a container is renamed?

Concept to retain: A container name is a manageable reference; renaming does not rebuild the image or change its filesystem.

My notes: _____

SC-169 · Containers · Inspect, rename, and identify a container

Situation: A teammate created a container with a confusing name and you need to document its identity.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List all containers and find the target.

```
docker ps -a
```

Verify / observe: Identify the exact container ID and current name.

Step 2 — Inspect its image and state.

```
docker inspect --format '{{.Name}} {{.Config.Image}} {{.State.Status}}' worker-9
```

Verify / observe: Confirm the name exists; substitute the actual name if necessary.

Step 3 — Rename the lab container.

```
docker rename worker-9 worker-renamed-9
```

Verify / observe: Verify the new name appears in docker ps -a.

Final conclusion: What remains unchanged when a container is renamed?

Concept to retain: A container name is a manageable reference; renaming does not rebuild the image or change its filesystem.

My notes: _____

SC-170 · Containers · Inspect, rename, and identify a container

Situation: A teammate created a container with a confusing name and you need to document its identity.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List all containers and find the target.

```
docker ps -a
```

Verify / observe: Identify the exact container ID and current name.

Step 2 — Inspect its image and state.

```
docker inspect --format '{{.Name}} {{.Config.Image}} {{.State.Status}}' worker-10
```

Verify / observe: Confirm the name exists; substitute the actual name if necessary.

Step 3 — Rename the lab container.

```
docker rename worker-10 worker-renamed-10
```

Verify / observe: Verify the new name appears in docker ps -a.

Final conclusion: What remains unchanged when a container is renamed?

Concept to retain: A container name is a manageable reference; renaming does not rebuild the image or change its filesystem.

My notes: _____

SC-171 · Containers · Execute a command in a running container

Situation: A running application container needs a quick diagnostic without restarting it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List running containers and identify the intended target.

```
docker ps
```

Verify / observe: Copy or note the exact name or ID.

Step 2 — Execute a non-interactive diagnostic.

```
docker exec app-1 sh -c 'id && pwd'
```

Verify / observe: Replace app-1 with an existing running container; inspect the output.

Step 3 — Open an interactive shell if the image supports it.

```
docker exec -it app-1 sh
```

Verify / observe: Exit the shell without stopping the main process.

Final conclusion: How does `docker exec` differ from `docker attach`?

Concept to retain: Exec starts a new process; attach connects to the container's main process.

My notes: _____

SC-172 · Containers · Execute a command in a running container

Situation: A running application container needs a quick diagnostic without restarting it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List running containers and identify the intended target.

```
docker ps
```

Verify / observe: Copy or note the exact name or ID.

Step 2 — Execute a non-interactive diagnostic.

```
docker exec app-2 sh -c 'id && pwd'
```

Verify / observe: Replace app-2 with an existing running container; inspect the output.

Step 3 — Open an interactive shell if the image supports it.

```
docker exec -it app-2 sh
```

Verify / observe: Exit the shell without stopping the main process.

Final conclusion: How does `docker exec` differ from `docker attach`?

Concept to retain: Exec starts a new process; attach connects to the container's main process.

My notes: _____

SC-173 · Containers · Execute a command in a running container

Situation: A running application container needs a quick diagnostic without restarting it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List running containers and identify the intended target.

```
docker ps
```

Verify / observe: Copy or note the exact name or ID.

Step 2 — Execute a non-interactive diagnostic.

```
docker exec app-3 sh -c 'id && pwd'
```

Verify / observe: Replace app-3 with an existing running container; inspect the output.

Step 3 — Open an interactive shell if the image supports it.

```
docker exec -it app-3 sh
```

Verify / observe: Exit the shell without stopping the main process.

Final conclusion: How does `docker exec` differ from `docker attach`?

Concept to retain: Exec starts a new process; attach connects to the container's main process.

My notes: _____

SC-174 · Containers · Execute a command in a running container

Situation: A running application container needs a quick diagnostic without restarting it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List running containers and identify the intended target.

```
docker ps
```

Verify / observe: Copy or note the exact name or ID.

Step 2 — Execute a non-interactive diagnostic.

```
docker exec app-4 sh -c 'id && pwd'
```

Verify / observe: Replace app-4 with an existing running container; inspect the output.

Step 3 — Open an interactive shell if the image supports it.

```
docker exec -it app-4 sh
```

Verify / observe: Exit the shell without stopping the main process.

Final conclusion: How does `docker exec` differ from `docker attach`?

Concept to retain: Exec starts a new process; attach connects to the container's main process.

My notes: _____

SC-175 · Containers · Execute a command in a running container

Situation: A running application container needs a quick diagnostic without restarting it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List running containers and identify the intended target.

```
docker ps
```

Verify / observe: Copy or note the exact name or ID.

Step 2 — Execute a non-interactive diagnostic.

```
docker exec app-5 sh -c 'id && pwd'
```

Verify / observe: Replace app-5 with an existing running container; inspect the output.

Step 3 — Open an interactive shell if the image supports it.

```
docker exec -it app-5 sh
```

Verify / observe: Exit the shell without stopping the main process.

Final conclusion: How does `docker exec` differ from `docker attach`?

Concept to retain: Exec starts a new process; attach connects to the container's main process.

My notes: _____

SC-176 · Containers · Execute a command in a running container

Situation: A running application container needs a quick diagnostic without restarting it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List running containers and identify the intended target.

```
docker ps
```

Verify / observe: Copy or note the exact name or ID.

Step 2 — Execute a non-interactive diagnostic.

```
docker exec app-6 sh -c 'id && pwd'
```

Verify / observe: Replace app-6 with an existing running container; inspect the output.

Step 3 — Open an interactive shell if the image supports it.

```
docker exec -it app-6 sh
```

Verify / observe: Exit the shell without stopping the main process.

Final conclusion: How does `docker exec` differ from `docker attach`?

Concept to retain: Exec starts a new process; attach connects to the container's main process.

My notes: _____

SC-177 · Containers · Execute a command in a running container

Situation: A running application container needs a quick diagnostic without restarting it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List running containers and identify the intended target.

```
docker ps
```

Verify / observe: Copy or note the exact name or ID.

Step 2 — Execute a non-interactive diagnostic.

```
docker exec app-7 sh -c 'id && pwd'
```

Verify / observe: Replace app-7 with an existing running container; inspect the output.

Step 3 — Open an interactive shell if the image supports it.

```
docker exec -it app-7 sh
```

Verify / observe: Exit the shell without stopping the main process.

Final conclusion: How does `docker exec` differ from `docker attach`?

Concept to retain: Exec starts a new process; attach connects to the container's main process.

My notes: _____

SC-178 · Containers · Execute a command in a running container

Situation: A running application container needs a quick diagnostic without restarting it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List running containers and identify the intended target.

```
docker ps
```

Verify / observe: Copy or note the exact name or ID.

Step 2 — Execute a non-interactive diagnostic.

```
docker exec app-8 sh -c 'id && pwd'
```

Verify / observe: Replace app-8 with an existing running container; inspect the output.

Step 3 — Open an interactive shell if the image supports it.

```
docker exec -it app-8 sh
```

Verify / observe: Exit the shell without stopping the main process.

Final conclusion: How does `docker exec` differ from `docker attach`?

Concept to retain: Exec starts a new process; attach connects to the container's main process.

My notes: _____

SC-179 · Containers · Execute a command in a running container

Situation: A running application container needs a quick diagnostic without restarting it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List running containers and identify the intended target.

```
docker ps
```

Verify / observe: Copy or note the exact name or ID.

Step 2 — Execute a non-interactive diagnostic.

```
docker exec app-9 sh -c 'id && pwd'
```

Verify / observe: Replace app-9 with an existing running container; inspect the output.

Step 3 — Open an interactive shell if the image supports it.

```
docker exec -it app-9 sh
```

Verify / observe: Exit the shell without stopping the main process.

Final conclusion: How does `docker exec` differ from `docker attach`?

Concept to retain: Exec starts a new process; attach connects to the container's main process.

My notes: _____

SC-180 · Containers · Execute a command in a running container

Situation: A running application container needs a quick diagnostic without restarting it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List running containers and identify the intended target.

```
docker ps
```

Verify / observe: Copy or note the exact name or ID.

Step 2 — Execute a non-interactive diagnostic.

```
docker exec app-10 sh -c 'id && pwd'
```

Verify / observe: Replace app-10 with an existing running container; inspect the output.

Step 3 — Open an interactive shell if the image supports it.

```
docker exec -it app-10 sh
```

Verify / observe: Exit the shell without stopping the main process.

Final conclusion: How does `docker exec` differ from `docker attach`?

Concept to retain: Exec starts a new process; attach connects to the container's main process.

My notes: _____

SC-181 · Containers · Copy a file out and into a container

Situation: You need to collect a configuration file from a running test container and copy a sample file into it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Copy a file from the container to the host.

```
docker cp app-1:/etc/os-release ./os-release-1.txt
```

Verify / observe: The source container must exist and contain the file.

Step 2 — Inspect the copied host file.

```
cat ./os-release-1.txt
```

Verify / observe: Read the copied content and confirm it is the expected file.

Step 3 — Copy a test file into a writable container path.

```
docker cp ./sample.txt app-1:/tmp/sample.txt
```

Verify / observe: Create sample.txt first and ensure the container path is writable.

Final conclusion: What permissions or path issues could make the copy fail?

Concept to retain: docker cp works between host and container filesystems, but source paths and permissions still matter.

My notes: _____

SC-182 · Containers · Copy a file out and into a container

Situation: You need to collect a configuration file from a running test container and copy a sample file into it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Copy a file from the container to the host.

```
docker cp app-2:/etc/os-release ./os-release-2.txt
```

Verify / observe: The source container must exist and contain the file.

Step 2 — Inspect the copied host file.

```
cat ./os-release-2.txt
```

Verify / observe: Read the copied content and confirm it is the expected file.

Step 3 — Copy a test file into a writable container path.

```
docker cp ./sample.txt app-2:/tmp/sample.txt
```

Verify / observe: Create sample.txt first and ensure the container path is writable.

Final conclusion: What permissions or path issues could make the copy fail?

Concept to retain: docker cp works between host and container filesystems, but source paths and permissions still matter.

My notes: _____

SC-183 · Containers · Copy a file out and into a container

Situation: You need to collect a configuration file from a running test container and copy a sample file into it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Copy a file from the container to the host.

```
docker cp app-3:/etc/os-release ./os-release-3.txt
```

Verify / observe: The source container must exist and contain the file.

Step 2 — Inspect the copied host file.

```
cat ./os-release-3.txt
```

Verify / observe: Read the copied content and confirm it is the expected file.

Step 3 — Copy a test file into a writable container path.

```
docker cp ./sample.txt app-3:/tmp/sample.txt
```

Verify / observe: Create sample.txt first and ensure the container path is writable.

Final conclusion: What permissions or path issues could make the copy fail?

Concept to retain: docker cp works between host and container filesystems, but source paths and permissions still matter.

My notes: _____

SC-184 · Containers · Copy a file out and into a container

Situation: You need to collect a configuration file from a running test container and copy a sample file into it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Copy a file from the container to the host.

```
docker cp app-4:/etc/os-release ./os-release-4.txt
```

Verify / observe: The source container must exist and contain the file.

Step 2 — Inspect the copied host file.

```
cat ./os-release-4.txt
```

Verify / observe: Read the copied content and confirm it is the expected file.

Step 3 — Copy a test file into a writable container path.

```
docker cp ./sample.txt app-4:/tmp/sample.txt
```

Verify / observe: Create sample.txt first and ensure the container path is writable.

Final conclusion: What permissions or path issues could make the copy fail?

Concept to retain: docker cp works between host and container filesystems, but source paths and permissions still matter.

My notes: _____

SC-185 · Containers · Copy a file out and into a container

Situation: You need to collect a configuration file from a running test container and copy a sample file into it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Copy a file from the container to the host.

```
docker cp app-5:/etc/os-release ./os-release-5.txt
```

Verify / observe: The source container must exist and contain the file.

Step 2 — Inspect the copied host file.

```
cat ./os-release-5.txt
```

Verify / observe: Read the copied content and confirm it is the expected file.

Step 3 — Copy a test file into a writable container path.

```
docker cp ./sample.txt app-5:/tmp/sample.txt
```

Verify / observe: Create sample.txt first and ensure the container path is writable.

Final conclusion: What permissions or path issues could make the copy fail?

Concept to retain: docker cp works between host and container filesystems, but source paths and permissions still matter.

My notes: _____

SC-186 · Containers · Copy a file out and into a container

Situation: You need to collect a configuration file from a running test container and copy a sample file into it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Copy a file from the container to the host.

```
docker cp app-6:/etc/os-release ./os-release-6.txt
```

Verify / observe: The source container must exist and contain the file.

Step 2 — Inspect the copied host file.

```
cat ./os-release-6.txt
```

Verify / observe: Read the copied content and confirm it is the expected file.

Step 3 — Copy a test file into a writable container path.

```
docker cp ./sample.txt app-6:/tmp/sample.txt
```

Verify / observe: Create sample.txt first and ensure the container path is writable.

Final conclusion: What permissions or path issues could make the copy fail?

Concept to retain: docker cp works between host and container filesystems, but source paths and permissions still matter.

My notes: _____

SC-187 · Containers · Copy a file out and into a container

Situation: You need to collect a configuration file from a running test container and copy a sample file into it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Copy a file from the container to the host.

```
docker cp app-7:/etc/os-release ./os-release-7.txt
```

Verify / observe: The source container must exist and contain the file.

Step 2 — Inspect the copied host file.

```
cat ./os-release-7.txt
```

Verify / observe: Read the copied content and confirm it is the expected file.

Step 3 — Copy a test file into a writable container path.

```
docker cp ./sample.txt app-7:/tmp/sample.txt
```

Verify / observe: Create sample.txt first and ensure the container path is writable.

Final conclusion: What permissions or path issues could make the copy fail?

Concept to retain: docker cp works between host and container filesystems, but source paths and permissions still matter.

My notes: _____

SC-188 · Containers · Copy a file out and into a container

Situation: You need to collect a configuration file from a running test container and copy a sample file into it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Copy a file from the container to the host.

```
docker cp app-8:/etc/os-release ./os-release-8.txt
```

Verify / observe: The source container must exist and contain the file.

Step 2 — Inspect the copied host file.

```
cat ./os-release-8.txt
```

Verify / observe: Read the copied content and confirm it is the expected file.

Step 3 — Copy a test file into a writable container path.

```
docker cp ./sample.txt app-8:/tmp/sample.txt
```

Verify / observe: Create sample.txt first and ensure the container path is writable.

Final conclusion: What permissions or path issues could make the copy fail?

Concept to retain: docker cp works between host and container filesystems, but source paths and permissions still matter.

My notes: _____

SC-189 · Containers · Copy a file out and into a container

Situation: You need to collect a configuration file from a running test container and copy a sample file into it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Copy a file from the container to the host.

```
docker cp app-9:/etc/os-release ./os-release-9.txt
```

Verify / observe: The source container must exist and contain the file.

Step 2 — Inspect the copied host file.

```
cat ./os-release-9.txt
```

Verify / observe: Read the copied content and confirm it is the expected file.

Step 3 — Copy a test file into a writable container path.

```
docker cp ./sample.txt app-9:/tmp/sample.txt
```

Verify / observe: Create sample.txt first and ensure the container path is writable.

Final conclusion: What permissions or path issues could make the copy fail?

Concept to retain: docker cp works between host and container filesystems, but source paths and permissions still matter.

My notes: _____

SC-190 · Containers · Copy a file out and into a container

Situation: You need to collect a configuration file from a running test container and copy a sample file into it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Copy a file from the container to the host.

```
docker cp app-10:/etc/os-release ./os-release-10.txt
```

Verify / observe: The source container must exist and contain the file.

Step 2 — Inspect the copied host file.

```
cat ./os-release-10.txt
```

Verify / observe: Read the copied content and confirm it is the expected file.

Step 3 — Copy a test file into a writable container path.

```
docker cp ./sample.txt app-10:/tmp/sample.txt
```

Verify / observe: Create sample.txt first and ensure the container path is writable.

Final conclusion: What permissions or path issues could make the copy fail?

Concept to retain: docker cp works between host and container filesystems, but source paths and permissions still matter.

My notes: _____

SC-191 · Containers · Graceful stop versus immediate kill

Situation: A process needs to be stopped safely, and you must compare graceful and immediate termination.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a long-running disposable container.

```
docker run -d --name signal-1 alpine:3.20 sh -c 'trap "echo got-term" TERM; while true; do sleep 2; done'
```

Verify / observe: Confirm the container is running.

Step 2 — Stop it with a timeout.

```
docker stop --time 10 signal-1
```

Verify / observe: Inspect its final state and logs.

Step 3 — For a second disposable container, use kill and compare.

```
docker run -d --name kill-1 alpine:3.20 sh -c 'sleep 300' && docker kill kill-1
```

Verify / observe: Confirm the second container stopped; remove both test containers when finished.

Final conclusion: Which operation gives the process an opportunity to handle a termination signal?

Concept to retain: Stop is the normal graceful path; kill terminates immediately with the chosen signal.

My notes: _____

SC-192 · Containers · Graceful stop versus immediate kill

Situation: A process needs to be stopped safely, and you must compare graceful and immediate termination.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a long-running disposable container.

```
docker run -d --name signal-2 alpine:3.20 sh -c 'trap "echo got-term" TERM; while true; do sleep 2; done'
```

Verify / observe: Confirm the container is running.

Step 2 — Stop it with a timeout.

```
docker stop --time 10 signal-2
```

Verify / observe: Inspect its final state and logs.

Step 3 — For a second disposable container, use kill and compare.

```
docker run -d --name kill-2 alpine:3.20 sh -c 'sleep 300' && docker kill kill-2
```

Verify / observe: Confirm the second container stopped; remove both test containers when finished.

Final conclusion: Which operation gives the process an opportunity to handle a termination signal?

Concept to retain: Stop is the normal graceful path; kill terminates immediately with the chosen signal.

My notes: _____

SC-193 · Containers · Graceful stop versus immediate kill

Situation: A process needs to be stopped safely, and you must compare graceful and immediate termination.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a long-running disposable container.

```
docker run -d --name signal-3 alpine:3.20 sh -c 'trap "echo got-term" TERM; while true; do sleep 2; done'
```

Verify / observe: Confirm the container is running.

Step 2 — Stop it with a timeout.

```
docker stop --time 10 signal-3
```

Verify / observe: Inspect its final state and logs.

Step 3 — For a second disposable container, use kill and compare.

```
docker run -d --name kill-3 alpine:3.20 sh -c 'sleep 300' && docker kill kill-3
```

Verify / observe: Confirm the second container stopped; remove both test containers when finished.

Final conclusion: Which operation gives the process an opportunity to handle a termination signal?

Concept to retain: Stop is the normal graceful path; kill terminates immediately with the chosen signal.

My notes: _____

SC-194 · Containers · Graceful stop versus immediate kill

Situation: A process needs to be stopped safely, and you must compare graceful and immediate termination.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a long-running disposable container.

```
docker run -d --name signal-4 alpine:3.20 sh -c 'trap "echo got-term" TERM; while true; do sleep 2; done'
```

Verify / observe: Confirm the container is running.

Step 2 — Stop it with a timeout.

```
docker stop --time 10 signal-4
```

Verify / observe: Inspect its final state and logs.

Step 3 — For a second disposable container, use kill and compare.

```
docker run -d --name kill-4 alpine:3.20 sh -c 'sleep 300' && docker kill kill-4
```

Verify / observe: Confirm the second container stopped; remove both test containers when finished.

Final conclusion: Which operation gives the process an opportunity to handle a termination signal?

Concept to retain: Stop is the normal graceful path; kill terminates immediately with the chosen signal.

My notes: _____

SC-195 · Containers · Graceful stop versus immediate kill

Situation: A process needs to be stopped safely, and you must compare graceful and immediate termination.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a long-running disposable container.

```
docker run -d --name signal-5 alpine:3.20 sh -c 'trap "echo got-term" TERM; while true; do sleep 2; done'
```

Verify / observe: Confirm the container is running.

Step 2 — Stop it with a timeout.

```
docker stop --time 10 signal-5
```

Verify / observe: Inspect its final state and logs.

Step 3 — For a second disposable container, use kill and compare.

```
docker run -d --name kill-5 alpine:3.20 sh -c 'sleep 300' && docker kill kill-5
```

Verify / observe: Confirm the second container stopped; remove both test containers when finished.

Final conclusion: Which operation gives the process an opportunity to handle a termination signal?

Concept to retain: Stop is the normal graceful path; kill terminates immediately with the chosen signal.

My notes: _____

SC-196 · Containers · Graceful stop versus immediate kill

Situation: A process needs to be stopped safely, and you must compare graceful and immediate termination.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a long-running disposable container.

```
docker run -d --name signal-6 alpine:3.20 sh -c 'trap "echo got-term" TERM; while true; do sleep 2; done'
```

Verify / observe: Confirm the container is running.

Step 2 — Stop it with a timeout.

```
docker stop --time 10 signal-6
```

Verify / observe: Inspect its final state and logs.

Step 3 — For a second disposable container, use kill and compare.

```
docker run -d --name kill-6 alpine:3.20 sh -c 'sleep 300' && docker kill kill-6
```

Verify / observe: Confirm the second container stopped; remove both test containers when finished.

Final conclusion: Which operation gives the process an opportunity to handle a termination signal?

Concept to retain: Stop is the normal graceful path; kill terminates immediately with the chosen signal.

My notes: _____

SC-197 · Containers · Graceful stop versus immediate kill

Situation: A process needs to be stopped safely, and you must compare graceful and immediate termination.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a long-running disposable container.

```
docker run -d --name signal-7 alpine:3.20 sh -c 'trap "echo got-term" TERM; while true; do sleep 2; done'
```

Verify / observe: Confirm the container is running.

Step 2 — Stop it with a timeout.

```
docker stop --time 10 signal-7
```

Verify / observe: Inspect its final state and logs.

Step 3 — For a second disposable container, use kill and compare.

```
docker run -d --name kill-7 alpine:3.20 sh -c 'sleep 300' && docker kill kill-7
```

Verify / observe: Confirm the second container stopped; remove both test containers when finished.

Final conclusion: Which operation gives the process an opportunity to handle a termination signal?

Concept to retain: Stop is the normal graceful path; kill terminates immediately with the chosen signal.

My notes: _____

SC-198 · Containers · Graceful stop versus immediate kill

Situation: A process needs to be stopped safely, and you must compare graceful and immediate termination.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a long-running disposable container.

```
docker run -d --name signal-8 alpine:3.20 sh -c 'trap "echo got-term" TERM; while true; do sleep 2; done'
```

Verify / observe: Confirm the container is running.

Step 2 — Stop it with a timeout.

```
docker stop --time 10 signal-8
```

Verify / observe: Inspect its final state and logs.

Step 3 — For a second disposable container, use kill and compare.

```
docker run -d --name kill-8 alpine:3.20 sh -c 'sleep 300' && docker kill kill-8
```

Verify / observe: Confirm the second container stopped; remove both test containers when finished.

Final conclusion: Which operation gives the process an opportunity to handle a termination signal?

Concept to retain: Stop is the normal graceful path; kill terminates immediately with the chosen signal.

My notes: _____

SC-199 · Containers · Graceful stop versus immediate kill

Situation: A process needs to be stopped safely, and you must compare graceful and immediate termination.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a long-running disposable container.

```
docker run -d --name signal-9 alpine:3.20 sh -c 'trap "echo got-term" TERM; while true; do sleep 2; done'
```

Verify / observe: Confirm the container is running.

Step 2 — Stop it with a timeout.

```
docker stop --time 10 signal-9
```

Verify / observe: Inspect its final state and logs.

Step 3 — For a second disposable container, use kill and compare.

```
docker run -d --name kill-9 alpine:3.20 sh -c 'sleep 300' && docker kill kill-9
```

Verify / observe: Confirm the second container stopped; remove both test containers when finished.

Final conclusion: Which operation gives the process an opportunity to handle a termination signal?

Concept to retain: Stop is the normal graceful path; kill terminates immediately with the chosen signal.

My notes: _____

SC-200 · Containers · Graceful stop versus immediate kill

Situation: A process needs to be stopped safely, and you must compare graceful and immediate termination.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a long-running disposable container.

```
docker run -d --name signal-10 alpine:3.20 sh -c 'trap "echo got-term" TERM; while true; do sleep 2; done'
```

Verify / observe: Confirm the container is running.

Step 2 — Stop it with a timeout.

```
docker stop --time 10 signal-10
```

Verify / observe: Inspect its final state and logs.

Step 3 — For a second disposable container, use kill and compare.

```
docker run -d --name kill-10 alpine:3.20 sh -c 'sleep 300' && docker kill kill-10
```

Verify / observe: Confirm the second container stopped; remove both test containers when finished.

Final conclusion: Which operation gives the process an opportunity to handle a termination signal?

Concept to retain: Stop is the normal graceful path; kill terminates immediately with the chosen signal.

My notes: _____

SC-201 · Containers · Set restart behavior and observe it

Situation: A lab service should restart after an unexpected process exit, but you need to understand policy limits.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start a container with a restart policy.

```
docker run -d --name restart-1 --restart unless-stopped alpine:3.20 sh -c 'while true; do sleep 5; done'
```

Verify / observe: Inspect the restart policy and running state.

Step 2 — Read the policy from container configuration.

```
docker inspect --format '{{.HostConfig.RestartPolicy.Name}}' restart-1
```

Verify / observe: Expect unless-stopped if the container was created correctly.

Step 3 — Stop it intentionally and inspect its state.

```
docker stop restart-1 && docker ps -a --filter name=restart-1
```

Verify / observe: Confirm it remains stopped because it was explicitly stopped.

Final conclusion: Why is a restart policy not the same as an application health check?

Concept to retain: Restart policies respond to process exits; health checks report application condition.

My notes: _____

SC-202 · Containers · Set restart behavior and observe it

Situation: A lab service should restart after an unexpected process exit, but you need to understand policy limits.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start a container with a restart policy.

```
docker run -d --name restart-2 --restart unless-stopped alpine:3.20 sh -c 'while true; do sleep 5; done'
```

Verify / observe: Inspect the restart policy and running state.

Step 2 — Read the policy from container configuration.

```
docker inspect --format '{{.HostConfig.RestartPolicy.Name}}' restart-2
```

Verify / observe: Expect unless-stopped if the container was created correctly.

Step 3 — Stop it intentionally and inspect its state.

```
docker stop restart-2 && docker ps -a --filter name=restart-2
```

Verify / observe: Confirm it remains stopped because it was explicitly stopped.

Final conclusion: Why is a restart policy not the same as an application health check?

Concept to retain: Restart policies respond to process exits; health checks report application condition.

My notes: _____

SC-203 · Containers · Set restart behavior and observe it

Situation: A lab service should restart after an unexpected process exit, but you need to understand policy limits.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start a container with a restart policy.

```
docker run -d --name restart-3 --restart unless-stopped alpine:3.20 sh -c 'while true; do sleep 5; done'
```

Verify / observe: Inspect the restart policy and running state.

Step 2 — Read the policy from container configuration.

```
docker inspect --format '{{.HostConfig.RestartPolicy.Name}}' restart-3
```

Verify / observe: Expect unless-stopped if the container was created correctly.

Step 3 — Stop it intentionally and inspect its state.

```
docker stop restart-3 && docker ps -a --filter name=restart-3
```

Verify / observe: Confirm it remains stopped because it was explicitly stopped.

Final conclusion: Why is a restart policy not the same as an application health check?

Concept to retain: Restart policies respond to process exits; health checks report application condition.

My notes: _____

SC-204 · Containers · Set restart behavior and observe it

Situation: A lab service should restart after an unexpected process exit, but you need to understand policy limits.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start a container with a restart policy.

```
docker run -d --name restart-4 --restart unless-stopped alpine:3.20 sh -c 'while true; do sleep 5; done'
```

Verify / observe: Inspect the restart policy and running state.

Step 2 — Read the policy from container configuration.

```
docker inspect --format '{{.HostConfig.RestartPolicy.Name}}' restart-4
```

Verify / observe: Expect unless-stopped if the container was created correctly.

Step 3 — Stop it intentionally and inspect its state.

```
docker stop restart-4 && docker ps -a --filter name=restart-4
```

Verify / observe: Confirm it remains stopped because it was explicitly stopped.

Final conclusion: Why is a restart policy not the same as an application health check?

Concept to retain: Restart policies respond to process exits; health checks report application condition.

My notes: _____

SC-205 · Containers · Set restart behavior and observe it

Situation: A lab service should restart after an unexpected process exit, but you need to understand policy limits.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start a container with a restart policy.

```
docker run -d --name restart-5 --restart unless-stopped alpine:3.20 sh -c 'while true; do sleep 5; done'
```

Verify / observe: Inspect the restart policy and running state.

Step 2 — Read the policy from container configuration.

```
docker inspect --format '{{.HostConfig.RestartPolicy.Name}}' restart-5
```

Verify / observe: Expect unless-stopped if the container was created correctly.

Step 3 — Stop it intentionally and inspect its state.

```
docker stop restart-5 && docker ps -a --filter name=restart-5
```

Verify / observe: Confirm it remains stopped because it was explicitly stopped.

Final conclusion: Why is a restart policy not the same as an application health check?

Concept to retain: Restart policies respond to process exits; health checks report application condition.

My notes: _____

SC-206 · Containers · Set restart behavior and observe it

Situation: A lab service should restart after an unexpected process exit, but you need to understand policy limits.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start a container with a restart policy.

```
docker run -d --name restart-6 --restart unless-stopped alpine:3.20 sh -c 'while true; do sleep 5; done'
```

Verify / observe: Inspect the restart policy and running state.

Step 2 — Read the policy from container configuration.

```
docker inspect --format '{{.HostConfig.RestartPolicy.Name}}' restart-6
```

Verify / observe: Expect unless-stopped if the container was created correctly.

Step 3 — Stop it intentionally and inspect its state.

```
docker stop restart-6 && docker ps -a --filter name=restart-6
```

Verify / observe: Confirm it remains stopped because it was explicitly stopped.

Final conclusion: Why is a restart policy not the same as an application health check?

Concept to retain: Restart policies respond to process exits; health checks report application condition.

My notes: _____

SC-207 · Containers · Set restart behavior and observe it

Situation: A lab service should restart after an unexpected process exit, but you need to understand policy limits.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start a container with a restart policy.

```
docker run -d --name restart-7 --restart unless-stopped alpine:3.20 sh -c 'while true; do sleep 5; done'
```

Verify / observe: Inspect the restart policy and running state.

Step 2 — Read the policy from container configuration.

```
docker inspect --format '{{.HostConfig.RestartPolicy.Name}}' restart-7
```

Verify / observe: Expect unless-stopped if the container was created correctly.

Step 3 — Stop it intentionally and inspect its state.

```
docker stop restart-7 && docker ps -a --filter name=restart-7
```

Verify / observe: Confirm it remains stopped because it was explicitly stopped.

Final conclusion: Why is a restart policy not the same as an application health check?

Concept to retain: Restart policies respond to process exits; health checks report application condition.

My notes: _____

SC-208 · Containers · Set restart behavior and observe it

Situation: A lab service should restart after an unexpected process exit, but you need to understand policy limits.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start a container with a restart policy.

```
docker run -d --name restart-8 --restart unless-stopped alpine:3.20 sh -c 'while true; do sleep 5; done'
```

Verify / observe: Inspect the restart policy and running state.

Step 2 — Read the policy from container configuration.

```
docker inspect --format '{{.HostConfig.RestartPolicy.Name}}' restart-8
```

Verify / observe: Expect unless-stopped if the container was created correctly.

Step 3 — Stop it intentionally and inspect its state.

```
docker stop restart-8 && docker ps -a --filter name=restart-8
```

Verify / observe: Confirm it remains stopped because it was explicitly stopped.

Final conclusion: Why is a restart policy not the same as an application health check?

Concept to retain: Restart policies respond to process exits; health checks report application condition.

My notes: _____

SC-209 · Containers · Set restart behavior and observe it

Situation: A lab service should restart after an unexpected process exit, but you need to understand policy limits.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start a container with a restart policy.

```
docker run -d --name restart-9 --restart unless-stopped alpine:3.20 sh -c 'while true; do sleep 5; done'
```

Verify / observe: Inspect the restart policy and running state.

Step 2 — Read the policy from container configuration.

```
docker inspect --format '{{.HostConfig.RestartPolicy.Name}}' restart-9
```

Verify / observe: Expect unless-stopped if the container was created correctly.

Step 3 — Stop it intentionally and inspect its state.

```
docker stop restart-9 && docker ps -a --filter name=restart-9
```

Verify / observe: Confirm it remains stopped because it was explicitly stopped.

Final conclusion: Why is a restart policy not the same as an application health check?

Concept to retain: Restart policies respond to process exits; health checks report application condition.

My notes: _____

SC-210 · Containers · Set restart behavior and observe it

Situation: A lab service should restart after an unexpected process exit, but you need to understand policy limits.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start a container with a restart policy.

```
docker run -d --name restart-10 --restart unless-stopped alpine:3.20 sh -c 'while true; do sleep 5; done'
```

Verify / observe: Inspect the restart policy and running state.

Step 2 — Read the policy from container configuration.

```
docker inspect --format '{{.HostConfig.RestartPolicy.Name}}' restart-10
```

Verify / observe: Expect unless-stopped if the container was created correctly.

Step 3 — Stop it intentionally and inspect its state.

```
docker stop restart-10 && docker ps -a --filter name=restart-10
```

Verify / observe: Confirm it remains stopped because it was explicitly stopped.

Final conclusion: Why is a restart policy not the same as an application health check?

Concept to retain: Restart policies respond to process exits; health checks report application condition.

My notes: _____

SC-211 · Containers · Apply CPU and memory limits

Situation: A test container must not consume unbounded CPU or memory on a shared development machine.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start a disposable container with resource limits.

```
docker run -d --name limit-1 --cpus 0.5 --memory 256m alpine:3.20 sh -c 'sleep 300'
```

Verify / observe: Confirm the container starts.

Step 2 — Inspect the configured limits.

```
docker inspect --format '{{.HostConfig.NanoCpus}} {{.HostConfig.Memory}}' limit-1
```

Verify / observe: Compare values with the requested CPU and memory limits.

Step 3 — Observe usage without a continuous stream.

```
docker stats --no-stream limit-1
```

Verify / observe: Read the CPU and memory usage columns.

Final conclusion: What is the difference between a configured limit and actual resource usage?

Concept to retain: Limits constrain resources; stats reports current observed usage.

My notes: _____

SC-212 · Containers · Apply CPU and memory limits

Situation: A test container must not consume unbounded CPU or memory on a shared development machine.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start a disposable container with resource limits.

```
docker run -d --name limit-2 --cpus 0.5 --memory 256m alpine:3.20 sh -c 'sleep 300'
```

Verify / observe: Confirm the container starts.

Step 2 — Inspect the configured limits.

```
docker inspect --format '{{.HostConfig.NanoCpus}} {{.HostConfig.Memory}}' limit-2
```

Verify / observe: Compare values with the requested CPU and memory limits.

Step 3 — Observe usage without a continuous stream.

```
docker stats --no-stream limit-2
```

Verify / observe: Read the CPU and memory usage columns.

Final conclusion: What is the difference between a configured limit and actual resource usage?

Concept to retain: Limits constrain resources; stats reports current observed usage.

My notes: _____

SC-213 · Containers · Apply CPU and memory limits

Situation: A test container must not consume unbounded CPU or memory on a shared development machine.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start a disposable container with resource limits.

```
docker run -d --name limit-3 --cpus 0.5 --memory 256m alpine:3.20 sh -c 'sleep 300'
```

Verify / observe: Confirm the container starts.

Step 2 — Inspect the configured limits.

```
docker inspect --format '{{.HostConfig.NanoCpus}} {{.HostConfig.Memory}}' limit-3
```

Verify / observe: Compare values with the requested CPU and memory limits.

Step 3 — Observe usage without a continuous stream.

```
docker stats --no-stream limit-3
```

Verify / observe: Read the CPU and memory usage columns.

Final conclusion: What is the difference between a configured limit and actual resource usage?

Concept to retain: Limits constrain resources; stats reports current observed usage.

My notes: _____

SC-214 · Containers · Apply CPU and memory limits

Situation: A test container must not consume unbounded CPU or memory on a shared development machine.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start a disposable container with resource limits.

```
docker run -d --name limit-4 --cpus 0.5 --memory 256m alpine:3.20 sh -c 'sleep 300'
```

Verify / observe: Confirm the container starts.

Step 2 — Inspect the configured limits.

```
docker inspect --format '{{.HostConfig.NanoCpus}} {{.HostConfig.Memory}}' limit-4
```

Verify / observe: Compare values with the requested CPU and memory limits.

Step 3 — Observe usage without a continuous stream.

```
docker stats --no-stream limit-4
```

Verify / observe: Read the CPU and memory usage columns.

Final conclusion: What is the difference between a configured limit and actual resource usage?

Concept to retain: Limits constrain resources; stats reports current observed usage.

My notes: _____

SC-215 · Containers · Apply CPU and memory limits

Situation: A test container must not consume unbounded CPU or memory on a shared development machine.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start a disposable container with resource limits.

```
docker run -d --name limit-5 --cpus 0.5 --memory 256m alpine:3.20 sh -c 'sleep 300'
```

Verify / observe: Confirm the container starts.

Step 2 — Inspect the configured limits.

```
docker inspect --format '{{.HostConfig.NanoCpus}} {{.HostConfig.Memory}}' limit-5
```

Verify / observe: Compare values with the requested CPU and memory limits.

Step 3 — Observe usage without a continuous stream.

```
docker stats --no-stream limit-5
```

Verify / observe: Read the CPU and memory usage columns.

Final conclusion: What is the difference between a configured limit and actual resource usage?

Concept to retain: Limits constrain resources; stats reports current observed usage.

My notes: _____

SC-216 · Containers · Apply CPU and memory limits

Situation: A test container must not consume unbounded CPU or memory on a shared development machine.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start a disposable container with resource limits.

```
docker run -d --name limit-6 --cpus 0.5 --memory 256m alpine:3.20 sh -c 'sleep 300'
```

Verify / observe: Confirm the container starts.

Step 2 — Inspect the configured limits.

```
docker inspect --format '{{.HostConfig.NanoCpus}} {{.HostConfig.Memory}}' limit-6
```

Verify / observe: Compare values with the requested CPU and memory limits.

Step 3 — Observe usage without a continuous stream.

```
docker stats --no-stream limit-6
```

Verify / observe: Read the CPU and memory usage columns.

Final conclusion: What is the difference between a configured limit and actual resource usage?

Concept to retain: Limits constrain resources; stats reports current observed usage.

My notes: _____

SC-217 · Containers · Apply CPU and memory limits

Situation: A test container must not consume unbounded CPU or memory on a shared development machine.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start a disposable container with resource limits.

```
docker run -d --name limit-7 --cpus 0.5 --memory 256m alpine:3.20 sh -c 'sleep 300'
```

Verify / observe: Confirm the container starts.

Step 2 — Inspect the configured limits.

```
docker inspect --format '{{.HostConfig.NanoCpus}} {{.HostConfig.Memory}}' limit-7
```

Verify / observe: Compare values with the requested CPU and memory limits.

Step 3 — Observe usage without a continuous stream.

```
docker stats --no-stream limit-7
```

Verify / observe: Read the CPU and memory usage columns.

Final conclusion: What is the difference between a configured limit and actual resource usage?

Concept to retain: Limits constrain resources; stats reports current observed usage.

My notes: _____

SC-218 · Containers · Apply CPU and memory limits

Situation: A test container must not consume unbounded CPU or memory on a shared development machine.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start a disposable container with resource limits.

```
docker run -d --name limit-8 --cpus 0.5 --memory 256m alpine:3.20 sh -c 'sleep 300'
```

Verify / observe: Confirm the container starts.

Step 2 — Inspect the configured limits.

```
docker inspect --format '{{.HostConfig.NanoCpus}} {{.HostConfig.Memory}}' limit-8
```

Verify / observe: Compare values with the requested CPU and memory limits.

Step 3 — Observe usage without a continuous stream.

```
docker stats --no-stream limit-8
```

Verify / observe: Read the CPU and memory usage columns.

Final conclusion: What is the difference between a configured limit and actual resource usage?

Concept to retain: Limits constrain resources; stats reports current observed usage.

My notes: _____

SC-219 · Containers · Apply CPU and memory limits

Situation: A test container must not consume unbounded CPU or memory on a shared development machine.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start a disposable container with resource limits.

```
docker run -d --name limit-9 --cpus 0.5 --memory 256m alpine:3.20 sh -c 'sleep 300'
```

Verify / observe: Confirm the container starts.

Step 2 — Inspect the configured limits.

```
docker inspect --format '{{.HostConfig.NanoCpus}} {{.HostConfig.Memory}}' limit-9
```

Verify / observe: Compare values with the requested CPU and memory limits.

Step 3 — Observe usage without a continuous stream.

```
docker stats --no-stream limit-9
```

Verify / observe: Read the CPU and memory usage columns.

Final conclusion: What is the difference between a configured limit and actual resource usage?

Concept to retain: Limits constrain resources; stats reports current observed usage.

My notes: _____

SC-220 · Containers · Apply CPU and memory limits

Situation: A test container must not consume unbounded CPU or memory on a shared development machine.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start a disposable container with resource limits.

```
docker run -d --name limit-10 --cpus 0.5 --memory 256m alpine:3.20 sh -c 'sleep 300'
```

Verify / observe: Confirm the container starts.

Step 2 — Inspect the configured limits.

```
docker inspect --format '{{.HostConfig.NanoCpus}} {{.HostConfig.Memory}}' limit-10
```

Verify / observe: Compare values with the requested CPU and memory limits.

Step 3 — Observe usage without a continuous stream.

```
docker stats --no-stream limit-10
```

Verify / observe: Read the CPU and memory usage columns.

Final conclusion: What is the difference between a configured limit and actual resource usage?

Concept to retain: Limits constrain resources; stats reports current observed usage.

My notes: _____

SC-221 · Containers · Run with a non-root user and fewer capabilities

Situation: You are hardening a simple workload by avoiding unnecessary privileges.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a test process with a numeric non-root UID/GID.

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Verify / observe: Check the UID and GID printed by id.

Step 2 — Drop all Linux capabilities for a basic command.

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Verify / observe: Confirm the basic command still runs.

Step 3 — Inspect security configuration of a lab container.

```
docker inspect --format '{{json .HostConfig.CapDrop}}' limit-1
```

Verify / observe: Use an existing lab container or adapt the target name.

Final conclusion: Which privileges does this workload demonstrably need, and how would you test that assumption?

Concept to retain: Least privilege reduces impact if a workload is compromised.

My notes: _____

SC-222 · Containers · Run with a non-root user and fewer capabilities

Situation: You are hardening a simple workload by avoiding unnecessary privileges.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a test process with a numeric non-root UID/GID.

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Verify / observe: Check the UID and GID printed by id.

Step 2 — Drop all Linux capabilities for a basic command.

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Verify / observe: Confirm the basic command still runs.

Step 3 — Inspect security configuration of a lab container.

```
docker inspect --format '{{json .HostConfig.CapDrop}}' limit-2
```

Verify / observe: Use an existing lab container or adapt the target name.

Final conclusion: Which privileges does this workload demonstrably need, and how would you test that assumption?

Concept to retain: Least privilege reduces impact if a workload is compromised.

My notes: _____

SC-223 · Containers · Run with a non-root user and fewer capabilities

Situation: You are hardening a simple workload by avoiding unnecessary privileges.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a test process with a numeric non-root UID/GID.

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Verify / observe: Check the UID and GID printed by id.

Step 2 — Drop all Linux capabilities for a basic command.

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Verify / observe: Confirm the basic command still runs.

Step 3 — Inspect security configuration of a lab container.

```
docker inspect --format '{{json .HostConfig.CapDrop}}' limit-3
```

Verify / observe: Use an existing lab container or adapt the target name.

Final conclusion: Which privileges does this workload demonstrably need, and how would you test that assumption?

Concept to retain: Least privilege reduces impact if a workload is compromised.

My notes: _____

SC-224 · Containers · Run with a non-root user and fewer capabilities

Situation: You are hardening a simple workload by avoiding unnecessary privileges.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a test process with a numeric non-root UID/GID.

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Verify / observe: Check the UID and GID printed by id.

Step 2 — Drop all Linux capabilities for a basic command.

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Verify / observe: Confirm the basic command still runs.

Step 3 — Inspect security configuration of a lab container.

```
docker inspect --format '{{json .HostConfig.CapDrop}}' limit-4
```

Verify / observe: Use an existing lab container or adapt the target name.

Final conclusion: Which privileges does this workload demonstrably need, and how would you test that assumption?

Concept to retain: Least privilege reduces impact if a workload is compromised.

My notes: _____

SC-225 · Containers · Run with a non-root user and fewer capabilities

Situation: You are hardening a simple workload by avoiding unnecessary privileges.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a test process with a numeric non-root UID/GID.

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Verify / observe: Check the UID and GID printed by id.

Step 2 — Drop all Linux capabilities for a basic command.

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Verify / observe: Confirm the basic command still runs.

Step 3 — Inspect security configuration of a lab container.

```
docker inspect --format '{{json .HostConfig.CapDrop}}' limit-5
```

Verify / observe: Use an existing lab container or adapt the target name.

Final conclusion: Which privileges does this workload demonstrably need, and how would you test that assumption?

Concept to retain: Least privilege reduces impact if a workload is compromised.

My notes: _____

SC-226 · Containers · Run with a non-root user and fewer capabilities

Situation: You are hardening a simple workload by avoiding unnecessary privileges.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a test process with a numeric non-root UID/GID.

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Verify / observe: Check the UID and GID printed by id.

Step 2 — Drop all Linux capabilities for a basic command.

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Verify / observe: Confirm the basic command still runs.

Step 3 — Inspect security configuration of a lab container.

```
docker inspect --format '{{json .HostConfig.CapDrop}}' limit-6
```

Verify / observe: Use an existing lab container or adapt the target name.

Final conclusion: Which privileges does this workload demonstrably need, and how would you test that assumption?

Concept to retain: Least privilege reduces impact if a workload is compromised.

My notes: _____

SC-227 · Containers · Run with a non-root user and fewer capabilities

Situation: You are hardening a simple workload by avoiding unnecessary privileges.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a test process with a numeric non-root UID/GID.

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Verify / observe: Check the UID and GID printed by id.

Step 2 — Drop all Linux capabilities for a basic command.

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Verify / observe: Confirm the basic command still runs.

Step 3 — Inspect security configuration of a lab container.

```
docker inspect --format '{{json .HostConfig.CapDrop}}' limit-7
```

Verify / observe: Use an existing lab container or adapt the target name.

Final conclusion: Which privileges does this workload demonstrably need, and how would you test that assumption?

Concept to retain: Least privilege reduces impact if a workload is compromised.

My notes: _____

SC-228 · Containers · Run with a non-root user and fewer capabilities

Situation: You are hardening a simple workload by avoiding unnecessary privileges.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a test process with a numeric non-root UID/GID.

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Verify / observe: Check the UID and GID printed by id.

Step 2 — Drop all Linux capabilities for a basic command.

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Verify / observe: Confirm the basic command still runs.

Step 3 — Inspect security configuration of a lab container.

```
docker inspect --format '{{json .HostConfig.CapDrop}}' limit-8
```

Verify / observe: Use an existing lab container or adapt the target name.

Final conclusion: Which privileges does this workload demonstrably need, and how would you test that assumption?

Concept to retain: Least privilege reduces impact if a workload is compromised.

My notes: _____

SC-229 · Containers · Run with a non-root user and fewer capabilities

Situation: You are hardening a simple workload by avoiding unnecessary privileges.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a test process with a numeric non-root UID/GID.

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Verify / observe: Check the UID and GID printed by id.

Step 2 — Drop all Linux capabilities for a basic command.

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Verify / observe: Confirm the basic command still runs.

Step 3 — Inspect security configuration of a lab container.

```
docker inspect --format '{{json .HostConfig.CapDrop}}' limit-9
```

Verify / observe: Use an existing lab container or adapt the target name.

Final conclusion: Which privileges does this workload demonstrably need, and how would you test that assumption?

Concept to retain: Least privilege reduces impact if a workload is compromised.

My notes: _____

SC-230 · Containers · Run with a non-root user and fewer capabilities

Situation: You are hardening a simple workload by avoiding unnecessary privileges.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run a test process with a numeric non-root UID/GID.

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Verify / observe: Check the UID and GID printed by id.

Step 2 — Drop all Linux capabilities for a basic command.

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Verify / observe: Confirm the basic command still runs.

Step 3 — Inspect security configuration of a lab container.

```
docker inspect --format '{{json .HostConfig.CapDrop}}' limit-10
```

Verify / observe: Use an existing lab container or adapt the target name.

Final conclusion: Which privileges does this workload demonstrably need, and how would you test that assumption?

Concept to retain: Least privilege reduces impact if a workload is compromised.

My notes: _____

SC-231 · Volumes · Prove that named-volume data persists

Situation: A service must retain uploaded data when one container is removed and another is created.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a named volume.

```
docker volume create persist-1
```

Verify / observe: Confirm the volume appears in docker volume ls.

Step 2 — Write a marker into the volume using a temporary container.

```
docker run --rm -v persist-1:/data alpine:3.20 sh -c 'echo persistent-1 > /data/marker.txt'
```

Verify / observe: Confirm the command succeeds.

Step 3 — Read the marker from a new container.

```
docker run --rm -v persist-1:/data alpine:3.20 cat /data/marker.txt
```

Verify / observe: The marker should still exist.

Final conclusion: What proves the data is stored outside the first container's writable layer?

Concept to retain: A named volume persists independently of a container's lifecycle.

My notes: _____

SC-232 · Volumes · Prove that named-volume data persists

Situation: A service must retain uploaded data when one container is removed and another is created.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a named volume.

```
docker volume create persist-2
```

Verify / observe: Confirm the volume appears in docker volume ls.

Step 2 — Write a marker into the volume using a temporary container.

```
docker run --rm -v persist-2:/data alpine:3.20 sh -c 'echo persistent-2 > /data/marker.txt'
```

Verify / observe: Confirm the command succeeds.

Step 3 — Read the marker from a new container.

```
docker run --rm -v persist-2:/data alpine:3.20 cat /data/marker.txt
```

Verify / observe: The marker should still exist.

Final conclusion: What proves the data is stored outside the first container's writable layer?

Concept to retain: A named volume persists independently of a container's lifecycle.

My notes: _____

SC-233 · Volumes · Prove that named-volume data persists

Situation: A service must retain uploaded data when one container is removed and another is created.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a named volume.

```
docker volume create persist-3
```

Verify / observe: Confirm the volume appears in docker volume ls.

Step 2 — Write a marker into the volume using a temporary container.

```
docker run --rm -v persist-3:/data alpine:3.20 sh -c 'echo persistent-3 > /data/marker.txt'
```

Verify / observe: Confirm the command succeeds.

Step 3 — Read the marker from a new container.

```
docker run --rm -v persist-3:/data alpine:3.20 cat /data/marker.txt
```

Verify / observe: The marker should still exist.

Final conclusion: What proves the data is stored outside the first container's writable layer?

Concept to retain: A named volume persists independently of a container's lifecycle.

My notes: _____

SC-234 · Volumes · Prove that named-volume data persists

Situation: A service must retain uploaded data when one container is removed and another is created.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a named volume.

```
docker volume create persist-4
```

Verify / observe: Confirm the volume appears in docker volume ls.

Step 2 — Write a marker into the volume using a temporary container.

```
docker run --rm -v persist-4:/data alpine:3.20 sh -c 'echo persistent-4 > /data/marker.txt'
```

Verify / observe: Confirm the command succeeds.

Step 3 — Read the marker from a new container.

```
docker run --rm -v persist-4:/data alpine:3.20 cat /data/marker.txt
```

Verify / observe: The marker should still exist.

Final conclusion: What proves the data is stored outside the first container's writable layer?

Concept to retain: A named volume persists independently of a container's lifecycle.

My notes: _____

SC-235 · Volumes · Prove that named-volume data persists

Situation: A service must retain uploaded data when one container is removed and another is created.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a named volume.

```
docker volume create persist-5
```

Verify / observe: Confirm the volume appears in docker volume ls.

Step 2 — Write a marker into the volume using a temporary container.

```
docker run --rm -v persist-5:/data alpine:3.20 sh -c 'echo persistent-5 > /data/marker.txt'
```

Verify / observe: Confirm the command succeeds.

Step 3 — Read the marker from a new container.

```
docker run --rm -v persist-5:/data alpine:3.20 cat /data/marker.txt
```

Verify / observe: The marker should still exist.

Final conclusion: What proves the data is stored outside the first container's writable layer?

Concept to retain: A named volume persists independently of a container's lifecycle.

My notes: _____

SC-236 · Volumes · Prove that named-volume data persists

Situation: A service must retain uploaded data when one container is removed and another is created.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a named volume.

```
docker volume create persist-6
```

Verify / observe: Confirm the volume appears in docker volume ls.

Step 2 — Write a marker into the volume using a temporary container.

```
docker run --rm -v persist-6:/data alpine:3.20 sh -c 'echo persistent-6 > /data/marker.txt'
```

Verify / observe: Confirm the command succeeds.

Step 3 — Read the marker from a new container.

```
docker run --rm -v persist-6:/data alpine:3.20 cat /data/marker.txt
```

Verify / observe: The marker should still exist.

Final conclusion: What proves the data is stored outside the first container's writable layer?

Concept to retain: A named volume persists independently of a container's lifecycle.

My notes: _____

SC-237 · Volumes · Prove that named-volume data persists

Situation: A service must retain uploaded data when one container is removed and another is created.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a named volume.

```
docker volume create persist-7
```

Verify / observe: Confirm the volume appears in docker volume ls.

Step 2 — Write a marker into the volume using a temporary container.

```
docker run --rm -v persist-7:/data alpine:3.20 sh -c 'echo persistent-7 > /data/marker.txt'
```

Verify / observe: Confirm the command succeeds.

Step 3 — Read the marker from a new container.

```
docker run --rm -v persist-7:/data alpine:3.20 cat /data/marker.txt
```

Verify / observe: The marker should still exist.

Final conclusion: What proves the data is stored outside the first container's writable layer?

Concept to retain: A named volume persists independently of a container's lifecycle.

My notes: _____

SC-238 · Volumes · Prove that named-volume data persists

Situation: A service must retain uploaded data when one container is removed and another is created.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a named volume.

```
docker volume create persist-8
```

Verify / observe: Confirm the volume appears in docker volume ls.

Step 2 — Write a marker into the volume using a temporary container.

```
docker run --rm -v persist-8:/data alpine:3.20 sh -c 'echo persistent-8 > /data/marker.txt'
```

Verify / observe: Confirm the command succeeds.

Step 3 — Read the marker from a new container.

```
docker run --rm -v persist-8:/data alpine:3.20 cat /data/marker.txt
```

Verify / observe: The marker should still exist.

Final conclusion: What proves the data is stored outside the first container's writable layer?

Concept to retain: A named volume persists independently of a container's lifecycle.

My notes: _____

SC-239 · Volumes · Prove that named-volume data persists

Situation: A service must retain uploaded data when one container is removed and another is created.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a named volume.

```
docker volume create persist-9
```

Verify / observe: Confirm the volume appears in docker volume ls.

Step 2 — Write a marker into the volume using a temporary container.

```
docker run --rm -v persist-9:/data alpine:3.20 sh -c 'echo persistent-9 > /data/marker.txt'
```

Verify / observe: Confirm the command succeeds.

Step 3 — Read the marker from a new container.

```
docker run --rm -v persist-9:/data alpine:3.20 cat /data/marker.txt
```

Verify / observe: The marker should still exist.

Final conclusion: What proves the data is stored outside the first container's writable layer?

Concept to retain: A named volume persists independently of a container's lifecycle.

My notes: _____

SC-240 · Volumes · Prove that named-volume data persists

Situation: A service must retain uploaded data when one container is removed and another is created.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a named volume.

```
docker volume create persist-10
```

Verify / observe: Confirm the volume appears in docker volume ls.

Step 2 — Write a marker into the volume using a temporary container.

```
docker run --rm -v persist-10:/data alpine:3.20 sh -c 'echo persistent-10 > /data/marker.txt'
```

Verify / observe: Confirm the command succeeds.

Step 3 — Read the marker from a new container.

```
docker run --rm -v persist-10:/data alpine:3.20 cat /data/marker.txt
```

Verify / observe: The marker should still exist.

Final conclusion: What proves the data is stored outside the first container's writable layer?

Concept to retain: A named volume persists independently of a container's lifecycle.

My notes: _____

SC-241 · Volumes · Inspect a volume before using it

Situation: You found a volume on a host and need to identify its driver and metadata before attaching it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List available volumes.

```
docker volume ls
```

Verify / observe: Find the exact volume name.

Step 2 — Inspect the selected volume.

```
docker volume inspect persist-1
```

Verify / observe: Review driver, mountpoint, and labels if present.

Step 3 — Check overall Docker disk use.

```
docker system df -v
```

Verify / observe: Use this as a general inventory, not as proof that the volume is safe to delete.

Final conclusion: What extra information would you need before removing or backing up this volume?

Concept to retain: Volume names and mountpoints alone do not establish data ownership or retention needs.

My notes: _____

SC-242 · Volumes · Inspect a volume before using it

Situation: You found a volume on a host and need to identify its driver and metadata before attaching it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List available volumes.

```
docker volume ls
```

Verify / observe: Find the exact volume name.

Step 2 — Inspect the selected volume.

```
docker volume inspect persist-2
```

Verify / observe: Review driver, mountpoint, and labels if present.

Step 3 — Check overall Docker disk use.

```
docker system df -v
```

Verify / observe: Use this as a general inventory, not as proof that the volume is safe to delete.

Final conclusion: What extra information would you need before removing or backing up this volume?

Concept to retain: Volume names and mountpoints alone do not establish data ownership or retention needs.

My notes: _____

SC-243 · Volumes · Inspect a volume before using it

Situation: You found a volume on a host and need to identify its driver and metadata before attaching it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List available volumes.

```
docker volume ls
```

Verify / observe: Find the exact volume name.

Step 2 — Inspect the selected volume.

```
docker volume inspect persist-3
```

Verify / observe: Review driver, mountpoint, and labels if present.

Step 3 — Check overall Docker disk use.

```
docker system df -v
```

Verify / observe: Use this as a general inventory, not as proof that the volume is safe to delete.

Final conclusion: What extra information would you need before removing or backing up this volume?

Concept to retain: Volume names and mountpoints alone do not establish data ownership or retention needs.

My notes: _____

SC-244 · Volumes · Inspect a volume before using it

Situation: You found a volume on a host and need to identify its driver and metadata before attaching it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List available volumes.

```
docker volume ls
```

Verify / observe: Find the exact volume name.

Step 2 — Inspect the selected volume.

```
docker volume inspect persist-4
```

Verify / observe: Review driver, mountpoint, and labels if present.

Step 3 — Check overall Docker disk use.

```
docker system df -v
```

Verify / observe: Use this as a general inventory, not as proof that the volume is safe to delete.

Final conclusion: What extra information would you need before removing or backing up this volume?

Concept to retain: Volume names and mountpoints alone do not establish data ownership or retention needs.

My notes: _____

SC-245 · Volumes · Inspect a volume before using it

Situation: You found a volume on a host and need to identify its driver and metadata before attaching it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List available volumes.

```
docker volume ls
```

Verify / observe: Find the exact volume name.

Step 2 — Inspect the selected volume.

```
docker volume inspect persist-5
```

Verify / observe: Review driver, mountpoint, and labels if present.

Step 3 — Check overall Docker disk use.

```
docker system df -v
```

Verify / observe: Use this as a general inventory, not as proof that the volume is safe to delete.

Final conclusion: What extra information would you need before removing or backing up this volume?

Concept to retain: Volume names and mountpoints alone do not establish data ownership or retention needs.

My notes: _____

SC-246 · Volumes · Inspect a volume before using it

Situation: You found a volume on a host and need to identify its driver and metadata before attaching it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List available volumes.

```
docker volume ls
```

Verify / observe: Find the exact volume name.

Step 2 — Inspect the selected volume.

```
docker volume inspect persist-6
```

Verify / observe: Review driver, mountpoint, and labels if present.

Step 3 — Check overall Docker disk use.

```
docker system df -v
```

Verify / observe: Use this as a general inventory, not as proof that the volume is safe to delete.

Final conclusion: What extra information would you need before removing or backing up this volume?

Concept to retain: Volume names and mountpoints alone do not establish data ownership or retention needs.

My notes: _____

SC-247 · Volumes · Inspect a volume before using it

Situation: You found a volume on a host and need to identify its driver and metadata before attaching it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List available volumes.

```
docker volume ls
```

Verify / observe: Find the exact volume name.

Step 2 — Inspect the selected volume.

```
docker volume inspect persist-7
```

Verify / observe: Review driver, mountpoint, and labels if present.

Step 3 — Check overall Docker disk use.

```
docker system df -v
```

Verify / observe: Use this as a general inventory, not as proof that the volume is safe to delete.

Final conclusion: What extra information would you need before removing or backing up this volume?

Concept to retain: Volume names and mountpoints alone do not establish data ownership or retention needs.

My notes: _____

SC-248 · Volumes · Inspect a volume before using it

Situation: You found a volume on a host and need to identify its driver and metadata before attaching it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List available volumes.

```
docker volume ls
```

Verify / observe: Find the exact volume name.

Step 2 — Inspect the selected volume.

```
docker volume inspect persist-8
```

Verify / observe: Review driver, mountpoint, and labels if present.

Step 3 — Check overall Docker disk use.

```
docker system df -v
```

Verify / observe: Use this as a general inventory, not as proof that the volume is safe to delete.

Final conclusion: What extra information would you need before removing or backing up this volume?

Concept to retain: Volume names and mountpoints alone do not establish data ownership or retention needs.

My notes: _____

SC-249 · Volumes · Inspect a volume before using it

Situation: You found a volume on a host and need to identify its driver and metadata before attaching it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List available volumes.

```
docker volume ls
```

Verify / observe: Find the exact volume name.

Step 2 — Inspect the selected volume.

```
docker volume inspect persist-9
```

Verify / observe: Review driver, mountpoint, and labels if present.

Step 3 — Check overall Docker disk use.

```
docker system df -v
```

Verify / observe: Use this as a general inventory, not as proof that the volume is safe to delete.

Final conclusion: What extra information would you need before removing or backing up this volume?

Concept to retain: Volume names and mountpoints alone do not establish data ownership or retention needs.

My notes: _____

SC-250 · Volumes · Inspect a volume before using it

Situation: You found a volume on a host and need to identify its driver and metadata before attaching it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List available volumes.

```
docker volume ls
```

Verify / observe: Find the exact volume name.

Step 2 — Inspect the selected volume.

```
docker volume inspect persist-10
```

Verify / observe: Review driver, mountpoint, and labels if present.

Step 3 — Check overall Docker disk use.

```
docker system df -v
```

Verify / observe: Use this as a general inventory, not as proof that the volume is safe to delete.

Final conclusion: What extra information would you need before removing or backing up this volume?

Concept to retain: Volume names and mountpoints alone do not establish data ownership or retention needs.

My notes: _____

SC-251 · Volumes · Back up a named volume to an archive

Situation: You need a simple archive of a disposable test volume before changing its contents.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Write a few test files into the volume.

```
docker run --rm -v backup-1:/data alpine:3.20 sh -c 'echo one > /data/one.txt; echo two > /data/two.txt'
```

Verify / observe: Use only a disposable lab volume.

Step 2 — Mount the volume and host backup directory into a helper container, then archive it.

```
docker run --rm -v backup-1:/src -v "$PWD":/backup alpine:3.20 tar -cf /backup/backup-1.tar -C /src .
```

Verify / observe: Check that the archive is created in the current host directory.

Step 3 — List the archive contents.

```
tar -tf backup-1.tar
```

Verify / observe: Confirm the expected test files are listed.

Final conclusion: What additional restore test would prove the backup is usable?

Concept to retain: A backup is not proven until you can restore it and verify the recovered data.

My notes: _____

SC-252 · Volumes · Back up a named volume to an archive

Situation: You need a simple archive of a disposable test volume before changing its contents.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Write a few test files into the volume.

```
docker run --rm -v backup-2:/data alpine:3.20 sh -c 'echo one > /data/one.txt; echo two > /data/two.txt'
```

Verify / observe: Use only a disposable lab volume.

Step 2 — Mount the volume and host backup directory into a helper container, then archive it.

```
docker run --rm -v backup-2:/src -v "$PWD":/backup alpine:3.20 tar -cf /backup/backup-2.tar -C /src .
```

Verify / observe: Check that the archive is created in the current host directory.

Step 3 — List the archive contents.

```
tar -tf backup-2.tar
```

Verify / observe: Confirm the expected test files are listed.

Final conclusion: What additional restore test would prove the backup is usable?

Concept to retain: A backup is not proven until you can restore it and verify the recovered data.

My notes: _____

SC-253 · Volumes · Back up a named volume to an archive

Situation: You need a simple archive of a disposable test volume before changing its contents.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Write a few test files into the volume.

```
docker run --rm -v backup-3:/data alpine:3.20 sh -c 'echo one > /data/one.txt; echo two > /data/two.txt'
```

Verify / observe: Use only a disposable lab volume.

Step 2 — Mount the volume and host backup directory into a helper container, then archive it.

```
docker run --rm -v backup-3:/src -v "$PWD":/backup alpine:3.20 tar -cf /backup/backup-3.tar -C /src .
```

Verify / observe: Check that the archive is created in the current host directory.

Step 3 — List the archive contents.

```
tar -tf backup-3.tar
```

Verify / observe: Confirm the expected test files are listed.

Final conclusion: What additional restore test would prove the backup is usable?

Concept to retain: A backup is not proven until you can restore it and verify the recovered data.

My notes: _____

SC-254 · Volumes · Back up a named volume to an archive

Situation: You need a simple archive of a disposable test volume before changing its contents.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Write a few test files into the volume.

```
docker run --rm -v backup-4:/data alpine:3.20 sh -c 'echo one > /data/one.txt; echo two > /data/two.txt'
```

Verify / observe: Use only a disposable lab volume.

Step 2 — Mount the volume and host backup directory into a helper container, then archive it.

```
docker run --rm -v backup-4:/src -v "$PWD":/backup alpine:3.20 tar -cf /backup/backup-4.tar -C /src .
```

Verify / observe: Check that the archive is created in the current host directory.

Step 3 — List the archive contents.

```
tar -tf backup-4.tar
```

Verify / observe: Confirm the expected test files are listed.

Final conclusion: What additional restore test would prove the backup is usable?

Concept to retain: A backup is not proven until you can restore it and verify the recovered data.

My notes: _____

SC-255 · Volumes · Back up a named volume to an archive

Situation: You need a simple archive of a disposable test volume before changing its contents.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Write a few test files into the volume.

```
docker run --rm -v backup-5:/data alpine:3.20 sh -c 'echo one > /data/one.txt; echo two > /data/two.txt'
```

Verify / observe: Use only a disposable lab volume.

Step 2 — Mount the volume and host backup directory into a helper container, then archive it.

```
docker run --rm -v backup-5:/src -v "$PWD":/backup alpine:3.20 tar -cf /backup/backup-5.tar -C /src .
```

Verify / observe: Check that the archive is created in the current host directory.

Step 3 — List the archive contents.

```
tar -tf backup-5.tar
```

Verify / observe: Confirm the expected test files are listed.

Final conclusion: What additional restore test would prove the backup is usable?

Concept to retain: A backup is not proven until you can restore it and verify the recovered data.

My notes: _____

SC-256 · Volumes · Back up a named volume to an archive

Situation: You need a simple archive of a disposable test volume before changing its contents.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Write a few test files into the volume.

```
docker run --rm -v backup-6:/data alpine:3.20 sh -c 'echo one > /data/one.txt; echo two > /data/two.txt'
```

Verify / observe: Use only a disposable lab volume.

Step 2 — Mount the volume and host backup directory into a helper container, then archive it.

```
docker run --rm -v backup-6:/src -v "$PWD":/backup alpine:3.20 tar -cf /backup/backup-6.tar -C /src .
```

Verify / observe: Check that the archive is created in the current host directory.

Step 3 — List the archive contents.

```
tar -tf backup-6.tar
```

Verify / observe: Confirm the expected test files are listed.

Final conclusion: What additional restore test would prove the backup is usable?

Concept to retain: A backup is not proven until you can restore it and verify the recovered data.

My notes: _____

SC-257 · Volumes · Back up a named volume to an archive

Situation: You need a simple archive of a disposable test volume before changing its contents.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Write a few test files into the volume.

```
docker run --rm -v backup-7:/data alpine:3.20 sh -c 'echo one > /data/one.txt; echo two > /data/two.txt'
```

Verify / observe: Use only a disposable lab volume.

Step 2 — Mount the volume and host backup directory into a helper container, then archive it.

```
docker run --rm -v backup-7:/src -v "$PWD":/backup alpine:3.20 tar -cf /backup/backup-7.tar -C /src .
```

Verify / observe: Check that the archive is created in the current host directory.

Step 3 — List the archive contents.

```
tar -tf backup-7.tar
```

Verify / observe: Confirm the expected test files are listed.

Final conclusion: What additional restore test would prove the backup is usable?

Concept to retain: A backup is not proven until you can restore it and verify the recovered data.

My notes: _____

SC-258 · Volumes · Back up a named volume to an archive

Situation: You need a simple archive of a disposable test volume before changing its contents.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Write a few test files into the volume.

```
docker run --rm -v backup-8:/data alpine:3.20 sh -c 'echo one > /data/one.txt; echo two > /data/two.txt'
```

Verify / observe: Use only a disposable lab volume.

Step 2 — Mount the volume and host backup directory into a helper container, then archive it.

```
docker run --rm -v backup-8:/src -v "$PWD":/backup alpine:3.20 tar -cf /backup/backup-8.tar -C /src .
```

Verify / observe: Check that the archive is created in the current host directory.

Step 3 — List the archive contents.

```
tar -tf backup-8.tar
```

Verify / observe: Confirm the expected test files are listed.

Final conclusion: What additional restore test would prove the backup is usable?

Concept to retain: A backup is not proven until you can restore it and verify the recovered data.

My notes: _____

SC-259 · Volumes · Back up a named volume to an archive

Situation: You need a simple archive of a disposable test volume before changing its contents.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Write a few test files into the volume.

```
docker run --rm -v backup-9:/data alpine:3.20 sh -c 'echo one > /data/one.txt; echo two > /data/two.txt'
```

Verify / observe: Use only a disposable lab volume.

Step 2 — Mount the volume and host backup directory into a helper container, then archive it.

```
docker run --rm -v backup-9:/src -v "$PWD":/backup alpine:3.20 tar -cf /backup/backup-9.tar -C /src .
```

Verify / observe: Check that the archive is created in the current host directory.

Step 3 — List the archive contents.

```
tar -tf backup-9.tar
```

Verify / observe: Confirm the expected test files are listed.

Final conclusion: What additional restore test would prove the backup is usable?

Concept to retain: A backup is not proven until you can restore it and verify the recovered data.

My notes: _____

SC-260 · Volumes · Back up a named volume to an archive

Situation: You need a simple archive of a disposable test volume before changing its contents.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Write a few test files into the volume.

```
docker run --rm -v backup-10:/data alpine:3.20 sh -c 'echo one > /data/one.txt; echo two > /data/two.txt'
```

Verify / observe: Use only a disposable lab volume.

Step 2 — Mount the volume and host backup directory into a helper container, then archive it.

```
docker run --rm -v backup-10:/src -v "$PWD":/backup alpine:3.20 tar -cf /backup/backup-10.tar -C /src .
```

Verify / observe: Check that the archive is created in the current host directory.

Step 3 — List the archive contents.

```
tar -tf backup-10.tar
```

Verify / observe: Confirm the expected test files are listed.

Final conclusion: What additional restore test would prove the backup is usable?

Concept to retain: A backup is not proven until you can restore it and verify the recovered data.

My notes: _____

SC-261 · Volumes · Restore a volume backup

Situation: You have a test archive and must restore it into a separate volume without overwriting the original.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a new restore target volume.

```
docker volume create restore-1
```

Verify / observe: Confirm the new volume exists.

Step 2 — Extract the archive into the restore volume.

```
docker run --rm -v restore-1:/dst -v "$PWD":/backup alpine:3.20 sh -c 'tar -xf /backup/backup-1.tar -C /dst'
```

Verify / observe: Ensure the archive is present in the mounted host directory.

Step 3 — Read a restored file.

```
docker run --rm -v restore-1:/data alpine:3.20 cat /data/one.txt
```

Verify / observe: Compare the restored content with the original.

Final conclusion: Did the restore recover the expected files? What metadata might this simple file archive not preserve?

Concept to retain: Restore testing catches missing files and incorrect archive paths before a real incident.

My notes: _____

SC-262 · Volumes · Restore a volume backup

Situation: You have a test archive and must restore it into a separate volume without overwriting the original.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a new restore target volume.

```
docker volume create restore-2
```

Verify / observe: Confirm the new volume exists.

Step 2 — Extract the archive into the restore volume.

```
docker run --rm -v restore-2:/dst -v "$PWD":/backup alpine:3.20 sh -c 'tar -xf /backup/backup-2.tar -C /dst'
```

Verify / observe: Ensure the archive is present in the mounted host directory.

Step 3 — Read a restored file.

```
docker run --rm -v restore-2:/data alpine:3.20 cat /data/one.txt
```

Verify / observe: Compare the restored content with the original.

Final conclusion: Did the restore recover the expected files? What metadata might this simple file archive not preserve?

Concept to retain: Restore testing catches missing files and incorrect archive paths before a real incident.

My notes: _____

SC-263 · Volumes · Restore a volume backup

Situation: You have a test archive and must restore it into a separate volume without overwriting the original.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a new restore target volume.

```
docker volume create restore-3
```

Verify / observe: Confirm the new volume exists.

Step 2 — Extract the archive into the restore volume.

```
docker run --rm -v restore-3:/dst -v "$PWD":/backup alpine:3.20 sh -c 'tar -xf /backup/backup-3.tar -C /dst'
```

Verify / observe: Ensure the archive is present in the mounted host directory.

Step 3 — Read a restored file.

```
docker run --rm -v restore-3:/data alpine:3.20 cat /data/one.txt
```

Verify / observe: Compare the restored content with the original.

Final conclusion: Did the restore recover the expected files? What metadata might this simple file archive not preserve?

Concept to retain: Restore testing catches missing files and incorrect archive paths before a real incident.

My notes: _____

SC-264 · Volumes · Restore a volume backup

Situation: You have a test archive and must restore it into a separate volume without overwriting the original.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a new restore target volume.

```
docker volume create restore-4
```

Verify / observe: Confirm the new volume exists.

Step 2 — Extract the archive into the restore volume.

```
docker run --rm -v restore-4:/dst -v "$PWD":/backup alpine:3.20 sh -c 'tar -xf /backup/backup-4.tar -C /dst'
```

Verify / observe: Ensure the archive is present in the mounted host directory.

Step 3 — Read a restored file.

```
docker run --rm -v restore-4:/data alpine:3.20 cat /data/one.txt
```

Verify / observe: Compare the restored content with the original.

Final conclusion: Did the restore recover the expected files? What metadata might this simple file archive not preserve?

Concept to retain: Restore testing catches missing files and incorrect archive paths before a real incident.

My notes: _____

SC-265 · Volumes · Restore a volume backup

Situation: You have a test archive and must restore it into a separate volume without overwriting the original.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a new restore target volume.

```
docker volume create restore-5
```

Verify / observe: Confirm the new volume exists.

Step 2 — Extract the archive into the restore volume.

```
docker run --rm -v restore-5:/dst -v "$PWD":/backup alpine:3.20 sh -c 'tar -xf /backup/backup-5.tar -C /dst'
```

Verify / observe: Ensure the archive is present in the mounted host directory.

Step 3 — Read a restored file.

```
docker run --rm -v restore-5:/data alpine:3.20 cat /data/one.txt
```

Verify / observe: Compare the restored content with the original.

Final conclusion: Did the restore recover the expected files? What metadata might this simple file archive not preserve?

Concept to retain: Restore testing catches missing files and incorrect archive paths before a real incident.

My notes: _____

SC-266 · Volumes · Restore a volume backup

Situation: You have a test archive and must restore it into a separate volume without overwriting the original.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a new restore target volume.

```
docker volume create restore-6
```

Verify / observe: Confirm the new volume exists.

Step 2 — Extract the archive into the restore volume.

```
docker run --rm -v restore-6:/dst -v "$PWD":/backup alpine:3.20 sh -c 'tar -xf /backup/backup-6.tar -C /dst'
```

Verify / observe: Ensure the archive is present in the mounted host directory.

Step 3 — Read a restored file.

```
docker run --rm -v restore-6:/data alpine:3.20 cat /data/one.txt
```

Verify / observe: Compare the restored content with the original.

Final conclusion: Did the restore recover the expected files? What metadata might this simple file archive not preserve?

Concept to retain: Restore testing catches missing files and incorrect archive paths before a real incident.

My notes: _____

SC-267 · Volumes · Restore a volume backup

Situation: You have a test archive and must restore it into a separate volume without overwriting the original.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a new restore target volume.

```
docker volume create restore-7
```

Verify / observe: Confirm the new volume exists.

Step 2 — Extract the archive into the restore volume.

```
docker run --rm -v restore-7:/dst -v "$PWD":/backup alpine:3.20 sh -c 'tar -xf /backup/backup-7.tar -C /dst'
```

Verify / observe: Ensure the archive is present in the mounted host directory.

Step 3 — Read a restored file.

```
docker run --rm -v restore-7:/data alpine:3.20 cat /data/one.txt
```

Verify / observe: Compare the restored content with the original.

Final conclusion: Did the restore recover the expected files? What metadata might this simple file archive not preserve?

Concept to retain: Restore testing catches missing files and incorrect archive paths before a real incident.

My notes: _____

SC-268 · Volumes · Restore a volume backup

Situation: You have a test archive and must restore it into a separate volume without overwriting the original.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a new restore target volume.

```
docker volume create restore-8
```

Verify / observe: Confirm the new volume exists.

Step 2 — Extract the archive into the restore volume.

```
docker run --rm -v restore-8:/dst -v "$PWD":/backup alpine:3.20 sh -c 'tar -xf /backup/backup-8.tar -C /dst'
```

Verify / observe: Ensure the archive is present in the mounted host directory.

Step 3 — Read a restored file.

```
docker run --rm -v restore-8:/data alpine:3.20 cat /data/one.txt
```

Verify / observe: Compare the restored content with the original.

Final conclusion: Did the restore recover the expected files? What metadata might this simple file archive not preserve?

Concept to retain: Restore testing catches missing files and incorrect archive paths before a real incident.

My notes: _____

SC-269 · Volumes · Restore a volume backup

Situation: You have a test archive and must restore it into a separate volume without overwriting the original.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a new restore target volume.

```
docker volume create restore-9
```

Verify / observe: Confirm the new volume exists.

Step 2 — Extract the archive into the restore volume.

```
docker run --rm -v restore-9:/dst -v "$PWD":/backup alpine:3.20 sh -c 'tar -xf /backup/backup-9.tar -C /dst'
```

Verify / observe: Ensure the archive is present in the mounted host directory.

Step 3 — Read a restored file.

```
docker run --rm -v restore-9:/data alpine:3.20 cat /data/one.txt
```

Verify / observe: Compare the restored content with the original.

Final conclusion: Did the restore recover the expected files? What metadata might this simple file archive not preserve?

Concept to retain: Restore testing catches missing files and incorrect archive paths before a real incident.

My notes: _____

SC-270 · Volumes · Restore a volume backup

Situation: You have a test archive and must restore it into a separate volume without overwriting the original.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a new restore target volume.

```
docker volume create restore-10
```

Verify / observe: Confirm the new volume exists.

Step 2 — Extract the archive into the restore volume.

```
docker run --rm -v restore-10:/dst -v "$PWD":/backup alpine:3.20 sh -c 'tar -xf /backup/backup-10.tar -C /dst'
```

Verify / observe: Ensure the archive is present in the mounted host directory.

Step 3 — Read a restored file.

```
docker run --rm -v restore-10:/data alpine:3.20 cat /data/one.txt
```

Verify / observe: Compare the restored content with the original.

Final conclusion: Did the restore recover the expected files? What metadata might this simple file archive not preserve?

Concept to retain: Restore testing catches missing files and incorrect archive paths before a real incident.

My notes: _____

SC-271 · Volumes · Compare bind mounts with named volumes

Situation: A developer wants to edit source files on the host while keeping database-like data persistent.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a host directory and a sample file.

```
mkdir -p ./src-lab-1 && echo hello > ./src-lab-1/index.txt
```

Verify / observe: Confirm the file exists on the host.

Step 2 — Mount the host directory as a bind mount.

```
docker run --rm -v "$PWD/src-lab-1":/src alpine:3.20 cat /src/index.txt
```

Verify / observe: The container should print hello.

Step 3 — Create and mount a named volume separately.

```
docker volume create named-1 && docker run --rm -v named-1:/data alpine:3.20 sh -c 'echo data > /data/value.txt'
```

Verify / observe: Confirm the named volume data can be read from a second container.

Final conclusion: Which storage type is best for editable source, and which is often better for container-managed persistent data?

Concept to retain: Bind mounts point to a host path; named volumes are managed by Docker.

My notes: _____

SC-272 · Volumes · Compare bind mounts with named volumes

Situation: A developer wants to edit source files on the host while keeping database-like data persistent.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a host directory and a sample file.

```
mkdir -p ./src-lab-2 && echo hello > ./src-lab-2/index.txt
```

Verify / observe: Confirm the file exists on the host.

Step 2 — Mount the host directory as a bind mount.

```
docker run --rm -v "$PWD/src-lab-2":/src alpine:3.20 cat /src/index.txt
```

Verify / observe: The container should print hello.

Step 3 — Create and mount a named volume separately.

```
docker volume create named-2 && docker run --rm -v named-2:/data alpine:3.20 sh -c 'echo data > /data/value.txt'
```

Verify / observe: Confirm the named volume data can be read from a second container.

Final conclusion: Which storage type is best for editable source, and which is often better for container-managed persistent data?

Concept to retain: Bind mounts point to a host path; named volumes are managed by Docker.

My notes: _____

SC-273 · Volumes · Compare bind mounts with named volumes

Situation: A developer wants to edit source files on the host while keeping database-like data persistent.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a host directory and a sample file.

```
mkdir -p ./src-lab-3 && echo hello > ./src-lab-3/index.txt
```

Verify / observe: Confirm the file exists on the host.

Step 2 — Mount the host directory as a bind mount.

```
docker run --rm -v "$PWD/src-lab-3":/src alpine:3.20 cat /src/index.txt
```

Verify / observe: The container should print hello.

Step 3 — Create and mount a named volume separately.

```
docker volume create named-3 && docker run --rm -v named-3:/data alpine:3.20 sh -c 'echo data > /data/value.txt'
```

Verify / observe: Confirm the named volume data can be read from a second container.

Final conclusion: Which storage type is best for editable source, and which is often better for container-managed persistent data?

Concept to retain: Bind mounts point to a host path; named volumes are managed by Docker.

My notes: _____

SC-274 · Volumes · Compare bind mounts with named volumes

Situation: A developer wants to edit source files on the host while keeping database-like data persistent.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a host directory and a sample file.

```
mkdir -p ./src-lab-4 && echo hello > ./src-lab-4/index.txt
```

Verify / observe: Confirm the file exists on the host.

Step 2 — Mount the host directory as a bind mount.

```
docker run --rm -v "$PWD/src-lab-4":/src alpine:3.20 cat /src/index.txt
```

Verify / observe: The container should print hello.

Step 3 — Create and mount a named volume separately.

```
docker volume create named-4 && docker run --rm -v named-4:/data alpine:3.20 sh -c 'echo data > /data/value.txt'
```

Verify / observe: Confirm the named volume data can be read from a second container.

Final conclusion: Which storage type is best for editable source, and which is often better for container-managed persistent data?

Concept to retain: Bind mounts point to a host path; named volumes are managed by Docker.

My notes: _____

SC-275 · Volumes · Compare bind mounts with named volumes

Situation: A developer wants to edit source files on the host while keeping database-like data persistent.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a host directory and a sample file.

```
mkdir -p ./src-lab-5 && echo hello > ./src-lab-5/index.txt
```

Verify / observe: Confirm the file exists on the host.

Step 2 — Mount the host directory as a bind mount.

```
docker run --rm -v "$PWD/src-lab-5":/src alpine:3.20 cat /src/index.txt
```

Verify / observe: The container should print hello.

Step 3 — Create and mount a named volume separately.

```
docker volume create named-5 && docker run --rm -v named-5:/data alpine:3.20 sh -c 'echo data > /data/value.txt'
```

Verify / observe: Confirm the named volume data can be read from a second container.

Final conclusion: Which storage type is best for editable source, and which is often better for container-managed persistent data?

Concept to retain: Bind mounts point to a host path; named volumes are managed by Docker.

My notes: _____

SC-276 · Volumes · Compare bind mounts with named volumes

Situation: A developer wants to edit source files on the host while keeping database-like data persistent.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a host directory and a sample file.

```
mkdir -p ./src-lab-6 && echo hello > ./src-lab-6/index.txt
```

Verify / observe: Confirm the file exists on the host.

Step 2 — Mount the host directory as a bind mount.

```
docker run --rm -v "$PWD/src-lab-6":/src alpine:3.20 cat /src/index.txt
```

Verify / observe: The container should print hello.

Step 3 — Create and mount a named volume separately.

```
docker volume create named-6 && docker run --rm -v named-6:/data alpine:3.20 sh -c 'echo data > /data/value.txt'
```

Verify / observe: Confirm the named volume data can be read from a second container.

Final conclusion: Which storage type is best for editable source, and which is often better for container-managed persistent data?

Concept to retain: Bind mounts point to a host path; named volumes are managed by Docker.

My notes: _____

SC-277 · Volumes · Compare bind mounts with named volumes

Situation: A developer wants to edit source files on the host while keeping database-like data persistent.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a host directory and a sample file.

```
mkdir -p ./src-lab-7 && echo hello > ./src-lab-7/index.txt
```

Verify / observe: Confirm the file exists on the host.

Step 2 — Mount the host directory as a bind mount.

```
docker run --rm -v "$PWD/src-lab-7":/src alpine:3.20 cat /src/index.txt
```

Verify / observe: The container should print hello.

Step 3 — Create and mount a named volume separately.

```
docker volume create named-7 && docker run --rm -v named-7:/data alpine:3.20 sh -c 'echo data > /data/value.txt'
```

Verify / observe: Confirm the named volume data can be read from a second container.

Final conclusion: Which storage type is best for editable source, and which is often better for container-managed persistent data?

Concept to retain: Bind mounts point to a host path; named volumes are managed by Docker.

My notes: _____

SC-278 · Volumes · Compare bind mounts with named volumes

Situation: A developer wants to edit source files on the host while keeping database-like data persistent.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a host directory and a sample file.

```
mkdir -p ./src-lab-8 && echo hello > ./src-lab-8/index.txt
```

Verify / observe: Confirm the file exists on the host.

Step 2 — Mount the host directory as a bind mount.

```
docker run --rm -v "$PWD/src-lab-8":/src alpine:3.20 cat /src/index.txt
```

Verify / observe: The container should print hello.

Step 3 — Create and mount a named volume separately.

```
docker volume create named-8 && docker run --rm -v named-8:/data alpine:3.20 sh -c 'echo data > /data/value.txt'
```

Verify / observe: Confirm the named volume data can be read from a second container.

Final conclusion: Which storage type is best for editable source, and which is often better for container-managed persistent data?

Concept to retain: Bind mounts point to a host path; named volumes are managed by Docker.

My notes: _____

SC-279 · Volumes · Compare bind mounts with named volumes

Situation: A developer wants to edit source files on the host while keeping database-like data persistent.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a host directory and a sample file.

```
mkdir -p ./src-lab-9 && echo hello > ./src-lab-9/index.txt
```

Verify / observe: Confirm the file exists on the host.

Step 2 — Mount the host directory as a bind mount.

```
docker run --rm -v "$PWD/src-lab-9":/src alpine:3.20 cat /src/index.txt
```

Verify / observe: The container should print hello.

Step 3 — Create and mount a named volume separately.

```
docker volume create named-9 && docker run --rm -v named-9:/data alpine:3.20 sh -c 'echo data > /data/value.txt'
```

Verify / observe: Confirm the named volume data can be read from a second container.

Final conclusion: Which storage type is best for editable source, and which is often better for container-managed persistent data?

Concept to retain: Bind mounts point to a host path; named volumes are managed by Docker.

My notes: _____

SC-280 · Volumes · Compare bind mounts with named volumes

Situation: A developer wants to edit source files on the host while keeping database-like data persistent.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a host directory and a sample file.

```
mkdir -p ./src-lab-10 && echo hello > ./src-lab-10/index.txt
```

Verify / observe: Confirm the file exists on the host.

Step 2 — Mount the host directory as a bind mount.

```
docker run --rm -v "$PWD/src-lab-10":/src alpine:3.20 cat /src/index.txt
```

Verify / observe: The container should print hello.

Step 3 — Create and mount a named volume separately.

```
docker volume create named-10 && docker run --rm -v named-10:/data alpine:3.20 sh -c 'echo data > /data/value.txt'
```

Verify / observe: Confirm the named volume data can be read from a second container.

Final conclusion: Which storage type is best for editable source, and which is often better for container-managed persistent data?

Concept to retain: Bind mounts point to a host path; named volumes are managed by Docker.

My notes: _____

SC-281 · Volumes · Clean up volume data safely

Situation: A lab is finished and you must remove only the volumes you created for it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect containers and identify any that use the test volume.

```
docker ps -a
```

Verify / observe: Do not proceed if a needed container still uses the volume.

Step 2 — Inspect the volume one final time.

```
docker volume inspect cleanup-1
```

Verify / observe: Confirm the exact name and that it is a disposable lab resource.

Step 3 — Remove the named test volume.

```
docker volume rm cleanup-1
```

Verify / observe: Verify the volume no longer appears in docker volume ls.

Final conclusion: Why is volume deletion more serious than deleting a stopped container?

Concept to retain: Volume removal can permanently delete persistent data.

My notes: _____

SC-282 · Volumes · Clean up volume data safely

Situation: A lab is finished and you must remove only the volumes you created for it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect containers and identify any that use the test volume.

```
docker ps -a
```

Verify / observe: Do not proceed if a needed container still uses the volume.

Step 2 — Inspect the volume one final time.

```
docker volume inspect cleanup-2
```

Verify / observe: Confirm the exact name and that it is a disposable lab resource.

Step 3 — Remove the named test volume.

```
docker volume rm cleanup-2
```

Verify / observe: Verify the volume no longer appears in docker volume ls.

Final conclusion: Why is volume deletion more serious than deleting a stopped container?

Concept to retain: Volume removal can permanently delete persistent data.

My notes: _____

SC-283 · Volumes · Clean up volume data safely

Situation: A lab is finished and you must remove only the volumes you created for it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect containers and identify any that use the test volume.

```
docker ps -a
```

Verify / observe: Do not proceed if a needed container still uses the volume.

Step 2 — Inspect the volume one final time.

```
docker volume inspect cleanup-3
```

Verify / observe: Confirm the exact name and that it is a disposable lab resource.

Step 3 — Remove the named test volume.

```
docker volume rm cleanup-3
```

Verify / observe: Verify the volume no longer appears in docker volume ls.

Final conclusion: Why is volume deletion more serious than deleting a stopped container?

Concept to retain: Volume removal can permanently delete persistent data.

My notes: _____

SC-284 · Volumes · Clean up volume data safely

Situation: A lab is finished and you must remove only the volumes you created for it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect containers and identify any that use the test volume.

```
docker ps -a
```

Verify / observe: Do not proceed if a needed container still uses the volume.

Step 2 — Inspect the volume one final time.

```
docker volume inspect cleanup-4
```

Verify / observe: Confirm the exact name and that it is a disposable lab resource.

Step 3 — Remove the named test volume.

```
docker volume rm cleanup-4
```

Verify / observe: Verify the volume no longer appears in docker volume ls.

Final conclusion: Why is volume deletion more serious than deleting a stopped container?

Concept to retain: Volume removal can permanently delete persistent data.

My notes: _____

SC-285 · Volumes · Clean up volume data safely

Situation: A lab is finished and you must remove only the volumes you created for it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect containers and identify any that use the test volume.

```
docker ps -a
```

Verify / observe: Do not proceed if a needed container still uses the volume.

Step 2 — Inspect the volume one final time.

```
docker volume inspect cleanup-5
```

Verify / observe: Confirm the exact name and that it is a disposable lab resource.

Step 3 — Remove the named test volume.

```
docker volume rm cleanup-5
```

Verify / observe: Verify the volume no longer appears in docker volume ls.

Final conclusion: Why is volume deletion more serious than deleting a stopped container?

Concept to retain: Volume removal can permanently delete persistent data.

My notes: _____

SC-286 · Volumes · Clean up volume data safely

Situation: A lab is finished and you must remove only the volumes you created for it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect containers and identify any that use the test volume.

```
docker ps -a
```

Verify / observe: Do not proceed if a needed container still uses the volume.

Step 2 — Inspect the volume one final time.

```
docker volume inspect cleanup-6
```

Verify / observe: Confirm the exact name and that it is a disposable lab resource.

Step 3 — Remove the named test volume.

```
docker volume rm cleanup-6
```

Verify / observe: Verify the volume no longer appears in docker volume ls.

Final conclusion: Why is volume deletion more serious than deleting a stopped container?

Concept to retain: Volume removal can permanently delete persistent data.

My notes: _____

SC-287 · Volumes · Clean up volume data safely

Situation: A lab is finished and you must remove only the volumes you created for it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect containers and identify any that use the test volume.

```
docker ps -a
```

Verify / observe: Do not proceed if a needed container still uses the volume.

Step 2 — Inspect the volume one final time.

```
docker volume inspect cleanup-7
```

Verify / observe: Confirm the exact name and that it is a disposable lab resource.

Step 3 — Remove the named test volume.

```
docker volume rm cleanup-7
```

Verify / observe: Verify the volume no longer appears in docker volume ls.

Final conclusion: Why is volume deletion more serious than deleting a stopped container?

Concept to retain: Volume removal can permanently delete persistent data.

My notes: _____

SC-288 · Volumes · Clean up volume data safely

Situation: A lab is finished and you must remove only the volumes you created for it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect containers and identify any that use the test volume.

```
docker ps -a
```

Verify / observe: Do not proceed if a needed container still uses the volume.

Step 2 — Inspect the volume one final time.

```
docker volume inspect cleanup-8
```

Verify / observe: Confirm the exact name and that it is a disposable lab resource.

Step 3 — Remove the named test volume.

```
docker volume rm cleanup-8
```

Verify / observe: Verify the volume no longer appears in docker volume ls.

Final conclusion: Why is volume deletion more serious than deleting a stopped container?

Concept to retain: Volume removal can permanently delete persistent data.

My notes: _____

SC-289 · Volumes · Clean up volume data safely

Situation: A lab is finished and you must remove only the volumes you created for it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect containers and identify any that use the test volume.

```
docker ps -a
```

Verify / observe: Do not proceed if a needed container still uses the volume.

Step 2 — Inspect the volume one final time.

```
docker volume inspect cleanup-9
```

Verify / observe: Confirm the exact name and that it is a disposable lab resource.

Step 3 — Remove the named test volume.

```
docker volume rm cleanup-9
```

Verify / observe: Verify the volume no longer appears in docker volume ls.

Final conclusion: Why is volume deletion more serious than deleting a stopped container?

Concept to retain: Volume removal can permanently delete persistent data.

My notes: _____

SC-290 · Volumes · Clean up volume data safely

Situation: A lab is finished and you must remove only the volumes you created for it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect containers and identify any that use the test volume.

```
docker ps -a
```

Verify / observe: Do not proceed if a needed container still uses the volume.

Step 2 — Inspect the volume one final time.

```
docker volume inspect cleanup-10
```

Verify / observe: Confirm the exact name and that it is a disposable lab resource.

Step 3 — Remove the named test volume.

```
docker volume rm cleanup-10
```

Verify / observe: Verify the volume no longer appears in docker volume ls.

Final conclusion: Why is volume deletion more serious than deleting a stopped container?

Concept to retain: Volume removal can permanently delete persistent data.

My notes: _____

SC-291 · Networks · Create a custom bridge network

Situation: Two containers in a lab need a private network separate from the default bridge.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a custom bridge network.

```
docker network create --driver bridge net-1
```

Verify / observe: Confirm Docker returns a network ID.

Step 2 — Inspect subnet and gateway details.

```
docker network inspect net-1
```

Verify / observe: Record the driver, subnet, gateway, and connected containers.

Step 3 — Start a test container attached to that network.

```
docker run -d --name netbox-1 --network net-1 alpine:3.20 sh -c 'sleep 300'
```

Verify / observe: Confirm the container is connected to the custom network.

Final conclusion: What does a user-defined bridge network provide compared with using no custom network?

Concept to retain: Custom networks provide explicit isolation and convenient container-name-based discovery.

My notes: _____

SC-292 · Networks · Create a custom bridge network

Situation: Two containers in a lab need a private network separate from the default bridge.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a custom bridge network.

```
docker network create --driver bridge net-2
```

Verify / observe: Confirm Docker returns a network ID.

Step 2 — Inspect subnet and gateway details.

```
docker network inspect net-2
```

Verify / observe: Record the driver, subnet, gateway, and connected containers.

Step 3 — Start a test container attached to that network.

```
docker run -d --name netbox-2 --network net-2 alpine:3.20 sh -c 'sleep 300'
```

Verify / observe: Confirm the container is connected to the custom network.

Final conclusion: What does a user-defined bridge network provide compared with using no custom network?

Concept to retain: Custom networks provide explicit isolation and convenient container-name-based discovery.

My notes: _____

SC-293 · Networks · Create a custom bridge network

Situation: Two containers in a lab need a private network separate from the default bridge.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a custom bridge network.

```
docker network create --driver bridge net-3
```

Verify / observe: Confirm Docker returns a network ID.

Step 2 — Inspect subnet and gateway details.

```
docker network inspect net-3
```

Verify / observe: Record the driver, subnet, gateway, and connected containers.

Step 3 — Start a test container attached to that network.

```
docker run -d --name netbox-3 --network net-3 alpine:3.20 sh -c 'sleep 300'
```

Verify / observe: Confirm the container is connected to the custom network.

Final conclusion: What does a user-defined bridge network provide compared with using no custom network?

Concept to retain: Custom networks provide explicit isolation and convenient container-name-based discovery.

My notes: _____

SC-294 · Networks · Create a custom bridge network

Situation: Two containers in a lab need a private network separate from the default bridge.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a custom bridge network.

```
docker network create --driver bridge net-4
```

Verify / observe: Confirm Docker returns a network ID.

Step 2 — Inspect subnet and gateway details.

```
docker network inspect net-4
```

Verify / observe: Record the driver, subnet, gateway, and connected containers.

Step 3 — Start a test container attached to that network.

```
docker run -d --name netbox-4 --network net-4 alpine:3.20 sh -c 'sleep 300'
```

Verify / observe: Confirm the container is connected to the custom network.

Final conclusion: What does a user-defined bridge network provide compared with using no custom network?

Concept to retain: Custom networks provide explicit isolation and convenient container-name-based discovery.

My notes: _____

SC-295 · Networks · Create a custom bridge network

Situation: Two containers in a lab need a private network separate from the default bridge.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a custom bridge network.

```
docker network create --driver bridge net-5
```

Verify / observe: Confirm Docker returns a network ID.

Step 2 — Inspect subnet and gateway details.

```
docker network inspect net-5
```

Verify / observe: Record the driver, subnet, gateway, and connected containers.

Step 3 — Start a test container attached to that network.

```
docker run -d --name netbox-5 --network net-5 alpine:3.20 sh -c 'sleep 300'
```

Verify / observe: Confirm the container is connected to the custom network.

Final conclusion: What does a user-defined bridge network provide compared with using no custom network?

Concept to retain: Custom networks provide explicit isolation and convenient container-name-based discovery.

My notes: _____

SC-296 · Networks · Create a custom bridge network

Situation: Two containers in a lab need a private network separate from the default bridge.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a custom bridge network.

```
docker network create --driver bridge net-6
```

Verify / observe: Confirm Docker returns a network ID.

Step 2 — Inspect subnet and gateway details.

```
docker network inspect net-6
```

Verify / observe: Record the driver, subnet, gateway, and connected containers.

Step 3 — Start a test container attached to that network.

```
docker run -d --name netbox-6 --network net-6 alpine:3.20 sh -c 'sleep 300'
```

Verify / observe: Confirm the container is connected to the custom network.

Final conclusion: What does a user-defined bridge network provide compared with using no custom network?

Concept to retain: Custom networks provide explicit isolation and convenient container-name-based discovery.

My notes: _____

SC-297 · Networks · Create a custom bridge network

Situation: Two containers in a lab need a private network separate from the default bridge.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a custom bridge network.

```
docker network create --driver bridge net-7
```

Verify / observe: Confirm Docker returns a network ID.

Step 2 — Inspect subnet and gateway details.

```
docker network inspect net-7
```

Verify / observe: Record the driver, subnet, gateway, and connected containers.

Step 3 — Start a test container attached to that network.

```
docker run -d --name netbox-7 --network net-7 alpine:3.20 sh -c 'sleep 300'
```

Verify / observe: Confirm the container is connected to the custom network.

Final conclusion: What does a user-defined bridge network provide compared with using no custom network?

Concept to retain: Custom networks provide explicit isolation and convenient container-name-based discovery.

My notes: _____

SC-298 · Networks · Create a custom bridge network

Situation: Two containers in a lab need a private network separate from the default bridge.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a custom bridge network.

```
docker network create --driver bridge net-8
```

Verify / observe: Confirm Docker returns a network ID.

Step 2 — Inspect subnet and gateway details.

```
docker network inspect net-8
```

Verify / observe: Record the driver, subnet, gateway, and connected containers.

Step 3 — Start a test container attached to that network.

```
docker run -d --name netbox-8 --network net-8 alpine:3.20 sh -c 'sleep 300'
```

Verify / observe: Confirm the container is connected to the custom network.

Final conclusion: What does a user-defined bridge network provide compared with using no custom network?

Concept to retain: Custom networks provide explicit isolation and convenient container-name-based discovery.

My notes: _____

SC-299 · Networks · Create a custom bridge network

Situation: Two containers in a lab need a private network separate from the default bridge.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a custom bridge network.

```
docker network create --driver bridge net-9
```

Verify / observe: Confirm Docker returns a network ID.

Step 2 — Inspect subnet and gateway details.

```
docker network inspect net-9
```

Verify / observe: Record the driver, subnet, gateway, and connected containers.

Step 3 — Start a test container attached to that network.

```
docker run -d --name netbox-9 --network net-9 alpine:3.20 sh -c 'sleep 300'
```

Verify / observe: Confirm the container is connected to the custom network.

Final conclusion: What does a user-defined bridge network provide compared with using no custom network?

Concept to retain: Custom networks provide explicit isolation and convenient container-name-based discovery.

My notes: _____

SC-300 · Networks · Create a custom bridge network

Situation: Two containers in a lab need a private network separate from the default bridge.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a custom bridge network.

```
docker network create --driver bridge net-10
```

Verify / observe: Confirm Docker returns a network ID.

Step 2 — Inspect subnet and gateway details.

```
docker network inspect net-10
```

Verify / observe: Record the driver, subnet, gateway, and connected containers.

Step 3 — Start a test container attached to that network.

```
docker run -d --name netbox-10 --network net-10 alpine:3.20 sh -c 'sleep 300'
```

Verify / observe: Confirm the container is connected to the custom network.

Final conclusion: What does a user-defined bridge network provide compared with using no custom network?

Concept to retain: Custom networks provide explicit isolation and convenient container-name-based discovery.

My notes: _____

SC-301 · Networks · Connect and disconnect a running container

Situation: A running lab container needs temporary access to a custom network.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a network if it does not exist.

```
docker network create attach-1
```

Verify / observe: If it already exists, inspect it rather than creating a duplicate.

Step 2 — Connect the test container.

```
docker network connect attach-1 app-1
```

Verify / observe: The target container must already exist.

Step 3 — Verify membership and disconnect when finished.

```
docker network inspect attach-1
```

Verify / observe: Confirm the container appears under Containers, then disconnect only the lab container.

Final conclusion: What application behavior could be interrupted by disconnecting a live container?

Concept to retain: Network membership can be changed independently, but live connectivity may be affected.

My notes: _____

SC-302 · Networks · Connect and disconnect a running container

Situation: A running lab container needs temporary access to a custom network.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a network if it does not exist.

```
docker network create attach-2
```

Verify / observe: If it already exists, inspect it rather than creating a duplicate.

Step 2 — Connect the test container.

```
docker network connect attach-2 app-2
```

Verify / observe: The target container must already exist.

Step 3 — Verify membership and disconnect when finished.

```
docker network inspect attach-2
```

Verify / observe: Confirm the container appears under Containers, then disconnect only the lab container.

Final conclusion: What application behavior could be interrupted by disconnecting a live container?

Concept to retain: Network membership can be changed independently, but live connectivity may be affected.

My notes: _____

SC-303 · Networks · Connect and disconnect a running container

Situation: A running lab container needs temporary access to a custom network.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a network if it does not exist.

```
docker network create attach-3
```

Verify / observe: If it already exists, inspect it rather than creating a duplicate.

Step 2 — Connect the test container.

```
docker network connect attach-3 app-3
```

Verify / observe: The target container must already exist.

Step 3 — Verify membership and disconnect when finished.

```
docker network inspect attach-3
```

Verify / observe: Confirm the container appears under Containers, then disconnect only the lab container.

Final conclusion: What application behavior could be interrupted by disconnecting a live container?

Concept to retain: Network membership can be changed independently, but live connectivity may be affected.

My notes: _____

SC-304 · Networks · Connect and disconnect a running container

Situation: A running lab container needs temporary access to a custom network.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a network if it does not exist.

```
docker network create attach-4
```

Verify / observe: If it already exists, inspect it rather than creating a duplicate.

Step 2 — Connect the test container.

```
docker network connect attach-4 app-4
```

Verify / observe: The target container must already exist.

Step 3 — Verify membership and disconnect when finished.

```
docker network inspect attach-4
```

Verify / observe: Confirm the container appears under Containers, then disconnect only the lab container.

Final conclusion: What application behavior could be interrupted by disconnecting a live container?

Concept to retain: Network membership can be changed independently, but live connectivity may be affected.

My notes: _____

SC-305 · Networks · Connect and disconnect a running container

Situation: A running lab container needs temporary access to a custom network.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a network if it does not exist.

```
docker network create attach-5
```

Verify / observe: If it already exists, inspect it rather than creating a duplicate.

Step 2 — Connect the test container.

```
docker network connect attach-5 app-5
```

Verify / observe: The target container must already exist.

Step 3 — Verify membership and disconnect when finished.

```
docker network inspect attach-5
```

Verify / observe: Confirm the container appears under Containers, then disconnect only the lab container.

Final conclusion: What application behavior could be interrupted by disconnecting a live container?

Concept to retain: Network membership can be changed independently, but live connectivity may be affected.

My notes: _____

SC-306 · Networks · Connect and disconnect a running container

Situation: A running lab container needs temporary access to a custom network.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a network if it does not exist.

```
docker network create attach-6
```

Verify / observe: If it already exists, inspect it rather than creating a duplicate.

Step 2 — Connect the test container.

```
docker network connect attach-6 app-6
```

Verify / observe: The target container must already exist.

Step 3 — Verify membership and disconnect when finished.

```
docker network inspect attach-6
```

Verify / observe: Confirm the container appears under Containers, then disconnect only the lab container.

Final conclusion: What application behavior could be interrupted by disconnecting a live container?

Concept to retain: Network membership can be changed independently, but live connectivity may be affected.

My notes: _____

SC-307 · Networks · Connect and disconnect a running container

Situation: A running lab container needs temporary access to a custom network.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a network if it does not exist.

```
docker network create attach-7
```

Verify / observe: If it already exists, inspect it rather than creating a duplicate.

Step 2 — Connect the test container.

```
docker network connect attach-7 app-7
```

Verify / observe: The target container must already exist.

Step 3 — Verify membership and disconnect when finished.

```
docker network inspect attach-7
```

Verify / observe: Confirm the container appears under Containers, then disconnect only the lab container.

Final conclusion: What application behavior could be interrupted by disconnecting a live container?

Concept to retain: Network membership can be changed independently, but live connectivity may be affected.

My notes: _____

SC-308 · Networks · Connect and disconnect a running container

Situation: A running lab container needs temporary access to a custom network.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a network if it does not exist.

```
docker network create attach-8
```

Verify / observe: If it already exists, inspect it rather than creating a duplicate.

Step 2 — Connect the test container.

```
docker network connect attach-8 app-8
```

Verify / observe: The target container must already exist.

Step 3 — Verify membership and disconnect when finished.

```
docker network inspect attach-8
```

Verify / observe: Confirm the container appears under Containers, then disconnect only the lab container.

Final conclusion: What application behavior could be interrupted by disconnecting a live container?

Concept to retain: Network membership can be changed independently, but live connectivity may be affected.

My notes: _____

SC-309 · Networks · Connect and disconnect a running container

Situation: A running lab container needs temporary access to a custom network.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a network if it does not exist.

```
docker network create attach-9
```

Verify / observe: If it already exists, inspect it rather than creating a duplicate.

Step 2 — Connect the test container.

```
docker network connect attach-9 app-9
```

Verify / observe: The target container must already exist.

Step 3 — Verify membership and disconnect when finished.

```
docker network inspect attach-9
```

Verify / observe: Confirm the container appears under Containers, then disconnect only the lab container.

Final conclusion: What application behavior could be interrupted by disconnecting a live container?

Concept to retain: Network membership can be changed independently, but live connectivity may be affected.

My notes: _____

SC-310 · Networks · Connect and disconnect a running container

Situation: A running lab container needs temporary access to a custom network.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Create a network if it does not exist.

```
docker network create attach-10
```

Verify / observe: If it already exists, inspect it rather than creating a duplicate.

Step 2 — Connect the test container.

```
docker network connect attach-10 app-10
```

Verify / observe: The target container must already exist.

Step 3 — Verify membership and disconnect when finished.

```
docker network inspect attach-10
```

Verify / observe: Confirm the container appears under Containers, then disconnect only the lab container.

Final conclusion: What application behavior could be interrupted by disconnecting a live container?

Concept to retain: Network membership can be changed independently, but live connectivity may be affected.

My notes: _____

SC-311 · Networks · Diagnose a published-port problem

Situation: A web container runs, but a developer cannot reach it from the host.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check container status and published ports.

```
docker ps --filter name=web-1
```

Verify / observe: Confirm the container is running and the expected port mapping exists.

Step 2 — Inspect the container's port configuration.

```
docker port web-1
```

Verify / observe: Confirm the host port and container port mapping.

Step 3 — Review recent container logs.

```
docker logs --tail 50 web-1
```

Verify / observe: Look for application startup errors; also consider host firewall and port conflicts.

Final conclusion: List at least three possible causes of a connection failure before changing the deployment.

Concept to retain: A running container alone does not prove the application is reachable.

My notes: _____

SC-312 · Networks · Diagnose a published-port problem

Situation: A web container runs, but a developer cannot reach it from the host.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check container status and published ports.

```
docker ps --filter name=web-2
```

Verify / observe: Confirm the container is running and the expected port mapping exists.

Step 2 — Inspect the container's port configuration.

```
docker port web-2
```

Verify / observe: Confirm the host port and container port mapping.

Step 3 — Review recent container logs.

```
docker logs --tail 50 web-2
```

Verify / observe: Look for application startup errors; also consider host firewall and port conflicts.

Final conclusion: List at least three possible causes of a connection failure before changing the deployment.

Concept to retain: A running container alone does not prove the application is reachable.

My notes: _____

SC-313 · Networks · Diagnose a published-port problem

Situation: A web container runs, but a developer cannot reach it from the host.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check container status and published ports.

```
docker ps --filter name=web-3
```

Verify / observe: Confirm the container is running and the expected port mapping exists.

Step 2 — Inspect the container's port configuration.

```
docker port web-3
```

Verify / observe: Confirm the host port and container port mapping.

Step 3 — Review recent container logs.

```
docker logs --tail 50 web-3
```

Verify / observe: Look for application startup errors; also consider host firewall and port conflicts.

Final conclusion: List at least three possible causes of a connection failure before changing the deployment.

Concept to retain: A running container alone does not prove the application is reachable.

My notes: _____

SC-314 · Networks · Diagnose a published-port problem

Situation: A web container runs, but a developer cannot reach it from the host.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check container status and published ports.

```
docker ps --filter name=web-4
```

Verify / observe: Confirm the container is running and the expected port mapping exists.

Step 2 — Inspect the container's port configuration.

```
docker port web-4
```

Verify / observe: Confirm the host port and container port mapping.

Step 3 — Review recent container logs.

```
docker logs --tail 50 web-4
```

Verify / observe: Look for application startup errors; also consider host firewall and port conflicts.

Final conclusion: List at least three possible causes of a connection failure before changing the deployment.

Concept to retain: A running container alone does not prove the application is reachable.

My notes: _____

SC-315 · Networks · Diagnose a published-port problem

Situation: A web container runs, but a developer cannot reach it from the host.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check container status and published ports.

```
docker ps --filter name=web-5
```

Verify / observe: Confirm the container is running and the expected port mapping exists.

Step 2 — Inspect the container's port configuration.

```
docker port web-5
```

Verify / observe: Confirm the host port and container port mapping.

Step 3 — Review recent container logs.

```
docker logs --tail 50 web-5
```

Verify / observe: Look for application startup errors; also consider host firewall and port conflicts.

Final conclusion: List at least three possible causes of a connection failure before changing the deployment.

Concept to retain: A running container alone does not prove the application is reachable.

My notes: _____

SC-316 · Networks · Diagnose a published-port problem

Situation: A web container runs, but a developer cannot reach it from the host.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check container status and published ports.

```
docker ps --filter name=web-6
```

Verify / observe: Confirm the container is running and the expected port mapping exists.

Step 2 — Inspect the container's port configuration.

```
docker port web-6
```

Verify / observe: Confirm the host port and container port mapping.

Step 3 — Review recent container logs.

```
docker logs --tail 50 web-6
```

Verify / observe: Look for application startup errors; also consider host firewall and port conflicts.

Final conclusion: List at least three possible causes of a connection failure before changing the deployment.

Concept to retain: A running container alone does not prove the application is reachable.

My notes: _____

SC-317 · Networks · Diagnose a published-port problem

Situation: A web container runs, but a developer cannot reach it from the host.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check container status and published ports.

```
docker ps --filter name=web-7
```

Verify / observe: Confirm the container is running and the expected port mapping exists.

Step 2 — Inspect the container's port configuration.

```
docker port web-7
```

Verify / observe: Confirm the host port and container port mapping.

Step 3 — Review recent container logs.

```
docker logs --tail 50 web-7
```

Verify / observe: Look for application startup errors; also consider host firewall and port conflicts.

Final conclusion: List at least three possible causes of a connection failure before changing the deployment.

Concept to retain: A running container alone does not prove the application is reachable.

My notes: _____

SC-318 · Networks · Diagnose a published-port problem

Situation: A web container runs, but a developer cannot reach it from the host.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check container status and published ports.

```
docker ps --filter name=web-8
```

Verify / observe: Confirm the container is running and the expected port mapping exists.

Step 2 — Inspect the container's port configuration.

```
docker port web-8
```

Verify / observe: Confirm the host port and container port mapping.

Step 3 — Review recent container logs.

```
docker logs --tail 50 web-8
```

Verify / observe: Look for application startup errors; also consider host firewall and port conflicts.

Final conclusion: List at least three possible causes of a connection failure before changing the deployment.

Concept to retain: A running container alone does not prove the application is reachable.

My notes: _____

SC-319 · Networks · Diagnose a published-port problem

Situation: A web container runs, but a developer cannot reach it from the host.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check container status and published ports.

```
docker ps --filter name=web-9
```

Verify / observe: Confirm the container is running and the expected port mapping exists.

Step 2 — Inspect the container's port configuration.

```
docker port web-9
```

Verify / observe: Confirm the host port and container port mapping.

Step 3 — Review recent container logs.

```
docker logs --tail 50 web-9
```

Verify / observe: Look for application startup errors; also consider host firewall and port conflicts.

Final conclusion: List at least three possible causes of a connection failure before changing the deployment.

Concept to retain: A running container alone does not prove the application is reachable.

My notes: _____

SC-320 · Networks · Diagnose a published-port problem

Situation: A web container runs, but a developer cannot reach it from the host.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check container status and published ports.

```
docker ps --filter name=web-10
```

Verify / observe: Confirm the container is running and the expected port mapping exists.

Step 2 — Inspect the container's port configuration.

```
docker port web-10
```

Verify / observe: Confirm the host port and container port mapping.

Step 3 — Review recent container logs.

```
docker logs --tail 50 web-10
```

Verify / observe: Look for application startup errors; also consider host firewall and port conflicts.

Final conclusion: List at least three possible causes of a connection failure before changing the deployment.

Concept to retain: A running container alone does not prove the application is reachable.

My notes: _____

SC-321 · Networks · Compare network isolation

Situation: You need to explain which containers can communicate and which ports are reachable from the host.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List networks.

```
docker network ls
```

Verify / observe: Identify built-in and custom networks.

Step 2 — Inspect a custom network.

```
docker network inspect net-1
```

Verify / observe: Review attached containers and network settings.

Step 3 — Inspect a web container's port mapping.

```
docker port web-1
```

Verify / observe: Separate container-to-container connectivity from host-published access.

Final conclusion: Draw a small network diagram showing host, published port, container, and custom network.

Concept to retain: Network membership and host port publishing solve different connectivity problems.

My notes: _____

SC-322 · Networks · Compare network isolation

Situation: You need to explain which containers can communicate and which ports are reachable from the host.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List networks.

```
docker network ls
```

Verify / observe: Identify built-in and custom networks.

Step 2 — Inspect a custom network.

```
docker network inspect net-2
```

Verify / observe: Review attached containers and network settings.

Step 3 — Inspect a web container's port mapping.

```
docker port web-2
```

Verify / observe: Separate container-to-container connectivity from host-published access.

Final conclusion: Draw a small network diagram showing host, published port, container, and custom network.

Concept to retain: Network membership and host port publishing solve different connectivity problems.

My notes: _____

SC-323 · Networks · Compare network isolation

Situation: You need to explain which containers can communicate and which ports are reachable from the host.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List networks.

```
docker network ls
```

Verify / observe: Identify built-in and custom networks.

Step 2 — Inspect a custom network.

```
docker network inspect net-3
```

Verify / observe: Review attached containers and network settings.

Step 3 — Inspect a web container's port mapping.

```
docker port web-3
```

Verify / observe: Separate container-to-container connectivity from host-published access.

Final conclusion: Draw a small network diagram showing host, published port, container, and custom network.

Concept to retain: Network membership and host port publishing solve different connectivity problems.

My notes: _____

SC-324 · Networks · Compare network isolation

Situation: You need to explain which containers can communicate and which ports are reachable from the host.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List networks.

```
docker network ls
```

Verify / observe: Identify built-in and custom networks.

Step 2 — Inspect a custom network.

```
docker network inspect net-4
```

Verify / observe: Review attached containers and network settings.

Step 3 — Inspect a web container's port mapping.

```
docker port web-4
```

Verify / observe: Separate container-to-container connectivity from host-published access.

Final conclusion: Draw a small network diagram showing host, published port, container, and custom network.

Concept to retain: Network membership and host port publishing solve different connectivity problems.

My notes: _____

SC-325 · Networks · Compare network isolation

Situation: You need to explain which containers can communicate and which ports are reachable from the host.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List networks.

```
docker network ls
```

Verify / observe: Identify built-in and custom networks.

Step 2 — Inspect a custom network.

```
docker network inspect net-5
```

Verify / observe: Review attached containers and network settings.

Step 3 — Inspect a web container's port mapping.

```
docker port web-5
```

Verify / observe: Separate container-to-container connectivity from host-published access.

Final conclusion: Draw a small network diagram showing host, published port, container, and custom network.

Concept to retain: Network membership and host port publishing solve different connectivity problems.

My notes: _____

SC-326 · Networks · Compare network isolation

Situation: You need to explain which containers can communicate and which ports are reachable from the host.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List networks.

```
docker network ls
```

Verify / observe: Identify built-in and custom networks.

Step 2 — Inspect a custom network.

```
docker network inspect net-6
```

Verify / observe: Review attached containers and network settings.

Step 3 — Inspect a web container's port mapping.

```
docker port web-6
```

Verify / observe: Separate container-to-container connectivity from host-published access.

Final conclusion: Draw a small network diagram showing host, published port, container, and custom network.

Concept to retain: Network membership and host port publishing solve different connectivity problems.

My notes: _____

SC-327 · Networks · Compare network isolation

Situation: You need to explain which containers can communicate and which ports are reachable from the host.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List networks.

```
docker network ls
```

Verify / observe: Identify built-in and custom networks.

Step 2 — Inspect a custom network.

```
docker network inspect net-7
```

Verify / observe: Review attached containers and network settings.

Step 3 — Inspect a web container's port mapping.

```
docker port web-7
```

Verify / observe: Separate container-to-container connectivity from host-published access.

Final conclusion: Draw a small network diagram showing host, published port, container, and custom network.

Concept to retain: Network membership and host port publishing solve different connectivity problems.

My notes: _____

SC-328 · Networks · Compare network isolation

Situation: You need to explain which containers can communicate and which ports are reachable from the host.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List networks.

```
docker network ls
```

Verify / observe: Identify built-in and custom networks.

Step 2 — Inspect a custom network.

```
docker network inspect net-8
```

Verify / observe: Review attached containers and network settings.

Step 3 — Inspect a web container's port mapping.

```
docker port web-8
```

Verify / observe: Separate container-to-container connectivity from host-published access.

Final conclusion: Draw a small network diagram showing host, published port, container, and custom network.

Concept to retain: Network membership and host port publishing solve different connectivity problems.

My notes: _____

SC-329 · Networks · Compare network isolation

Situation: You need to explain which containers can communicate and which ports are reachable from the host.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List networks.

```
docker network ls
```

Verify / observe: Identify built-in and custom networks.

Step 2 — Inspect a custom network.

```
docker network inspect net-9
```

Verify / observe: Review attached containers and network settings.

Step 3 — Inspect a web container's port mapping.

```
docker port web-9
```

Verify / observe: Separate container-to-container connectivity from host-published access.

Final conclusion: Draw a small network diagram showing host, published port, container, and custom network.

Concept to retain: Network membership and host port publishing solve different connectivity problems.

My notes: _____

SC-330 · Networks · Compare network isolation

Situation: You need to explain which containers can communicate and which ports are reachable from the host.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List networks.

```
docker network ls
```

Verify / observe: Identify built-in and custom networks.

Step 2 — Inspect a custom network.

```
docker network inspect net-10
```

Verify / observe: Review attached containers and network settings.

Step 3 — Inspect a web container's port mapping.

```
docker port web-10
```

Verify / observe: Separate container-to-container connectivity from host-published access.

Final conclusion: Draw a small network diagram showing host, published port, container, and custom network.

Concept to retain: Network membership and host port publishing solve different connectivity problems.

My notes: _____

SC-331 · Networks · Remove an unused custom network

Situation: A test network is no longer needed, but you must avoid disconnecting other workloads.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the network and its attached containers.

```
docker network inspect net-1
```

Verify / observe: Check the Containers section.

Step 2 — List all containers for cross-checking.

```
docker ps -a
```

Verify / observe: Confirm the network belongs only to this lab.

Step 3 — Remove the network after disconnecting or removing its lab containers.

```
docker network rm net-1
```

Verify / observe: Verify the network is absent from docker network ls.

Final conclusion: What would you do if the network still had containers attached?

Concept to retain: Networks should be removed only after their dependencies are understood.

My notes: _____

SC-332 · Networks · Remove an unused custom network

Situation: A test network is no longer needed, but you must avoid disconnecting other workloads.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the network and its attached containers.

```
docker network inspect net-2
```

Verify / observe: Check the Containers section.

Step 2 — List all containers for cross-checking.

```
docker ps -a
```

Verify / observe: Confirm the network belongs only to this lab.

Step 3 — Remove the network after disconnecting or removing its lab containers.

```
docker network rm net-2
```

Verify / observe: Verify the network is absent from docker network ls.

Final conclusion: What would you do if the network still had containers attached?

Concept to retain: Networks should be removed only after their dependencies are understood.

My notes: _____

SC-333 · Networks · Remove an unused custom network

Situation: A test network is no longer needed, but you must avoid disconnecting other workloads.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the network and its attached containers.

```
docker network inspect net-3
```

Verify / observe: Check the Containers section.

Step 2 — List all containers for cross-checking.

```
docker ps -a
```

Verify / observe: Confirm the network belongs only to this lab.

Step 3 — Remove the network after disconnecting or removing its lab containers.

```
docker network rm net-3
```

Verify / observe: Verify the network is absent from docker network ls.

Final conclusion: What would you do if the network still had containers attached?

Concept to retain: Networks should be removed only after their dependencies are understood.

My notes: _____

SC-334 · Networks · Remove an unused custom network

Situation: A test network is no longer needed, but you must avoid disconnecting other workloads.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the network and its attached containers.

```
docker network inspect net-4
```

Verify / observe: Check the Containers section.

Step 2 — List all containers for cross-checking.

```
docker ps -a
```

Verify / observe: Confirm the network belongs only to this lab.

Step 3 — Remove the network after disconnecting or removing its lab containers.

```
docker network rm net-4
```

Verify / observe: Verify the network is absent from docker network ls.

Final conclusion: What would you do if the network still had containers attached?

Concept to retain: Networks should be removed only after their dependencies are understood.

My notes: _____

SC-335 · Networks · Remove an unused custom network

Situation: A test network is no longer needed, but you must avoid disconnecting other workloads.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the network and its attached containers.

```
docker network inspect net-5
```

Verify / observe: Check the Containers section.

Step 2 — List all containers for cross-checking.

```
docker ps -a
```

Verify / observe: Confirm the network belongs only to this lab.

Step 3 — Remove the network after disconnecting or removing its lab containers.

```
docker network rm net-5
```

Verify / observe: Verify the network is absent from docker network ls.

Final conclusion: What would you do if the network still had containers attached?

Concept to retain: Networks should be removed only after their dependencies are understood.

My notes: _____

SC-336 · Networks · Remove an unused custom network

Situation: A test network is no longer needed, but you must avoid disconnecting other workloads.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the network and its attached containers.

```
docker network inspect net-6
```

Verify / observe: Check the Containers section.

Step 2 — List all containers for cross-checking.

```
docker ps -a
```

Verify / observe: Confirm the network belongs only to this lab.

Step 3 — Remove the network after disconnecting or removing its lab containers.

```
docker network rm net-6
```

Verify / observe: Verify the network is absent from docker network ls.

Final conclusion: What would you do if the network still had containers attached?

Concept to retain: Networks should be removed only after their dependencies are understood.

My notes: _____

SC-337 · Networks · Remove an unused custom network

Situation: A test network is no longer needed, but you must avoid disconnecting other workloads.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the network and its attached containers.

```
docker network inspect net-7
```

Verify / observe: Check the Containers section.

Step 2 — List all containers for cross-checking.

```
docker ps -a
```

Verify / observe: Confirm the network belongs only to this lab.

Step 3 — Remove the network after disconnecting or removing its lab containers.

```
docker network rm net-7
```

Verify / observe: Verify the network is absent from docker network ls.

Final conclusion: What would you do if the network still had containers attached?

Concept to retain: Networks should be removed only after their dependencies are understood.

My notes: _____

SC-338 · Networks · Remove an unused custom network

Situation: A test network is no longer needed, but you must avoid disconnecting other workloads.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the network and its attached containers.

```
docker network inspect net-8
```

Verify / observe: Check the Containers section.

Step 2 — List all containers for cross-checking.

```
docker ps -a
```

Verify / observe: Confirm the network belongs only to this lab.

Step 3 — Remove the network after disconnecting or removing its lab containers.

```
docker network rm net-8
```

Verify / observe: Verify the network is absent from docker network ls.

Final conclusion: What would you do if the network still had containers attached?

Concept to retain: Networks should be removed only after their dependencies are understood.

My notes: _____

SC-339 · Networks · Remove an unused custom network

Situation: A test network is no longer needed, but you must avoid disconnecting other workloads.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the network and its attached containers.

```
docker network inspect net-9
```

Verify / observe: Check the Containers section.

Step 2 — List all containers for cross-checking.

```
docker ps -a
```

Verify / observe: Confirm the network belongs only to this lab.

Step 3 — Remove the network after disconnecting or removing its lab containers.

```
docker network rm net-9
```

Verify / observe: Verify the network is absent from docker network ls.

Final conclusion: What would you do if the network still had containers attached?

Concept to retain: Networks should be removed only after their dependencies are understood.

My notes: _____

SC-340 · Networks · Remove an unused custom network

Situation: A test network is no longer needed, but you must avoid disconnecting other workloads.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the network and its attached containers.

```
docker network inspect net-10
```

Verify / observe: Check the Containers section.

Step 2 — List all containers for cross-checking.

```
docker ps -a
```

Verify / observe: Confirm the network belongs only to this lab.

Step 3 — Remove the network after disconnecting or removing its lab containers.

```
docker network rm net-10
```

Verify / observe: Verify the network is absent from docker network ls.

Final conclusion: What would you do if the network still had containers attached?

Concept to retain: Networks should be removed only after their dependencies are understood.

My notes: _____

SC-341 · Builds · Build an image from a Dockerfile

Situation: A small app has a Dockerfile and must be turned into a locally tagged image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review the Dockerfile and build context.

```
ls -la
```

Verify / observe: Ensure the Dockerfile and required source files are in the current directory.

Step 2 — Build the image with a unique tag.

```
docker build -t local/app-1:v1 .
```

Verify / observe: The final dot is the build context.

Step 3 — Inspect the result.

```
docker image inspect local/app-1:v1
```

Verify / observe: Confirm the tag exists and review the image metadata.

Final conclusion: What files are available to COPY during the build, and why does the build context matter?

Concept to retain: Build context controls which files are sent to the builder.

My notes: _____

SC-342 · Builds · Build an image from a Dockerfile

Situation: A small app has a Dockerfile and must be turned into a locally tagged image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review the Dockerfile and build context.

```
ls -la
```

Verify / observe: Ensure the Dockerfile and required source files are in the current directory.

Step 2 — Build the image with a unique tag.

```
docker build -t local/app-2:v1 .
```

Verify / observe: The final dot is the build context.

Step 3 — Inspect the result.

```
docker image inspect local/app-2:v1
```

Verify / observe: Confirm the tag exists and review the image metadata.

Final conclusion: What files are available to COPY during the build, and why does the build context matter?

Concept to retain: Build context controls which files are sent to the builder.

My notes: _____

SC-343 · Builds · Build an image from a Dockerfile

Situation: A small app has a Dockerfile and must be turned into a locally tagged image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review the Dockerfile and build context.

```
ls -la
```

Verify / observe: Ensure the Dockerfile and required source files are in the current directory.

Step 2 — Build the image with a unique tag.

```
docker build -t local/app-3:v1 .
```

Verify / observe: The final dot is the build context.

Step 3 — Inspect the result.

```
docker image inspect local/app-3:v1
```

Verify / observe: Confirm the tag exists and review the image metadata.

Final conclusion: What files are available to COPY during the build, and why does the build context matter?

Concept to retain: Build context controls which files are sent to the builder.

My notes: _____

SC-344 · Builds · Build an image from a Dockerfile

Situation: A small app has a Dockerfile and must be turned into a locally tagged image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review the Dockerfile and build context.

```
ls -la
```

Verify / observe: Ensure the Dockerfile and required source files are in the current directory.

Step 2 — Build the image with a unique tag.

```
docker build -t local/app-4:v1 .
```

Verify / observe: The final dot is the build context.

Step 3 — Inspect the result.

```
docker image inspect local/app-4:v1
```

Verify / observe: Confirm the tag exists and review the image metadata.

Final conclusion: What files are available to COPY during the build, and why does the build context matter?

Concept to retain: Build context controls which files are sent to the builder.

My notes: _____

SC-345 · Builds · Build an image from a Dockerfile

Situation: A small app has a Dockerfile and must be turned into a locally tagged image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review the Dockerfile and build context.

```
ls -la
```

Verify / observe: Ensure the Dockerfile and required source files are in the current directory.

Step 2 — Build the image with a unique tag.

```
docker build -t local/app-5:v1 .
```

Verify / observe: The final dot is the build context.

Step 3 — Inspect the result.

```
docker image inspect local/app-5:v1
```

Verify / observe: Confirm the tag exists and review the image metadata.

Final conclusion: What files are available to COPY during the build, and why does the build context matter?

Concept to retain: Build context controls which files are sent to the builder.

My notes: _____

SC-346 · Builds · Build an image from a Dockerfile

Situation: A small app has a Dockerfile and must be turned into a locally tagged image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review the Dockerfile and build context.

```
ls -la
```

Verify / observe: Ensure the Dockerfile and required source files are in the current directory.

Step 2 — Build the image with a unique tag.

```
docker build -t local/app-6:v1 .
```

Verify / observe: The final dot is the build context.

Step 3 — Inspect the result.

```
docker image inspect local/app-6:v1
```

Verify / observe: Confirm the tag exists and review the image metadata.

Final conclusion: What files are available to COPY during the build, and why does the build context matter?

Concept to retain: Build context controls which files are sent to the builder.

My notes: _____

SC-347 · Builds · Build an image from a Dockerfile

Situation: A small app has a Dockerfile and must be turned into a locally tagged image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review the Dockerfile and build context.

```
ls -la
```

Verify / observe: Ensure the Dockerfile and required source files are in the current directory.

Step 2 — Build the image with a unique tag.

```
docker build -t local/app-7:v1 .
```

Verify / observe: The final dot is the build context.

Step 3 — Inspect the result.

```
docker image inspect local/app-7:v1
```

Verify / observe: Confirm the tag exists and review the image metadata.

Final conclusion: What files are available to COPY during the build, and why does the build context matter?

Concept to retain: Build context controls which files are sent to the builder.

My notes: _____

SC-348 · Builds · Build an image from a Dockerfile

Situation: A small app has a Dockerfile and must be turned into a locally tagged image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review the Dockerfile and build context.

```
ls -la
```

Verify / observe: Ensure the Dockerfile and required source files are in the current directory.

Step 2 — Build the image with a unique tag.

```
docker build -t local/app-8:v1 .
```

Verify / observe: The final dot is the build context.

Step 3 — Inspect the result.

```
docker image inspect local/app-8:v1
```

Verify / observe: Confirm the tag exists and review the image metadata.

Final conclusion: What files are available to COPY during the build, and why does the build context matter?

Concept to retain: Build context controls which files are sent to the builder.

My notes: _____

SC-349 · Builds · Build an image from a Dockerfile

Situation: A small app has a Dockerfile and must be turned into a locally tagged image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review the Dockerfile and build context.

```
ls -la
```

Verify / observe: Ensure the Dockerfile and required source files are in the current directory.

Step 2 — Build the image with a unique tag.

```
docker build -t local/app-9:v1 .
```

Verify / observe: The final dot is the build context.

Step 3 — Inspect the result.

```
docker image inspect local/app-9:v1
```

Verify / observe: Confirm the tag exists and review the image metadata.

Final conclusion: What files are available to COPY during the build, and why does the build context matter?

Concept to retain: Build context controls which files are sent to the builder.

My notes: _____

SC-350 · Builds · Build an image from a Dockerfile

Situation: A small app has a Dockerfile and must be turned into a locally tagged image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review the Dockerfile and build context.

```
ls -la
```

Verify / observe: Ensure the Dockerfile and required source files are in the current directory.

Step 2 — Build the image with a unique tag.

```
docker build -t local/app-10:v1 .
```

Verify / observe: The final dot is the build context.

Step 3 — Inspect the result.

```
docker image inspect local/app-10:v1
```

Verify / observe: Confirm the tag exists and review the image metadata.

Final conclusion: What files are available to COPY during the build, and why does the build context matter?

Concept to retain: Build context controls which files are sent to the builder.

My notes: _____

SC-351 · Builds · Diagnose a failed Docker build

Situation: A build fails during a COPY instruction. You must isolate the cause before rewriting the Dockerfile.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List files in the build directory.

```
find . -maxdepth 2 -type f
```

Verify / observe: Confirm the expected source file exists in the context.

Step 2 — Review the Dockerfile.

```
cat Dockerfile
```

Verify / observe: Check COPY paths, working directory, and stage names.

Step 3 — Re-run the build with plain progress output.

```
docker build --progress=plain -t local/debug-1:latest .
```

Verify / observe: Read the first failing instruction and its error message.

Final conclusion: Is the problem a missing file, an incorrect path, an ignored file, or a different issue? Support your conclusion with the build output.

Concept to retain: The earliest failing instruction usually provides the strongest diagnostic clue.

My notes: _____

SC-352 · Builds · Diagnose a failed Docker build

Situation: A build fails during a COPY instruction. You must isolate the cause before rewriting the Dockerfile.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List files in the build directory.

```
find . -maxdepth 2 -type f
```

Verify / observe: Confirm the expected source file exists in the context.

Step 2 — Review the Dockerfile.

```
cat Dockerfile
```

Verify / observe: Check COPY paths, working directory, and stage names.

Step 3 — Re-run the build with plain progress output.

```
docker build --progress=plain -t local/debug-2:latest .
```

Verify / observe: Read the first failing instruction and its error message.

Final conclusion: Is the problem a missing file, an incorrect path, an ignored file, or a different issue? Support your conclusion with the build output.

Concept to retain: The earliest failing instruction usually provides the strongest diagnostic clue.

My notes: _____

SC-353 · Builds · Diagnose a failed Docker build

Situation: A build fails during a COPY instruction. You must isolate the cause before rewriting the Dockerfile.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List files in the build directory.

```
find . -maxdepth 2 -type f
```

Verify / observe: Confirm the expected source file exists in the context.

Step 2 — Review the Dockerfile.

```
cat Dockerfile
```

Verify / observe: Check COPY paths, working directory, and stage names.

Step 3 — Re-run the build with plain progress output.

```
docker build --progress=plain -t local/debug-3:latest .
```

Verify / observe: Read the first failing instruction and its error message.

Final conclusion: Is the problem a missing file, an incorrect path, an ignored file, or a different issue? Support your conclusion with the build output.

Concept to retain: The earliest failing instruction usually provides the strongest diagnostic clue.

My notes: _____

SC-354 · Builds · Diagnose a failed Docker build

Situation: A build fails during a COPY instruction. You must isolate the cause before rewriting the Dockerfile.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List files in the build directory.

```
find . -maxdepth 2 -type f
```

Verify / observe: Confirm the expected source file exists in the context.

Step 2 — Review the Dockerfile.

```
cat Dockerfile
```

Verify / observe: Check COPY paths, working directory, and stage names.

Step 3 — Re-run the build with plain progress output.

```
docker build --progress=plain -t local/debug-4:latest .
```

Verify / observe: Read the first failing instruction and its error message.

Final conclusion: Is the problem a missing file, an incorrect path, an ignored file, or a different issue? Support your conclusion with the build output.

Concept to retain: The earliest failing instruction usually provides the strongest diagnostic clue.

My notes: _____

SC-355 · Builds · Diagnose a failed Docker build

Situation: A build fails during a COPY instruction. You must isolate the cause before rewriting the Dockerfile.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List files in the build directory.

```
find . -maxdepth 2 -type f
```

Verify / observe: Confirm the expected source file exists in the context.

Step 2 — Review the Dockerfile.

```
cat Dockerfile
```

Verify / observe: Check COPY paths, working directory, and stage names.

Step 3 — Re-run the build with plain progress output.

```
docker build --progress=plain -t local/debug-5:latest .
```

Verify / observe: Read the first failing instruction and its error message.

Final conclusion: Is the problem a missing file, an incorrect path, an ignored file, or a different issue? Support your conclusion with the build output.

Concept to retain: The earliest failing instruction usually provides the strongest diagnostic clue.

My notes: _____

SC-356 · Builds · Diagnose a failed Docker build

Situation: A build fails during a COPY instruction. You must isolate the cause before rewriting the Dockerfile.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List files in the build directory.

```
find . -maxdepth 2 -type f
```

Verify / observe: Confirm the expected source file exists in the context.

Step 2 — Review the Dockerfile.

```
cat Dockerfile
```

Verify / observe: Check COPY paths, working directory, and stage names.

Step 3 — Re-run the build with plain progress output.

```
docker build --progress=plain -t local/debug-6:latest .
```

Verify / observe: Read the first failing instruction and its error message.

Final conclusion: Is the problem a missing file, an incorrect path, an ignored file, or a different issue? Support your conclusion with the build output.

Concept to retain: The earliest failing instruction usually provides the strongest diagnostic clue.

My notes: _____

SC-357 · Builds · Diagnose a failed Docker build

Situation: A build fails during a COPY instruction. You must isolate the cause before rewriting the Dockerfile.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List files in the build directory.

```
find . -maxdepth 2 -type f
```

Verify / observe: Confirm the expected source file exists in the context.

Step 2 — Review the Dockerfile.

```
cat Dockerfile
```

Verify / observe: Check COPY paths, working directory, and stage names.

Step 3 — Re-run the build with plain progress output.

```
docker build --progress=plain -t local/debug-7:latest .
```

Verify / observe: Read the first failing instruction and its error message.

Final conclusion: Is the problem a missing file, an incorrect path, an ignored file, or a different issue? Support your conclusion with the build output.

Concept to retain: The earliest failing instruction usually provides the strongest diagnostic clue.

My notes: _____

SC-358 · Builds · Diagnose a failed Docker build

Situation: A build fails during a COPY instruction. You must isolate the cause before rewriting the Dockerfile.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List files in the build directory.

```
find . -maxdepth 2 -type f
```

Verify / observe: Confirm the expected source file exists in the context.

Step 2 — Review the Dockerfile.

```
cat Dockerfile
```

Verify / observe: Check COPY paths, working directory, and stage names.

Step 3 — Re-run the build with plain progress output.

```
docker build --progress=plain -t local/debug-8:latest .
```

Verify / observe: Read the first failing instruction and its error message.

Final conclusion: Is the problem a missing file, an incorrect path, an ignored file, or a different issue? Support your conclusion with the build output.

Concept to retain: The earliest failing instruction usually provides the strongest diagnostic clue.

My notes: _____

SC-359 · Builds · Diagnose a failed Docker build

Situation: A build fails during a COPY instruction. You must isolate the cause before rewriting the Dockerfile.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List files in the build directory.

```
find . -maxdepth 2 -type f
```

Verify / observe: Confirm the expected source file exists in the context.

Step 2 — Review the Dockerfile.

```
cat Dockerfile
```

Verify / observe: Check COPY paths, working directory, and stage names.

Step 3 — Re-run the build with plain progress output.

```
docker build --progress=plain -t local/debug-9:latest .
```

Verify / observe: Read the first failing instruction and its error message.

Final conclusion: Is the problem a missing file, an incorrect path, an ignored file, or a different issue? Support your conclusion with the build output.

Concept to retain: The earliest failing instruction usually provides the strongest diagnostic clue.

My notes: _____

SC-360 · Builds · Diagnose a failed Docker build

Situation: A build fails during a COPY instruction. You must isolate the cause before rewriting the Dockerfile.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List files in the build directory.

```
find . -maxdepth 2 -type f
```

Verify / observe: Confirm the expected source file exists in the context.

Step 2 — Review the Dockerfile.

```
cat Dockerfile
```

Verify / observe: Check COPY paths, working directory, and stage names.

Step 3 — Re-run the build with plain progress output.

```
docker build --progress=plain -t local/debug-10:latest .
```

Verify / observe: Read the first failing instruction and its error message.

Final conclusion: Is the problem a missing file, an incorrect path, an ignored file, or a different issue? Support your conclusion with the build output.

Concept to retain: The earliest failing instruction usually provides the strongest diagnostic clue.

My notes: _____

SC-361 · Builds · Understand build cache behavior

Situation: A team wants faster builds but needs to understand when cached layers are reused.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Build the image normally.

```
docker build -t local/cache-1:first .
```

Verify / observe: Record which steps are marked cached.

Step 2 — Build the same context again.

```
docker build -t local/cache-1:second .
```

Verify / observe: Compare cache reuse with the first build.

Step 3 — Run a no-cache build for comparison.

```
docker build --no-cache -t local/cache-1:fresh .
```

Verify / observe: Confirm steps are rebuilt instead of reused from cache.

Final conclusion: Which changes invalidate cache, and why can a no-cache build take longer?

Concept to retain: Cache improves speed, but cache invalidation depends on inputs and instruction order.

My notes: _____

SC-362 · Builds · Understand build cache behavior

Situation: A team wants faster builds but needs to understand when cached layers are reused.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Build the image normally.

```
docker build -t local/cache-2:first .
```

Verify / observe: Record which steps are marked cached.

Step 2 — Build the same context again.

```
docker build -t local/cache-2:second .
```

Verify / observe: Compare cache reuse with the first build.

Step 3 — Run a no-cache build for comparison.

```
docker build --no-cache -t local/cache-2:fresh .
```

Verify / observe: Confirm steps are rebuilt instead of reused from cache.

Final conclusion: Which changes invalidate cache, and why can a no-cache build take longer?

Concept to retain: Cache improves speed, but cache invalidation depends on inputs and instruction order.

My notes: _____

SC-363 · Builds · Understand build cache behavior

Situation: A team wants faster builds but needs to understand when cached layers are reused.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Build the image normally.

```
docker build -t local/cache-3:first .
```

Verify / observe: Record which steps are marked cached.

Step 2 — Build the same context again.

```
docker build -t local/cache-3:second .
```

Verify / observe: Compare cache reuse with the first build.

Step 3 — Run a no-cache build for comparison.

```
docker build --no-cache -t local/cache-3:fresh .
```

Verify / observe: Confirm steps are rebuilt instead of reused from cache.

Final conclusion: Which changes invalidate cache, and why can a no-cache build take longer?

Concept to retain: Cache improves speed, but cache invalidation depends on inputs and instruction order.

My notes: _____

SC-364 · Builds · Understand build cache behavior

Situation: A team wants faster builds but needs to understand when cached layers are reused.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Build the image normally.

```
docker build -t local/cache-4:first .
```

Verify / observe: Record which steps are marked cached.

Step 2 — Build the same context again.

```
docker build -t local/cache-4:second .
```

Verify / observe: Compare cache reuse with the first build.

Step 3 — Run a no-cache build for comparison.

```
docker build --no-cache -t local/cache-4:fresh .
```

Verify / observe: Confirm steps are rebuilt instead of reused from cache.

Final conclusion: Which changes invalidate cache, and why can a no-cache build take longer?

Concept to retain: Cache improves speed, but cache invalidation depends on inputs and instruction order.

My notes: _____

SC-365 · Builds · Understand build cache behavior

Situation: A team wants faster builds but needs to understand when cached layers are reused.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Build the image normally.

```
docker build -t local/cache-5:first .
```

Verify / observe: Record which steps are marked cached.

Step 2 — Build the same context again.

```
docker build -t local/cache-5:second .
```

Verify / observe: Compare cache reuse with the first build.

Step 3 — Run a no-cache build for comparison.

```
docker build --no-cache -t local/cache-5:fresh .
```

Verify / observe: Confirm steps are rebuilt instead of reused from cache.

Final conclusion: Which changes invalidate cache, and why can a no-cache build take longer?

Concept to retain: Cache improves speed, but cache invalidation depends on inputs and instruction order.

My notes: _____

SC-366 · Builds · Understand build cache behavior

Situation: A team wants faster builds but needs to understand when cached layers are reused.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Build the image normally.

```
docker build -t local/cache-6:first .
```

Verify / observe: Record which steps are marked cached.

Step 2 — Build the same context again.

```
docker build -t local/cache-6:second .
```

Verify / observe: Compare cache reuse with the first build.

Step 3 — Run a no-cache build for comparison.

```
docker build --no-cache -t local/cache-6:fresh .
```

Verify / observe: Confirm steps are rebuilt instead of reused from cache.

Final conclusion: Which changes invalidate cache, and why can a no-cache build take longer?

Concept to retain: Cache improves speed, but cache invalidation depends on inputs and instruction order.

My notes: _____

SC-367 · Builds · Understand build cache behavior

Situation: A team wants faster builds but needs to understand when cached layers are reused.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Build the image normally.

```
docker build -t local/cache-7:first .
```

Verify / observe: Record which steps are marked cached.

Step 2 — Build the same context again.

```
docker build -t local/cache-7:second .
```

Verify / observe: Compare cache reuse with the first build.

Step 3 — Run a no-cache build for comparison.

```
docker build --no-cache -t local/cache-7:fresh .
```

Verify / observe: Confirm steps are rebuilt instead of reused from cache.

Final conclusion: Which changes invalidate cache, and why can a no-cache build take longer?

Concept to retain: Cache improves speed, but cache invalidation depends on inputs and instruction order.

My notes: _____

SC-368 · Builds · Understand build cache behavior

Situation: A team wants faster builds but needs to understand when cached layers are reused.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Build the image normally.

```
docker build -t local/cache-8:first .
```

Verify / observe: Record which steps are marked cached.

Step 2 — Build the same context again.

```
docker build -t local/cache-8:second .
```

Verify / observe: Compare cache reuse with the first build.

Step 3 — Run a no-cache build for comparison.

```
docker build --no-cache -t local/cache-8:fresh .
```

Verify / observe: Confirm steps are rebuilt instead of reused from cache.

Final conclusion: Which changes invalidate cache, and why can a no-cache build take longer?

Concept to retain: Cache improves speed, but cache invalidation depends on inputs and instruction order.

My notes: _____

SC-369 · Builds · Understand build cache behavior

Situation: A team wants faster builds but needs to understand when cached layers are reused.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Build the image normally.

```
docker build -t local/cache-9:first .
```

Verify / observe: Record which steps are marked cached.

Step 2 — Build the same context again.

```
docker build -t local/cache-9:second .
```

Verify / observe: Compare cache reuse with the first build.

Step 3 — Run a no-cache build for comparison.

```
docker build --no-cache -t local/cache-9:fresh .
```

Verify / observe: Confirm steps are rebuilt instead of reused from cache.

Final conclusion: Which changes invalidate cache, and why can a no-cache build take longer?

Concept to retain: Cache improves speed, but cache invalidation depends on inputs and instruction order.

My notes: _____

SC-370 · Builds · Understand build cache behavior

Situation: A team wants faster builds but needs to understand when cached layers are reused.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Build the image normally.

```
docker build -t local/cache-10:first .
```

Verify / observe: Record which steps are marked cached.

Step 2 — Build the same context again.

```
docker build -t local/cache-10:second .
```

Verify / observe: Compare cache reuse with the first build.

Step 3 — Run a no-cache build for comparison.

```
docker build --no-cache -t local/cache-10:fresh .
```

Verify / observe: Confirm steps are rebuilt instead of reused from cache.

Final conclusion: Which changes invalidate cache, and why can a no-cache build take longer?

Concept to retain: Cache improves speed, but cache invalidation depends on inputs and instruction order.

My notes: _____

SC-371 · Builds · Use build arguments correctly

Situation: You need to parameterize a build with an application version without confusing it with runtime configuration.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the Dockerfile declares the expected ARG.

```
grep -n 'ARG\|APP_VERSION' Dockerfile
```

Verify / observe: The Dockerfile must define and use APP_VERSION for the argument to affect output.

Step 2 — Build with a chosen argument.

```
docker build --build-arg APP_VERSION=lab-1 -t local/arg-1:latest .
```

Verify / observe: Watch the build output for the argument-dependent stage.

Step 3 — Inspect the final image configuration.

```
docker image inspect local/arg-1:latest
```

Verify / observe: Confirm only the intended configuration is present.

Final conclusion: Why is ARG not a safe place for passwords or long-lived secrets?

Concept to retain: Build arguments parameterize a build; they should not be treated as a secret store.

My notes: _____

SC-372 · Builds · Use build arguments correctly

Situation: You need to parameterize a build with an application version without confusing it with runtime configuration.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the Dockerfile declares the expected ARG.

```
grep -n 'ARG\|APP_VERSION' Dockerfile
```

Verify / observe: The Dockerfile must define and use APP_VERSION for the argument to affect output.

Step 2 — Build with a chosen argument.

```
docker build --build-arg APP_VERSION=lab-2 -t local/arg-2:latest .
```

Verify / observe: Watch the build output for the argument-dependent stage.

Step 3 — Inspect the final image configuration.

```
docker image inspect local/arg-2:latest
```

Verify / observe: Confirm only the intended configuration is present.

Final conclusion: Why is ARG not a safe place for passwords or long-lived secrets?

Concept to retain: Build arguments parameterize a build; they should not be treated as a secret store.

My notes: _____

SC-373 · Builds · Use build arguments correctly

Situation: You need to parameterize a build with an application version without confusing it with runtime configuration.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the Dockerfile declares the expected ARG.

```
grep -n 'ARG\|APP_VERSION' Dockerfile
```

Verify / observe: The Dockerfile must define and use APP_VERSION for the argument to affect output.

Step 2 — Build with a chosen argument.

```
docker build --build-arg APP_VERSION=lab-3 -t local/arg-3:latest .
```

Verify / observe: Watch the build output for the argument-dependent stage.

Step 3 — Inspect the final image configuration.

```
docker image inspect local/arg-3:latest
```

Verify / observe: Confirm only the intended configuration is present.

Final conclusion: Why is ARG not a safe place for passwords or long-lived secrets?

Concept to retain: Build arguments parameterize a build; they should not be treated as a secret store.

My notes: _____

SC-374 · Builds · Use build arguments correctly

Situation: You need to parameterize a build with an application version without confusing it with runtime configuration.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the Dockerfile declares the expected ARG.

```
grep -n 'ARG\|APP_VERSION' Dockerfile
```

Verify / observe: The Dockerfile must define and use APP_VERSION for the argument to affect output.

Step 2 — Build with a chosen argument.

```
docker build --build-arg APP_VERSION=lab-4 -t local/arg-4:latest .
```

Verify / observe: Watch the build output for the argument-dependent stage.

Step 3 — Inspect the final image configuration.

```
docker image inspect local/arg-4:latest
```

Verify / observe: Confirm only the intended configuration is present.

Final conclusion: Why is ARG not a safe place for passwords or long-lived secrets?

Concept to retain: Build arguments parameterize a build; they should not be treated as a secret store.

My notes: _____

SC-375 · Builds · Use build arguments correctly

Situation: You need to parameterize a build with an application version without confusing it with runtime configuration.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the Dockerfile declares the expected ARG.

```
grep -n 'ARG\|APP_VERSION' Dockerfile
```

Verify / observe: The Dockerfile must define and use APP_VERSION for the argument to affect output.

Step 2 — Build with a chosen argument.

```
docker build --build-arg APP_VERSION=lab-5 -t local/arg-5:latest .
```

Verify / observe: Watch the build output for the argument-dependent stage.

Step 3 — Inspect the final image configuration.

```
docker image inspect local/arg-5:latest
```

Verify / observe: Confirm only the intended configuration is present.

Final conclusion: Why is ARG not a safe place for passwords or long-lived secrets?

Concept to retain: Build arguments parameterize a build; they should not be treated as a secret store.

My notes: _____

SC-376 · Builds · Use build arguments correctly

Situation: You need to parameterize a build with an application version without confusing it with runtime configuration.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the Dockerfile declares the expected ARG.

```
grep -n 'ARG\|APP_VERSION' Dockerfile
```

Verify / observe: The Dockerfile must define and use APP_VERSION for the argument to affect output.

Step 2 — Build with a chosen argument.

```
docker build --build-arg APP_VERSION=lab-6 -t local/arg-6:latest .
```

Verify / observe: Watch the build output for the argument-dependent stage.

Step 3 — Inspect the final image configuration.

```
docker image inspect local/arg-6:latest
```

Verify / observe: Confirm only the intended configuration is present.

Final conclusion: Why is ARG not a safe place for passwords or long-lived secrets?

Concept to retain: Build arguments parameterize a build; they should not be treated as a secret store.

My notes: _____

SC-377 · Builds · Use build arguments correctly

Situation: You need to parameterize a build with an application version without confusing it with runtime configuration.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the Dockerfile declares the expected ARG.

```
grep -n 'ARG\|APP_VERSION' Dockerfile
```

Verify / observe: The Dockerfile must define and use APP_VERSION for the argument to affect output.

Step 2 — Build with a chosen argument.

```
docker build --build-arg APP_VERSION=lab-7 -t local/arg-7:latest .
```

Verify / observe: Watch the build output for the argument-dependent stage.

Step 3 — Inspect the final image configuration.

```
docker image inspect local/arg-7:latest
```

Verify / observe: Confirm only the intended configuration is present.

Final conclusion: Why is ARG not a safe place for passwords or long-lived secrets?

Concept to retain: Build arguments parameterize a build; they should not be treated as a secret store.

My notes: _____

SC-378 · Builds · Use build arguments correctly

Situation: You need to parameterize a build with an application version without confusing it with runtime configuration.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the Dockerfile declares the expected ARG.

```
grep -n 'ARG\|APP_VERSION' Dockerfile
```

Verify / observe: The Dockerfile must define and use APP_VERSION for the argument to affect output.

Step 2 — Build with a chosen argument.

```
docker build --build-arg APP_VERSION=lab-8 -t local/arg-8:latest .
```

Verify / observe: Watch the build output for the argument-dependent stage.

Step 3 — Inspect the final image configuration.

```
docker image inspect local/arg-8:latest
```

Verify / observe: Confirm only the intended configuration is present.

Final conclusion: Why is ARG not a safe place for passwords or long-lived secrets?

Concept to retain: Build arguments parameterize a build; they should not be treated as a secret store.

My notes: _____

SC-379 · Builds · Use build arguments correctly

Situation: You need to parameterize a build with an application version without confusing it with runtime configuration.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the Dockerfile declares the expected ARG.

```
grep -n 'ARG\|APP_VERSION' Dockerfile
```

Verify / observe: The Dockerfile must define and use APP_VERSION for the argument to affect output.

Step 2 — Build with a chosen argument.

```
docker build --build-arg APP_VERSION=lab-9 -t local/arg-9:latest .
```

Verify / observe: Watch the build output for the argument-dependent stage.

Step 3 — Inspect the final image configuration.

```
docker image inspect local/arg-9:latest
```

Verify / observe: Confirm only the intended configuration is present.

Final conclusion: Why is ARG not a safe place for passwords or long-lived secrets?

Concept to retain: Build arguments parameterize a build; they should not be treated as a secret store.

My notes: _____

SC-380 · Builds · Use build arguments correctly

Situation: You need to parameterize a build with an application version without confusing it with runtime configuration.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the Dockerfile declares the expected ARG.

```
grep -n 'ARG\|APP_VERSION' Dockerfile
```

Verify / observe: The Dockerfile must define and use APP_VERSION for the argument to affect output.

Step 2 — Build with a chosen argument.

```
docker build --build-arg APP_VERSION=lab-10 -t local/arg-10:latest .
```

Verify / observe: Watch the build output for the argument-dependent stage.

Step 3 — Inspect the final image configuration.

```
docker image inspect local/arg-10:latest
```

Verify / observe: Confirm only the intended configuration is present.

Final conclusion: Why is ARG not a safe place for passwords or long-lived secrets?

Concept to retain: Build arguments parameterize a build; they should not be treated as a secret store.

My notes: _____

SC-381 · Builds · Build a named multi-stage target

Situation: A Dockerfile contains a stage named test, and you need to build that stage separately.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Find stage names in the Dockerfile.

```
grep -n 'FROM\|AS ' Dockerfile
```

Verify / observe: Locate the stage named test.

Step 2 — Build only the test target.

```
docker build --target test -t local/test-1:latest .
```

Verify / observe: The build fails if no stage named test exists.

Step 3 — Inspect the resulting tag.

```
docker image inspect local/test-1:latest
```

Verify / observe: Confirm the intended target was built.

Final conclusion: Why can multi-stage builds keep compilers and build tools out of the final runtime image?

Concept to retain: A named target lets you build a particular stage; the final image can copy only required artifacts.

My notes: _____

SC-382 · Builds · Build a named multi-stage target

Situation: A Dockerfile contains a stage named test, and you need to build that stage separately.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Find stage names in the Dockerfile.

```
grep -n 'FROM\|AS ' Dockerfile
```

Verify / observe: Locate the stage named test.

Step 2 — Build only the test target.

```
docker build --target test -t local/test-2:latest .
```

Verify / observe: The build fails if no stage named test exists.

Step 3 — Inspect the resulting tag.

```
docker image inspect local/test-2:latest
```

Verify / observe: Confirm the intended target was built.

Final conclusion: Why can multi-stage builds keep compilers and build tools out of the final runtime image?

Concept to retain: A named target lets you build a particular stage; the final image can copy only required artifacts.

My notes: _____

SC-383 · Builds · Build a named multi-stage target

Situation: A Dockerfile contains a stage named test, and you need to build that stage separately.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Find stage names in the Dockerfile.

```
grep -n 'FROM\|AS ' Dockerfile
```

Verify / observe: Locate the stage named test.

Step 2 — Build only the test target.

```
docker build --target test -t local/test-3:latest .
```

Verify / observe: The build fails if no stage named test exists.

Step 3 — Inspect the resulting tag.

```
docker image inspect local/test-3:latest
```

Verify / observe: Confirm the intended target was built.

Final conclusion: Why can multi-stage builds keep compilers and build tools out of the final runtime image?

Concept to retain: A named target lets you build a particular stage; the final image can copy only required artifacts.

My notes: _____

SC-384 · Builds · Build a named multi-stage target

Situation: A Dockerfile contains a stage named test, and you need to build that stage separately.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Find stage names in the Dockerfile.

```
grep -n 'FROM\|AS ' Dockerfile
```

Verify / observe: Locate the stage named test.

Step 2 — Build only the test target.

```
docker build --target test -t local/test-4:latest .
```

Verify / observe: The build fails if no stage named test exists.

Step 3 — Inspect the resulting tag.

```
docker image inspect local/test-4:latest
```

Verify / observe: Confirm the intended target was built.

Final conclusion: Why can multi-stage builds keep compilers and build tools out of the final runtime image?

Concept to retain: A named target lets you build a particular stage; the final image can copy only required artifacts.

My notes: _____

SC-385 · Builds · Build a named multi-stage target

Situation: A Dockerfile contains a stage named test, and you need to build that stage separately.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Find stage names in the Dockerfile.

```
grep -n 'FROM\|AS ' Dockerfile
```

Verify / observe: Locate the stage named test.

Step 2 — Build only the test target.

```
docker build --target test -t local/test-5:latest .
```

Verify / observe: The build fails if no stage named test exists.

Step 3 — Inspect the resulting tag.

```
docker image inspect local/test-5:latest
```

Verify / observe: Confirm the intended target was built.

Final conclusion: Why can multi-stage builds keep compilers and build tools out of the final runtime image?

Concept to retain: A named target lets you build a particular stage; the final image can copy only required artifacts.

My notes: _____

SC-386 · Builds · Build a named multi-stage target

Situation: A Dockerfile contains a stage named test, and you need to build that stage separately.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Find stage names in the Dockerfile.

```
grep -n 'FROM\|AS ' Dockerfile
```

Verify / observe: Locate the stage named test.

Step 2 — Build only the test target.

```
docker build --target test -t local/test-6:latest .
```

Verify / observe: The build fails if no stage named test exists.

Step 3 — Inspect the resulting tag.

```
docker image inspect local/test-6:latest
```

Verify / observe: Confirm the intended target was built.

Final conclusion: Why can multi-stage builds keep compilers and build tools out of the final runtime image?

Concept to retain: A named target lets you build a particular stage; the final image can copy only required artifacts.

My notes: _____

SC-387 · Builds · Build a named multi-stage target

Situation: A Dockerfile contains a stage named test, and you need to build that stage separately.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Find stage names in the Dockerfile.

```
grep -n 'FROM\|AS ' Dockerfile
```

Verify / observe: Locate the stage named test.

Step 2 — Build only the test target.

```
docker build --target test -t local/test-7:latest .
```

Verify / observe: The build fails if no stage named test exists.

Step 3 — Inspect the resulting tag.

```
docker image inspect local/test-7:latest
```

Verify / observe: Confirm the intended target was built.

Final conclusion: Why can multi-stage builds keep compilers and build tools out of the final runtime image?

Concept to retain: A named target lets you build a particular stage; the final image can copy only required artifacts.

My notes: _____

SC-388 · Builds · Build a named multi-stage target

Situation: A Dockerfile contains a stage named test, and you need to build that stage separately.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Find stage names in the Dockerfile.

```
grep -n 'FROM\|AS ' Dockerfile
```

Verify / observe: Locate the stage named test.

Step 2 — Build only the test target.

```
docker build --target test -t local/test-8:latest .
```

Verify / observe: The build fails if no stage named test exists.

Step 3 — Inspect the resulting tag.

```
docker image inspect local/test-8:latest
```

Verify / observe: Confirm the intended target was built.

Final conclusion: Why can multi-stage builds keep compilers and build tools out of the final runtime image?

Concept to retain: A named target lets you build a particular stage; the final image can copy only required artifacts.

My notes: _____

SC-389 · Builds · Build a named multi-stage target

Situation: A Dockerfile contains a stage named test, and you need to build that stage separately.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Find stage names in the Dockerfile.

```
grep -n 'FROM\|AS ' Dockerfile
```

Verify / observe: Locate the stage named test.

Step 2 — Build only the test target.

```
docker build --target test -t local/test-9:latest .
```

Verify / observe: The build fails if no stage named test exists.

Step 3 — Inspect the resulting tag.

```
docker image inspect local/test-9:latest
```

Verify / observe: Confirm the intended target was built.

Final conclusion: Why can multi-stage builds keep compilers and build tools out of the final runtime image?

Concept to retain: A named target lets you build a particular stage; the final image can copy only required artifacts.

My notes: _____

SC-390 · Builds · Build a named multi-stage target

Situation: A Dockerfile contains a stage named test, and you need to build that stage separately.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Find stage names in the Dockerfile.

```
grep -n 'FROM\|AS ' Dockerfile
```

Verify / observe: Locate the stage named test.

Step 2 — Build only the test target.

```
docker build --target test -t local/test-10:latest .
```

Verify / observe: The build fails if no stage named test exists.

Step 3 — Inspect the resulting tag.

```
docker image inspect local/test-10:latest
```

Verify / observe: Confirm the intended target was built.

Final conclusion: Why can multi-stage builds keep compilers and build tools out of the final runtime image?

Concept to retain: A named target lets you build a particular stage; the final image can copy only required artifacts.

My notes: _____

SC-391 · Builds · Reduce unnecessary files in the build context

Situation: A build sends too many files, including local artifacts that should not be copied into the image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the build context file list.

```
du -sh . && find . -maxdepth 2 -type f
```

Verify / observe: Identify caches, dependency directories, and local output files.

Step 2 — Review ignore rules.

```
cat .dockerignore
```

Verify / observe: Check whether large or sensitive files are excluded.

Step 3 — Rebuild and compare output.

```
docker build --progress=plain -t local/context-1:latest .
```

Verify / observe: Compare transfer size and build behavior.

Final conclusion: Which files should be excluded, and how could an overbroad ignore rule break a build?

Concept to retain: A small, intentional build context reduces transfer time and accidental inclusion.

My notes: _____

SC-392 · Builds · Reduce unnecessary files in the build context

Situation: A build sends too many files, including local artifacts that should not be copied into the image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the build context file list.

```
du -sh . && find . -maxdepth 2 -type f
```

Verify / observe: Identify caches, dependency directories, and local output files.

Step 2 — Review ignore rules.

```
cat .dockerignore
```

Verify / observe: Check whether large or sensitive files are excluded.

Step 3 — Rebuild and compare output.

```
docker build --progress=plain -t local/context-2:latest .
```

Verify / observe: Compare transfer size and build behavior.

Final conclusion: Which files should be excluded, and how could an overbroad ignore rule break a build?

Concept to retain: A small, intentional build context reduces transfer time and accidental inclusion.

My notes: _____

SC-393 · Builds · Reduce unnecessary files in the build context

Situation: A build sends too many files, including local artifacts that should not be copied into the image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the build context file list.

```
du -sh . && find . -maxdepth 2 -type f
```

Verify / observe: Identify caches, dependency directories, and local output files.

Step 2 — Review ignore rules.

```
cat .dockerignore
```

Verify / observe: Check whether large or sensitive files are excluded.

Step 3 — Rebuild and compare output.

```
docker build --progress=plain -t local/context-3:latest .
```

Verify / observe: Compare transfer size and build behavior.

Final conclusion: Which files should be excluded, and how could an overbroad ignore rule break a build?

Concept to retain: A small, intentional build context reduces transfer time and accidental inclusion.

My notes: _____

SC-394 · Builds · Reduce unnecessary files in the build context

Situation: A build sends too many files, including local artifacts that should not be copied into the image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the build context file list.

```
du -sh . && find . -maxdepth 2 -type f
```

Verify / observe: Identify caches, dependency directories, and local output files.

Step 2 — Review ignore rules.

```
cat .dockerignore
```

Verify / observe: Check whether large or sensitive files are excluded.

Step 3 — Rebuild and compare output.

```
docker build --progress=plain -t local/context-4:latest .
```

Verify / observe: Compare transfer size and build behavior.

Final conclusion: Which files should be excluded, and how could an overbroad ignore rule break a build?

Concept to retain: A small, intentional build context reduces transfer time and accidental inclusion.

My notes: _____

SC-395 · Builds · Reduce unnecessary files in the build context

Situation: A build sends too many files, including local artifacts that should not be copied into the image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the build context file list.

```
du -sh . && find . -maxdepth 2 -type f
```

Verify / observe: Identify caches, dependency directories, and local output files.

Step 2 — Review ignore rules.

```
cat .dockerignore
```

Verify / observe: Check whether large or sensitive files are excluded.

Step 3 — Rebuild and compare output.

```
docker build --progress=plain -t local/context-5:latest .
```

Verify / observe: Compare transfer size and build behavior.

Final conclusion: Which files should be excluded, and how could an overbroad ignore rule break a build?

Concept to retain: A small, intentional build context reduces transfer time and accidental inclusion.

My notes: _____

SC-396 · Builds · Reduce unnecessary files in the build context

Situation: A build sends too many files, including local artifacts that should not be copied into the image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the build context file list.

```
du -sh . && find . -maxdepth 2 -type f
```

Verify / observe: Identify caches, dependency directories, and local output files.

Step 2 — Review ignore rules.

```
cat .dockerignore
```

Verify / observe: Check whether large or sensitive files are excluded.

Step 3 — Rebuild and compare output.

```
docker build --progress=plain -t local/context-6:latest .
```

Verify / observe: Compare transfer size and build behavior.

Final conclusion: Which files should be excluded, and how could an overbroad ignore rule break a build?

Concept to retain: A small, intentional build context reduces transfer time and accidental inclusion.

My notes: _____

SC-397 · Builds · Reduce unnecessary files in the build context

Situation: A build sends too many files, including local artifacts that should not be copied into the image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the build context file list.

```
du -sh . && find . -maxdepth 2 -type f
```

Verify / observe: Identify caches, dependency directories, and local output files.

Step 2 — Review ignore rules.

```
cat .dockerignore
```

Verify / observe: Check whether large or sensitive files are excluded.

Step 3 — Rebuild and compare output.

```
docker build --progress=plain -t local/context-7:latest .
```

Verify / observe: Compare transfer size and build behavior.

Final conclusion: Which files should be excluded, and how could an overbroad ignore rule break a build?

Concept to retain: A small, intentional build context reduces transfer time and accidental inclusion.

My notes: _____

SC-398 · Builds · Reduce unnecessary files in the build context

Situation: A build sends too many files, including local artifacts that should not be copied into the image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the build context file list.

```
du -sh . && find . -maxdepth 2 -type f
```

Verify / observe: Identify caches, dependency directories, and local output files.

Step 2 — Review ignore rules.

```
cat .dockerignore
```

Verify / observe: Check whether large or sensitive files are excluded.

Step 3 — Rebuild and compare output.

```
docker build --progress=plain -t local/context-8:latest .
```

Verify / observe: Compare transfer size and build behavior.

Final conclusion: Which files should be excluded, and how could an overbroad ignore rule break a build?

Concept to retain: A small, intentional build context reduces transfer time and accidental inclusion.

My notes: _____

SC-399 · Builds · Reduce unnecessary files in the build context

Situation: A build sends too many files, including local artifacts that should not be copied into the image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the build context file list.

```
du -sh . && find . -maxdepth 2 -type f
```

Verify / observe: Identify caches, dependency directories, and local output files.

Step 2 — Review ignore rules.

```
cat .dockerignore
```

Verify / observe: Check whether large or sensitive files are excluded.

Step 3 — Rebuild and compare output.

```
docker build --progress=plain -t local/context-9:latest .
```

Verify / observe: Compare transfer size and build behavior.

Final conclusion: Which files should be excluded, and how could an overbroad ignore rule break a build?

Concept to retain: A small, intentional build context reduces transfer time and accidental inclusion.

My notes: _____

SC-400 · Builds · Reduce unnecessary files in the build context

Situation: A build sends too many files, including local artifacts that should not be copied into the image.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Inspect the build context file list.

```
du -sh . && find . -maxdepth 2 -type f
```

Verify / observe: Identify caches, dependency directories, and local output files.

Step 2 — Review ignore rules.

```
cat .dockerignore
```

Verify / observe: Check whether large or sensitive files are excluded.

Step 3 — Rebuild and compare output.

```
docker build --progress=plain -t local/context-10:latest .
```

Verify / observe: Compare transfer size and build behavior.

Final conclusion: Which files should be excluded, and how could an overbroad ignore rule break a build?

Concept to retain: A small, intentional build context reduces transfer time and accidental inclusion.

My notes: _____

SC-401 · Builds · Inspect a Buildx builder

Situation: A multi-platform build may be needed, so you must discover the builder and supported platforms first.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List configured builders.

```
docker buildx ls
```

Verify / observe: Identify the active builder and its driver.

Step 2 — Inspect the selected builder.

```
docker buildx inspect
```

Verify / observe: Review supported platforms and status.

Step 3 — Review available builder cache usage.

```
docker buildx du
```

Verify / observe: Record current cache usage before considering cleanup.

Final conclusion: What must be true before attempting a multi-platform build and publishing it?

Concept to retain: Builder capabilities vary by driver and host setup; inspect before relying on them.

My notes: _____

SC-402 · Builds · Inspect a Buildx builder

Situation: A multi-platform build may be needed, so you must discover the builder and supported platforms first.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List configured builders.

```
docker buildx ls
```

Verify / observe: Identify the active builder and its driver.

Step 2 — Inspect the selected builder.

```
docker buildx inspect
```

Verify / observe: Review supported platforms and status.

Step 3 — Review available builder cache usage.

```
docker buildx du
```

Verify / observe: Record current cache usage before considering cleanup.

Final conclusion: What must be true before attempting a multi-platform build and publishing it?

Concept to retain: Builder capabilities vary by driver and host setup; inspect before relying on them.

My notes: _____

SC-403 · Builds · Inspect a Buildx builder

Situation: A multi-platform build may be needed, so you must discover the builder and supported platforms first.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List configured builders.

```
docker buildx ls
```

Verify / observe: Identify the active builder and its driver.

Step 2 — Inspect the selected builder.

```
docker buildx inspect
```

Verify / observe: Review supported platforms and status.

Step 3 — Review available builder cache usage.

```
docker buildx du
```

Verify / observe: Record current cache usage before considering cleanup.

Final conclusion: What must be true before attempting a multi-platform build and publishing it?

Concept to retain: Builder capabilities vary by driver and host setup; inspect before relying on them.

My notes: _____

SC-404 · Builds · Inspect a Buildx builder

Situation: A multi-platform build may be needed, so you must discover the builder and supported platforms first.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List configured builders.

```
docker buildx ls
```

Verify / observe: Identify the active builder and its driver.

Step 2 — Inspect the selected builder.

```
docker buildx inspect
```

Verify / observe: Review supported platforms and status.

Step 3 — Review available builder cache usage.

```
docker buildx du
```

Verify / observe: Record current cache usage before considering cleanup.

Final conclusion: What must be true before attempting a multi-platform build and publishing it?

Concept to retain: Builder capabilities vary by driver and host setup; inspect before relying on them.

My notes: _____

SC-405 · Builds · Inspect a Buildx builder

Situation: A multi-platform build may be needed, so you must discover the builder and supported platforms first.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List configured builders.

```
docker buildx ls
```

Verify / observe: Identify the active builder and its driver.

Step 2 — Inspect the selected builder.

```
docker buildx inspect
```

Verify / observe: Review supported platforms and status.

Step 3 — Review available builder cache usage.

```
docker buildx du
```

Verify / observe: Record current cache usage before considering cleanup.

Final conclusion: What must be true before attempting a multi-platform build and publishing it?

Concept to retain: Builder capabilities vary by driver and host setup; inspect before relying on them.

My notes: _____

SC-406 · Builds · Inspect a Buildx builder

Situation: A multi-platform build may be needed, so you must discover the builder and supported platforms first.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List configured builders.

```
docker buildx ls
```

Verify / observe: Identify the active builder and its driver.

Step 2 — Inspect the selected builder.

```
docker buildx inspect
```

Verify / observe: Review supported platforms and status.

Step 3 — Review available builder cache usage.

```
docker buildx du
```

Verify / observe: Record current cache usage before considering cleanup.

Final conclusion: What must be true before attempting a multi-platform build and publishing it?

Concept to retain: Builder capabilities vary by driver and host setup; inspect before relying on them.

My notes: _____

SC-407 · Builds · Inspect a Buildx builder

Situation: A multi-platform build may be needed, so you must discover the builder and supported platforms first.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List configured builders.

```
docker buildx ls
```

Verify / observe: Identify the active builder and its driver.

Step 2 — Inspect the selected builder.

```
docker buildx inspect
```

Verify / observe: Review supported platforms and status.

Step 3 — Review available builder cache usage.

```
docker buildx du
```

Verify / observe: Record current cache usage before considering cleanup.

Final conclusion: What must be true before attempting a multi-platform build and publishing it?

Concept to retain: Builder capabilities vary by driver and host setup; inspect before relying on them.

My notes: _____

SC-408 · Builds · Inspect a Buildx builder

Situation: A multi-platform build may be needed, so you must discover the builder and supported platforms first.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List configured builders.

```
docker buildx ls
```

Verify / observe: Identify the active builder and its driver.

Step 2 — Inspect the selected builder.

```
docker buildx inspect
```

Verify / observe: Review supported platforms and status.

Step 3 — Review available builder cache usage.

```
docker buildx du
```

Verify / observe: Record current cache usage before considering cleanup.

Final conclusion: What must be true before attempting a multi-platform build and publishing it?

Concept to retain: Builder capabilities vary by driver and host setup; inspect before relying on them.

My notes: _____

SC-409 · Builds · Inspect a Buildx builder

Situation: A multi-platform build may be needed, so you must discover the builder and supported platforms first.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List configured builders.

```
docker buildx ls
```

Verify / observe: Identify the active builder and its driver.

Step 2 — Inspect the selected builder.

```
docker buildx inspect
```

Verify / observe: Review supported platforms and status.

Step 3 — Review available builder cache usage.

```
docker buildx du
```

Verify / observe: Record current cache usage before considering cleanup.

Final conclusion: What must be true before attempting a multi-platform build and publishing it?

Concept to retain: Builder capabilities vary by driver and host setup; inspect before relying on them.

My notes: _____

SC-410 · Builds · Inspect a Buildx builder

Situation: A multi-platform build may be needed, so you must discover the builder and supported platforms first.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List configured builders.

```
docker buildx ls
```

Verify / observe: Identify the active builder and its driver.

Step 2 — Inspect the selected builder.

```
docker buildx inspect
```

Verify / observe: Review supported platforms and status.

Step 3 — Review available builder cache usage.

```
docker buildx du
```

Verify / observe: Record current cache usage before considering cleanup.

Final conclusion: What must be true before attempting a multi-platform build and publishing it?

Concept to retain: Builder capabilities vary by driver and host setup; inspect before relying on them.

My notes: _____

SC-411 · Compose · Validate a Compose project before launch

Situation: A team has a compose.yaml file and wants to catch configuration errors before starting services.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Render and validate the model.

```
docker compose -f compose.yaml config
```

Verify / observe: Review the resolved configuration and any validation errors.

Step 2 — List services from the model.

```
docker compose -f compose.yaml config --services
```

Verify / observe: Record service names exactly.

Step 3 — Review service images and build declarations.

```
docker compose -f compose.yaml config --images
```

Verify / observe: Compare image references with the deployment plan.

Final conclusion: What configuration error would block deployment, and which resolved values need careful handling?

Concept to retain: Compose config validates and renders a model, but resolved output may contain sensitive environment values.

My notes: _____

SC-412 · Compose · Validate a Compose project before launch

Situation: A team has a compose.yaml file and wants to catch configuration errors before starting services.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Render and validate the model.

```
docker compose -f compose.yaml config
```

Verify / observe: Review the resolved configuration and any validation errors.

Step 2 — List services from the model.

```
docker compose -f compose.yaml config --services
```

Verify / observe: Record service names exactly.

Step 3 — Review service images and build declarations.

```
docker compose -f compose.yaml config --images
```

Verify / observe: Compare image references with the deployment plan.

Final conclusion: What configuration error would block deployment, and which resolved values need careful handling?

Concept to retain: Compose config validates and renders a model, but resolved output may contain sensitive environment values.

My notes: _____

SC-413 · Compose · Validate a Compose project before launch

Situation: A team has a compose.yaml file and wants to catch configuration errors before starting services.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Render and validate the model.

```
docker compose -f compose.yaml config
```

Verify / observe: Review the resolved configuration and any validation errors.

Step 2 — List services from the model.

```
docker compose -f compose.yaml config --services
```

Verify / observe: Record service names exactly.

Step 3 — Review service images and build declarations.

```
docker compose -f compose.yaml config --images
```

Verify / observe: Compare image references with the deployment plan.

Final conclusion: What configuration error would block deployment, and which resolved values need careful handling?

Concept to retain: Compose config validates and renders a model, but resolved output may contain sensitive environment values.

My notes: _____

SC-414 · Compose · Validate a Compose project before launch

Situation: A team has a compose.yaml file and wants to catch configuration errors before starting services.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Render and validate the model.

```
docker compose -f compose.yaml config
```

Verify / observe: Review the resolved configuration and any validation errors.

Step 2 — List services from the model.

```
docker compose -f compose.yaml config --services
```

Verify / observe: Record service names exactly.

Step 3 — Review service images and build declarations.

```
docker compose -f compose.yaml config --images
```

Verify / observe: Compare image references with the deployment plan.

Final conclusion: What configuration error would block deployment, and which resolved values need careful handling?

Concept to retain: Compose config validates and renders a model, but resolved output may contain sensitive environment values.

My notes: _____

SC-415 · Compose · Validate a Compose project before launch

Situation: A team has a compose.yaml file and wants to catch configuration errors before starting services.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Render and validate the model.

```
docker compose -f compose.yaml config
```

Verify / observe: Review the resolved configuration and any validation errors.

Step 2 — List services from the model.

```
docker compose -f compose.yaml config --services
```

Verify / observe: Record service names exactly.

Step 3 — Review service images and build declarations.

```
docker compose -f compose.yaml config --images
```

Verify / observe: Compare image references with the deployment plan.

Final conclusion: What configuration error would block deployment, and which resolved values need careful handling?

Concept to retain: Compose config validates and renders a model, but resolved output may contain sensitive environment values.

My notes: _____

SC-416 · Compose · Validate a Compose project before launch

Situation: A team has a compose.yaml file and wants to catch configuration errors before starting services.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Render and validate the model.

```
docker compose -f compose.yaml config
```

Verify / observe: Review the resolved configuration and any validation errors.

Step 2 — List services from the model.

```
docker compose -f compose.yaml config --services
```

Verify / observe: Record service names exactly.

Step 3 — Review service images and build declarations.

```
docker compose -f compose.yaml config --images
```

Verify / observe: Compare image references with the deployment plan.

Final conclusion: What configuration error would block deployment, and which resolved values need careful handling?

Concept to retain: Compose config validates and renders a model, but resolved output may contain sensitive environment values.

My notes: _____

SC-417 · Compose · Validate a Compose project before launch

Situation: A team has a compose.yaml file and wants to catch configuration errors before starting services.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Render and validate the model.

```
docker compose -f compose.yaml config
```

Verify / observe: Review the resolved configuration and any validation errors.

Step 2 — List services from the model.

```
docker compose -f compose.yaml config --services
```

Verify / observe: Record service names exactly.

Step 3 — Review service images and build declarations.

```
docker compose -f compose.yaml config --images
```

Verify / observe: Compare image references with the deployment plan.

Final conclusion: What configuration error would block deployment, and which resolved values need careful handling?

Concept to retain: Compose config validates and renders a model, but resolved output may contain sensitive environment values.

My notes: _____

SC-418 · Compose · Validate a Compose project before launch

Situation: A team has a compose.yaml file and wants to catch configuration errors before starting services.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Render and validate the model.

```
docker compose -f compose.yaml config
```

Verify / observe: Review the resolved configuration and any validation errors.

Step 2 — List services from the model.

```
docker compose -f compose.yaml config --services
```

Verify / observe: Record service names exactly.

Step 3 — Review service images and build declarations.

```
docker compose -f compose.yaml config --images
```

Verify / observe: Compare image references with the deployment plan.

Final conclusion: What configuration error would block deployment, and which resolved values need careful handling?

Concept to retain: Compose config validates and renders a model, but resolved output may contain sensitive environment values.

My notes: _____

SC-419 · Compose · Validate a Compose project before launch

Situation: A team has a compose.yaml file and wants to catch configuration errors before starting services.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Render and validate the model.

```
docker compose -f compose.yaml config
```

Verify / observe: Review the resolved configuration and any validation errors.

Step 2 — List services from the model.

```
docker compose -f compose.yaml config --services
```

Verify / observe: Record service names exactly.

Step 3 — Review service images and build declarations.

```
docker compose -f compose.yaml config --images
```

Verify / observe: Compare image references with the deployment plan.

Final conclusion: What configuration error would block deployment, and which resolved values need careful handling?

Concept to retain: Compose config validates and renders a model, but resolved output may contain sensitive environment values.

My notes: _____

SC-420 · Compose · Validate a Compose project before launch

Situation: A team has a compose.yaml file and wants to catch configuration errors before starting services.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Render and validate the model.

```
docker compose -f compose.yaml config
```

Verify / observe: Review the resolved configuration and any validation errors.

Step 2 — List services from the model.

```
docker compose -f compose.yaml config --services
```

Verify / observe: Record service names exactly.

Step 3 — Review service images and build declarations.

```
docker compose -f compose.yaml config --images
```

Verify / observe: Compare image references with the deployment plan.

Final conclusion: What configuration error would block deployment, and which resolved values need careful handling?

Concept to retain: Compose config validates and renders a model, but resolved output may contain sensitive environment values.

My notes: _____

SC-421 · Compose · Bring up a multi-service app

Situation: A local app has a Compose file and must be started in a repeatable way.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Validate the Compose model.

```
docker compose config
```

Verify / observe: Resolve syntax and environment interpolation issues first.

Step 2 — Build and start services in the background.

```
docker compose up -d --build
```

Verify / observe: Wait for the services to be created.

Step 3 — Check service and container state.

```
docker compose ps --all
```

Verify / observe: Look for exited services or missing replicas.

Step 4 — Read logs for a specific service.

```
docker compose logs --tail 80 api
```

Verify / observe: Replace api with a service name from the Compose model.

Final conclusion: Which service is healthy, which is failing, and what evidence supports your conclusion?

Concept to retain: Compose coordinates related services; state and logs should be checked after startup.

My notes: _____

SC-422 · Compose · Bring up a multi-service app

Situation: A local app has a Compose file and must be started in a repeatable way.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Validate the Compose model.

```
docker compose config
```

Verify / observe: Resolve syntax and environment interpolation issues first.

Step 2 — Build and start services in the background.

```
docker compose up -d --build
```

Verify / observe: Wait for the services to be created.

Step 3 — Check service and container state.

```
docker compose ps --all
```

Verify / observe: Look for exited services or missing replicas.

Step 4 — Read logs for a specific service.

```
docker compose logs --tail 80 api
```

Verify / observe: Replace api with a service name from the Compose model.

Final conclusion: Which service is healthy, which is failing, and what evidence supports your conclusion?

Concept to retain: Compose coordinates related services; state and logs should be checked after startup.

My notes: _____

SC-423 · Compose · Bring up a multi-service app

Situation: A local app has a Compose file and must be started in a repeatable way.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Validate the Compose model.

```
docker compose config
```

Verify / observe: Resolve syntax and environment interpolation issues first.

Step 2 — Build and start services in the background.

```
docker compose up -d --build
```

Verify / observe: Wait for the services to be created.

Step 3 — Check service and container state.

```
docker compose ps --all
```

Verify / observe: Look for exited services or missing replicas.

Step 4 — Read logs for a specific service.

```
docker compose logs --tail 80 api
```

Verify / observe: Replace api with a service name from the Compose model.

Final conclusion: Which service is healthy, which is failing, and what evidence supports your conclusion?

Concept to retain: Compose coordinates related services; state and logs should be checked after startup.

My notes: _____

SC-424 · Compose · Bring up a multi-service app

Situation: A local app has a Compose file and must be started in a repeatable way.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Validate the Compose model.

```
docker compose config
```

Verify / observe: Resolve syntax and environment interpolation issues first.

Step 2 — Build and start services in the background.

```
docker compose up -d --build
```

Verify / observe: Wait for the services to be created.

Step 3 — Check service and container state.

```
docker compose ps --all
```

Verify / observe: Look for exited services or missing replicas.

Step 4 — Read logs for a specific service.

```
docker compose logs --tail 80 api
```

Verify / observe: Replace api with a service name from the Compose model.

Final conclusion: Which service is healthy, which is failing, and what evidence supports your conclusion?

Concept to retain: Compose coordinates related services; state and logs should be checked after startup.

My notes: _____

SC-425 · Compose · Bring up a multi-service app

Situation: A local app has a Compose file and must be started in a repeatable way.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Validate the Compose model.

```
docker compose config
```

Verify / observe: Resolve syntax and environment interpolation issues first.

Step 2 — Build and start services in the background.

```
docker compose up -d --build
```

Verify / observe: Wait for the services to be created.

Step 3 — Check service and container state.

```
docker compose ps --all
```

Verify / observe: Look for exited services or missing replicas.

Step 4 — Read logs for a specific service.

```
docker compose logs --tail 80 api
```

Verify / observe: Replace api with a service name from the Compose model.

Final conclusion: Which service is healthy, which is failing, and what evidence supports your conclusion?

Concept to retain: Compose coordinates related services; state and logs should be checked after startup.

My notes: _____

SC-426 · Compose · Bring up a multi-service app

Situation: A local app has a Compose file and must be started in a repeatable way.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Validate the Compose model.

```
docker compose config
```

Verify / observe: Resolve syntax and environment interpolation issues first.

Step 2 — Build and start services in the background.

```
docker compose up -d --build
```

Verify / observe: Wait for the services to be created.

Step 3 — Check service and container state.

```
docker compose ps --all
```

Verify / observe: Look for exited services or missing replicas.

Step 4 — Read logs for a specific service.

```
docker compose logs --tail 80 api
```

Verify / observe: Replace api with a service name from the Compose model.

Final conclusion: Which service is healthy, which is failing, and what evidence supports your conclusion?

Concept to retain: Compose coordinates related services; state and logs should be checked after startup.

My notes: _____

SC-427 · Compose · Bring up a multi-service app

Situation: A local app has a Compose file and must be started in a repeatable way.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Validate the Compose model.

```
docker compose config
```

Verify / observe: Resolve syntax and environment interpolation issues first.

Step 2 — Build and start services in the background.

```
docker compose up -d --build
```

Verify / observe: Wait for the services to be created.

Step 3 — Check service and container state.

```
docker compose ps --all
```

Verify / observe: Look for exited services or missing replicas.

Step 4 — Read logs for a specific service.

```
docker compose logs --tail 80 api
```

Verify / observe: Replace api with a service name from the Compose model.

Final conclusion: Which service is healthy, which is failing, and what evidence supports your conclusion?

Concept to retain: Compose coordinates related services; state and logs should be checked after startup.

My notes: _____

SC-428 · Compose · Bring up a multi-service app

Situation: A local app has a Compose file and must be started in a repeatable way.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Validate the Compose model.

```
docker compose config
```

Verify / observe: Resolve syntax and environment interpolation issues first.

Step 2 — Build and start services in the background.

```
docker compose up -d --build
```

Verify / observe: Wait for the services to be created.

Step 3 — Check service and container state.

```
docker compose ps --all
```

Verify / observe: Look for exited services or missing replicas.

Step 4 — Read logs for a specific service.

```
docker compose logs --tail 80 api
```

Verify / observe: Replace api with a service name from the Compose model.

Final conclusion: Which service is healthy, which is failing, and what evidence supports your conclusion?

Concept to retain: Compose coordinates related services; state and logs should be checked after startup.

My notes: _____

SC-429 · Compose · Bring up a multi-service app

Situation: A local app has a Compose file and must be started in a repeatable way.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Validate the Compose model.

```
docker compose config
```

Verify / observe: Resolve syntax and environment interpolation issues first.

Step 2 — Build and start services in the background.

```
docker compose up -d --build
```

Verify / observe: Wait for the services to be created.

Step 3 — Check service and container state.

```
docker compose ps --all
```

Verify / observe: Look for exited services or missing replicas.

Step 4 — Read logs for a specific service.

```
docker compose logs --tail 80 api
```

Verify / observe: Replace api with a service name from the Compose model.

Final conclusion: Which service is healthy, which is failing, and what evidence supports your conclusion?

Concept to retain: Compose coordinates related services; state and logs should be checked after startup.

My notes: _____

SC-430 · Compose · Bring up a multi-service app

Situation: A local app has a Compose file and must be started in a repeatable way.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Validate the Compose model.

```
docker compose config
```

Verify / observe: Resolve syntax and environment interpolation issues first.

Step 2 — Build and start services in the background.

```
docker compose up -d --build
```

Verify / observe: Wait for the services to be created.

Step 3 — Check service and container state.

```
docker compose ps --all
```

Verify / observe: Look for exited services or missing replicas.

Step 4 — Read logs for a specific service.

```
docker compose logs --tail 80 api
```

Verify / observe: Replace api with a service name from the Compose model.

Final conclusion: Which service is healthy, which is failing, and what evidence supports your conclusion?

Concept to retain: Compose coordinates related services; state and logs should be checked after startup.

My notes: _____

SC-431 · Compose · Diagnose a service that exits

Situation: One Compose service exits after startup while the rest remain running.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List all services and container states.

```
docker compose ps --all
```

Verify / observe: Identify the service that exited.

Step 2 — Read its logs.

```
docker compose logs --tail 100 worker
```

Verify / observe: Replace worker with the failing service name.

Step 3 — Inspect the rendered service configuration.

```
docker compose config
```

Verify / observe: Check command, environment, volumes, dependencies, and profiles.

Final conclusion: Is the exit expected for a one-shot job, or is it an application failure? Explain how you know.

Concept to retain: An exited container is not automatically an error; service intent and exit code matter.

My notes: _____

SC-432 · Compose · Diagnose a service that exits

Situation: One Compose service exits after startup while the rest remain running.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List all services and container states.

```
docker compose ps --all
```

Verify / observe: Identify the service that exited.

Step 2 — Read its logs.

```
docker compose logs --tail 100 worker
```

Verify / observe: Replace worker with the failing service name.

Step 3 — Inspect the rendered service configuration.

```
docker compose config
```

Verify / observe: Check command, environment, volumes, dependencies, and profiles.

Final conclusion: Is the exit expected for a one-shot job, or is it an application failure? Explain how you know.

Concept to retain: An exited container is not automatically an error; service intent and exit code matter.

My notes: _____

SC-433 · Compose · Diagnose a service that exits

Situation: One Compose service exits after startup while the rest remain running.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List all services and container states.

```
docker compose ps --all
```

Verify / observe: Identify the service that exited.

Step 2 — Read its logs.

```
docker compose logs --tail 100 worker
```

Verify / observe: Replace worker with the failing service name.

Step 3 — Inspect the rendered service configuration.

```
docker compose config
```

Verify / observe: Check command, environment, volumes, dependencies, and profiles.

Final conclusion: Is the exit expected for a one-shot job, or is it an application failure? Explain how you know.

Concept to retain: An exited container is not automatically an error; service intent and exit code matter.

My notes: _____

SC-434 · Compose · Diagnose a service that exits

Situation: One Compose service exits after startup while the rest remain running.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List all services and container states.

```
docker compose ps --all
```

Verify / observe: Identify the service that exited.

Step 2 — Read its logs.

```
docker compose logs --tail 100 worker
```

Verify / observe: Replace worker with the failing service name.

Step 3 — Inspect the rendered service configuration.

```
docker compose config
```

Verify / observe: Check command, environment, volumes, dependencies, and profiles.

Final conclusion: Is the exit expected for a one-shot job, or is it an application failure? Explain how you know.

Concept to retain: An exited container is not automatically an error; service intent and exit code matter.

My notes: _____

SC-435 · Compose · Diagnose a service that exits

Situation: One Compose service exits after startup while the rest remain running.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List all services and container states.

```
docker compose ps --all
```

Verify / observe: Identify the service that exited.

Step 2 — Read its logs.

```
docker compose logs --tail 100 worker
```

Verify / observe: Replace worker with the failing service name.

Step 3 — Inspect the rendered service configuration.

```
docker compose config
```

Verify / observe: Check command, environment, volumes, dependencies, and profiles.

Final conclusion: Is the exit expected for a one-shot job, or is it an application failure? Explain how you know.

Concept to retain: An exited container is not automatically an error; service intent and exit code matter.

My notes: _____

SC-436 · Compose · Diagnose a service that exits

Situation: One Compose service exits after startup while the rest remain running.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List all services and container states.

```
docker compose ps --all
```

Verify / observe: Identify the service that exited.

Step 2 — Read its logs.

```
docker compose logs --tail 100 worker
```

Verify / observe: Replace worker with the failing service name.

Step 3 — Inspect the rendered service configuration.

```
docker compose config
```

Verify / observe: Check command, environment, volumes, dependencies, and profiles.

Final conclusion: Is the exit expected for a one-shot job, or is it an application failure? Explain how you know.

Concept to retain: An exited container is not automatically an error; service intent and exit code matter.

My notes: _____

SC-437 · Compose · Diagnose a service that exits

Situation: One Compose service exits after startup while the rest remain running.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List all services and container states.

```
docker compose ps --all
```

Verify / observe: Identify the service that exited.

Step 2 — Read its logs.

```
docker compose logs --tail 100 worker
```

Verify / observe: Replace worker with the failing service name.

Step 3 — Inspect the rendered service configuration.

```
docker compose config
```

Verify / observe: Check command, environment, volumes, dependencies, and profiles.

Final conclusion: Is the exit expected for a one-shot job, or is it an application failure? Explain how you know.

Concept to retain: An exited container is not automatically an error; service intent and exit code matter.

My notes: _____

SC-438 · Compose · Diagnose a service that exits

Situation: One Compose service exits after startup while the rest remain running.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List all services and container states.

```
docker compose ps --all
```

Verify / observe: Identify the service that exited.

Step 2 — Read its logs.

```
docker compose logs --tail 100 worker
```

Verify / observe: Replace worker with the failing service name.

Step 3 — Inspect the rendered service configuration.

```
docker compose config
```

Verify / observe: Check command, environment, volumes, dependencies, and profiles.

Final conclusion: Is the exit expected for a one-shot job, or is it an application failure? Explain how you know.

Concept to retain: An exited container is not automatically an error; service intent and exit code matter.

My notes: _____

SC-439 · Compose · Diagnose a service that exits

Situation: One Compose service exits after startup while the rest remain running.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List all services and container states.

```
docker compose ps --all
```

Verify / observe: Identify the service that exited.

Step 2 — Read its logs.

```
docker compose logs --tail 100 worker
```

Verify / observe: Replace worker with the failing service name.

Step 3 — Inspect the rendered service configuration.

```
docker compose config
```

Verify / observe: Check command, environment, volumes, dependencies, and profiles.

Final conclusion: Is the exit expected for a one-shot job, or is it an application failure? Explain how you know.

Concept to retain: An exited container is not automatically an error; service intent and exit code matter.

My notes: _____

SC-440 · Compose · Diagnose a service that exits

Situation: One Compose service exits after startup while the rest remain running.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List all services and container states.

```
docker compose ps --all
```

Verify / observe: Identify the service that exited.

Step 2 — Read its logs.

```
docker compose logs --tail 100 worker
```

Verify / observe: Replace worker with the failing service name.

Step 3 — Inspect the rendered service configuration.

```
docker compose config
```

Verify / observe: Check command, environment, volumes, dependencies, and profiles.

Final conclusion: Is the exit expected for a one-shot job, or is it an application failure? Explain how you know.

Concept to retain: An exited container is not automatically an error; service intent and exit code matter.

My notes: _____

SC-441 · Compose · Execute a command in a running service

Situation: A running API service needs a quick diagnostic without recreating the service.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the service is running.

```
docker compose ps
```

Verify / observe: Identify the exact service name.

Step 2 — Run a non-interactive command inside it.

```
docker compose exec api id
```

Verify / observe: Replace api with a running service that has the id utility.

Step 3 — Inspect logs after the check.

```
docker compose logs --tail 50 api
```

Verify / observe: Check whether the diagnostic affected the service.

Final conclusion: Why use `compose exec` instead of creating a new one-off container?

Concept to retain: Exec targets a running service container; run creates a one-off container from a service definition.

My notes: _____

SC-442 · Compose · Execute a command in a running service

Situation: A running API service needs a quick diagnostic without recreating the service.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the service is running.

```
docker compose ps
```

Verify / observe: Identify the exact service name.

Step 2 — Run a non-interactive command inside it.

```
docker compose exec api id
```

Verify / observe: Replace api with a running service that has the id utility.

Step 3 — Inspect logs after the check.

```
docker compose logs --tail 50 api
```

Verify / observe: Check whether the diagnostic affected the service.

Final conclusion: Why use `compose exec` instead of creating a new one-off container?

Concept to retain: Exec targets a running service container; run creates a one-off container from a service definition.

My notes: _____

SC-443 · Compose · Execute a command in a running service

Situation: A running API service needs a quick diagnostic without recreating the service.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the service is running.

```
docker compose ps
```

Verify / observe: Identify the exact service name.

Step 2 — Run a non-interactive command inside it.

```
docker compose exec api id
```

Verify / observe: Replace api with a running service that has the id utility.

Step 3 — Inspect logs after the check.

```
docker compose logs --tail 50 api
```

Verify / observe: Check whether the diagnostic affected the service.

Final conclusion: Why use `compose exec` instead of creating a new one-off container?

Concept to retain: Exec targets a running service container; run creates a one-off container from a service definition.

My notes: _____

SC-444 · Compose · Execute a command in a running service

Situation: A running API service needs a quick diagnostic without recreating the service.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the service is running.

```
docker compose ps
```

Verify / observe: Identify the exact service name.

Step 2 — Run a non-interactive command inside it.

```
docker compose exec api id
```

Verify / observe: Replace api with a running service that has the id utility.

Step 3 — Inspect logs after the check.

```
docker compose logs --tail 50 api
```

Verify / observe: Check whether the diagnostic affected the service.

Final conclusion: Why use `compose exec` instead of creating a new one-off container?

Concept to retain: Exec targets a running service container; run creates a one-off container from a service definition.

My notes: _____

SC-445 · Compose · Execute a command in a running service

Situation: A running API service needs a quick diagnostic without recreating the service.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the service is running.

```
docker compose ps
```

Verify / observe: Identify the exact service name.

Step 2 — Run a non-interactive command inside it.

```
docker compose exec api id
```

Verify / observe: Replace api with a running service that has the id utility.

Step 3 — Inspect logs after the check.

```
docker compose logs --tail 50 api
```

Verify / observe: Check whether the diagnostic affected the service.

Final conclusion: Why use `compose exec` instead of creating a new one-off container?

Concept to retain: Exec targets a running service container; run creates a one-off container from a service definition.

My notes: _____

SC-446 · Compose · Execute a command in a running service

Situation: A running API service needs a quick diagnostic without recreating the service.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the service is running.

```
docker compose ps
```

Verify / observe: Identify the exact service name.

Step 2 — Run a non-interactive command inside it.

```
docker compose exec api id
```

Verify / observe: Replace api with a running service that has the id utility.

Step 3 — Inspect logs after the check.

```
docker compose logs --tail 50 api
```

Verify / observe: Check whether the diagnostic affected the service.

Final conclusion: Why use `compose exec` instead of creating a new one-off container?

Concept to retain: Exec targets a running service container; run creates a one-off container from a service definition.

My notes: _____

SC-447 · Compose · Execute a command in a running service

Situation: A running API service needs a quick diagnostic without recreating the service.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the service is running.

```
docker compose ps
```

Verify / observe: Identify the exact service name.

Step 2 — Run a non-interactive command inside it.

```
docker compose exec api id
```

Verify / observe: Replace api with a running service that has the id utility.

Step 3 — Inspect logs after the check.

```
docker compose logs --tail 50 api
```

Verify / observe: Check whether the diagnostic affected the service.

Final conclusion: Why use `compose exec` instead of creating a new one-off container?

Concept to retain: Exec targets a running service container; run creates a one-off container from a service definition.

My notes: _____

SC-448 · Compose · Execute a command in a running service

Situation: A running API service needs a quick diagnostic without recreating the service.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the service is running.

```
docker compose ps
```

Verify / observe: Identify the exact service name.

Step 2 — Run a non-interactive command inside it.

```
docker compose exec api id
```

Verify / observe: Replace api with a running service that has the id utility.

Step 3 — Inspect logs after the check.

```
docker compose logs --tail 50 api
```

Verify / observe: Check whether the diagnostic affected the service.

Final conclusion: Why use `compose exec` instead of creating a new one-off container?

Concept to retain: Exec targets a running service container; run creates a one-off container from a service definition.

My notes: _____

SC-449 · Compose · Execute a command in a running service

Situation: A running API service needs a quick diagnostic without recreating the service.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the service is running.

```
docker compose ps
```

Verify / observe: Identify the exact service name.

Step 2 — Run a non-interactive command inside it.

```
docker compose exec api id
```

Verify / observe: Replace api with a running service that has the id utility.

Step 3 — Inspect logs after the check.

```
docker compose logs --tail 50 api
```

Verify / observe: Check whether the diagnostic affected the service.

Final conclusion: Why use `compose exec` instead of creating a new one-off container?

Concept to retain: Exec targets a running service container; run creates a one-off container from a service definition.

My notes: _____

SC-450 · Compose · Execute a command in a running service

Situation: A running API service needs a quick diagnostic without recreating the service.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the service is running.

```
docker compose ps
```

Verify / observe: Identify the exact service name.

Step 2 — Run a non-interactive command inside it.

```
docker compose exec api id
```

Verify / observe: Replace api with a running service that has the id utility.

Step 3 — Inspect logs after the check.

```
docker compose logs --tail 50 api
```

Verify / observe: Check whether the diagnostic affected the service.

Final conclusion: Why use `compose exec` instead of creating a new one-off container?

Concept to retain: Exec targets a running service container; run creates a one-off container from a service definition.

My notes: _____

SC-451 · Compose · Run a one-off migration or maintenance job

Situation: A database-backed app needs a one-time task using the same service configuration.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review service names and definitions.

```
docker compose config --services
```

Verify / observe: Identify the worker or migration service.

Step 2 — Run the one-off command and remove the temporary container afterward.

```
docker compose run --rm worker sh -c 'echo migration-check-1'
```

Verify / observe: Replace worker with a suitable service; the command is a safe placeholder.

Step 3 — Check the regular project state.

```
docker compose ps --all
```

Verify / observe: Confirm the one-off container was removed and long-running services remain.

Final conclusion: Why is a one-off `run` often preferable to changing the service's normal startup command?

Concept to retain: One-off jobs can reuse service configuration without altering the normal service lifecycle.

My notes: _____

SC-452 · Compose · Run a one-off migration or maintenance job

Situation: A database-backed app needs a one-time task using the same service configuration.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review service names and definitions.

```
docker compose config --services
```

Verify / observe: Identify the worker or migration service.

Step 2 — Run the one-off command and remove the temporary container afterward.

```
docker compose run --rm worker sh -c 'echo migration-check-2'
```

Verify / observe: Replace worker with a suitable service; the command is a safe placeholder.

Step 3 — Check the regular project state.

```
docker compose ps --all
```

Verify / observe: Confirm the one-off container was removed and long-running services remain.

Final conclusion: Why is a one-off `run` often preferable to changing the service's normal startup command?

Concept to retain: One-off jobs can reuse service configuration without altering the normal service lifecycle.

My notes: _____

SC-453 · Compose · Run a one-off migration or maintenance job

Situation: A database-backed app needs a one-time task using the same service configuration.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review service names and definitions.

```
docker compose config --services
```

Verify / observe: Identify the worker or migration service.

Step 2 — Run the one-off command and remove the temporary container afterward.

```
docker compose run --rm worker sh -c 'echo migration-check-3'
```

Verify / observe: Replace worker with a suitable service; the command is a safe placeholder.

Step 3 — Check the regular project state.

```
docker compose ps --all
```

Verify / observe: Confirm the one-off container was removed and long-running services remain.

Final conclusion: Why is a one-off `run` often preferable to changing the service's normal startup command?

Concept to retain: One-off jobs can reuse service configuration without altering the normal service lifecycle.

My notes: _____

SC-454 · Compose · Run a one-off migration or maintenance job

Situation: A database-backed app needs a one-time task using the same service configuration.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review service names and definitions.

```
docker compose config --services
```

Verify / observe: Identify the worker or migration service.

Step 2 — Run the one-off command and remove the temporary container afterward.

```
docker compose run --rm worker sh -c 'echo migration-check-4'
```

Verify / observe: Replace worker with a suitable service; the command is a safe placeholder.

Step 3 — Check the regular project state.

```
docker compose ps --all
```

Verify / observe: Confirm the one-off container was removed and long-running services remain.

Final conclusion: Why is a one-off `run` often preferable to changing the service's normal startup command?

Concept to retain: One-off jobs can reuse service configuration without altering the normal service lifecycle.

My notes: _____

SC-455 · Compose · Run a one-off migration or maintenance job

Situation: A database-backed app needs a one-time task using the same service configuration.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review service names and definitions.

```
docker compose config --services
```

Verify / observe: Identify the worker or migration service.

Step 2 — Run the one-off command and remove the temporary container afterward.

```
docker compose run --rm worker sh -c 'echo migration-check-5'
```

Verify / observe: Replace worker with a suitable service; the command is a safe placeholder.

Step 3 — Check the regular project state.

```
docker compose ps --all
```

Verify / observe: Confirm the one-off container was removed and long-running services remain.

Final conclusion: Why is a one-off `run` often preferable to changing the service's normal startup command?

Concept to retain: One-off jobs can reuse service configuration without altering the normal service lifecycle.

My notes: _____

SC-456 · Compose · Run a one-off migration or maintenance job

Situation: A database-backed app needs a one-time task using the same service configuration.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review service names and definitions.

```
docker compose config --services
```

Verify / observe: Identify the worker or migration service.

Step 2 — Run the one-off command and remove the temporary container afterward.

```
docker compose run --rm worker sh -c 'echo migration-check-6'
```

Verify / observe: Replace worker with a suitable service; the command is a safe placeholder.

Step 3 — Check the regular project state.

```
docker compose ps --all
```

Verify / observe: Confirm the one-off container was removed and long-running services remain.

Final conclusion: Why is a one-off `run` often preferable to changing the service's normal startup command?

Concept to retain: One-off jobs can reuse service configuration without altering the normal service lifecycle.

My notes: _____

SC-457 · Compose · Run a one-off migration or maintenance job

Situation: A database-backed app needs a one-time task using the same service configuration.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review service names and definitions.

```
docker compose config --services
```

Verify / observe: Identify the worker or migration service.

Step 2 — Run the one-off command and remove the temporary container afterward.

```
docker compose run --rm worker sh -c 'echo migration-check-7'
```

Verify / observe: Replace worker with a suitable service; the command is a safe placeholder.

Step 3 — Check the regular project state.

```
docker compose ps --all
```

Verify / observe: Confirm the one-off container was removed and long-running services remain.

Final conclusion: Why is a one-off `run` often preferable to changing the service's normal startup command?

Concept to retain: One-off jobs can reuse service configuration without altering the normal service lifecycle.

My notes: _____

SC-458 · Compose · Run a one-off migration or maintenance job

Situation: A database-backed app needs a one-time task using the same service configuration.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review service names and definitions.

```
docker compose config --services
```

Verify / observe: Identify the worker or migration service.

Step 2 — Run the one-off command and remove the temporary container afterward.

```
docker compose run --rm worker sh -c 'echo migration-check-8'
```

Verify / observe: Replace worker with a suitable service; the command is a safe placeholder.

Step 3 — Check the regular project state.

```
docker compose ps --all
```

Verify / observe: Confirm the one-off container was removed and long-running services remain.

Final conclusion: Why is a one-off `run` often preferable to changing the service's normal startup command?

Concept to retain: One-off jobs can reuse service configuration without altering the normal service lifecycle.

My notes: _____

SC-459 · Compose · Run a one-off migration or maintenance job

Situation: A database-backed app needs a one-time task using the same service configuration.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review service names and definitions.

```
docker compose config --services
```

Verify / observe: Identify the worker or migration service.

Step 2 — Run the one-off command and remove the temporary container afterward.

```
docker compose run --rm worker sh -c 'echo migration-check-9'
```

Verify / observe: Replace worker with a suitable service; the command is a safe placeholder.

Step 3 — Check the regular project state.

```
docker compose ps --all
```

Verify / observe: Confirm the one-off container was removed and long-running services remain.

Final conclusion: Why is a one-off `run` often preferable to changing the service's normal startup command?

Concept to retain: One-off jobs can reuse service configuration without altering the normal service lifecycle.

My notes: _____

SC-460 · Compose · Run a one-off migration or maintenance job

Situation: A database-backed app needs a one-time task using the same service configuration.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Review service names and definitions.

```
docker compose config --services
```

Verify / observe: Identify the worker or migration service.

Step 2 — Run the one-off command and remove the temporary container afterward.

```
docker compose run --rm worker sh -c 'echo migration-check-10'
```

Verify / observe: Replace worker with a suitable service; the command is a safe placeholder.

Step 3 — Check the regular project state.

```
docker compose ps --all
```

Verify / observe: Confirm the one-off container was removed and long-running services remain.

Final conclusion: Why is a one-off `run` often preferable to changing the service's normal startup command?

Concept to retain: One-off jobs can reuse service configuration without altering the normal service lifecycle.

My notes: _____

SC-461 · Compose · Update a Compose service after a configuration change

Situation: A service definition or image changed and you need to recreate the intended service.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Validate the updated model.

```
docker compose config
```

Verify / observe: Confirm the service and image or build settings are correct.

Step 2 — Rebuild and apply the update.

```
docker compose up -d --build api
```

Verify / observe: Replace api with the service being changed.

Step 3 — Check the new service state and logs.

```
docker compose ps && docker compose logs --tail 50 api
```

Verify / observe: Confirm the updated container is running and inspect startup output.

Final conclusion: What evidence shows that the updated version is active?

Concept to retain: Compose up reconciles the project with the current model; verify the resulting state.

My notes: _____

SC-462 · Compose · Update a Compose service after a configuration change

Situation: A service definition or image changed and you need to recreate the intended service.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Validate the updated model.

```
docker compose config
```

Verify / observe: Confirm the service and image or build settings are correct.

Step 2 — Rebuild and apply the update.

```
docker compose up -d --build api
```

Verify / observe: Replace api with the service being changed.

Step 3 — Check the new service state and logs.

```
docker compose ps && docker compose logs --tail 50 api
```

Verify / observe: Confirm the updated container is running and inspect startup output.

Final conclusion: What evidence shows that the updated version is active?

Concept to retain: Compose up reconciles the project with the current model; verify the resulting state.

My notes: _____

SC-463 · Compose · Update a Compose service after a configuration change

Situation: A service definition or image changed and you need to recreate the intended service.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Validate the updated model.

```
docker compose config
```

Verify / observe: Confirm the service and image or build settings are correct.

Step 2 — Rebuild and apply the update.

```
docker compose up -d --build api
```

Verify / observe: Replace api with the service being changed.

Step 3 — Check the new service state and logs.

```
docker compose ps && docker compose logs --tail 50 api
```

Verify / observe: Confirm the updated container is running and inspect startup output.

Final conclusion: What evidence shows that the updated version is active?

Concept to retain: Compose up reconciles the project with the current model; verify the resulting state.

My notes: _____

SC-464 · Compose · Update a Compose service after a configuration change

Situation: A service definition or image changed and you need to recreate the intended service.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Validate the updated model.

```
docker compose config
```

Verify / observe: Confirm the service and image or build settings are correct.

Step 2 — Rebuild and apply the update.

```
docker compose up -d --build api
```

Verify / observe: Replace api with the service being changed.

Step 3 — Check the new service state and logs.

```
docker compose ps && docker compose logs --tail 50 api
```

Verify / observe: Confirm the updated container is running and inspect startup output.

Final conclusion: What evidence shows that the updated version is active?

Concept to retain: Compose up reconciles the project with the current model; verify the resulting state.

My notes: _____

SC-465 · Compose · Update a Compose service after a configuration change

Situation: A service definition or image changed and you need to recreate the intended service.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Validate the updated model.

```
docker compose config
```

Verify / observe: Confirm the service and image or build settings are correct.

Step 2 — Rebuild and apply the update.

```
docker compose up -d --build api
```

Verify / observe: Replace api with the service being changed.

Step 3 — Check the new service state and logs.

```
docker compose ps && docker compose logs --tail 50 api
```

Verify / observe: Confirm the updated container is running and inspect startup output.

Final conclusion: What evidence shows that the updated version is active?

Concept to retain: Compose up reconciles the project with the current model; verify the resulting state.

My notes: _____

SC-466 · Compose · Update a Compose service after a configuration change

Situation: A service definition or image changed and you need to recreate the intended service.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Validate the updated model.

```
docker compose config
```

Verify / observe: Confirm the service and image or build settings are correct.

Step 2 — Rebuild and apply the update.

```
docker compose up -d --build api
```

Verify / observe: Replace api with the service being changed.

Step 3 — Check the new service state and logs.

```
docker compose ps && docker compose logs --tail 50 api
```

Verify / observe: Confirm the updated container is running and inspect startup output.

Final conclusion: What evidence shows that the updated version is active?

Concept to retain: Compose up reconciles the project with the current model; verify the resulting state.

My notes: _____

SC-467 · Compose · Update a Compose service after a configuration change

Situation: A service definition or image changed and you need to recreate the intended service.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Validate the updated model.

```
docker compose config
```

Verify / observe: Confirm the service and image or build settings are correct.

Step 2 — Rebuild and apply the update.

```
docker compose up -d --build api
```

Verify / observe: Replace api with the service being changed.

Step 3 — Check the new service state and logs.

```
docker compose ps && docker compose logs --tail 50 api
```

Verify / observe: Confirm the updated container is running and inspect startup output.

Final conclusion: What evidence shows that the updated version is active?

Concept to retain: Compose up reconciles the project with the current model; verify the resulting state.

My notes: _____

SC-468 · Compose · Update a Compose service after a configuration change

Situation: A service definition or image changed and you need to recreate the intended service.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Validate the updated model.

```
docker compose config
```

Verify / observe: Confirm the service and image or build settings are correct.

Step 2 — Rebuild and apply the update.

```
docker compose up -d --build api
```

Verify / observe: Replace api with the service being changed.

Step 3 — Check the new service state and logs.

```
docker compose ps && docker compose logs --tail 50 api
```

Verify / observe: Confirm the updated container is running and inspect startup output.

Final conclusion: What evidence shows that the updated version is active?

Concept to retain: Compose up reconciles the project with the current model; verify the resulting state.

My notes: _____

SC-469 · Compose · Update a Compose service after a configuration change

Situation: A service definition or image changed and you need to recreate the intended service.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Validate the updated model.

```
docker compose config
```

Verify / observe: Confirm the service and image or build settings are correct.

Step 2 — Rebuild and apply the update.

```
docker compose up -d --build api
```

Verify / observe: Replace api with the service being changed.

Step 3 — Check the new service state and logs.

```
docker compose ps && docker compose logs --tail 50 api
```

Verify / observe: Confirm the updated container is running and inspect startup output.

Final conclusion: What evidence shows that the updated version is active?

Concept to retain: Compose up reconciles the project with the current model; verify the resulting state.

My notes: _____

SC-470 · Compose · Update a Compose service after a configuration change

Situation: A service definition or image changed and you need to recreate the intended service.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Validate the updated model.

```
docker compose config
```

Verify / observe: Confirm the service and image or build settings are correct.

Step 2 — Rebuild and apply the update.

```
docker compose up -d --build api
```

Verify / observe: Replace api with the service being changed.

Step 3 — Check the new service state and logs.

```
docker compose ps && docker compose logs --tail 50 api
```

Verify / observe: Confirm the updated container is running and inspect startup output.

Final conclusion: What evidence shows that the updated version is active?

Concept to retain: Compose up reconciles the project with the current model; verify the resulting state.

My notes: _____

SC-471 · Compose · Tear down a lab Compose project safely

Situation: A practice deployment is complete and its containers and network should be removed without deleting persistent data.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Record current project resources.

```
docker compose ps --all
```

Verify / observe: Confirm the directory/project identity.

Step 2 — Stop and remove project containers and default network.

```
docker compose down
```

Verify / observe: Do not add --volumes unless you intentionally want to remove project volumes.

Step 3 — Verify project containers are gone.

```
docker compose ps --all
```

Verify / observe: Confirm the project has no remaining containers.

Final conclusion: Which resources are removed by default, and which persistent resources are retained?

Concept to retain: Default teardown is safer than adding volume-removal flags without a specific reason.

My notes: _____

SC-472 · Compose · Tear down a lab Compose project safely

Situation: A practice deployment is complete and its containers and network should be removed without deleting persistent data.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Record current project resources.

```
docker compose ps --all
```

Verify / observe: Confirm the directory/project identity.

Step 2 — Stop and remove project containers and default network.

```
docker compose down
```

Verify / observe: Do not add --volumes unless you intentionally want to remove project volumes.

Step 3 — Verify project containers are gone.

```
docker compose ps --all
```

Verify / observe: Confirm the project has no remaining containers.

Final conclusion: Which resources are removed by default, and which persistent resources are retained?

Concept to retain: Default teardown is safer than adding volume-removal flags without a specific reason.

My notes: _____

SC-473 · Compose · Tear down a lab Compose project safely

Situation: A practice deployment is complete and its containers and network should be removed without deleting persistent data.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Record current project resources.

```
docker compose ps --all
```

Verify / observe: Confirm the directory/project identity.

Step 2 — Stop and remove project containers and default network.

```
docker compose down
```

Verify / observe: Do not add --volumes unless you intentionally want to remove project volumes.

Step 3 — Verify project containers are gone.

```
docker compose ps --all
```

Verify / observe: Confirm the project has no remaining containers.

Final conclusion: Which resources are removed by default, and which persistent resources are retained?

Concept to retain: Default teardown is safer than adding volume-removal flags without a specific reason.

My notes: _____

SC-474 · Compose · Tear down a lab Compose project safely

Situation: A practice deployment is complete and its containers and network should be removed without deleting persistent data.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Record current project resources.

```
docker compose ps --all
```

Verify / observe: Confirm the directory/project identity.

Step 2 — Stop and remove project containers and default network.

```
docker compose down
```

Verify / observe: Do not add --volumes unless you intentionally want to remove project volumes.

Step 3 — Verify project containers are gone.

```
docker compose ps --all
```

Verify / observe: Confirm the project has no remaining containers.

Final conclusion: Which resources are removed by default, and which persistent resources are retained?

Concept to retain: Default teardown is safer than adding volume-removal flags without a specific reason.

My notes: _____

SC-475 · Compose · Tear down a lab Compose project safely

Situation: A practice deployment is complete and its containers and network should be removed without deleting persistent data.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Record current project resources.

```
docker compose ps --all
```

Verify / observe: Confirm the directory/project identity.

Step 2 — Stop and remove project containers and default network.

```
docker compose down
```

Verify / observe: Do not add --volumes unless you intentionally want to remove project volumes.

Step 3 — Verify project containers are gone.

```
docker compose ps --all
```

Verify / observe: Confirm the project has no remaining containers.

Final conclusion: Which resources are removed by default, and which persistent resources are retained?

Concept to retain: Default teardown is safer than adding volume-removal flags without a specific reason.

My notes: _____

SC-476 · Compose · Tear down a lab Compose project safely

Situation: A practice deployment is complete and its containers and network should be removed without deleting persistent data.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Record current project resources.

```
docker compose ps --all
```

Verify / observe: Confirm the directory/project identity.

Step 2 — Stop and remove project containers and default network.

```
docker compose down
```

Verify / observe: Do not add --volumes unless you intentionally want to remove project volumes.

Step 3 — Verify project containers are gone.

```
docker compose ps --all
```

Verify / observe: Confirm the project has no remaining containers.

Final conclusion: Which resources are removed by default, and which persistent resources are retained?

Concept to retain: Default teardown is safer than adding volume-removal flags without a specific reason.

My notes: _____

SC-477 · Compose · Tear down a lab Compose project safely

Situation: A practice deployment is complete and its containers and network should be removed without deleting persistent data.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Record current project resources.

```
docker compose ps --all
```

Verify / observe: Confirm the directory/project identity.

Step 2 — Stop and remove project containers and default network.

```
docker compose down
```

Verify / observe: Do not add --volumes unless you intentionally want to remove project volumes.

Step 3 — Verify project containers are gone.

```
docker compose ps --all
```

Verify / observe: Confirm the project has no remaining containers.

Final conclusion: Which resources are removed by default, and which persistent resources are retained?

Concept to retain: Default teardown is safer than adding volume-removal flags without a specific reason.

My notes: _____

SC-478 · Compose · Tear down a lab Compose project safely

Situation: A practice deployment is complete and its containers and network should be removed without deleting persistent data.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Record current project resources.

```
docker compose ps --all
```

Verify / observe: Confirm the directory/project identity.

Step 2 — Stop and remove project containers and default network.

```
docker compose down
```

Verify / observe: Do not add --volumes unless you intentionally want to remove project volumes.

Step 3 — Verify project containers are gone.

```
docker compose ps --all
```

Verify / observe: Confirm the project has no remaining containers.

Final conclusion: Which resources are removed by default, and which persistent resources are retained?

Concept to retain: Default teardown is safer than adding volume-removal flags without a specific reason.

My notes: _____

SC-479 · Compose · Tear down a lab Compose project safely

Situation: A practice deployment is complete and its containers and network should be removed without deleting persistent data.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Record current project resources.

```
docker compose ps --all
```

Verify / observe: Confirm the directory/project identity.

Step 2 — Stop and remove project containers and default network.

```
docker compose down
```

Verify / observe: Do not add --volumes unless you intentionally want to remove project volumes.

Step 3 — Verify project containers are gone.

```
docker compose ps --all
```

Verify / observe: Confirm the project has no remaining containers.

Final conclusion: Which resources are removed by default, and which persistent resources are retained?

Concept to retain: Default teardown is safer than adding volume-removal flags without a specific reason.

My notes: _____

SC-480 · Compose · Tear down a lab Compose project safely

Situation: A practice deployment is complete and its containers and network should be removed without deleting persistent data.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Record current project resources.

```
docker compose ps --all
```

Verify / observe: Confirm the directory/project identity.

Step 2 — Stop and remove project containers and default network.

```
docker compose down
```

Verify / observe: Do not add --volumes unless you intentionally want to remove project volumes.

Step 3 — Verify project containers are gone.

```
docker compose ps --all
```

Verify / observe: Confirm the project has no remaining containers.

Final conclusion: Which resources are removed by default, and which persistent resources are retained?

Concept to retain: Default teardown is safer than adding volume-removal flags without a specific reason.

My notes: _____

SC-481 · Compose · Separate projects with project names

Situation: Two developers run similar Compose files and need resource names that do not collide.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start the first project with a unique project name.

```
docker compose -p trainee-1-a up -d
```

Verify / observe: Run from the directory containing the intended Compose file.

Step 2 — Start a second isolated project with another name.

```
docker compose -p trainee-1-b up -d
```

Verify / observe: Use the same file only if its services can safely run twice and ports do not collide.

Step 3 — List project resources and compare names.

```
docker ps -a
```

Verify / observe: Check that the two projects use distinct generated resource names.

Final conclusion: What other resource conflicts, such as published ports or fixed volume names, could still occur?

Concept to retain: Project names isolate generated resources but cannot eliminate every explicit-name or port conflict.

My notes: _____

SC-482 · Compose · Separate projects with project names

Situation: Two developers run similar Compose files and need resource names that do not collide.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start the first project with a unique project name.

```
docker compose -p trainee-2-a up -d
```

Verify / observe: Run from the directory containing the intended Compose file.

Step 2 — Start a second isolated project with another name.

```
docker compose -p trainee-2-b up -d
```

Verify / observe: Use the same file only if its services can safely run twice and ports do not collide.

Step 3 — List project resources and compare names.

```
docker ps -a
```

Verify / observe: Check that the two projects use distinct generated resource names.

Final conclusion: What other resource conflicts, such as published ports or fixed volume names, could still occur?

Concept to retain: Project names isolate generated resources but cannot eliminate every explicit-name or port conflict.

My notes: _____

SC-483 · Compose · Separate projects with project names

Situation: Two developers run similar Compose files and need resource names that do not collide.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start the first project with a unique project name.

```
docker compose -p trainee-3-a up -d
```

Verify / observe: Run from the directory containing the intended Compose file.

Step 2 — Start a second isolated project with another name.

```
docker compose -p trainee-3-b up -d
```

Verify / observe: Use the same file only if its services can safely run twice and ports do not collide.

Step 3 — List project resources and compare names.

```
docker ps -a
```

Verify / observe: Check that the two projects use distinct generated resource names.

Final conclusion: What other resource conflicts, such as published ports or fixed volume names, could still occur?

Concept to retain: Project names isolate generated resources but cannot eliminate every explicit-name or port conflict.

My notes: _____

SC-484 · Compose · Separate projects with project names

Situation: Two developers run similar Compose files and need resource names that do not collide.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start the first project with a unique project name.

```
docker compose -p trainee-4-a up -d
```

Verify / observe: Run from the directory containing the intended Compose file.

Step 2 — Start a second isolated project with another name.

```
docker compose -p trainee-4-b up -d
```

Verify / observe: Use the same file only if its services can safely run twice and ports do not collide.

Step 3 — List project resources and compare names.

```
docker ps -a
```

Verify / observe: Check that the two projects use distinct generated resource names.

Final conclusion: What other resource conflicts, such as published ports or fixed volume names, could still occur?

Concept to retain: Project names isolate generated resources but cannot eliminate every explicit-name or port conflict.

My notes: _____

SC-485 · Compose · Separate projects with project names

Situation: Two developers run similar Compose files and need resource names that do not collide.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start the first project with a unique project name.

```
docker compose -p trainee-5-a up -d
```

Verify / observe: Run from the directory containing the intended Compose file.

Step 2 — Start a second isolated project with another name.

```
docker compose -p trainee-5-b up -d
```

Verify / observe: Use the same file only if its services can safely run twice and ports do not collide.

Step 3 — List project resources and compare names.

```
docker ps -a
```

Verify / observe: Check that the two projects use distinct generated resource names.

Final conclusion: What other resource conflicts, such as published ports or fixed volume names, could still occur?

Concept to retain: Project names isolate generated resources but cannot eliminate every explicit-name or port conflict.

My notes: _____

SC-486 · Compose · Separate projects with project names

Situation: Two developers run similar Compose files and need resource names that do not collide.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start the first project with a unique project name.

```
docker compose -p trainee-6-a up -d
```

Verify / observe: Run from the directory containing the intended Compose file.

Step 2 — Start a second isolated project with another name.

```
docker compose -p trainee-6-b up -d
```

Verify / observe: Use the same file only if its services can safely run twice and ports do not collide.

Step 3 — List project resources and compare names.

```
docker ps -a
```

Verify / observe: Check that the two projects use distinct generated resource names.

Final conclusion: What other resource conflicts, such as published ports or fixed volume names, could still occur?

Concept to retain: Project names isolate generated resources but cannot eliminate every explicit-name or port conflict.

My notes: _____

SC-487 · Compose · Separate projects with project names

Situation: Two developers run similar Compose files and need resource names that do not collide.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start the first project with a unique project name.

```
docker compose -p trainee-7-a up -d
```

Verify / observe: Run from the directory containing the intended Compose file.

Step 2 — Start a second isolated project with another name.

```
docker compose -p trainee-7-b up -d
```

Verify / observe: Use the same file only if its services can safely run twice and ports do not collide.

Step 3 — List project resources and compare names.

```
docker ps -a
```

Verify / observe: Check that the two projects use distinct generated resource names.

Final conclusion: What other resource conflicts, such as published ports or fixed volume names, could still occur?

Concept to retain: Project names isolate generated resources but cannot eliminate every explicit-name or port conflict.

My notes: _____

SC-488 · Compose · Separate projects with project names

Situation: Two developers run similar Compose files and need resource names that do not collide.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start the first project with a unique project name.

```
docker compose -p trainee-8-a up -d
```

Verify / observe: Run from the directory containing the intended Compose file.

Step 2 — Start a second isolated project with another name.

```
docker compose -p trainee-8-b up -d
```

Verify / observe: Use the same file only if its services can safely run twice and ports do not collide.

Step 3 — List project resources and compare names.

```
docker ps -a
```

Verify / observe: Check that the two projects use distinct generated resource names.

Final conclusion: What other resource conflicts, such as published ports or fixed volume names, could still occur?

Concept to retain: Project names isolate generated resources but cannot eliminate every explicit-name or port conflict.

My notes: _____

SC-489 · Compose · Separate projects with project names

Situation: Two developers run similar Compose files and need resource names that do not collide.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start the first project with a unique project name.

```
docker compose -p trainee-9-a up -d
```

Verify / observe: Run from the directory containing the intended Compose file.

Step 2 — Start a second isolated project with another name.

```
docker compose -p trainee-9-b up -d
```

Verify / observe: Use the same file only if its services can safely run twice and ports do not collide.

Step 3 — List project resources and compare names.

```
docker ps -a
```

Verify / observe: Check that the two projects use distinct generated resource names.

Final conclusion: What other resource conflicts, such as published ports or fixed volume names, could still occur?

Concept to retain: Project names isolate generated resources but cannot eliminate every explicit-name or port conflict.

My notes: _____

SC-490 · Compose · Separate projects with project names

Situation: Two developers run similar Compose files and need resource names that do not collide.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Start the first project with a unique project name.

```
docker compose -p trainee-10-a up -d
```

Verify / observe: Run from the directory containing the intended Compose file.

Step 2 — Start a second isolated project with another name.

```
docker compose -p trainee-10-b up -d
```

Verify / observe: Use the same file only if its services can safely run twice and ports do not collide.

Step 3 — List project resources and compare names.

```
docker ps -a
```

Verify / observe: Check that the two projects use distinct generated resource names.

Final conclusion: What other resource conflicts, such as published ports or fixed volume names, could still occur?

Concept to retain: Project names isolate generated resources but cannot eliminate every explicit-name or port conflict.

My notes: _____

SC-491 · Registries · Diagnose an image-pull failure

Situation: A deployment cannot pull its requested image. You must distinguish a bad reference from authentication or network trouble.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check the exact image reference in the deployment file.

```
docker image ls
```

Verify / observe: Local presence does not prove the remote reference is valid.

Step 2 — Attempt to pull the exact image and tag.

```
docker pull REGISTRY/USERNAME/IMAGE:TAG
```

Verify / observe: Replace all placeholders with the real registry and reference.

Step 3 — Review authentication state and the error text.

```
docker login REGISTRY
```

Verify / observe: Authenticate interactively only if the registry requires it; never paste a password into the command.

Final conclusion: Classify the failure as a reference/tag problem, permissions issue, or connectivity issue based on evidence.

Concept to retain: The exact error message helps separate registry authentication from name resolution and network problems.

My notes: _____

SC-492 · Registries · Diagnose an image-pull failure

Situation: A deployment cannot pull its requested image. You must distinguish a bad reference from authentication or network trouble.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check the exact image reference in the deployment file.

```
docker image ls
```

Verify / observe: Local presence does not prove the remote reference is valid.

Step 2 — Attempt to pull the exact image and tag.

```
docker pull REGISTRY/USERNAME/IMAGE:TAG
```

Verify / observe: Replace all placeholders with the real registry and reference.

Step 3 — Review authentication state and the error text.

```
docker login REGISTRY
```

Verify / observe: Authenticate interactively only if the registry requires it; never paste a password into the command.

Final conclusion: Classify the failure as a reference/tag problem, permissions issue, or connectivity issue based on evidence.

Concept to retain: The exact error message helps separate registry authentication from name resolution and network problems.

My notes: _____

SC-493 · Registries · Diagnose an image-pull failure

Situation: A deployment cannot pull its requested image. You must distinguish a bad reference from authentication or network trouble.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check the exact image reference in the deployment file.

```
docker image ls
```

Verify / observe: Local presence does not prove the remote reference is valid.

Step 2 — Attempt to pull the exact image and tag.

```
docker pull REGISTRY/USERNAME/IMAGE:TAG
```

Verify / observe: Replace all placeholders with the real registry and reference.

Step 3 — Review authentication state and the error text.

```
docker login REGISTRY
```

Verify / observe: Authenticate interactively only if the registry requires it; never paste a password into the command.

Final conclusion: Classify the failure as a reference/tag problem, permissions issue, or connectivity issue based on evidence.

Concept to retain: The exact error message helps separate registry authentication from name resolution and network problems.

My notes: _____

SC-494 · Registries · Diagnose an image-pull failure

Situation: A deployment cannot pull its requested image. You must distinguish a bad reference from authentication or network trouble.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check the exact image reference in the deployment file.

```
docker image ls
```

Verify / observe: Local presence does not prove the remote reference is valid.

Step 2 — Attempt to pull the exact image and tag.

```
docker pull REGISTRY/USERNAME/IMAGE:TAG
```

Verify / observe: Replace all placeholders with the real registry and reference.

Step 3 — Review authentication state and the error text.

```
docker login REGISTRY
```

Verify / observe: Authenticate interactively only if the registry requires it; never paste a password into the command.

Final conclusion: Classify the failure as a reference/tag problem, permissions issue, or connectivity issue based on evidence.

Concept to retain: The exact error message helps separate registry authentication from name resolution and network problems.

My notes: _____

SC-495 · Registries · Diagnose an image-pull failure

Situation: A deployment cannot pull its requested image. You must distinguish a bad reference from authentication or network trouble.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check the exact image reference in the deployment file.

```
docker image ls
```

Verify / observe: Local presence does not prove the remote reference is valid.

Step 2 — Attempt to pull the exact image and tag.

```
docker pull REGISTRY/USERNAME/IMAGE:TAG
```

Verify / observe: Replace all placeholders with the real registry and reference.

Step 3 — Review authentication state and the error text.

```
docker login REGISTRY
```

Verify / observe: Authenticate interactively only if the registry requires it; never paste a password into the command.

Final conclusion: Classify the failure as a reference/tag problem, permissions issue, or connectivity issue based on evidence.

Concept to retain: The exact error message helps separate registry authentication from name resolution and network problems.

My notes: _____

SC-496 · Registries · Diagnose an image-pull failure

Situation: A deployment cannot pull its requested image. You must distinguish a bad reference from authentication or network trouble.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check the exact image reference in the deployment file.

```
docker image ls
```

Verify / observe: Local presence does not prove the remote reference is valid.

Step 2 — Attempt to pull the exact image and tag.

```
docker pull REGISTRY/USERNAME/IMAGE:TAG
```

Verify / observe: Replace all placeholders with the real registry and reference.

Step 3 — Review authentication state and the error text.

```
docker login REGISTRY
```

Verify / observe: Authenticate interactively only if the registry requires it; never paste a password into the command.

Final conclusion: Classify the failure as a reference/tag problem, permissions issue, or connectivity issue based on evidence.

Concept to retain: The exact error message helps separate registry authentication from name resolution and network problems.

My notes: _____

SC-497 · Registries · Diagnose an image-pull failure

Situation: A deployment cannot pull its requested image. You must distinguish a bad reference from authentication or network trouble.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check the exact image reference in the deployment file.

```
docker image ls
```

Verify / observe: Local presence does not prove the remote reference is valid.

Step 2 — Attempt to pull the exact image and tag.

```
docker pull REGISTRY/USERNAME/IMAGE:TAG
```

Verify / observe: Replace all placeholders with the real registry and reference.

Step 3 — Review authentication state and the error text.

```
docker login REGISTRY
```

Verify / observe: Authenticate interactively only if the registry requires it; never paste a password into the command.

Final conclusion: Classify the failure as a reference/tag problem, permissions issue, or connectivity issue based on evidence.

Concept to retain: The exact error message helps separate registry authentication from name resolution and network problems.

My notes: _____

SC-498 · Registries · Diagnose an image-pull failure

Situation: A deployment cannot pull its requested image. You must distinguish a bad reference from authentication or network trouble.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check the exact image reference in the deployment file.

```
docker image ls
```

Verify / observe: Local presence does not prove the remote reference is valid.

Step 2 — Attempt to pull the exact image and tag.

```
docker pull REGISTRY/USERNAME/IMAGE:TAG
```

Verify / observe: Replace all placeholders with the real registry and reference.

Step 3 — Review authentication state and the error text.

```
docker login REGISTRY
```

Verify / observe: Authenticate interactively only if the registry requires it; never paste a password into the command.

Final conclusion: Classify the failure as a reference/tag problem, permissions issue, or connectivity issue based on evidence.

Concept to retain: The exact error message helps separate registry authentication from name resolution and network problems.

My notes: _____

SC-499 · Registries · Diagnose an image-pull failure

Situation: A deployment cannot pull its requested image. You must distinguish a bad reference from authentication or network trouble.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check the exact image reference in the deployment file.

```
docker image ls
```

Verify / observe: Local presence does not prove the remote reference is valid.

Step 2 — Attempt to pull the exact image and tag.

```
docker pull REGISTRY/USERNAME/IMAGE:TAG
```

Verify / observe: Replace all placeholders with the real registry and reference.

Step 3 — Review authentication state and the error text.

```
docker login REGISTRY
```

Verify / observe: Authenticate interactively only if the registry requires it; never paste a password into the command.

Final conclusion: Classify the failure as a reference/tag problem, permissions issue, or connectivity issue based on evidence.

Concept to retain: The exact error message helps separate registry authentication from name resolution and network problems.

My notes: _____

SC-500 · Registries · Diagnose an image-pull failure

Situation: A deployment cannot pull its requested image. You must distinguish a bad reference from authentication or network trouble.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check the exact image reference in the deployment file.

```
docker image ls
```

Verify / observe: Local presence does not prove the remote reference is valid.

Step 2 — Attempt to pull the exact image and tag.

```
docker pull REGISTRY/USERNAME/IMAGE:TAG
```

Verify / observe: Replace all placeholders with the real registry and reference.

Step 3 — Review authentication state and the error text.

```
docker login REGISTRY
```

Verify / observe: Authenticate interactively only if the registry requires it; never paste a password into the command.

Final conclusion: Classify the failure as a reference/tag problem, permissions issue, or connectivity issue based on evidence.

Concept to retain: The exact error message helps separate registry authentication from name resolution and network problems.

My notes: _____

SC-501 · Registries · Promote an image from test to release tag

Situation: A tested local image needs a release tag without rebuilding it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the test image exists.

```
docker image ls local/app-1
```

Verify / observe: Record its image ID.

Step 2 — Create a release tag on the same image.

```
docker image tag local/app-1:test REGISTRY/USERNAME/app:release-1
```

Verify / observe: Replace placeholders and ensure the source tag is correct.

Step 3 — Verify the new tag before pushing.

```
docker image inspect REGISTRY/USERNAME/app:release-1
```

Verify / observe: Compare image IDs where the local registry tag is visible.

Final conclusion: What checks prevent accidentally promoting the wrong image?

Concept to retain: Explicit tags and image-ID verification reduce release mistakes.

My notes: _____

SC-502 · Registries · Promote an image from test to release tag

Situation: A tested local image needs a release tag without rebuilding it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the test image exists.

```
docker image ls local/app-2
```

Verify / observe: Record its image ID.

Step 2 — Create a release tag on the same image.

```
docker image tag local/app-2:test REGISTRY/USERNAME/app:release-2
```

Verify / observe: Replace placeholders and ensure the source tag is correct.

Step 3 — Verify the new tag before pushing.

```
docker image inspect REGISTRY/USERNAME/app:release-2
```

Verify / observe: Compare image IDs where the local registry tag is visible.

Final conclusion: What checks prevent accidentally promoting the wrong image?

Concept to retain: Explicit tags and image-ID verification reduce release mistakes.

My notes: _____

SC-503 · Registries · Promote an image from test to release tag

Situation: A tested local image needs a release tag without rebuilding it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the test image exists.

```
docker image ls local/app-3
```

Verify / observe: Record its image ID.

Step 2 — Create a release tag on the same image.

```
docker image tag local/app-3:test REGISTRY/USERNAME/app:release-3
```

Verify / observe: Replace placeholders and ensure the source tag is correct.

Step 3 — Verify the new tag before pushing.

```
docker image inspect REGISTRY/USERNAME/app:release-3
```

Verify / observe: Compare image IDs where the local registry tag is visible.

Final conclusion: What checks prevent accidentally promoting the wrong image?

Concept to retain: Explicit tags and image-ID verification reduce release mistakes.

My notes: _____

SC-504 · Registries · Promote an image from test to release tag

Situation: A tested local image needs a release tag without rebuilding it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the test image exists.

```
docker image ls local/app-4
```

Verify / observe: Record its image ID.

Step 2 — Create a release tag on the same image.

```
docker image tag local/app-4:test REGISTRY/USERNAME/app:release-4
```

Verify / observe: Replace placeholders and ensure the source tag is correct.

Step 3 — Verify the new tag before pushing.

```
docker image inspect REGISTRY/USERNAME/app:release-4
```

Verify / observe: Compare image IDs where the local registry tag is visible.

Final conclusion: What checks prevent accidentally promoting the wrong image?

Concept to retain: Explicit tags and image-ID verification reduce release mistakes.

My notes: _____

SC-505 · Registries · Promote an image from test to release tag

Situation: A tested local image needs a release tag without rebuilding it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the test image exists.

```
docker image ls local/app-5
```

Verify / observe: Record its image ID.

Step 2 — Create a release tag on the same image.

```
docker image tag local/app-5:test REGISTRY/USERNAME/app:release-5
```

Verify / observe: Replace placeholders and ensure the source tag is correct.

Step 3 — Verify the new tag before pushing.

```
docker image inspect REGISTRY/USERNAME/app:release-5
```

Verify / observe: Compare image IDs where the local registry tag is visible.

Final conclusion: What checks prevent accidentally promoting the wrong image?

Concept to retain: Explicit tags and image-ID verification reduce release mistakes.

My notes: _____

SC-506 · Registries · Promote an image from test to release tag

Situation: A tested local image needs a release tag without rebuilding it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the test image exists.

```
docker image ls local/app-6
```

Verify / observe: Record its image ID.

Step 2 — Create a release tag on the same image.

```
docker image tag local/app-6:test REGISTRY/USERNAME/app:release-6
```

Verify / observe: Replace placeholders and ensure the source tag is correct.

Step 3 — Verify the new tag before pushing.

```
docker image inspect REGISTRY/USERNAME/app:release-6
```

Verify / observe: Compare image IDs where the local registry tag is visible.

Final conclusion: What checks prevent accidentally promoting the wrong image?

Concept to retain: Explicit tags and image-ID verification reduce release mistakes.

My notes: _____

SC-507 · Registries · Promote an image from test to release tag

Situation: A tested local image needs a release tag without rebuilding it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the test image exists.

```
docker image ls local/app-7
```

Verify / observe: Record its image ID.

Step 2 — Create a release tag on the same image.

```
docker image tag local/app-7:test REGISTRY/USERNAME/app:release-7
```

Verify / observe: Replace placeholders and ensure the source tag is correct.

Step 3 — Verify the new tag before pushing.

```
docker image inspect REGISTRY/USERNAME/app:release-7
```

Verify / observe: Compare image IDs where the local registry tag is visible.

Final conclusion: What checks prevent accidentally promoting the wrong image?

Concept to retain: Explicit tags and image-ID verification reduce release mistakes.

My notes: _____

SC-508 · Registries · Promote an image from test to release tag

Situation: A tested local image needs a release tag without rebuilding it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the test image exists.

```
docker image ls local/app-8
```

Verify / observe: Record its image ID.

Step 2 — Create a release tag on the same image.

```
docker image tag local/app-8:test REGISTRY/USERNAME/app:release-8
```

Verify / observe: Replace placeholders and ensure the source tag is correct.

Step 3 — Verify the new tag before pushing.

```
docker image inspect REGISTRY/USERNAME/app:release-8
```

Verify / observe: Compare image IDs where the local registry tag is visible.

Final conclusion: What checks prevent accidentally promoting the wrong image?

Concept to retain: Explicit tags and image-ID verification reduce release mistakes.

My notes: _____

SC-509 · Registries · Promote an image from test to release tag

Situation: A tested local image needs a release tag without rebuilding it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the test image exists.

```
docker image ls local/app-9
```

Verify / observe: Record its image ID.

Step 2 — Create a release tag on the same image.

```
docker image tag local/app-9:test REGISTRY/USERNAME/app:release-9
```

Verify / observe: Replace placeholders and ensure the source tag is correct.

Step 3 — Verify the new tag before pushing.

```
docker image inspect REGISTRY/USERNAME/app:release-9
```

Verify / observe: Compare image IDs where the local registry tag is visible.

Final conclusion: What checks prevent accidentally promoting the wrong image?

Concept to retain: Explicit tags and image-ID verification reduce release mistakes.

My notes: _____

SC-510 · Registries · Promote an image from test to release tag

Situation: A tested local image needs a release tag without rebuilding it.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Confirm the test image exists.

```
docker image ls local/app-10
```

Verify / observe: Record its image ID.

Step 2 — Create a release tag on the same image.

```
docker image tag local/app-10:test REGISTRY/USERNAME/app:release-10
```

Verify / observe: Replace placeholders and ensure the source tag is correct.

Step 3 — Verify the new tag before pushing.

```
docker image inspect REGISTRY/USERNAME/app:release-10
```

Verify / observe: Compare image IDs where the local registry tag is visible.

Final conclusion: What checks prevent accidentally promoting the wrong image?

Concept to retain: Explicit tags and image-ID verification reduce release mistakes.

My notes: _____

SC-511 · Security · Run a container without extra privileges

Situation: A simple utility container needs no special capabilities or privileged host access.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run the utility with a non-root user.

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Verify / observe: Confirm the UID/GID in output.

Step 2 — Drop all capabilities for the same simple task.

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Verify / observe: Confirm the command succeeds.

Step 3 — Compare with the default configuration of a disposable container.

```
docker inspect --format '{{json .HostConfig.CapAdd}} {{json .HostConfig.CapDrop}}' app-1
```

Verify / observe: Use an existing lab container or adapt the target.

Final conclusion: What evidence would justify adding back a capability or running with elevated privileges?

Concept to retain: Grant only the privileges the application demonstrably needs.

My notes: _____

SC-512 · Security · Run a container without extra privileges

Situation: A simple utility container needs no special capabilities or privileged host access.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run the utility with a non-root user.

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Verify / observe: Confirm the UID/GID in output.

Step 2 — Drop all capabilities for the same simple task.

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Verify / observe: Confirm the command succeeds.

Step 3 — Compare with the default configuration of a disposable container.

```
docker inspect --format '{{json .HostConfig.CapAdd}} {{json .HostConfig.CapDrop}}' app-2
```

Verify / observe: Use an existing lab container or adapt the target.

Final conclusion: What evidence would justify adding back a capability or running with elevated privileges?

Concept to retain: Grant only the privileges the application demonstrably needs.

My notes: _____

SC-513 · Security · Run a container without extra privileges

Situation: A simple utility container needs no special capabilities or privileged host access.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run the utility with a non-root user.

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Verify / observe: Confirm the UID/GID in output.

Step 2 — Drop all capabilities for the same simple task.

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Verify / observe: Confirm the command succeeds.

Step 3 — Compare with the default configuration of a disposable container.

```
docker inspect --format '{{json .HostConfig.CapAdd}} {{json .HostConfig.CapDrop}}' app-3
```

Verify / observe: Use an existing lab container or adapt the target.

Final conclusion: What evidence would justify adding back a capability or running with elevated privileges?

Concept to retain: Grant only the privileges the application demonstrably needs.

My notes: _____

SC-514 · Security · Run a container without extra privileges

Situation: A simple utility container needs no special capabilities or privileged host access.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run the utility with a non-root user.

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Verify / observe: Confirm the UID/GID in output.

Step 2 — Drop all capabilities for the same simple task.

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Verify / observe: Confirm the command succeeds.

Step 3 — Compare with the default configuration of a disposable container.

```
docker inspect --format '{{json .HostConfig.CapAdd}} {{json .HostConfig.CapDrop}}' app-4
```

Verify / observe: Use an existing lab container or adapt the target.

Final conclusion: What evidence would justify adding back a capability or running with elevated privileges?

Concept to retain: Grant only the privileges the application demonstrably needs.

My notes: _____

SC-515 · Security · Run a container without extra privileges

Situation: A simple utility container needs no special capabilities or privileged host access.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run the utility with a non-root user.

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Verify / observe: Confirm the UID/GID in output.

Step 2 — Drop all capabilities for the same simple task.

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Verify / observe: Confirm the command succeeds.

Step 3 — Compare with the default configuration of a disposable container.

```
docker inspect --format '{{json .HostConfig.CapAdd}} {{json .HostConfig.CapDrop}}' app-5
```

Verify / observe: Use an existing lab container or adapt the target.

Final conclusion: What evidence would justify adding back a capability or running with elevated privileges?

Concept to retain: Grant only the privileges the application demonstrably needs.

My notes: _____

SC-516 · Security · Run a container without extra privileges

Situation: A simple utility container needs no special capabilities or privileged host access.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run the utility with a non-root user.

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Verify / observe: Confirm the UID/GID in output.

Step 2 — Drop all capabilities for the same simple task.

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Verify / observe: Confirm the command succeeds.

Step 3 — Compare with the default configuration of a disposable container.

```
docker inspect --format '{{json .HostConfig.CapAdd}} {{json .HostConfig.CapDrop}}' app-6
```

Verify / observe: Use an existing lab container or adapt the target.

Final conclusion: What evidence would justify adding back a capability or running with elevated privileges?

Concept to retain: Grant only the privileges the application demonstrably needs.

My notes: _____

SC-517 · Security · Run a container without extra privileges

Situation: A simple utility container needs no special capabilities or privileged host access.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run the utility with a non-root user.

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Verify / observe: Confirm the UID/GID in output.

Step 2 — Drop all capabilities for the same simple task.

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Verify / observe: Confirm the command succeeds.

Step 3 — Compare with the default configuration of a disposable container.

```
docker inspect --format '{{json .HostConfig.CapAdd}} {{json .HostConfig.CapDrop}}' app-7
```

Verify / observe: Use an existing lab container or adapt the target.

Final conclusion: What evidence would justify adding back a capability or running with elevated privileges?

Concept to retain: Grant only the privileges the application demonstrably needs.

My notes: _____

SC-518 · Security · Run a container without extra privileges

Situation: A simple utility container needs no special capabilities or privileged host access.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run the utility with a non-root user.

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Verify / observe: Confirm the UID/GID in output.

Step 2 — Drop all capabilities for the same simple task.

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Verify / observe: Confirm the command succeeds.

Step 3 — Compare with the default configuration of a disposable container.

```
docker inspect --format '{{json .HostConfig.CapAdd}} {{json .HostConfig.CapDrop}}' app-8
```

Verify / observe: Use an existing lab container or adapt the target.

Final conclusion: What evidence would justify adding back a capability or running with elevated privileges?

Concept to retain: Grant only the privileges the application demonstrably needs.

My notes: _____

SC-519 · Security · Run a container without extra privileges

Situation: A simple utility container needs no special capabilities or privileged host access.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run the utility with a non-root user.

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Verify / observe: Confirm the UID/GID in output.

Step 2 — Drop all capabilities for the same simple task.

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Verify / observe: Confirm the command succeeds.

Step 3 — Compare with the default configuration of a disposable container.

```
docker inspect --format '{{json .HostConfig.CapAdd}} {{json .HostConfig.CapDrop}}' app-9
```

Verify / observe: Use an existing lab container or adapt the target.

Final conclusion: What evidence would justify adding back a capability or running with elevated privileges?

Concept to retain: Grant only the privileges the application demonstrably needs.

My notes: _____

SC-520 · Security · Run a container without extra privileges

Situation: A simple utility container needs no special capabilities or privileged host access.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Run the utility with a non-root user.

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Verify / observe: Confirm the UID/GID in output.

Step 2 — Drop all capabilities for the same simple task.

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Verify / observe: Confirm the command succeeds.

Step 3 — Compare with the default configuration of a disposable container.

```
docker inspect --format '{{json .HostConfig.CapAdd}} {{json .HostConfig.CapDrop}}' app-10
```

Verify / observe: Use an existing lab container or adapt the target.

Final conclusion: What evidence would justify adding back a capability or running with elevated privileges?

Concept to retain: Grant only the privileges the application demonstrably needs.

My notes: _____

SC-521 · Security · Avoid embedding credentials in images

Situation: A Dockerfile or build command appears to include a password. You need a safer approach.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Search the Dockerfile and build scripts for credential-like literals.

```
grep -RniE 'password|token|secret|apikey' Dockerfile .dockerignore . 2>/dev/null
```

Verify / observe: Review results carefully; avoid printing or sharing real secret values.

Step 2 — Check whether sensitive files enter the build context.

```
cat .dockerignore
```

Verify / observe: Ensure secret files are excluded from the context.

Step 3 — Review the official build-secret mechanism available in your builder.

```
docker buildx build --help
```

Verify / observe: Look for secret-mount support in the installed Buildx version and consult its documentation.

Final conclusion: Explain why ARG, ENV, and copied credential files can leak into build metadata or layers.

Concept to retain: Use dedicated secret mechanisms and keep sensitive files out of build contexts.

My notes: _____

SC-522 · Security · Avoid embedding credentials in images

Situation: A Dockerfile or build command appears to include a password. You need a safer approach.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Search the Dockerfile and build scripts for credential-like literals.

```
grep -RniE 'password|token|secret|apikey' Dockerfile .dockerignore . 2>/dev/null
```

Verify / observe: Review results carefully; avoid printing or sharing real secret values.

Step 2 — Check whether sensitive files enter the build context.

```
cat .dockerignore
```

Verify / observe: Ensure secret files are excluded from the context.

Step 3 — Review the official build-secret mechanism available in your builder.

```
docker buildx build --help
```

Verify / observe: Look for secret-mount support in the installed Buildx version and consult its documentation.

Final conclusion: Explain why ARG, ENV, and copied credential files can leak into build metadata or layers.

Concept to retain: Use dedicated secret mechanisms and keep sensitive files out of build contexts.

My notes: _____

SC-523 · Security · Avoid embedding credentials in images

Situation: A Dockerfile or build command appears to include a password. You need a safer approach.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Search the Dockerfile and build scripts for credential-like literals.

```
grep -RniE 'password|token|secret|apikey' Dockerfile .dockerignore . 2>/dev/null
```

Verify / observe: Review results carefully; avoid printing or sharing real secret values.

Step 2 — Check whether sensitive files enter the build context.

```
cat .dockerignore
```

Verify / observe: Ensure secret files are excluded from the context.

Step 3 — Review the official build-secret mechanism available in your builder.

```
docker buildx build --help
```

Verify / observe: Look for secret-mount support in the installed Buildx version and consult its documentation.

Final conclusion: Explain why ARG, ENV, and copied credential files can leak into build metadata or layers.

Concept to retain: Use dedicated secret mechanisms and keep sensitive files out of build contexts.

My notes: _____

SC-524 · Security · Avoid embedding credentials in images

Situation: A Dockerfile or build command appears to include a password. You need a safer approach.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Search the Dockerfile and build scripts for credential-like literals.

```
grep -RniE 'password|token|secret|apikey' Dockerfile .dockerignore . 2>/dev/null
```

Verify / observe: Review results carefully; avoid printing or sharing real secret values.

Step 2 — Check whether sensitive files enter the build context.

```
cat .dockerignore
```

Verify / observe: Ensure secret files are excluded from the context.

Step 3 — Review the official build-secret mechanism available in your builder.

```
docker buildx build --help
```

Verify / observe: Look for secret-mount support in the installed Buildx version and consult its documentation.

Final conclusion: Explain why ARG, ENV, and copied credential files can leak into build metadata or layers.

Concept to retain: Use dedicated secret mechanisms and keep sensitive files out of build contexts.

My notes: _____

SC-525 · Security · Avoid embedding credentials in images

Situation: A Dockerfile or build command appears to include a password. You need a safer approach.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Search the Dockerfile and build scripts for credential-like literals.

```
grep -RniE 'password|token|secret|apikey' Dockerfile .dockerignore . 2>/dev/null
```

Verify / observe: Review results carefully; avoid printing or sharing real secret values.

Step 2 — Check whether sensitive files enter the build context.

```
cat .dockerignore
```

Verify / observe: Ensure secret files are excluded from the context.

Step 3 — Review the official build-secret mechanism available in your builder.

```
docker buildx build --help
```

Verify / observe: Look for secret-mount support in the installed Buildx version and consult its documentation.

Final conclusion: Explain why ARG, ENV, and copied credential files can leak into build metadata or layers.

Concept to retain: Use dedicated secret mechanisms and keep sensitive files out of build contexts.

My notes: _____

SC-526 · Security · Avoid embedding credentials in images

Situation: A Dockerfile or build command appears to include a password. You need a safer approach.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Search the Dockerfile and build scripts for credential-like literals.

```
grep -RniE 'password|token|secret|apikey' Dockerfile .dockerignore . 2>/dev/null
```

Verify / observe: Review results carefully; avoid printing or sharing real secret values.

Step 2 — Check whether sensitive files enter the build context.

```
cat .dockerignore
```

Verify / observe: Ensure secret files are excluded from the context.

Step 3 — Review the official build-secret mechanism available in your builder.

```
docker buildx build --help
```

Verify / observe: Look for secret-mount support in the installed Buildx version and consult its documentation.

Final conclusion: Explain why ARG, ENV, and copied credential files can leak into build metadata or layers.

Concept to retain: Use dedicated secret mechanisms and keep sensitive files out of build contexts.

My notes: _____

SC-527 · Security · Avoid embedding credentials in images

Situation: A Dockerfile or build command appears to include a password. You need a safer approach.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Search the Dockerfile and build scripts for credential-like literals.

```
grep -RniE 'password|token|secret|apikey' Dockerfile .dockerignore . 2>/dev/null
```

Verify / observe: Review results carefully; avoid printing or sharing real secret values.

Step 2 — Check whether sensitive files enter the build context.

```
cat .dockerignore
```

Verify / observe: Ensure secret files are excluded from the context.

Step 3 — Review the official build-secret mechanism available in your builder.

```
docker buildx build --help
```

Verify / observe: Look for secret-mount support in the installed Buildx version and consult its documentation.

Final conclusion: Explain why ARG, ENV, and copied credential files can leak into build metadata or layers.

Concept to retain: Use dedicated secret mechanisms and keep sensitive files out of build contexts.

My notes: _____

SC-528 · Security · Avoid embedding credentials in images

Situation: A Dockerfile or build command appears to include a password. You need a safer approach.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Search the Dockerfile and build scripts for credential-like literals.

```
grep -RniE 'password|token|secret|apikey' Dockerfile .dockerignore . 2>/dev/null
```

Verify / observe: Review results carefully; avoid printing or sharing real secret values.

Step 2 — Check whether sensitive files enter the build context.

```
cat .dockerignore
```

Verify / observe: Ensure secret files are excluded from the context.

Step 3 — Review the official build-secret mechanism available in your builder.

```
docker buildx build --help
```

Verify / observe: Look for secret-mount support in the installed Buildx version and consult its documentation.

Final conclusion: Explain why ARG, ENV, and copied credential files can leak into build metadata or layers.

Concept to retain: Use dedicated secret mechanisms and keep sensitive files out of build contexts.

My notes: _____

SC-529 · Security · Avoid embedding credentials in images

Situation: A Dockerfile or build command appears to include a password. You need a safer approach.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Search the Dockerfile and build scripts for credential-like literals.

```
grep -RniE 'password|token|secret|apikey' Dockerfile .dockerignore . 2>/dev/null
```

Verify / observe: Review results carefully; avoid printing or sharing real secret values.

Step 2 — Check whether sensitive files enter the build context.

```
cat .dockerignore
```

Verify / observe: Ensure secret files are excluded from the context.

Step 3 — Review the official build-secret mechanism available in your builder.

```
docker buildx build --help
```

Verify / observe: Look for secret-mount support in the installed Buildx version and consult its documentation.

Final conclusion: Explain why ARG, ENV, and copied credential files can leak into build metadata or layers.

Concept to retain: Use dedicated secret mechanisms and keep sensitive files out of build contexts.

My notes: _____

SC-530 · Security · Avoid embedding credentials in images

Situation: A Dockerfile or build command appears to include a password. You need a safer approach.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Search the Dockerfile and build scripts for credential-like literals.

```
grep -RniE 'password|token|secret|apikey' Dockerfile .dockerignore . 2>/dev/null
```

Verify / observe: Review results carefully; avoid printing or sharing real secret values.

Step 2 — Check whether sensitive files enter the build context.

```
cat .dockerignore
```

Verify / observe: Ensure secret files are excluded from the context.

Step 3 — Review the official build-secret mechanism available in your builder.

```
docker buildx build --help
```

Verify / observe: Look for secret-mount support in the installed Buildx version and consult its documentation.

Final conclusion: Explain why ARG, ENV, and copied credential files can leak into build metadata or layers.

Concept to retain: Use dedicated secret mechanisms and keep sensitive files out of build contexts.

My notes: _____

SC-531 · Security · Audit exposed ports and running containers

Situation: Before a demo, you must determine which services are running and which ports are published.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List running containers with port mappings.

```
docker ps
```

Verify / observe: Record container names, images, and published ports.

Step 2 — Inspect a specific container's port configuration.

```
docker port web-1
```

Verify / observe: Use an existing lab web container.

Step 3 — Inspect its security and runtime configuration.

```
docker inspect web-1
```

Verify / observe: Review user, mounts, network mode, capabilities, and privileged setting.

Final conclusion: Which published ports are necessary, and which runtime settings deserve follow-up?

Concept to retain: An exposure review should combine port mappings with the container's runtime configuration.

My notes: _____

SC-532 · Security · Audit exposed ports and running containers

Situation: Before a demo, you must determine which services are running and which ports are published.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List running containers with port mappings.

```
docker ps
```

Verify / observe: Record container names, images, and published ports.

Step 2 — Inspect a specific container's port configuration.

```
docker port web-2
```

Verify / observe: Use an existing lab web container.

Step 3 — Inspect its security and runtime configuration.

```
docker inspect web-2
```

Verify / observe: Review user, mounts, network mode, capabilities, and privileged setting.

Final conclusion: Which published ports are necessary, and which runtime settings deserve follow-up?

Concept to retain: An exposure review should combine port mappings with the container's runtime configuration.

My notes: _____

SC-533 · Security · Audit exposed ports and running containers

Situation: Before a demo, you must determine which services are running and which ports are published.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List running containers with port mappings.

```
docker ps
```

Verify / observe: Record container names, images, and published ports.

Step 2 — Inspect a specific container's port configuration.

```
docker port web-3
```

Verify / observe: Use an existing lab web container.

Step 3 — Inspect its security and runtime configuration.

```
docker inspect web-3
```

Verify / observe: Review user, mounts, network mode, capabilities, and privileged setting.

Final conclusion: Which published ports are necessary, and which runtime settings deserve follow-up?

Concept to retain: An exposure review should combine port mappings with the container's runtime configuration.

My notes: _____

SC-534 · Security · Audit exposed ports and running containers

Situation: Before a demo, you must determine which services are running and which ports are published.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List running containers with port mappings.

```
docker ps
```

Verify / observe: Record container names, images, and published ports.

Step 2 — Inspect a specific container's port configuration.

```
docker port web-4
```

Verify / observe: Use an existing lab web container.

Step 3 — Inspect its security and runtime configuration.

```
docker inspect web-4
```

Verify / observe: Review user, mounts, network mode, capabilities, and privileged setting.

Final conclusion: Which published ports are necessary, and which runtime settings deserve follow-up?

Concept to retain: An exposure review should combine port mappings with the container's runtime configuration.

My notes: _____

SC-535 · Security · Audit exposed ports and running containers

Situation: Before a demo, you must determine which services are running and which ports are published.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List running containers with port mappings.

```
docker ps
```

Verify / observe: Record container names, images, and published ports.

Step 2 — Inspect a specific container's port configuration.

```
docker port web-5
```

Verify / observe: Use an existing lab web container.

Step 3 — Inspect its security and runtime configuration.

```
docker inspect web-5
```

Verify / observe: Review user, mounts, network mode, capabilities, and privileged setting.

Final conclusion: Which published ports are necessary, and which runtime settings deserve follow-up?

Concept to retain: An exposure review should combine port mappings with the container's runtime configuration.

My notes: _____

SC-536 · Security · Audit exposed ports and running containers

Situation: Before a demo, you must determine which services are running and which ports are published.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List running containers with port mappings.

```
docker ps
```

Verify / observe: Record container names, images, and published ports.

Step 2 — Inspect a specific container's port configuration.

```
docker port web-6
```

Verify / observe: Use an existing lab web container.

Step 3 — Inspect its security and runtime configuration.

```
docker inspect web-6
```

Verify / observe: Review user, mounts, network mode, capabilities, and privileged setting.

Final conclusion: Which published ports are necessary, and which runtime settings deserve follow-up?

Concept to retain: An exposure review should combine port mappings with the container's runtime configuration.

My notes: _____

SC-537 · Security · Audit exposed ports and running containers

Situation: Before a demo, you must determine which services are running and which ports are published.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List running containers with port mappings.

```
docker ps
```

Verify / observe: Record container names, images, and published ports.

Step 2 — Inspect a specific container's port configuration.

```
docker port web-7
```

Verify / observe: Use an existing lab web container.

Step 3 — Inspect its security and runtime configuration.

```
docker inspect web-7
```

Verify / observe: Review user, mounts, network mode, capabilities, and privileged setting.

Final conclusion: Which published ports are necessary, and which runtime settings deserve follow-up?

Concept to retain: An exposure review should combine port mappings with the container's runtime configuration.

My notes: _____

SC-538 · Security · Audit exposed ports and running containers

Situation: Before a demo, you must determine which services are running and which ports are published.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List running containers with port mappings.

```
docker ps
```

Verify / observe: Record container names, images, and published ports.

Step 2 — Inspect a specific container's port configuration.

```
docker port web-8
```

Verify / observe: Use an existing lab web container.

Step 3 — Inspect its security and runtime configuration.

```
docker inspect web-8
```

Verify / observe: Review user, mounts, network mode, capabilities, and privileged setting.

Final conclusion: Which published ports are necessary, and which runtime settings deserve follow-up?

Concept to retain: An exposure review should combine port mappings with the container's runtime configuration.

My notes: _____

SC-539 · Security · Audit exposed ports and running containers

Situation: Before a demo, you must determine which services are running and which ports are published.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List running containers with port mappings.

```
docker ps
```

Verify / observe: Record container names, images, and published ports.

Step 2 — Inspect a specific container's port configuration.

```
docker port web-9
```

Verify / observe: Use an existing lab web container.

Step 3 — Inspect its security and runtime configuration.

```
docker inspect web-9
```

Verify / observe: Review user, mounts, network mode, capabilities, and privileged setting.

Final conclusion: Which published ports are necessary, and which runtime settings deserve follow-up?

Concept to retain: An exposure review should combine port mappings with the container's runtime configuration.

My notes: _____

SC-540 · Security · Audit exposed ports and running containers

Situation: Before a demo, you must determine which services are running and which ports are published.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — List running containers with port mappings.

```
docker ps
```

Verify / observe: Record container names, images, and published ports.

Step 2 — Inspect a specific container's port configuration.

```
docker port web-10
```

Verify / observe: Use an existing lab web container.

Step 3 — Inspect its security and runtime configuration.

```
docker inspect web-10
```

Verify / observe: Review user, mounts, network mode, capabilities, and privileged setting.

Final conclusion: Which published ports are necessary, and which runtime settings deserve follow-up?

Concept to retain: An exposure review should combine port mappings with the container's runtime configuration.

My notes: _____

SC-541 · Security · Check health status instead of only process status

Situation: A service process is running but users report errors. You need to distinguish running from healthy.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check whether the container is running.

```
docker ps --filter name=app-1
```

Verify / observe: Running state only reports the container process state.

Step 2 — Inspect health status if the image defines a health check.

```
docker inspect --format '{{if .State.Health}}{{.State.Health.Status}}{{else}}no-healthcheck{{end}}' app-1
```

Verify / observe: A missing health check is different from an unhealthy result.

Step 3 — Read recent logs and health-check details.

```
docker inspect app-1
```

Verify / observe: Review State.Health.Log if present and compare with application logs.

Final conclusion: Can you conclude the application is healthy from running state alone? Explain the evidence needed.

Concept to retain: Container liveness and application health are different signals.

My notes: _____

SC-542 · Security · Check health status instead of only process status

Situation: A service process is running but users report errors. You need to distinguish running from healthy.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check whether the container is running.

```
docker ps --filter name=app-2
```

Verify / observe: Running state only reports the container process state.

Step 2 — Inspect health status if the image defines a health check.

```
docker inspect --format '{{if .State.Health}}{{.State.Health.Status}}{{else}}no-healthcheck{{end}}' app-2
```

Verify / observe: A missing health check is different from an unhealthy result.

Step 3 — Read recent logs and health-check details.

```
docker inspect app-2
```

Verify / observe: Review State.Health.Log if present and compare with application logs.

Final conclusion: Can you conclude the application is healthy from running state alone? Explain the evidence needed.

Concept to retain: Container liveness and application health are different signals.

My notes: _____

SC-543 · Security · Check health status instead of only process status

Situation: A service process is running but users report errors. You need to distinguish running from healthy.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check whether the container is running.

```
docker ps --filter name=app-3
```

Verify / observe: Running state only reports the container process state.

Step 2 — Inspect health status if the image defines a health check.

```
docker inspect --format '{{if .State.Health}}{{.State.Health.Status}}{{else}}no-healthcheck{{end}}' app-3
```

Verify / observe: A missing health check is different from an unhealthy result.

Step 3 — Read recent logs and health-check details.

```
docker inspect app-3
```

Verify / observe: Review State.Health.Log if present and compare with application logs.

Final conclusion: Can you conclude the application is healthy from running state alone? Explain the evidence needed.

Concept to retain: Container liveness and application health are different signals.

My notes: _____

SC-544 · Security · Check health status instead of only process status

Situation: A service process is running but users report errors. You need to distinguish running from healthy.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check whether the container is running.

```
docker ps --filter name=app-4
```

Verify / observe: Running state only reports the container process state.

Step 2 — Inspect health status if the image defines a health check.

```
docker inspect --format '{{if .State.Health}}{{.State.Health.Status}}{{else}}no-healthcheck{{end}}' app-4
```

Verify / observe: A missing health check is different from an unhealthy result.

Step 3 — Read recent logs and health-check details.

```
docker inspect app-4
```

Verify / observe: Review State.Health.Log if present and compare with application logs.

Final conclusion: Can you conclude the application is healthy from running state alone? Explain the evidence needed.

Concept to retain: Container liveness and application health are different signals.

My notes: _____

SC-545 · Security · Check health status instead of only process status

Situation: A service process is running but users report errors. You need to distinguish running from healthy.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check whether the container is running.

```
docker ps --filter name=app-5
```

Verify / observe: Running state only reports the container process state.

Step 2 — Inspect health status if the image defines a health check.

```
docker inspect --format '{{if .State.Health}}{{.State.Health.Status}}{{else}}no-healthcheck{{end}}' app-5
```

Verify / observe: A missing health check is different from an unhealthy result.

Step 3 — Read recent logs and health-check details.

```
docker inspect app-5
```

Verify / observe: Review State.Health.Log if present and compare with application logs.

Final conclusion: Can you conclude the application is healthy from running state alone? Explain the evidence needed.

Concept to retain: Container liveness and application health are different signals.

My notes: _____

SC-546 · Security · Check health status instead of only process status

Situation: A service process is running but users report errors. You need to distinguish running from healthy.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check whether the container is running.

```
docker ps --filter name=app-6
```

Verify / observe: Running state only reports the container process state.

Step 2 — Inspect health status if the image defines a health check.

```
docker inspect --format '{{if .State.Health}}{{.State.Health.Status}}{{else}}no-healthcheck{{end}}' app-6
```

Verify / observe: A missing health check is different from an unhealthy result.

Step 3 — Read recent logs and health-check details.

```
docker inspect app-6
```

Verify / observe: Review State.Health.Log if present and compare with application logs.

Final conclusion: Can you conclude the application is healthy from running state alone? Explain the evidence needed.

Concept to retain: Container liveness and application health are different signals.

My notes: _____

SC-547 · Security · Check health status instead of only process status

Situation: A service process is running but users report errors. You need to distinguish running from healthy.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check whether the container is running.

```
docker ps --filter name=app-7
```

Verify / observe: Running state only reports the container process state.

Step 2 — Inspect health status if the image defines a health check.

```
docker inspect --format '{{if .State.Health}}{{.State.Health.Status}}{{else}}no-healthcheck{{end}}' app-7
```

Verify / observe: A missing health check is different from an unhealthy result.

Step 3 — Read recent logs and health-check details.

```
docker inspect app-7
```

Verify / observe: Review State.Health.Log if present and compare with application logs.

Final conclusion: Can you conclude the application is healthy from running state alone? Explain the evidence needed.

Concept to retain: Container liveness and application health are different signals.

My notes: _____

SC-548 · Security · Check health status instead of only process status

Situation: A service process is running but users report errors. You need to distinguish running from healthy.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check whether the container is running.

```
docker ps --filter name=app-8
```

Verify / observe: Running state only reports the container process state.

Step 2 — Inspect health status if the image defines a health check.

```
docker inspect --format '{{if .State.Health}}{{.State.Health.Status}}{{else}}no-healthcheck{{end}}' app-8
```

Verify / observe: A missing health check is different from an unhealthy result.

Step 3 — Read recent logs and health-check details.

```
docker inspect app-8
```

Verify / observe: Review State.Health.Log if present and compare with application logs.

Final conclusion: Can you conclude the application is healthy from running state alone? Explain the evidence needed.

Concept to retain: Container liveness and application health are different signals.

My notes: _____

SC-549 · Security · Check health status instead of only process status

Situation: A service process is running but users report errors. You need to distinguish running from healthy.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check whether the container is running.

```
docker ps --filter name=app-9
```

Verify / observe: Running state only reports the container process state.

Step 2 — Inspect health status if the image defines a health check.

```
docker inspect --format '{{if .State.Health}}{{.State.Health.Status}}{{else}}no-healthcheck{{end}}' app-9
```

Verify / observe: A missing health check is different from an unhealthy result.

Step 3 — Read recent logs and health-check details.

```
docker inspect app-9
```

Verify / observe: Review State.Health.Log if present and compare with application logs.

Final conclusion: Can you conclude the application is healthy from running state alone? Explain the evidence needed.

Concept to retain: Container liveness and application health are different signals.

My notes: _____

SC-550 · Security · Check health status instead of only process status

Situation: A service process is running but users report errors. You need to distinguish running from healthy.

Your mission: Complete the sequence below in order. Keep notes about command output and any errors.

Step 1 — Check whether the container is running.

```
docker ps --filter name=app-10
```

Verify / observe: Running state only reports the container process state.

Step 2 — Inspect health status if the image defines a health check.

```
docker inspect --format '{{if .State.Health}}{{.State.Health.Status}}{{else}}no-healthcheck{{end}}' app-10
```

Verify / observe: A missing health check is different from an unhealthy result.

Step 3 — Read recent logs and health-check details.

```
docker inspect app-10
```

Verify / observe: Review State.Health.Log if present and compare with application logs.

Final conclusion: Can you conclude the application is healthy from running state alone? Explain the evidence needed.

Concept to retain: Container liveness and application health are different signals.

My notes: _____

Final practical assessment

- ■ I verify the active Docker context before running destructive commands.
- ■ I can distinguish an image, a container, a volume, and a network.
- ■ I can explain the evidence from docker ps, inspect, logs, events, stats, and system df.
- ■ I can build, tag, save, load, pull, and publish images with explicit references.
- ■ I can diagnose port publishing, DNS, container exit, and health-check issues.
- ■ I can demonstrate volume persistence and test a backup restore.
- ■ I can validate, start, inspect, scale, and safely tear down a Compose project.
- ■ I understand the difference between standalone containers and Swarm services/stacks.
- ■ I can identify risky privileges, mounts, published ports, and secret-handling practices.
- ■ I can write an evidence-based conclusion separating facts, assumptions, and next steps.

Useful built-in discovery commands

```
docker --help
docker run --help
docker image --help
docker container --help
docker volume --help
docker network --help
docker compose --help
docker buildx --help
docker service --help
docker swarm --help
```

Official documentation: https://docs.docker.com/ · Docker CLI reference: https://docs.docker.com/reference/cli/docker/